RS =150/=

SPRING NOTES

BY

SHEKHAR SIR

SATYATECHNOLOGIES

SRI RAGHAVENDRA XEROX

Software Languages Material Available

Beside Bangalore Ayyangar Bakery, Opp. C DAC, Ameerpet, Hyderabad.

Cell: 9951596199

Spring

2-2-2015

C, C++, Java are Programming languages

Jdbc, Servlet, JSP, EJP, are technologies.

Struts, Spring, Hibernate are framework

- framework is a special software i.e built on technology having the ability to generate common logic of the Appln dynamically.
- framework is a special s/w that provides abstraction layer on existing technology to simplifies the application development process

Plane Jdbc Appln

Register Jdbc Driver
Establish the conn.
Create Statement object
} (Common logics)

Send & execute query
Gather & process result
} Appln specific logics

Exception handling
Close Jdbc Objects
} Common logics

Boiler Plate code Problem

Spring-Jdbc App

→ Get JdbcTemplate class object.
→ Send & execute query.
→ Gather & process the results

While coorking with technology we should develop both Common & Appln specific logic this use Boiler Plate code Problem. writing same logic in multiple Appln of same categories rises boiler plate code Problem

→ While working with framework we just need to take care of Appln specific logic development, because common logic of the appln will be generated Automatically by framework

There are 2 types of framework

(1) Invasive framework :- Here the classes of framework Appln extend or implement class or interface of framework API that means we can execute these classes outside the framework enviment eg. Struts 1.x.

(2) non-invasive framework :- Here classes of framework Appln do not implement or extend framework API interfaces, class that means we can execute these classes outside of the framework enviment eg Spring, Hibernate, Struts 2.x, JSF etc.

Diff. categories Java framework.

(a) Web frameworks

→ Provides abstraction layer on servlet, jsp, technologies & allows to developers mvc based web Appln

eg. Struts, jsf, webwork & etc ...

(b) ORM Frameworks:

→ Provides abstraction layer on jdbc technology & allows to develop objects based o-r mapping persistence logic

eg. hibernate, iBatis, AJB & etc

(c) Application framework | Java JEE framework

⇒ Provides abstraction layer multiple java, jee technologies (jdbc, servlet, jsp, ejb, jms & etc) & allows to develop all kinds of Applications.

eg. Spring.

Nataraz@gmail.com

3-2-2015

Spring Provides Abstraction to all technology.

Spring

type: Application framework

Version: 3.x | 4.x (compatable with JdK 1.6 ↑)

Vendor : Interface21

creator : Mr. Rod Jhonson

open source

To Download : download as zip file from www.springframework.org
www.io.spring.org.

Zip file : → spring-framework-3.0.5.RELEASE-with-docs.zip)

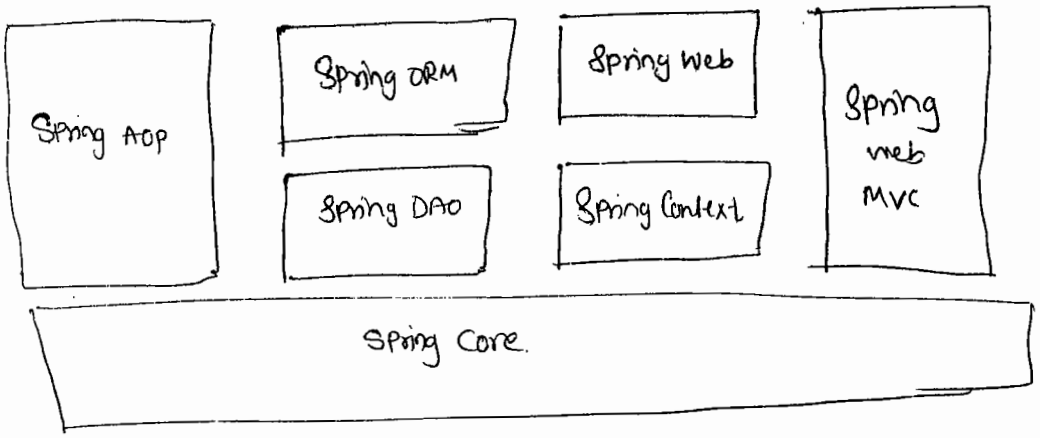
To install s/w : Extract zip file

Online tutorials : roseindia, javas4s, tutorialpoint.

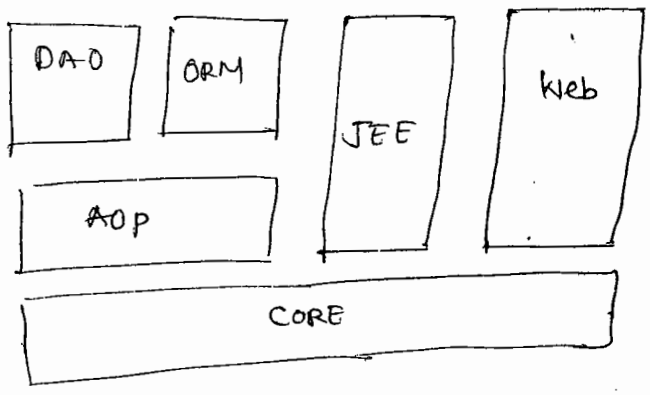
Ref. Books : Pro spring
Spring in Action

<SP_home>\dist --> gives spring libraries (jar file)
<SP_home>\docs --> gives api docs, reference doc
<SP_home>\Project → gives sample apps, Projects.
<SP_home>\src → gives source code.

Spring 1.x framework overview.



Spring 2.x overview diagram



Spring 2.x web = Spring 1.x web + Spring 1.x web mvc modules

Spring 2.x JEE = Spring 1.x Context module.

Spring Core: Provides IOC containers to Perform Dependency Injections (Assigning values to the resources dynamically)

Spring DAO → Provides abstraction layer on JDBC to develop Persistence logic.

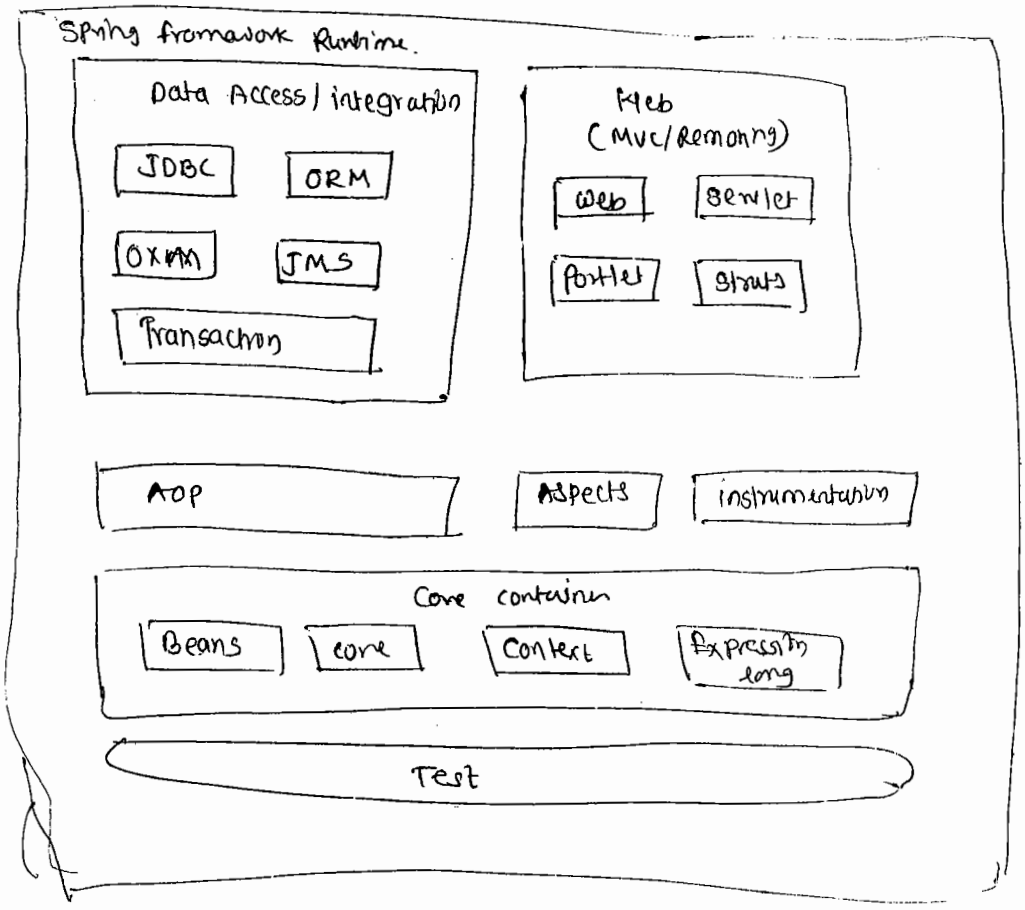
Spring ORM : Provides abstraction layer on ORM frameworks like hibernate, toplink and etc. to develope O-R mapping Persistence logic

Spring web : a) Provides Plugins for Struts+ spring, jsf + Spring integration
b) Provides Spring webmvc as web f|w to develop web Applications

Spring JEE : Provides abstraction layer on multiple java, JEE technologies like Jndi, jms, rmi, jta, f etc. to develope JEE apps

Spring Aop : Provides a different mechanism or methodology to develope & apply middleware services like logging, security, Transaction & etc.

3.x Overview diagram



Def^{n} Spring is an open source, light weight, non-invasive, loosly coupled aspect oriented, dependency injection based, Java Appln framework to develop all kinds of applications.

→ along with spring S/w we get it source code this makes Spring as ~~light~~ open source.

→ Spring is lightweight Reasons, ⓐ Spring framework size is very less, ⓑ Spring container can be activated as in memory containers without any physical server support

ⓒ Spring resources can be developed without Spring API

→ Spring is non-invasive framework, because the classes of spring app can be developed with out extending or implention Spring api classes / interfaces

4-2-2015

⇒ Spring framework is not replacement or alternate for JSE, ~~JEE~~ JEE technologies, fact it is compliment to these 2 technologies.

→ Spring framework is a loosly coupled because

a) we can use one or another module of spring · without having dependency with other modules.

e.g. we can use spring core + jdbc modules to develop persitence logic without using other modules

b) the dependent value of spring resource can be configured through xml file without hardcoding them directly in resources (classes)

c) we can use only spring to develop complete Project or we can use Spring along with other frameworks or java technologies to develop ~~with~~ Project

→ Aspect oriented Programming is a methodology of developing middleware services logic separately from main stream business logic and linking them with business logic dynamically at run time.

→ middleware services are like security transaction etc.. since Spring gives support for this AOP, We say Spring is Aspect oriented.

→ if the underlying server or container or framework or runtime environment dynamically assign value to the resources then it is a called dependency injection. in this underlying server or container pushes the value to resource. eg:- the way JVM called constructor to initialize object when object is created is called as dependency injection

eg: if course material assigned to Student the moment he registers for Course is called dependency injection.

there are two modes dependency injections.

a) Setter injection
b) Constructor Injection

→ while developing Project generally come across diff. types of classes

a) POJO class: the class that can be executed in JDK environment without taking support of third Party libraries is called POJO class

b) Java Bean: the class that follows some standards is called Java bean. this class does not contain business logic but contains accessor methods (Setter & getter) the standards are

a) must be a Private class
b) must have a Private member variables (Properties)
c) must have public getter & setter methods for each Property
d) must have 0-Param constructor directly or indirectly
e) must implement java.io.Serizable.

note:→ Java Bean is a POJO class, but every POJO class need not be a Java Bean.

c) Component class/Bean class:- the class that uses its membership in its member methods while developing business logics is called component class/Bean class.

→ If bean class is managed by Spring Container then it is called Spring bean

→ if bean class is managed by EJB Container then is called EJB Bean.

→ bean class can be developed as POJO class or as non-POJO class.

```
class Test {
    int a;
    int b;
  Public void bms() {
    --- b. logic using JDK apis
    ...
  }
}
```

"Test" is POJO class & Bean class / component class

```
Public class Test implements Serializable {
    Private int sno;
    Private String sname;
      // write Setter & getter methods.
      ...
      ...
    }
```

"Test" is Java bean & Pojo class.

```
Public class Test {
  int age;
  Public int calcPension() {
      --uses Spring api for b-logic
      ...
    }
}
```

"Test" is not a POJO class. Test is Bean class.

Container is s/w appln that can manage whole lifecycle of given resources it is like an aquarium. Spring framework gives by default in built container as light weight in memory container without having support of any physical server like Tomcat

There are

a) Beanfactory container (basic container)

b) Application Context container (Extension of Beanfactory container)

=) both these containers manage & executes Spring beans & also capable Performing dependency injection.

Spring

9-2-2015

Q: Why framework?

→ ① When Java is released, there are no Predefined APIs for Connecting with database or for connecting with mail server, or for transaction mgmt etc

② In early days of Java, a Programmer itself started coding everything in a Project, it made burdon on the Programmer

③ to reduce the burdon Sun microsystem release a set of APIs for connecting with databases, mail server, transaction etc. with a name J2EE

④ J2EE has lots of API and it is more burdon on developer because remembering & integrating J2EE APIs in a Project is Complex

⑤ to get out of burdon on directly working with J2EE APIs, 3rd Party vendors started Providing framework in Java

⑥ frameworks are considered as a top layers (Abstraction layers) on top of J2EE APIs

⑦ the following of the benefits of directly working with frameworks

① frameworks remove the burdon of writing repeated code. so burdon of the Programmer is reduced.

② frameworks will help to finish a Project as early as possible so Projects can be deliver fastly. to the clients.

③ A Project can be maintained easily it means a only with minimum changes. we can add new feature to the Project.

Rod Johnson

↓

Interface21 → Spring

↓

SpringSource

J2SE → Java API

10-2-2015

J2EE → API's

↓

Servlet API, JSP API, JDBC, JNDI, EJB, JTA, JMS

Spring → Modular framework.

the below or the major reasons for choosing Spring framework by the industry.

(1) Spring framework can be used to develop all layers of an application it means we can develop core java application, Web application, remoting applications etc. using Spring framework

(2) Spring framework is versatile framework it means we can easily integrate our Spring application with application developed using other framework

(3) Spring framework suggesting to use ordinary java classes and interfaces for creating an application. this is also called POJI / POJO model.

(4) Spring framework has given its own container called IOC container (Inversion of control) so we no need to depend on ~~third party~~ containers provided by the third party vendors.

Q | What is a Non-invasive framework?

A → (1) If framework is not forcing programmers to extend the a class from a superclass or a interface which is given by a framework then it is called a non-invasive f/w.

(2) If a framework is forcing programmers to extend or implement a java class from a superclass or interface given by framework, then it is called invasive framework.

for eg: Spring & hibernate & JSF & Struts2.x → non-invasive framework.

Struts1.x → invasive framework.

Struts 2.x → non-invasive framework

Q Why spring fw is called light weight framework?

Ans → Spring framework is divided into six modules a programmer can choose particular module only to develop a spring application, it is not compulsary to choose all the module to develop of an appln. so Spring Framework is called a light weight framework

11-2-2015

Spring → Modular framework.
- light weight
- non-invasive.

Modules

1. Spring core
2. Spring AOP
3. Spring MVC
4. Spring JEE (Services)
5. Spring Data Access.
6. Spring test

Modules

1. Spring core
2. Spring AOP
3. Spring MVC
4. Spring J'EE (services)
5. Spring Data Access
6. Spring Test.

1. **Core module:**

→ this core module is the fundamental to the Spring framework

→ In Spring, all the remaining modules are developed on the base Spring core module.

→ in core module we can get the following information

① How to configure the application classes in Spring configuration file.
② How to utilize the dependency injection mechanism
③ How to obtain a Spring IOC container
④ How to modify object graph at runtime using Spring Expression Language. (SpEL).

2. AOP (Aspect Oriented Programming)

```
p c withdraw
{
    p v withdrawMoney(acno, amount)
    {
        // Security → cross-cutting concern, Service logic, Secondary logic
        // B.logic → Primary logic
    }
}
```

→ When developing application, the business logic which are created or called Primary logic.

→ In order to make Business logic as efficient, some Service logic are attached to Business logic. these Service logic are called Secondary logics.

→ In Spring framework Secondary logic are also called cross-cutting concerns

→ In a Project, if Primary & Secondary logic are combinely develop, then we loose reusability of the services to the multiple Primary logic.

→ In Spring framework, AOP module is given for separating the Primary and secondary logic and then infecting (or combining) Secondary logic to the Primary at run time.

3. MVC

→ this mvc module is given to develop
1. Presentation layer (Web layer) in a Project by following MVC architecture.

→ this MVC module Provide all the infrastructure like a controller, tag libraries, annotation etc. for creating web layer of a Project.

→ this MVC module of the Spring framework is a almost equal to Struts framework.

4. Spring JEE (Services)

AOP tells to How to Separate Blogic & Service logic.

JEE tells to Implementing Service logic.

Implementing Middleware Services.

↓

transaction, Security, Mailing, Messaging

12-2-2015

Core modules

configure.

Servlet ⟶ web.xml.

Spring beans → xml.
(POJO's)

→ this Module of the Spring framework tells a Programmer about How to implement the middleware Services to the Business logic of ~~Java~~ Java Projects

→ middleware services are transaction, Security, mailing, messaging, Scheduling etc.

→ the difference between Spring AOP module and Spring JEE module is, AOP module tells How to separate B.logic & services and How to injecting or combining at run time. but JEE module tells about How to implement Services.

Spring Data Access module.

① Spring-jdbc
↓
abstraction layer on top of jdbc tech.

② Spring ORM

→ this module is divided into two submodule.

① Spring-jdbc
② Spring-ORM

→ spring jdbc module is an abstraction layer on top of jdbc technology this layer avoids the Boiler Plate code used in jdbc Programming

→ spring ORM module is an abstraction layer on top ORM tools.

→ When working with ORM tools like a hibernate again we have a Boiler plate code this spring ORM layers avoids boiler plate code of ORM tools.

Spring Test Module.

* difference bet Mock object & Proxy object ?

Junit Tool
TestNG
JMeter
} Testing tools — for testing Mock object is necessary which is created by Spring f/w

→ this module is given for conducting unit testing on the Spring code
→ Unit testing is the primary Responsibility of a java Programmer.
→ to conduct unit testing we use a tools like junit, TestNG, or JMeter etc.
→ While testing by creating test cases some times we need Mock objects.
→ the benefit of the this Spring test module is framework it self creates a mock objects.
→ In Real time, this Spring test module will be used in very less number of cases.

SPRING - CORE Module

18-2-15

Tight coupling & loose coupling.

Why has-relation preferred or better or than IS-a

Ex. Car is a vehicle

```
Class A {
    M1()
}
```

is-a relation means inheritance.

```
Class B extends A
{
}
```

Ex. car has-a engine.

```
class A {
    M1() {
        m2();
    }
```

```
Class B
{
    Aa = new A();
}
```

Has-A Relation means composition.

```
Class Empl. extends Addr. {
}
```

→ Tight coupling & Loose coupling between objects.

Q) Why HAS-A relationship is better than IS-A relationship

A → ① Reason-1

→ if we apply IS-a relationship between two classes then and object of subclass can get functionality of a super class. for only once. It is not possible to get functionality for more than once.

→ If we apply Has-A relation then it is possible to get functionality of one class in an object of another class.

→ for eg. We have class Employee extends Address class Here one employee object can get functionality of

Address class for once only.

→ If we create Address class object in Employee class then it is possible to get functionality of the Address into Employee for multiple time

```
Class Employee extends Address (IS-A)
  {

  }
```

→ Class Employee ~~Address~~ (HAS-A)

```
{
    Address temporary = new Address();
    Address Permanent = new Address();
}
```

Reason: 2

→ if we apply is-a relationship then all functionality of Superclass will be inherited into Subclass. If we want some functionality the remaining functionality should be made as a Private. if another Subclass want Private functionality of super class then it is not possible

→ If it is a HAS-A relationship then we no need make the functionality of a class as a private. we can called only required functionality by using an Object of the class

```
class B
{
  A a = new A();
  void x()
    {
      a.m1();
      a.m4();
    }
}
```

```
class C
{
  A a = new A();
  void y()
    {
      a.m2();
      a.m3();
    }
}
```

Reason 3

```
Class B extends A
{
}
```

dev.1 → B, dev.2 → A → (Is-A)

dev 1 (B) → testing

A → tested automatically.

```
Class B
{
  A a = new A();
}
```

mock object → possible in HAS-A, not in Is-A.

→ In Is-A Relation, when we are testing the subclass, its superclass is also be tested.

→ If any problem occurs in a superclass then subclass testing ~~is fail~~ will be failed.

→ in case of HAS-A relationship, we can create & inject mock object of the class into another class and then we can test that class only. So for unit testing also HAS-A relation is better than Is-A relation

```
Class Travel   (caller class)
{
  Car c = new Car();
  void Journey()
  {
    c.go();
    c.move();
  }
}
```

```
Class Car   (dependent class)
{
  void go() move()
  {
  }
}
```

Connection (Con)

c: Reference variable which pointing the object of implemented class

→ returns the object of implemented class

16-2-15

tight coupling

→ tight coupling bet^n two classes will occur in the following situation

① If functionality (method) name is changed in the dependent class name then we need to modify caller class also.

```
Class Travel  => caller class
{
  Car c = new Car();
  void journey()
  {
    c.go();   // go -> move
  }
}
```

```
class Car  --> dependent class
{
  void go()   // move()
  {
  }
}
```

→ In the above Travel class is depending on car, in car functionality name changed to move so in Travel ~~ed~~ class also functionality name is need changed to move. It is a tight coupling.

② if a caller class want to change its dependency to another similar type of class then also we need to change or modify caller class.

```
Class Travel
{
  ~~Car c = new Car();~~
  Bike b = new Bike();

  void journey()
  {
    ~~c.go();~~
    b.ride();
  }
}
```

```
class Car
{
  void go()
  {
  }
}
```

```
class Bike
{
  void ride()
  {
  }
}
```

→ In the above when we want to change dependency from car class to Bike class then we have changed caller class ~~Travel~~ called Travel. this is also called tight coupling.

Loose Coupling

17-2-2015

→ loose coupling bet^{n} two classes can be achieved in the following two ways.

① by following POJI/ POJO model.

② by applying dependency injection mechanism.

IOC → Inversion of control
↓
3rd person

→ By following POJI/ POJO model, a group of similar classes are going to have common functionality names.

→ By taking the reference variable of an interface, we can store any one of its implementation class object.

→ Invarsion of control indicates the an external classes managing the classes of an application

→ an external class means like a container manages the collaboration between or dependency bet^{n} classes of application.

→ IOC is of 2 types

① Dependency look-up

② Dependency injection.

→ In Dependency look-up, a class explicitly ask a container to get its dependent object or to take its dependent object.

→ In Dependency injection, a container only injects the dependency to a class

→ Spring fw Promotes dependent injection type of IOC.

→ ① by following POJI/POJO model.

② by using dependency injection mechanism. we can get the loose coupling bet^{n} the 2 classes or object.

Types of Dependency Injection

→ In Spring FW there are 4 types of Dependency injections

1. constructor injection.
2. setter injection.
3. Interface injection.
4. Lookup method / method injection.

→ Constructor and setter injection are used very frequently for injecting the dependencies.

→ Interface & Lookup method injection are used very rarely in Spring Application

→ If a dependency object is injected to the caller by calling a constructor in the caller class, then it is called constructor injection.

→ If a dependency object is created to a caller by calling setter method defined in a caller class. then it is called setter injection

```
Public class Travel
{
   Private Vehicle v;
   Public Travel (Vehicle v) ← called constructor
      {
        this.v = v;
      }
   Public void journey (){
      =
   }
}
```

constructor injection

⇒ In the above class a constructor is defined to inject its dependency so it is called constructor injection.

```
Public class Travel
{
   Private Vehicle V;
   Public void setV(Vehicle v)
   {
      this.v = v;
   }
   Public void journey()
   {
      ≡
   }
}
```

→ in the above class a setter method is defined to inject a its dependency so it is called setter injection.

Pojo, Java Beans, Spring Beans

→ a Pojo class means it is a class which doesn't exceed the boundary of Java API

→ no exceeding the Boundary of Java API means a class is compiled directly with Java compiler. by without taking the support of any jar file.

→ a Java Bean is also a Java class which follows the below rules.

① class must be a Public class.
② class must contain default constructor.
③ a Private variable should contain either setter, getter or both methods
④ a class can atmost implements Java.io.Serilizable interface.

Ex. 1

```
class A
{
   int k;
   A(int k)
   {
      this.k = k;
   }
}
```

→ PoJo ✓
Java Beans ✗

Ex. 2

```
Public class A
{
  Private int k;
  Public A (int K)
  {
    ---
  }
  P v setK (int K)
  {
    =
  }
}
```

Java Bean ✗ no default constructor.
POJO ✓

Ex. 3

```
Public class A
{
  Private int k;
  Public A ()
  {
    ---
  }
  Public void setK (int K)
  {
    ---
  }
}
```

Java Bean ✓
POJO ✓

Ex. 4

```
Public class A extends Exception
{
  ---
}
```

POJO ✓
Java Bean ✗ because extending Exception class.

Ex. 5

```
Public class A
{
}
```

POJO ✓
Java Bean ✓

Ex. 6

```
Public class A implements Serializable
{
    void m1()
    {
    =
    }
}
```

POJO ✓
Java Bean ✓

Ex. 4

```
Public class MyServlet extends HttpServlet
{
  =
}
```

POJO X
Java Bean X

* Every Java Bean class is definitely POJO class Also but every POJO class is not a Java Bean class.

→ As part of a Spring Application, whatever the classes we developed they are called Spring Beans.

→ In most of the cases a Spring Bean is also a POJO. Sometimes Spring Bean is Java Bean also

→ In exceptional cases a Spring bean will be not a POJO or Java Bean

Ex. 1

```
Public class A
{
    Private int i;
    Public void setI (int i)
    {
      this.i = i;
    }
}
```

POJO ✓
Java Bean ✓
Spring Bean ✓

Ex.2

```
Public class A
{
    Private int i;
    Public A (int i)
    {
        this.i = i;
    }
}
```

POJO ✓

Java Bean ✗

Spring Bean ✓

Ex.3

```
Public class A implements InitializingBean
{
    ≡
}
```

↳ spring framework Interface

POJO ✗

Java Bean ✗

Spring Bean ✓

When Beanfactory Container is activated the entries of Spring Configuration file will be verified by using SAX xml Parser
↳ Sample API for xml.

```
Object obj = factory.getBean("db");
```
Bean id

① Beanfactory container to load "Bean" class based on the given Bean id "db" using configurations done in spring cfg file.

② makes Beanfactory Container to create "DemoBean" class object using -Param Constructor.

factory.getBean("db"); method returns "~~DemoBean~~" Bean class object and we are refering that object using java.lang.Object class with ref. "obj"

Note :- getBean() method is the Predefined method in Predefined class "object" if we want to get the Object from Beanfactory Container we use this getBean() method. If we call Business method, that method is not Part of the Predefined class object it is Part of the implement class of ~~Spring Interface~~ SpringInterface, so we must typecast i.e

```
(classname) cn = (classname) obj;
```

Naming convention.

X → Spring interface name.
XBean → Spring Bean class name.
X cfg.xml → Spring cfg file name.
X client → client App name.

for Example Put X → Test

18-2-2015

Spring Configuration file

<anyname>.xml

demo.xml

```
<beans>
    <bean   id = "id1"   class = "com.sathya.beans.Travel">
    =
    </bean>
    <bean   id = "id2"   class = "com.sathya.beans.Car">
    =
    </bean>
</beans>
```

(attributes. — id; Fully qualified name. — class)

→ only class is configured using spring xml tags not interface.

class may be concrete, abstract, final

~~id → can't be duplicated for same~~

① In order to tell classes of our application to the spring Ioc container we configure our classes in spring configuration file

② a spring configuration file is a xml file and the it can be named as <anyname>.xml

③ if we write information about our classes to the another file then it is called configuration.

④ In web applications we configure our servlet classes in deployment descriptor file (web.xml). Similarly in spring application we configure spring beans in a spring configuration file.

⑤ a Spring Configuration file contains root element as <beans> & each class is configured using <bean> tag.

<anyname-xml>

```
<beans>
    <bean id = "xxxx" class = "fully qualified classname">
        ≡
    </bean>
    <bean id = "xxxx" class = "fully qualified classname">
        ≡
    </bean>
</beans>
```

Imp Statements

① using <bean> tag, only classes can be configured but not interfaces

② a class can be concrete class, abstract class, or it may be final class.

<bean id = "id1" class = "com.satnya.beans.Travel" /> → valid
(Travel → concrete class)

<bean id = "id2" class = "~~com.satnya~~.java.util.Calender"> → valid
(Calender → abstract class.)

<bean id = "id3" class = "java.lang.Runnable"> Invalid
(Runnable → interface)

③ we can configure same class for multiple times with different ids

<bean id = "id1" class = "com.satnya.beans.Travel" /> ✓

<bean id = "id2" class = "com.satnya.beans.Travel" /> ✓

④ ~~id attributes follows~~

④ id attribute follows xml id rules.

i) same id should not be duplicated in entire xml.

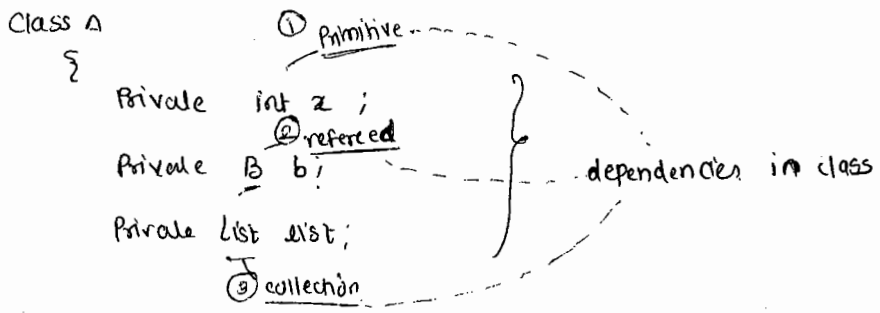
ii) id should not contain special characters. It can have only alphanumeric characters.

<bean id = "id1" class = "com.sathya.beans.Travel" />

<bean id = "id1" class = "com.sathya.beans.Travel" /> → Invalid
↳ duplicated hence invalid.

<bean id = "@id1" class = "com.sathya.beans.Travel" /> → Invalid. because of special character.
↑ special character.

<bean id = "_id" class = "com.sathya.beans.Travel" /> → valid



Types of dependencies in a bean

→ ① a bean can have a maximum of 3 types of dependencies

① Primitive type.
② referenced type.
③ collection type.

→

```
class A
{
    Private int x ;  → Primitive type
    Private B b;     → reference type
    Private List list ; → collection type.
    - - -
    - - -
    - - -
}
```

② In a bean class if we defined a constructor for injecting a dependencies then we need to configure <constructor-arg> tag in a xml file.

③ If a setter method is defined for injecting a dependency then we need to configure <Property> tag in a xml file

Example 1

```
P C A
{
    Private int x;
    Public void setX (int x)  → setter method
    {
        this.x = x;
    }
}
```

When SpringContainer is activated by giving this spring cfg file the Spring Container creates A class object having "id1" as the object Name & calls the method.

demo-xml

```
<beans>
    <bean id = "id1" class = "A">
*       < Property name = "x"
                value = "100" />
    <bean>
</ beans>
```

id | obj.name

Bean class

Bean Property

Value to be injected

Ex.2

```
P C A
{
  Private int x;
  Public A (int x)        → constructor.
  {
    this.x = x;
  }
}
```

While configuring Spring Bean to Place only <Property> tags under <bean> tag the Spring container uses ()-Param constructor for Spring Bean class instantiated & also uses setter xxx(-) methods for setter Injection on bean properties.

demo.xml

```
<beans>
  <bean id = "id1" class = "A">
    < constructor-arg value = "100" />
  </bean>
</beans>
```

While configuring SpringBean if you place only <constructor-arg> tag under <bean> tag then Spring container uses Parameterized constructor to create SpringBean class object & to perform Constructor Injection on Bean Properties.

Note: In <Property> tag name attribute is manadatory. but in <constructor-arg> tag name attribute is not allowed.

Ex.3

```
P C A
{
  Private int x;
  Private int y;
  Public A (int x)
  {
    this.x = x;
  }
  Public void setY(int y)
  {
    this.y = y;
  }
}
```

demo.xml

```
<beans>
  <bean id = "id1" class = "A">
    <constructor-arg value = "100" />
    <Property name = "y" value = "200" />
  </bean>
</beans>
```

Bean with Multiple Constructors

19-2-2015

→ a spring bean can have more than one constructor, because a constructor can be overloaded.

→ if bean class contain many constructor then we need to configure bean class for multiple times in xml file.

for Example

in this Example dependency is Primitive type

int x;
int y;

```
Public Class A
{
    Private int x;
    Private int y;
    Public A (int x)        // with 1 argument
    {
        this.x = x;
    }
    Public A (int x, int y) // with 2 argument
    {
        this.x = x;
        this.y = y;
    }
}
```

spring.xml

```
<beans>
    <bean id = "a1" class = "A">
        <constructor-arg value = "100" />
    </bean>
    <bean id = "a2" class = "A">
        <constructor-arg value = "50" />
        <constructor-arg value = "70" />
    </bean>
</beans>
```

Value → indicates dependency is Primitives type

ref → indicates dependency is reference type.

dependency of reference type.

→ if a dependency of bean class is reference type the IOC container injects object.

→ In the configuration file we use "ref" attribute to configure the referenced type of dependency.

ex:

```
Public class A
  {
    Private B b;
    Public void setB(B b)
      {
        this.b = b;
      }
  }
```

Xml file

```
<beans>
  <bean id = "id1" class = "A">
    <Property name= "b"
              ref = "id2" />
  </bean>
  <bean id = "id2" class = "B" />
</beans>
```

ref → indicates injects object. as dependency is reference type

Note ① <Property> tag has the below attributes

1. name
2. value
3. ref

② name attribute is mandatory, in value & ref, we can use only one at a time.

* <ref> tag :

→ when dependency is reference type then in xml file we can configure ref as attribute or ref as tag.

→ If we configure ref as tag then it should have either local or parent or bean has an attribute.

① → if we configure local attribute with ~~ref~~ <ref> tag then the Spring container will search for dependent bean in same xml.

Example

Spring.xml

```
<beans>
    <bean id = "id1"  class = "A">
        <property name = "b">
            <ref local = "id2" />
        </property>
    </bean>
    <bean id = "id2" class = "B" />
</beans>
```

② If we configure ~~de~~ Parent attribute with <ref> tag then Spring Container checks for dependent bean in Parent xml

Example

Parent.xml

```
<beans>
    <bean id = "id2" class = "B" />
</beans>
```

child.xml

```
<beans>
    <bean id = "id1" class = "A">
        <Property name = "b">
            <ref Parent = "id2"/>
        </Property>
    </bean>
</beans>
```

(8) → If we configure bean attribute with <ref> tag then Spring Container will search for dependent bean in the local xml, if not found then container will search for dependent bean in Parent xml.

→ If we use ref as attribute then it will be equal to <ref> tag with bean attribute.

```
Class A
{
  void m1()
  {
  }
}
```

class B Provide runtime support to class A
Hence it is container class

```
class B → container class
{
  p s v m (String[]args)
  {
    A a = new A();
    a.m1();
  }
}
```

Ioc container.

1. BeanFactory (I)
2. ApplicationContext (I)

XmlBeanFactory (C)

```
BeanFactory factory = new XmlBeanFactory();
```

20-2-15

Spring Container

① Beanfactory interface (super)

↑ extends

② ApplicationContext interface (sub) → is better than Beanfactory interface

① Beanfactory factory = new XmlBeanfactory(); ← implementation class of Beanfactory
by creating its object spring IOC container started in an application

② ApplicationContext ctx = new ClassPathXmlApplicationContext();

② Object getBean(String id); → returning object of Particular class in Object format so we need type cast

② Object o = factory.getBeans("id2");

③ A a = (A) o;

Spring Container

→ a container is a class which provides runtime support to other classes

→ If we create class with a main method then it will provide the runtime support to the other classes of a console Application. So a main class is called as container

→ In AWT Programming a frame class provides runtime support to other classes Button, Label, etc. so frame class is a container class

→ in Spring framework a Spring IOC container means, it is an object of implementation class of either Beanfactory interface or ApplicationContext interface.

→ ApplicationContext interface is a sub interface of a Beanfactory, so Comparitively ApplicationContext container is better than Beanfactory.

→ XmlBeanfactory is an implementation class of Beanfactory interface so if we create an object of XmlBeanfactory then Spring IOC container started in an application.

```
BeanFactory factory = new XmlBeanFactory();
```

→ ClassPathXmlApplicationContext is an implementation class of ~~ApplicationContext~~ Interface. so If we create an object of ~~this~~ Implementation class then Spring Ioc container is started in application.

```
ApplicationContext ctx = new ClassPathXmlApplicationContext();
```

→ when creating Spring Ioc container object we need to pass Spring configuration file as a Parameter, so that Spring Container reads Bean definition from xml file.

→ while creating BeanFactory ~~Object~~ Container Object, we need to create Resource object for xml file and then we need to pass a resource object as Parameter.

```
Interface
Resource ~~res~~ res = new ClassPathResource("demo.xml");
                          Implementation class

BeanFactory factory = new XmlBeanFactory (*res);
```

→ when creating ApplicationContext Container Object we can directly pass xml file name as a Parameter.

```
ApplicationContext ctx = new ClassPathXmlApplicationContext("demo.xml");
```

→ To call methods of Bean class from a main class we need to follow below steps.

① we need to Create a Spring Ioc Container Object

② We need to get the Object of bean based on its id from Spring container Object.

③ we need to invoke/call methods of the bean.

Spring Software

→ a framework software is a group of jar files not a setup file

→ To download Spring software we need to visit the following website

repo.spring.io/release/org/springframework/spring/

21-2-2015

FirstApp

→ DemoBean.java
Spring-Config.xml
main.java

xsd file
nameSpace → element, attribute,
xmlns → keyword to importing namespace into the xml file.

BeanNameSpace.

Path → related to O.S i.e exe file.
ClassPath → related to Java.
Javac reads classpath

→ To run .exe O.S is needed, so place in Path

// DemoBean.java

```
Public class DemoBean
{
  Private String msg;
  Public void setMsg (String msg)
  {
     this.msg = msg;
  }
  Public void display ()
  {
     System.out.println(msg);
  }
};
```

Spring-config.xml

```
<beans   xmlns = "http://www.springframework.org/schema/beans"
         xmlns:xsi = "http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation = "http://www.springframework.org/schema/beans
         http://www.springframework.org/schema/beans/spring-beans-4.0.dtd">

    <bean   id = "demoBean"   class = "DemoBean">
        <property   name = "msg"   value = "Welcome to Spring" />
    </bean>
</beans>
```

// Main.java

```
import org.springframework.core.io.Resource;
import org.springframework.core.io.ClassPathResource;
import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.xml.XmlBeanFactory;

class Main
{
    public static void main (String[] args)
    {
        Resource res = new ClassPathResource ("spring-config.xml");
        BeanFactory factory = new XmlBeanFactory (res);
        Object o = factory.getBean ("demoBean");
        DemoBean db = (DemoBean) o;
        db.display();
    }
}
```

→ DemoBean.java is a POJO class so it can be compiled directly.

→ In main.java we have imported and used spring framework API, so when we compiled we will get compiled time errors. the reason is java compiler doesn't know spring-framework API

→ In order to resolve the errors we need to set following 3 jar files in ClassPath

① Spring-core-4.1.2.RELEASE.jar
② Spring-beans-4.1.2.RELEASE.jar
③ commons-logging-1.0.4.jar

> javac *.java
> java Main

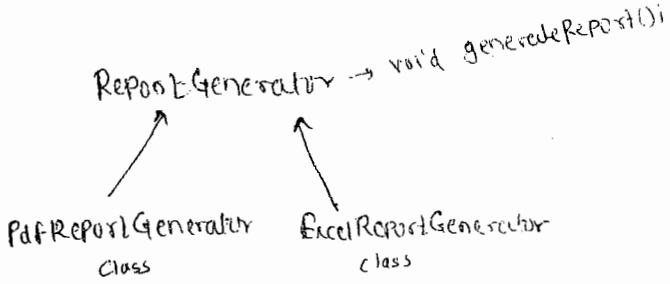
o/p → Welcome to Spring

Second Example

→ in this Example, we have a class ReportService and it depends on another object of Type ReportGenerator.

→ for ReportGenerator interface, we are creating 2 implementation classes PdfReportGenerator & ExcelReportGenerator.

Second-App
- ReportService.java ① (Bean)
- interface ReportGenerator.java ②
- (c) PdfReportGenerator.java ③
- (c) ExcelReportGenerator.java ④
- ~~Spring.xml~~ spring.xml ⑤
- Main.java ⑥



// ReportService.java

```
Public class ReportService
  {
    Private ReportGenerator rg;   (interface)
    Public void setRg ( ReportGenerator rg)
      {
        this.rg = rg;
      }
    Public void showReport()
      {
        rg. generateReport();
      }
  };
```

// ReportGenerator.java ✓

```
Public interface ReportGenerator
  {
    void generateReport ();
  }
```

// PdfReportGenerator.java ✓

```
Public class PdfReportGenerator implements ReportGenerator
  {
    Public void generateReport()
      {
        System.out.println ("Report in Pdf format");
      }
  };
```

// ExcelReportGenerator.java

```
Public class ExcelReportGenerator implements ReportGenerator
    {
      Public void generateReport ()
        {
          SOPln (" Report in Excel format");
        }
    };
```

Spring.xml

```
<beans
    =
    =>

  <bean id = " reportService " class = " ReportService ">
      <Property name = " rg " ref = " Pdf "/>
  </bean>

  <bean id = " Pdf " class = " PdfReportGenerator "/>
  <bean id = " excel " class = " ExcelReportGenerator "/>

</beans>
```

// Main.java

```
class main
    {
      main
        {
          Resource res = new ClassPathResource (" Spring.xml ");
          BeanFactory factory = new XmlBeanfactory (res);
          Object o = factory.getBean ("reportService");
          ReportService rs = (ReportService) o;
          rs. showReport ();
        }
    }
```

O/P → Report in Pdf format

23-2-2015

```
Str= theList.getClass().getName()
sopln(str);
```

Dependency In the form Of a Collection.

→ In a bean class if we take dependency as collection type of reference variable then internally Spring IOC container creates a collection objects and injects the dependency.

→ In a Spring Bean class we can take the dependency as a one of the following 4 types of collection.

1) java.util.List
2) java.util.Set
3) java.util.Map
4) java.util.Properties.

→ In Spring configuration file to configure collection type we use the following tags

List → <list>

Set → <set>

Map → <map>

Properties → <props>

① **<list> tag:**

① this tag can be configure in xml, for reference variable of the following tags

1) java.util.List interface
2) java.util.Vector class
3) java.util.ArrayList class

② If dependency of type List or ArrayList then internally IOC container creates and injects ArrayList class object

③ If dependency is Vector then container creates vector class object.

④ under <list> tag, we can configure sub-elements <value> and <ref>

Example

```
Public class A
 {
    Private List theList;
    Public void setTheList (List theList)
      {
        this.theList = theList;
      }
 }
```

Spring.xml

```
<beans>
   <bean id = "a" class = "A">
      <Property name = "theList">
        <list>
            <value> 100 </value>
            <value> Sathya </value>
            <value> 100 </value>
            <ref bean = "b" />
        </list>
      </Property>
```

```
</bean>
<bean id="b" class="B" />
</beans>
```

→ internally spring container will do the following work

```
A a = new A();
List theList = new ArrayList();
theList.add(100);
theList.add("Sathya");
theList.add(100);
B b = new B();
theList.add(b);
a.setTheList(theList);
```

② <set> tag

→ we configure this <set> tag in xml file for the following tags.

1) java.util.Set
2) java.util.HashSet
3) java.util.LinkedHashSet

→ for all the above 3 types, internally spring container creates ~~linked~~ LinkedHashSet object.

→ under <set> tag we can configure sub-elements as <value> & <ref>

→ if duplicate value configured then it will not be added to Set. It means container ignores the duplicates but doesn't throw an exception.

Ex:

```
Public class A
    {
      Private Set theSet;
      Public void setTheSet (Set theSet)
       {
          this.theSet = theSet;
       }
    }
```

Spring.xml

```
<beans>
    <bean id = "a"  class = "A">
        <Property name = "theSet">
          <set>
             <value> 100 </value>
             <value> Sathya </value>
             <value> 100 </value>
          </set>
        </Property>
    </bean>
    <bean id = "b"  class = "B" />
</beans>
```

→ Internally Spring container will do the following work for injecting the dependency.

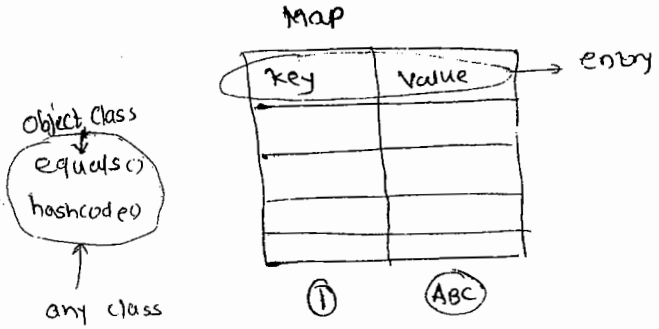
```
A a = new A();
Set theSet = new LinkedHashSet();
theSet.add (100);
theSet.add ("Sathya");
B b = new B();          ← 100 omitted as repeated
theSet.add (b);
a.setTheSet (theSet);
```

24-2-2015

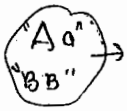
<map> tag

Map
HashMap } LinkedHashMap
LinkedHashMap

Hashtable → Hashtable



check if this string



→ contain same hashCode
i.e 2112

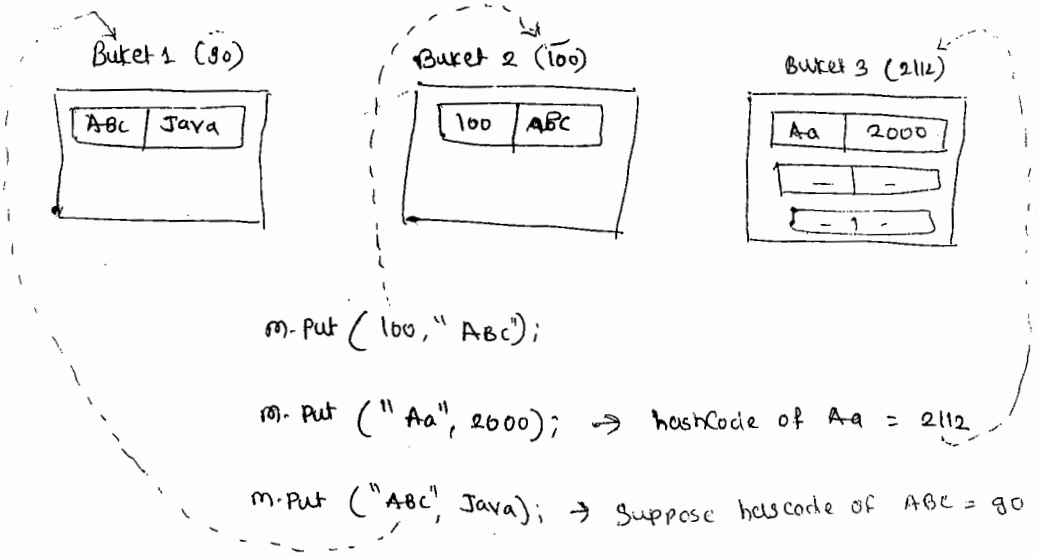
String S1 = "Aa";
String S2 = "BB";
SOP (S1.hashCode()); → 2112
SOP (S2.hashCode()); → 2112

```
Class A
{
  void m1(){}
}

 A a = new A();
SOP (a.hashcode()); → int value because hashCode() ~~doesn't~~ not override here.
                       ↳ memory address converted int bit format)

Class A
 {
   P int hashCode()
    {
      return 100;
    }
    void m1()
     { }
 }
    A  a = new A();
   SOP (a.hashCode()); → 100, because hashCode() method is override
```

```
A a1 = new A (10,20);
A a2 = new A (10,20);
```



~~Here~~ Store in the form of hashcode bucket

m.Put (100, "ABC");

m.Put ("Aa", 2000); → hashCode of Aa = 2112

m.Put ("ABC", Java); → Suppose hascode of ABC = 90

Object o = m.get ("Aa");

↓

hashCode() → finding right bucket if not found then Create.

↓

equals() → finding right key

<Map> tag

→ We configured <map> tag in spring xml file for the following collections type.

1. Map
2. HashMap
3. LinkedHashMap

} LinkedHashMap

4. Hashtable } Hashtable.

→ For Map, HashMap & LinkedHashMap collection type, the Spring IOC container creates an object of LinkedHashMap

→ for Hashtable, container creates a Hashtable object

→ TreeMap is not supported

→ in xml file under <map> tag we configure <entry> tags

→ in Map collection a ~~key-pair~~ key-value pair is called one entry.

→ to use an object as key, that class must override hashCode() & equals() methods

→ internally a data of Map will be stored in hashCode buckets. So to store a ~~key/val~~ key/value pair in right bucket, hashCode() method on the key will be called.

→ when reading the value from Map object based on the key, then first hashCode() methods call to find the right bucket, and then equals() method call to find the value in a the bucket.

→ the possible combination of attributes in <entry> tags are

① key, value
② key, value-ref.
③ key-ref, value.
④ key-ref, value-ref.

ex:

```
Public class A
{
    Private Map theMap;
    Public void setTheMap(Map theMap)
    {
        this.theMap = theMap;
    }
}
```

Spring.xml

```
<beans>
    <bean id="a" class="A">
        <property name="theMap">
            <map>
                <entry key="1" value="1000" />  ⟶ key, value
                <entry key="k1" value-ref="id3" />  ⟶ key, value-ref
```

```
<entry key-ref = "id1" value = "java" /> → key-ref, value
<entry key-ref = "id2" value-ref = "id4" /> → key-ref, value-ref

        </map>
    </property>
</bean>
<bean id = "id1" class = "x" />
<bean id = "id2" class = "y" />
<bean id = "id3" class = "z" />
<bean id = "id4" class = "M" />
</beans>
```

internally IOC container will do the following

```
A a = new A();
Map theMap = new LinkedHashMap();
theMap.put (1, 1000);
Z id3 = new Z();
theMap.put ("k1", id3);
X id1 = new X();
theMap.put (id1, "Java");
Y id2 = new Y();
M id4 = new M();
theMap.put (id2, id4);
a.setTheMap (theMap);
```

<props> tag

→ If the dependency collection is Property class then in xml we configure <Props> tag

→ Property class object can store the data in String as key and String as value.

→ other type of objects are not allowed.

→ under <Props> tag we configure a subelement <Prop> tag

ex:

```
Public class A
 {
   Private Properties theProps;
   Public void setTheProps (Properties theProps)
    {
      this.theProps = theProps;
    }
 }
```

Spring.xml

```
<beans>
   <bean id = "a" class = "A">
     <Property name = "theProps">
        <Props>
           <Prop key = "1"> 106 </Prop>
           <Prop key = "k1"> Sathya </Prop>
           <Prop key = "15.67"> abc </Prop>
        </Props>
     </Property>
   </bean>
</beans>
```

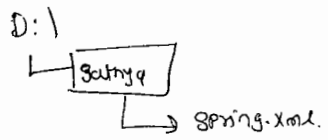
Internally IoC Container will do the following.

```
A a = new A();
Properties theProps = new Properties();
theProps.setProperty("1", "100");
theProps.setProperty("k1", "Sathya");
theProps.setProperty("15.67", "abc");
a.setTheProps(theProps);
```

ApplicationContext Container.

25-2-2015






```
ApplicationContext ctx = new FileSystemXmlApplicationContext("D:\Sathya\Spring.xml");
```

~~Application~~ Context

→ ApplicationContext is subinterface of BeanFactory

→ the 3 important implementation classes of ~~inter~~ ApplicationContext interface are

① ClassPathXmlApplicationContext

② FileSystemXmlApplicationContext

③ XmlWebApplicationContext → used in spring MVC Application

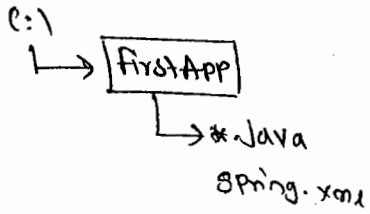
→ XmlWebApplicationContext will be used in Spring MVC applications

→ ClassPathXmlApplicationContext loads the spring configuration file from classpath location.

→ classpath location means, either the xml file is present in the same location of class in which that xml file is loaded or it exist in

in jar file i.e added in classpath.

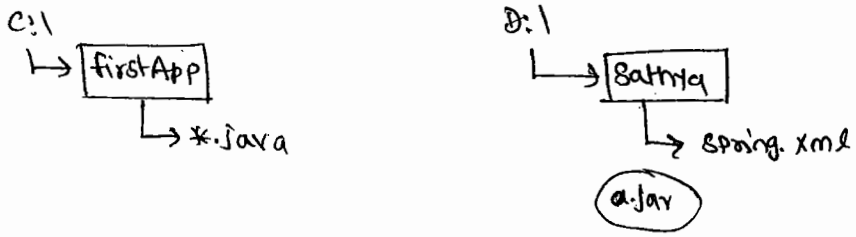
Example 1



C:\firstApp > javac main ↵

→ Here Spring.xml is loaded in main class and both main class & Spring.xml are in the same directory. so xml file is in classpath location

Example 2

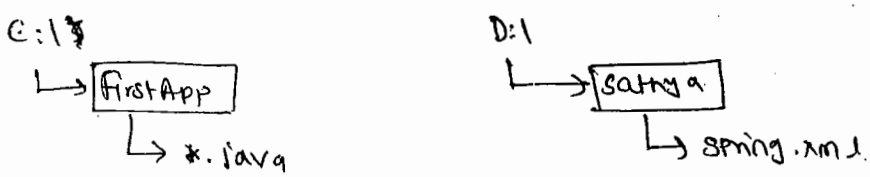


C:\firstApp> set classpath = D:\a.jar ; ↵
C:\firstApp > javac Main ↵

→ Here the xml file is in jar file & that jar is added to classpath so, the xml file is in classpath location.

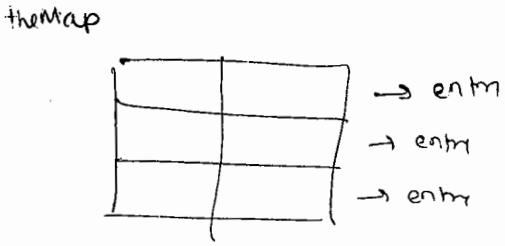
→ if a class which is loading the xml file and the xml file are in two different location then we use FileSystemXmlApplicationContext for loading the xml file.

Example



ApplicationContext ctx = new FileSystemXmlApplicationContext ("D:\Sathya\Spring.xml");

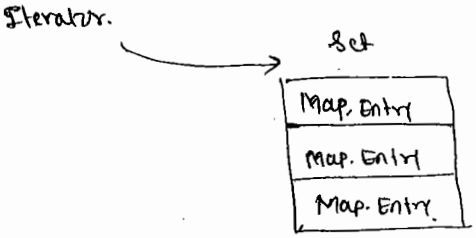
Map is best collection of java



Set s = theMap.entrySet();

↓

Map.Entry obj created



(eclipse)

CollectionTest

└ src

└ CollectionBean.java
main.java
applicationContext.xml

// CollectionBean.java

```
import java.util.Iterator;
import java.util.Map;
import java.util.Set;
```

ctrl + shift + O → for import

```
Public class CollectionBean
  {
    Private Set theSet;
    Private Map theMap;
    Public void setTheSet (Set theSet)
     {
       this.theSet = theSet;
     }
```

```
{
    Public void setTheMap (Map theMap)
    {
        this.theMap = theMap;
    }

    Public void displayTheSet()
    {
        Iterator it = theSet.iterator();
        While (it.hasNext())
        {
            Object o = it.next();
            Sopln (o.toString());
        }
    }

    Public void displayTheMap()
    {
        Set s = theMap.entrySet();
        Iterator it = s.iterator();
        while (it.hasNext())
        {
            Map.Entry me = (Map.Entry) it.next();
            Sopln (me.getKey() + " : " + me.getValue());
        }
    }
}
```

→ Configure the following jar ~~following build~~ file to the Build Path of the Project

Commons-logging-1.0.4
Spring-beans-4.1.2
Spring-Context-4.1.2
Spring.context-support-4.1.2.jar
Spring-core-4.1.2
Spring-expression-4.1.2

ApplicationContext.xml

```
<beans>
    <bean id = "collectionBean"  class = "CollectionBean">
        <Property name = "theSet">
            <set>
                <value> 100 </value>
                <value> sathya </value>
                <value> 100 </value>
            </set>
        </Property>
        <~~bean~~> <Property name = "theMap">
            <map>
                <entry key = "1" value = "1000" />
                <entry key = "k1" value = "Java" />
                <entry key = "15.15" value = "200.45" />
            </map>
        </Property>
    </bean>
</beans>
```

// Main.java

```
import org.springframework.context.ApplicationContext;
       .support.ClassPathXmlApplicationContext;

Public class main
{
    Public static void main (String args[])
    {
        ApplicationContext ctx = new ClassPathXmlApplicationContext
                                 ("applicationContext.xml");

        ~~Collection~~ Object o = ctx.getBean ("collectionBean");

        CollectionBean cb = (CollectionBean)o;
        SOPln ("set values:");
```

```
        cb.displayTheSet();

        SOPln("Map values:");
        cb.displayTheMap();
    }
}
```

Op

```
Set Values:
100
Sathya

Map values
1:1000
k1:java
15.15:200.45
```

27-2-15

Circular Dependency Issue

→ If a class contains dependency as reference of another class & if it has a setter injection then spring IOC container first creates an object of caller Bean then creates an object of dependency Bean

→ if a constructor injection is define then container first creates of an object of dependency Bean & then creates an obj. of caller Bean.

→ If A & B are two Beans, where A depends on B, B depends on A and in the both Beans constructor injection is defined then circular dependency Problem occurs

dependency as reference	Caller Bean	dependency Bean
Setter inj	1 obj	2nd obj
constructor	2nd obj	1st obj

```
class A
{
  Private B b;
  Public A(B b)
   {
     this.b = b;
   }
}
```

```
class B
{
  Private A a;
  Public B(A a)
   {
     this.a = a
   }
}
```

→ for creating an object of class A, Container is waiting for class B object and the for creating B object, container is waiting for A. finally container throws BeansInCreationException.

→ To resolve above circular dependency Problem, either in caller class or in a dependent class, we need to change the dependency injection type from constructor to setter.

→ for example if we change the dependency injection type in class A, from constructor to setter, then container will do the following

① Creates a Mock object of A

② injects Mock object to B through constructor.

③ finally injects object of B to A through setter.

```
Class A
{
  Private B b;
  Public void setB(B b)
  {
    this.b = b;
  }
}
```

```
class B
{
  Private A a;
  Public B(A a)
  {
    this.a = a;
  }
}
```

Differences Bet^n Constructor & Setter injections

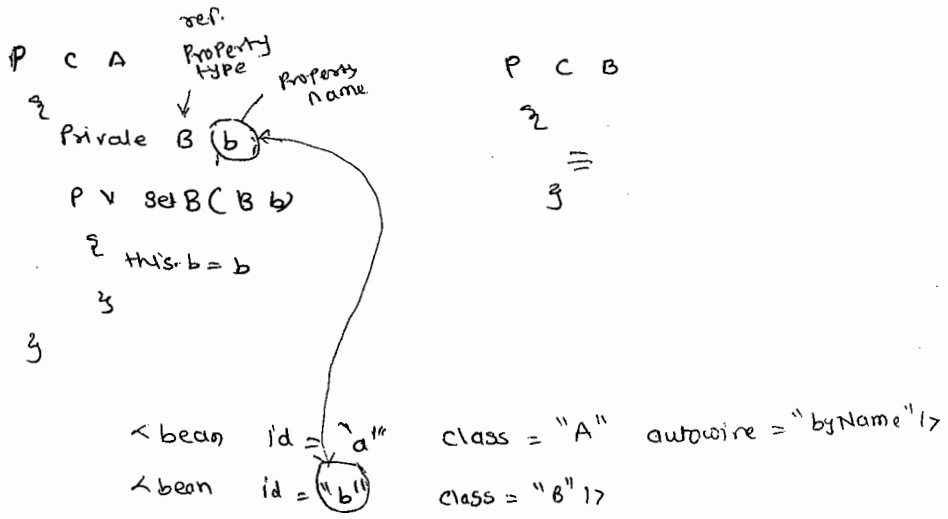
Constructor	Setter
① with constructor injections we cannot solve circular dependency problem.	① with a setter injection we can solve circular dependency problem
② if dependency is of final then we need constructor for injecting it	② for the dependency of final types, we cannot define setter.
③ if you want to use dependency in Constructor also, then we choose Constructor Injection.	③ if we don't want to use dependency in a constructor then we choose setter method.
④ if a dependency is a mandatory then we define constructor injection.	④ if a dependency is optional then we choose setter injection.
⑤ In this case, Constructor injection is highly recommended, as we can inject all dependencies with in 3 to 4 line [i mean calling one constructor]	⑤ If we have more dependences for example 15 to 20 are there in our bean class then in this case setter injection is not recommended as we need to write almost 20 setter right, bean length will be increased.

* V Imp

Bean autowiring

Bean autowiring not Applicable for primitive & collection type.
Applicable for ref. type.

28-2-2015



→ the process of combining our bean classes with spring container is called bean wiring.

→ for bean wiring, we need to configure our beans with there dependencies in an xml file.

→ In bean autowiring, a spring container automatically inject the collaborators (dependencies) of the bean by without writing <construction-arg> or <Property> tags in Spring Configuration file.

→ bydefault autowiring on bean is disabled, if we want to enbaled then we need to add autowire attribute in <bean> tag.

→ the Possible values of autowire attribute are

1. no (default value)
2. byName
3. byType
4. Constructor

② byName

→ if autowire Strategie is a byName then a spring container verifies a bean id in xml is matched with Property name or not. if matched then container injects an object of Bean class by calling the setter method.

For example

Bean id & Property name are checked.

```
P  C  A                          P  C  B
{                                {
   Private B b;                     ≡
   P v setB(B b)                 }
   {
      this.b = b;
   }
}
```

```
<bean id = "a"  class = "A"  autowire = "byName" />
<bean id = "b"  class = "B" />
```

→ if a bean id is not matched with a property name in xml then Spring container will not apply dependency injection. So that property remains with its default value "null".

→ if a bean id is matched with a property name, but its class is not matched then at the time of injecting the dependency java.lang.ClassCastException is thrown.

③ **byType** Bean class & Property type both are same.

→ In this autowiring strategi, Spring container verifies a bean class in xml is matched with property type. or not, if matched then container injects by calling setter method.

→

```
P  C  A                          P  C  B
{                                {
   Private B b;                     ≡
   P v setB(B b)                 }
   {
      this.b = b;
   }
}
```

```
<bean id = "a"  class = "A"  autowire = "byType" />
<bean id = "b"  class = "B" />
```

→ If a bean class in xml is matched with property type for more than one's, spring container throws org.springframework.beans.factory.UnsatisfiedDependencyException

→ If a bean class in xml is not matched with property type then container will not inject that dependency, so that property remains with default value as "null".

④ Constructor

→ if autowiring strategy is constructor then Spring Container checks a bean class in xml is matched with property type or not. If matched then Container injects the dependency by calling constructor of the bean.

→ the autowiring strategy byType & constructor has same verification strategy, but dependency injection type is different.

→ the autowiring strategy byName & byType has same injection type i.e setter, but verification is different.

```
P C A
{
  Private B b;
  P A(B b)
  {
    this.b = b;
  }
}
```

```
P C B
{
  ≡
}
```

```
<bean id="a" class="A" autowire="constructor" />
<bean id="b" class="B" />
```

→ if a bean class is matched than more than one's, then Container throws UnsatisfiedDependencyException.

→ if bean class in xml is not matched with a property type, then container call search for default constructor, if exist then an object of caller class created by calling the default constructor.

→ If a default constructor is not present, spring container throws Exception.

Q In which Autowiring Strategy unsatisfiedDependencyException doesn't occurs?

→ byName

```
P  C  MovieLister
{
   Private MovieFinder finder;
```

Q What is a drawback of Autowiring?

→ autowiring is only applicable for reference type but not applicable primitives types & collections type.

Initializing a bean

1-3-2015

Initializing a Bean Class

→ Before an object of a Bean class is going to become into usable state, if we want to execute any initialising logic on it then framework has given 2 options for initializing

① We can implement our Bean class from InitializingBean interface.

② We can define a custom init method in Bean class.

→ if we implement initializing interface then we need override its abstract method after Properties Set, for defining initialization logic

Example

```
P C CurdBean implements InitializingBean   — interface
{
   Private DataSource ds;
   => setter method
@Override
Public void afterPropertiesSet()   — method
{
   // Connection open logic
}

P V insert()
{ =
}
P V delete () { }
P V updat() { }
P V select () { }
}
```

→ In above Spring container automatically calls afterPropertiesSet() method, after the dependency is injected

→ in the above our Bean class implementing framework given interface, It means a our Bean class is acting as invasive class

→ If we want to make our class as non-invasive class, we can define a custom init() method in our Bean class

```
Public class CurdBean
{
   Private DataSource ds;
   => setter method.
   Public void setUp () ──→ [custom init method]
     {
       // connection open logic
     }
   Public void insert () {}
   Public void update () {}
   Public void delete () {}
   Public void select () {}
}
```

→ to tell the Spring container that a method of our Bean class as init() method, we need to configure init-method attribute with Bean tag in xml.

xml configuration:

```
<bean id = "CurdBean" class = "CurdBean" autowire = "byName"
                                          init-method = "setUp" />
<bean id = "ds" class = "BasicDataSource" />
```

Disposing in bean class:

→ before an object of an bean class is going to be garbage collector, if we want to execute some destruction logic of the bean by Ioc container then the framework has given 2 options for defining the destruction logic.

① by implementing our bean class from DisposalBean interface.

② by defining a custom destroy method.

→ If we implement DisposableBean interface then we need to override its abstract method destroy() for defining the destruction logic.

```
P c CardBean implements InitializingBean, DisposableBean
{
    Private DataSource ds;
      ⇒ setter method
    @ Override
   Public void afterPropertiesSet()
      {
          // Conn. Open logic
      }

    @ Override
     Public void destroy()
      {
          // Conn. close
      }
     ==
     ==
}
```

→ If we want to make our bean class non-invasive class, we can define a custom destroy method & we can configure that method in destroy-method attribute in bean tag.

```
Public class CurdBean
{
  Private DataSource ds;
   ⇒ setter method
  Public void setUp () → custom init method
    {
      // Conn. open logic
    }
  Public void tearDown () → custom destroy method
    {
      // Connection close
    }
}
```

Xml Configuration.

```
<bean id = "curdBean" class = "CurdBean" autowire = "byName"
                                  init-method = "setUp" destroy-method = "tearDown"/>

<bean id = "ds" class = "BasicDataSource"/>
```

Interface Injection in Spring

Interfaces.		Method
① BeanNameAware	→	setBeanName (String id)
② BeanFactoryAware	→	setBeanFactory (BeanFactory factory)
③ ApplicationContextAware	→	setApplicationContext (ApplicationContext ctx)

① this method is called by container for injecting bean id

→ if an interface given method is called for injecting a dependency into a bean class then it is called interface injection.

→ interface injection is a not possible for the user defined interfaces.

→ interface injection can not configured in xml file.

→ interface injection is possible only in the following 3 cases

① when we want to inject a bean id into the bean class.

② when we want to inject a BeanFactory container object into the bean class

③ when we want to inject a ApplicationContext container object into the bean class

→ Spring framework has provided 3 Aware interfaces of interface injection

→ ① BeanNameAware
② BeanFactoryAware
③ ApplicationContextAware

① → if a bean wants its id configure in xml then we need to implement that class from BeanNameAware interface.

→ the Spring container calls setBeanName method of BeanNameAware interface, for injecting a bean id.

for example → NextPage

2-3-2015

```
P C MyBean implements BeanNameAware
{
    Private String id;
    @Override
        Public void setBeanName(String id)
        {
            this.id = id;
        }
}
```

---

② → if a bean wants a BeanFactory object then we need to implement our bean class from BeanFactoryAware interface.

```
P C MyBean implements BeanFactoryAware
{
    Private Beanfactory factory;
    @Override
    Public void setBeanfactory (Beanfactory factory)
    {
        this.factory = factory;
    }
}
```

→ setBeanfactory() method of BeanfactoryAware interface will be called by Spring Container for injecting a Beanfactory object.

---

③ → If a bean want ApplicationContext object then we need to implement that bean class from ApplicationContextAware interface

```
P C MyBean implements ApplicationContextAware
{
    Private ApplicationContext ctx;
    @Override
```

```
Public void setApplicationContext (ApplicationContext ctx)
{
    this.ctx = ctx;
}
}
```

→ setApplicationContext method of ApplicationContextAware interface called by container for injecting ApplicationContext object.

Scope → lifetime of object

* Bean Scopes:

→ a scope indicates lifetime of an object or value within a container.

→ in order to set a scope for bean, we need to configure scope attribute along with <bean> tag

→ if we do not configure scope attribute then by default spring container reads the scope as singleton.

→ the available scopes for bean in spring are

1. singleton
2. Prototype
3. request } MVC
4. session } MVC
5. global session. → Portlet

→ global session scope can be used spring with portlet.

→ request and session scope can be set for a bean only in MVC applications.

→ If we set bean scope as a singleton then spring container will creates one object of that bean class with respect to the id and returns same object for multiple times.

For example (1)

```
<bean id = "id1" class = "A" scope = "singleton" />

A a1 = (A) ctx.getBean ("id1");
A a2 = (A) ctx.getBean ("id1");

a1 == a2 ⟹ true
```

for one id one object in singleton
i.e for id1 it will create one object
& for id2 it will create one object

(2) For example

```
<bean id = "id1" class = "A" scope = "singleton" />
<bean id = "id2" class = "A" scope = "singleton" />

A a1 = (A) ctx.getBean ("id1");
A a2 = (A) ctx.getBean ("id2");
A a3 = (A) ctx.getBean ("id1");
A a4 = (A) ctx.getBean ("id2");

a1 == a2 ⟹ false.
a3 == a4 ⟹ false.
a1 == a3 ⟹ true.
a2 == a4 ⟹ true.
```

Note :→ Spring Container make bean object as singleton cort to id. It means when we call multiple times getBean() method by passing the same id, the container returns same object of the bean.

→ if the bean scope is **prototype** then Spring Container creates & returns a new object whenever getBean() method is called

3-3-2015

```
<bean id="id1" class="A" scope="Prototype" />
<bean id="id2" class="A" scope="Prototype" />
```

```
A a1 = (A) ctx.getBean("id1");
A a2 = (A) ctx.getBean("id1");
```

a1 == a2 → false

```
A a3 = (A) ctx.getBean("id2");
```

a1 == a3 → false

a2 == a3 → false

* VV Imp

Making a Java class as Singleton

→ if we created a Java class in an application with fixed data and if multiple objects are created to the java class then memory for the objects will be wasted.

→ to solve this memory Problem, one object of the Java class should be created and it should be made as sharable to all other classes.

→ To make a java class as 100% Singleton class we need to follow below rules.

① We should not allow other java classes to create an object of a singleton class. to restrict the, we need to make constructor of that class as Private

② we need to create a global access point (factory access method) to read a object of singleton class by other classes.

③ In multithreading application, if two threads calls the factory method concurrently or simultaneously then two object of the singleton class will created. to restrict two thread at a time we need to make a factory method as Synchronized method.

④ a singleton class object should not be cloned, to avoid cloning we need to override clone method and throw CloneNotSupportedException

```
Public class A implements Cloneable
{
① Private A () { }
② Private static A a;
③ ② Public static synchronized A getInstance()
    {
      if (a==null)
      {
        a = new A ();
      }
    return a;
    }
④ Public Object clone() throws CloneNotSupportedException
    {
      throw new CloneNotSupportedException();
    }
```

→ In a spring configuration file, we need to configure our singleton class like the following.

<bean id = "id1" class = "A" factory-method = "getInstance" scop= "Prototype" />

→ even though scope is Prototype but the bean class is always singleton, because Spring container also calls factory-method only to read the Object

~~A a1 = (A) ctx.getBean("id1");~~

A a1 = (A) ctx.getBean("id1");

A a2 = (A) ctx.getBean("id1");

a1 == a2 → true.

look-up injection

4-3-2015

Look-up Method injection

① Suppose in an application, we have two bean classes A & B where A depends on B

② A is a singleton bean and B is a Prototype Bean.

③ In class A, we have taken reference variable of class B as dependency and the we defined setter or constructor for injecting dependency.

Example

```
Public class A
{
  Privale B b;
 Public void setB(B b)
  {
    this.b = b;
  }
}
```

```
Public class B
{
  Public void m2()
  {
  }
}
```

XML configuration

```
<bean id = "id1" class = "A" scope = "singleton">
    <property name = "b" ref = "id2" />
</bean>
<bean id = "id2" class = "B" scope = "Prototype" />
```

→ In the above, even though the scope of class B is Prototype but Spring container injects one object of class B only to class A. because dependency of a class are injected for once, whenever its object is created.

↔ if we want to inject multiple objects of Prototype bean to one object of singleton bean then instead of constructor or setter injection, we need to apply look-up method injection.

→ for injecting a multiple objects of Prototype bean to one object of singleton bean using lookup method injection we need to do the following.

① we need to create an abstract method in class A with return type as class B

② we need to make class A as a abstract because of abstract method

③ In methods of class A we need to call abstract method to get an object of class B

```
abstract Public class A
{
    Public abstract B getObjectOfB();
    Public void m1()
    {
        getObjectOfB().m2();
    }
}
```

→ at runtime Spring container creates a Proxy class by extending class A & in that Proxy class it overrides abstract method and returns a new object of class B.

→ in the above if m1() method of class is called two times or twice then getObjectOfB() method returns an object of class B for twice.

→ in xml configuration file we need to configure the abstract method of class A as look-up method like the following.

```
<bean id = "id1" class = "A" scope = "singleton">
    <lookup-method name = "getObjectOfB" bean = "id2" />
</bean>
<bean id = "id2" class = "B" scope = "Prototype" />
```

→ Singleton bean is depend on Prototype bean

Q How can we inject an object created using Proxy design pattern into object created using Singleton design pattern.

→ using lookup-method injection.

5-3-2015

P- namespace, C- namespace & util-namespace.

→ these namespaces are included by spring in 2.5 version.

→ P- namespace is given to avoid to remove <Property> tags from an xml file.

→ C namespace is given to avoid to remove <constructor-arg> tags from xml file.

→ util namespace is given to configure a collection in xml separately to make it as reusable for multiple beans.

→ by using above 3 namespaces a lot of xml is reduced.

→ if we want to use ~~this~~ those namespaces then we need to import the namespaces into a xml file like the following.

```
xmlns:p = "http://www.springframework.org/schema/p"
xmlns:c = "http://www.springframework.org/schema/c"
xmlns:util = "http://www.springframework.org/schema/util"
```

Ex:

```
Public class A
{
    Private int k;  → dependency is Primitive type
    Private B b;  → reference type / object type
    Private List theList;  → collection type

Public A (int k)
    {
        this.k = k;
    }

Public void setB (B b)
    {
        this.b = b;
    }
```

```
    Public void setTheList (List theList)
    {   this.theList = theList;
    }
}
```

xml Configuration without using namespace

```
<bean  id = "id1"  class = "A">
    <Constructor-arg  value = 100 />

<bean  id = "id2"  class = "A"
    <property  name = "b"  ref = "id2" />

id = "id3"  class = "A"
    <Property  name = "theList">
        <list>
            <value = 10 />   |   <value>10</value>
            <value = 20 />   |   <value>20</value>
        </list>
</bean>  </property>
</bean>  </bean>
<bean  id = "id2"  class = "B" />
```

xml Configuration with namespace

```
<bean  id = "id1"  class = "A"  c:K = "100"
        P:b-ref = "id2"  p:theList-ref = "li" />

<bean  id = "id2"  class = "B" />
<util:list  id = "li">
    <value> 10 </value>
    <value> 20 </value>
</util:list>
```

→ the following are the 4 tags of util namespace.

① <util : List>

② <util : Set>

③ <util : Map>

④ <util : Properties>

→ if we configure util : List tag then by default the Spring container creates of an object of ArrayList (java.util.ArrayList).

→ If we want to customize class name then we need to configure list - class attribute

```
<util : list  id = "li"
              list-class = "java.util.LinkedList">

</util:list>
```

→ If we configure <util : set> then container creates bydefault an object of LinkedHashSet, if we want to customize then we need to add set - class Attribute

```
<util : set  id = "s"  set-class = "java.util.TreeSet">
   - - -
</util:set>
```

→ If we configure <util : Map> then by default Container creates an object of LinkedHashMap, if we want to customize then we need to add map - class Attribute

```
<util:map  id = "m"  map-class = "java.util.TreeMap">
   - - -
</util:map>
```

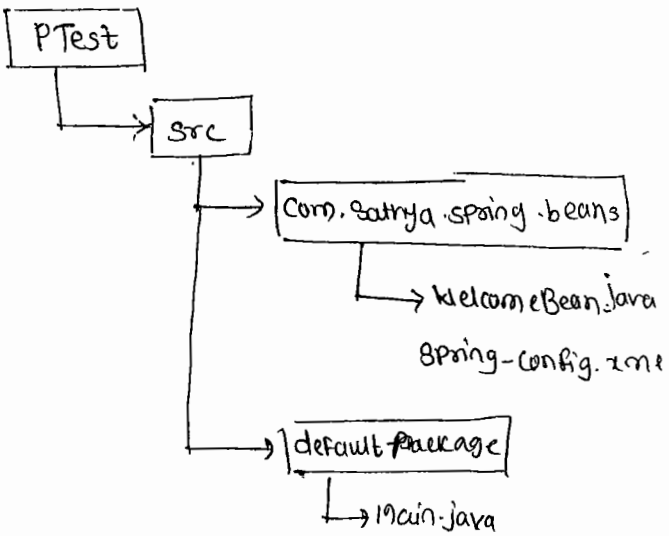
→ for <util: Properties> tags, the container creates a Properties class object Properties is a already a class only so we cannot customize class again.

```
<util:Properties id="p">
    <Prob key="d"> 1000 </Prob>
    <Prob key="a"> satya </Prob>
</util:Properties>
```

```
Map m = new HashMap();
m.put("a", "100");
m.put("b", "200");
sop(m);  → {a=100, b=200}
```

for Set & List o/p in [ ]
for Map (key, value) o/p in { }

6-3-2015



// WelcomeBean.java

```
Package com.sathya.spring.beans;
import java.util.Map;
Public class WelcomeBean
    {
      Private String message;
      Private Map theMap;

    Public void setTheMap (Map theMap)
      {
        this.theMap = theMap;
      }
    Public void setMessage (String message)
      {
        this.message = message;
      }

     Public void showMessage()
        {
          SOMn(message);
        }

     Public void showTheMap()
        {
          SOPln (theMap.getClass().getName());
          SOPln (theMap);
        }
}
```

<-- spring-config.xml -->

```
<beans xmlns = "http://www.springframework.org/schema/beans"
xmlns:p = "http://www.springframework.org/schema/p"
xmlns:util = "http://www.springframework.org/schema/util"
xmlns:xsi = "http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation = "http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-4.0.xsd
                                                    /util
                                                    /util/spring-util-4.0.xsd">

<bean id = "welcomeBean" class = "com.sathya.spring.beans.WelcomeBean"
                                  p:message = "welcome to spring" />
<util:map id = "m1" map-class = "java.util.HashMap"> p:theMap-ref = "m1" />
    <entry key = "j" value = "java" />
    <entry key = "1" value = "100" />
</util:map>
</beans>
```

/ → file
. → Java class

```
// Main
class Main
{
    main {
        ApplicationContext ctx = new ClassPathXmlApplicationContext ("spring-config.xml");
        Object o = ctx.getBean ("welcomeBean");
        WelcomeBean welcomeBean = (WelcomeBean) o;
        welcomeBean.showMessage ();
        sopln ("====");
        welcomeBean.showTheMap ();
    }
}
```

"com/sathya/spring/beans/spring-config.xml"
write fully qualified name

```
O/P
welcome to spring
==========
java.util.HashMap
{1=100, j=java}
```

BeanPostProcessor interface

Common bean initializing logic

```
p c MyBeanPostProcessor implements BeanPostProcessor
{
   @Override
      p Object postProcessBeforeInitialization (Object bean, String name)
      {
         "
        }

   @ Override
      p Object postProcessAfterInitialization (Object bean, String name)
      {
        "
        }
}
```

→ In a Spring Application, if a multiple beans are having common initialization logic and if it is define in each bean separately then duplication of the code occurs in a application.

→ In order to avoid duplication of initialization logic, framework has given BeanPostProcessor interface.

→ instead of duplicating Common initialization logic we need to create a separate class by implementing BeanPostProcessor interface to define a Common initialization logic.

→ for Example

```
Public class MyBeanPostProcessor implements BeanPostProcessor
{
```

@Override

```
Public Object PostProcessBeforeInitialization (Object bean, String name)
{
    ==
}
```

@Override

```
Public Object postProcessAfterInitialization (Object bean, String name)
{
    ==
}
```

}

→ As Part initialization of Spring bean Container 1st calls ~~PostProcess~~ postProcessBeforeInitialization method. After that initialization method defined in the Bean class will be called. ~~Finally~~ finally PostProcessAfterInitialization method will be called.

for example

```
Public class A
{
    Public void setUp()
    {
        ==
    }
}

Public class B
{
    Public void init()
    {
        ==
    }
}
```

Order of initialization

① PostProcessBeforeInitialization()
② setUp()
③ PostProcessAfterInitialization()

Order of initialization

① PostProcessBeforeInitialization
② init()
③ PostProcessAfterInitialization

Bean life cycle

7-3-2015

- life cycle is for object.
- Object is created by JVM by new keyword.
- JVM calls constructor

→ If we initialize object in method, we have to call that method.

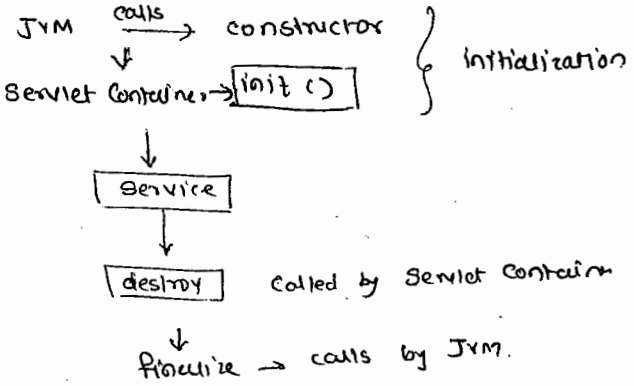
Constructor
→ ^ to indicate a JVM is finished object creation.

JVM
↓
create object
↓
calling constructor
for initialization

```
P c Δ
{
  int x, y

  void init()
    { x=10;
      y=20;
    }
}
```

Servlet → class Servlet Container internally calls JVM to create object



→ life cycle of a Java class contains 5 well defined stages.

1. does not exist
2. Instantiation.
3. Initialization
4. Service.
5. destroy

→ Actually life cycle exists for object, not for a class. but we call object life cycle as class life cycle.

→ for every class in Java that is for POJO class, Servlet class, or Spring bean class etc. there is a life cycle.

→ **Pojo life cycle:**

```
Public class A
{
  void m1 () { }
  void m2 () { }
}
```

① before an object of class A is created, its life cycle stage is doesnot exist

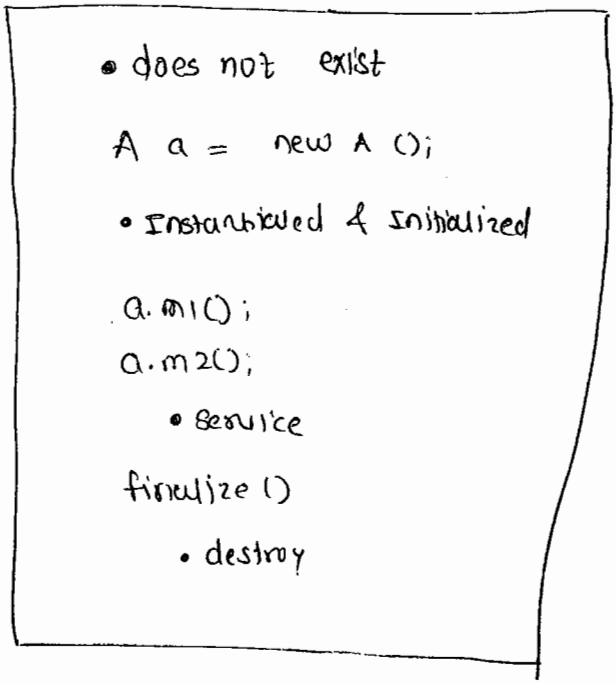
② each Java class will be instantiated by JVM only.

③ after instantiation (memory allocation) JVM replies that instantiation is completed by calling the constructor.

④ If we defined any initialization logic in constructor then it will be executed. If initialization logic defined in any other method then that method is need to be called to finish initialization.

⑤ after initialization that object will provide the service, by calling the methods.

⑥ finally, before an object is going to garbage collector finalize() method will be called by JVM, to indicate object is destroyed



Servlet life Cycle:

→ In servlet we say that life cycle will be managed by Servlet container.

→ Servlet container also internally calls JVM only for instantiating a servlet nothing but creating object.

→ JVM replies servlet container that a servlet is instantiated by calling constructor. After that a servlet container calls init() method to finish initialization.

→ after initialization service() method is called for each request to provide service to the clients.

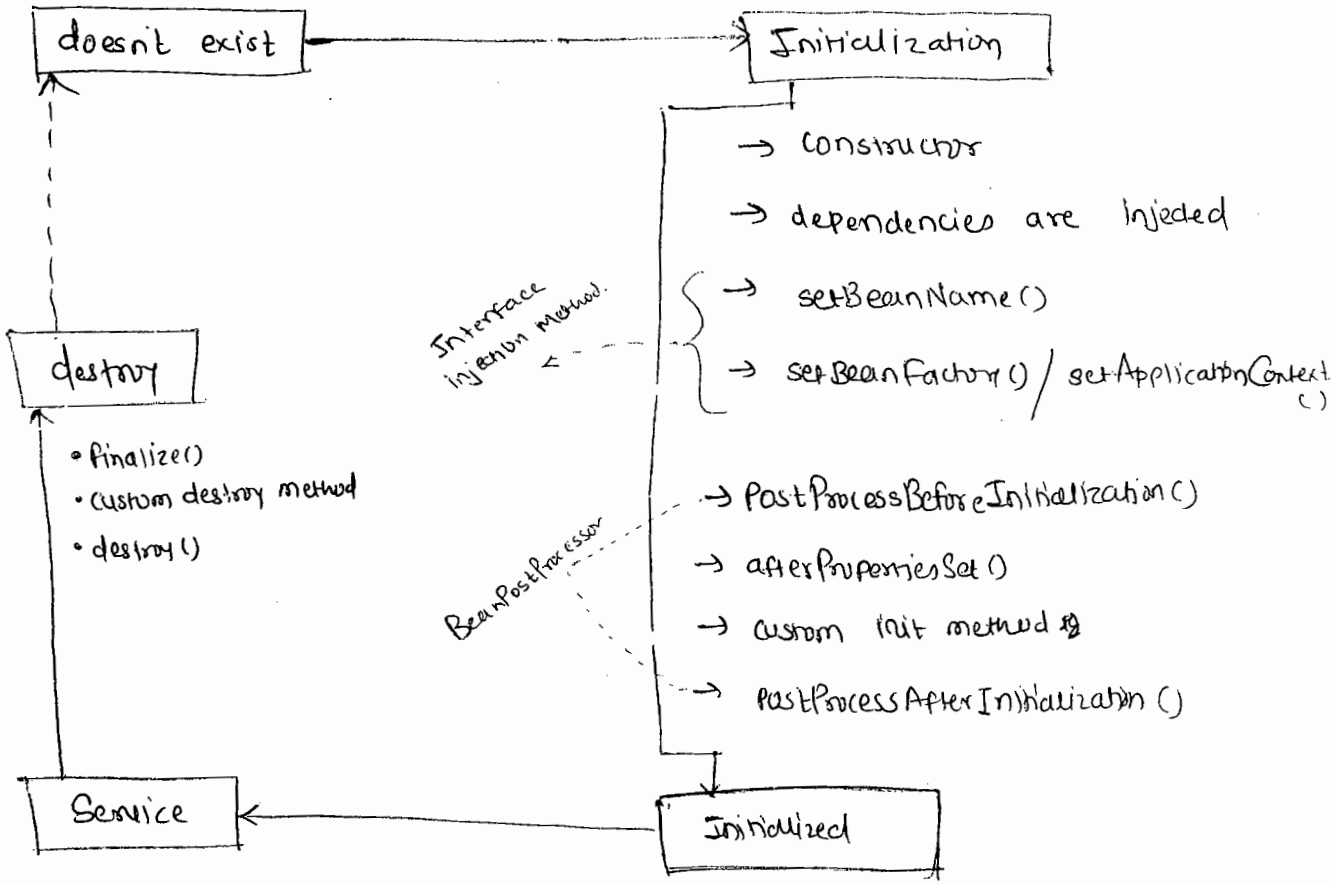
→ a servlet container releases object to JVM by calling destroy() method and later JVM releases to garbage collector by calling finalize() method.

→ So a servlet object is completely destroyed.

→ In the life cycle of the servlet also we have the same 5 stages

8-3-2015

Bean Life cycle



→ In Spring framework, we say that, Spring Container creates a a Bean Object but Spring container internally calls JVM only for creating an object.

→ after instantiating bean class by JVM, the JVM calls constructor of the Bean class, so that initialization begins.

→ after constructor is executed, the Spring Container injects the dependencies required to Bean object.

→ the Spring Container checks wheather a Bean class implements BeanNameAware Interface or not, if yes then calls setBeanName() method.

→ a Spring Container checks a bean class implements BeanFactoryAware or ApplicationContextAware interface or not.

→ If BeanFactoryAware interface is implemented then calls setBeanfactory() method & If ApplicationContextAware interface is implemented then calls setApplicationContext() method.

→ If BeanPostProcessor class is created then container calls PostProcessBeforeInitization() method.

→ If a bean class implements InitializingBean interface then container calls afterPropertiesSet() method.

→ If a custom init method is defined then it will called

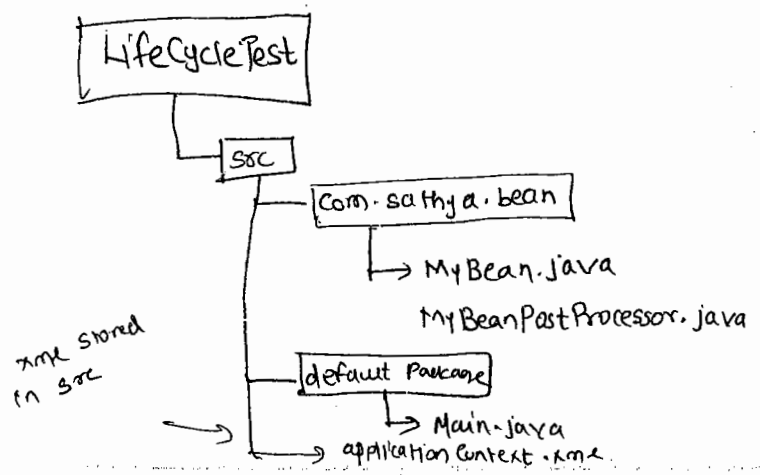
→ PostProcessAfterInitialization() of a BeanPostProcessor class is called.

→ Now the bean is initialized and ready to provide to service.

→ Before removing a bean object from container, it checks wheather a bean class implements DisposableBean interface or not. if yes then it calls destroy() method.

→ If a custom destroy method is defined then it will be called.

→ Before a bean object is going to be garbage collected, jvm call finalize() method. now the bean object is completely destroyed.



ApplicationContextAware

Public class MyBean implements BeanNameAware, ~~ApplicationContext~~, InitializingBean,

DisposableBean

{

Private String msg;

Public MyBean()

{

Sopln ("constructor called");

}

Public void setMsg (String msg)

{

this.msg = msg;

Sopln ("dependency injected.");

}

@Override

Public void destroy () throws Exception {

Sopln ("destroy method called");

}

@Override

Public void afterPropertiesSet () throws Exception {

Sopln ("afterPropertiesSet method called.");

}

@Override

Public void setApplicationContext (ApplicationContext arg0)

zero

throws BeansException {

Sopln ("setApplicationContext method called");

}

@Override

public void setBeanName (String arg0) {

Sopln ("setBeanName method called.");

}

```
Public void setUp()
{
    SOPln ("custom init method called");
}

Public void tearDown()
  {
     SOPln (" custom destroy method called.");
  }

 Public void m1()
    {
      SOPln ("m1 method called");
    }
}
```

// MyBeanPostProcessor.java

```
Public class MyBeanPostProcessor implements BeanPostProcessor

  {
   @Override
  Public Object postProcessAfterInitialization ( Object arg0, String arg1)
       throws BeansException {
       SOPln (" postProcessAfterInitialization method called");
       return arg0;
      }

  @Override
  Public Object postProcessAfterInitialization (Object arg0, String arg1)
       throws BeansException
      {
        SOPln (" postProcessBeforeInitialization method called");
        return arg0;
      }
}
```

applicationContext.xml

```
<beans>
    <bean id = "MyBean" class = "com.sathya.bean.MyBean" init-method = "setUp"
                                                destroy-method = "tearDown" >
        <Property name = "msg" value = "sathya" />
    </bean>
    <bean class = "com.sathya.bean.MyBeanPostProcessor" />
</beans>
```

// Main.java

```
Public class Main {
    Main {
        ClassPathXmlApplicationContext ctx = new ClassPathXmlApplicationContext
                                            ("applicationContext.xml");
        MyBean myBean = (MyBean) ctx.getBean ("MyBean");
        myBean.m1();
        -- -> ctx.close();
        }
}
```

ctx.close() → method is not in ApplicationContext interface, thats why ClassPathXmlApplicationContext is taken

o/p

Constructor called
dependency injected.
SetBeanName method called.
SetApplicationContext method called.
PostProcessBeforeInitialization method called
afterPropertiesSet method called
Custom init method called.
PostProcessAfterInitialization method called
m1 method
destroy method called
custom destroy method called

✓

9-3-2015

Core Module annotations.

① Stereo type annotations
② autowiring annotations
③ Miscelleneous annotations

xml → Meta information.
xml has extensibility, flexibility.

→ When working with Java technologies & Java frameworks, a Programmer is need to write some configuration data in xml file.

→ When size of the project is increasing, the size of xml files are also going to be increase.

→ to cutdown the size of xml files, annotations are introduced in Java

→ with the help of annotations, we can also define meta data in source code only. So there is no need to define metadata in xml file seperately.

→ In spring framework the framework has given annotations for each module to reduce the size of the spring configuration file

→ the annotations of a core module are divided into 3 types.

① Stero type annotations.
② autowiring annotations
③ Miscelleneous annotations

Stereo type Annotations ①

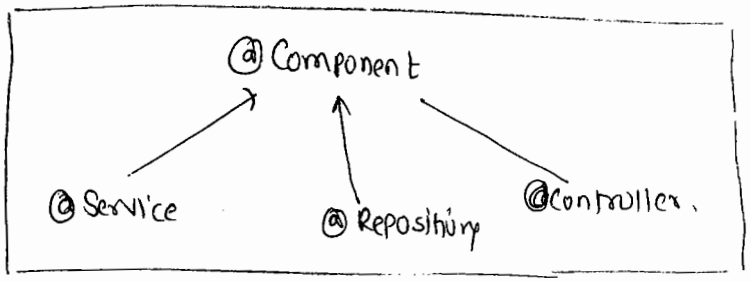
→ Stereo type Annotations tells about roll of a class in an application

→ a Programmer can identified wheather a class containing or consisting either business logic or Persistence logic or controller logic easily with the help of Stereo type Annotation.

→ the framework has given 4 Stereo type annotations

{ ① @Component → tells role of class
② @Service → business logic
③ @Repository → Persistence logic
④ @Controller → controller bean of MVC application

→ @Component annotation is a base for the remaining 3 annotations



→ if we add @Component annotation on the top of class then it tells the a class is a component class. it means it is class consisting some logic.

→ @Component annotations doesn't tell weather a class is specifically containing business or Persistence or Controller logic. So we don't use directly this @Component annotation.

→ if we add @Service annotation then it tells the class consisting business logic.

→ if we add @Repository annotation on top of class then it tells the a class consisting persistence logic.

→ if we add @Controller then it tells that a class is acting as Controller bean of MVC application.

for example

for ex:

```
Package com.sattya.beans;
@Service
Public class TestBean
  {
    Public void m1()
      {
        // Business code
      }
  }
```

```
Package com.sattya.beans;
@Repository
Public class CurdBean
  {
    P V insert()
      {
        // Persistence logic
      }
  }
```

→ the classes which are added with Stereo type annotations are going to be ~~visible~~ visible to spring container in a Component-Scanning

→ Component Scanning is a feature added in Spring 2.5 where ~~where~~ a Spring Container will automatically Configure the beans at runtime, by Scanning a given Package.

→ the benefit of Component Scanning is we no need to explicitly configure the beans in xml file so the size of the xml will be reduced.

→ In component scanning a container identifies a class of Package

as a bean, if a stereo type annotation is added on top of the class.

→ To enable component scan feature, we need to configure <context: component-scan> tag with package name in xml file

for example

< context: component-scan base-package = "com.sathya.beans" />

→ the spring container scans com.sathya.beans package & also its sub packages for the components.

→ the container configures a component as bean at a runtime with unqualified class name as id, by starting it with a lower case letter

for example the id configured for com.sathya.beans.TestBean is testBean

(2) Autowiring Annotations

10-3-2015

→ bean Autowiring is a ~~fasci~~ fasility provided by spring container, for automatically injecting dependencies or reference types., by without explicitly configuring it id

→ autowiring facility is only applicable for dependencies of reference types.

→ Annotation for Autowiring.

① ~~@Autowired~~ ① @Autowired

② ~~@Qualifier~~ ② @Qualifier.

→ these autowired annotation is applicable for ~~beans~~ fields (variables), methods, & for constructors.

→ Stereotype Annotation are class level annotations, It means they are only applicable for classes.

→ if we add @Autowired annotation then a Spring container checks for a bean class in xml is matched with Property type or not. If ~~not~~ matched then injects a that bean object by calling either setter or a constructor

→ if not matched then throws an exception.

Example

```
// Movielister.java

Package com.sathya.beans;
@Service
    Public class Movielister
      {
          @Autowired
          Private Moviefinder finder;
          Public void setfinder (Moviefinder finder)
           {
              this.finder = finder;
           }
      }

// Moviefinder Java
Package com.sathya.beans;
@Service
   Public class Moviefinder
    { }

Xml Configuration

   <Context: component-scan  base-package = "com.sathya.beans"/>
```

→ In the above example @ at the time of component scanning, the container finds a ~~@~~ ~~Movie~~ Moviefinder which is matched with the Property type in MovieLister. So it injects the an object of Moviefinder bean by calling setter method.

Required attribute

→ If a we want to tell the container that if bean class in xml is not matched with property type then don't throw an exception then we need to add required element to the @Autowired Annotation.

```
Public class MovieLister
{
   @Autowired (required = false)  [default value is true]
                                  — i.e if throws the exception.
   Private Moviefinder finder;
   ---
   ---
}
```

→ by passing required = false we are telling a spring container that a dependency is optional.

→ if constructor injection is defined then, even though required = false, container throws an exception to ~~overcome~~ overcome it we need to defined default constructor also

```
Public class MovieLister
{
   @Autowired (required = false)
   Private Moviefinder finder;
   Public MovieLister()  [default constructor]
   {}
   Public MovieLister (MovieFinder finder)
   {
      this.finder = finder;
   }
}
```

@Qualifier

→ if a bean class in a xml is matched for more than one with property type, then ambiguity occurs and container throws an Exception. to resolve it, we will use @Qualifier Annotation.

→ @Annotation @Qualifier Annotation always used with combination with @Autowired annotation, it cannot be used independently

```
Public class MovieLister
{
    @ Autowired
    @ Qualifier("id2")
    Rivate MovieFinder finder;
    ==
}
```

Xml Configuration

```
<context: component-scan base-Package = "com.sathya.beans"/>
<bean id = "id2" class = "com.sathya.beans.moviefinder" />
```

③ Miscellaneous annotations:

① @Scope:

→ this annotation is used to set the scope for a bean class

```
@Service
@Scope (value = "Prototype")
Public class A
  {
  }
```

②&③ @PostConstruct & @PreDestroy

→ these annotations are used to tell the container about ~~Constructor~~ custom init method & custom destroy method of bean class to the container

```
@Service
  Public class A
    {
      @PostConstruct
       Public void setUp()
        {
          // initialization logic
        }
      @PreDestroy
       Public void tearDown()
        {
          // destruction logic
        }
    }
```

Core module Annotation

Stereotype

- @Component
- @Service
- @Repository
- @Controller

Autowiring

- only for ref. type.
- @Autowired
- @Qualifier

Miscellenous.

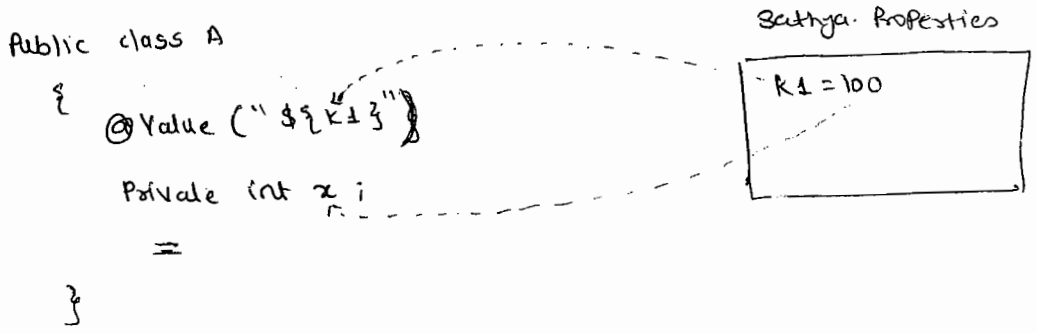
- @Scope
- @PostConstruct &
- @PreDestroy.
- @Value
- @Primary

11-3-2015

④ @Value:

→ this annotation is given for injecting a value to a Primitive type or simple type property, by reading it from a resource container.

→ To configure the location of the resource ~~bundle~~ bundle. we need to configure <context: Property-placeholder> tag in spring configuration file



Xml Configuration

<context: Property-Placeholder location = "Sathya.Properties" />

⑤ @Primary:

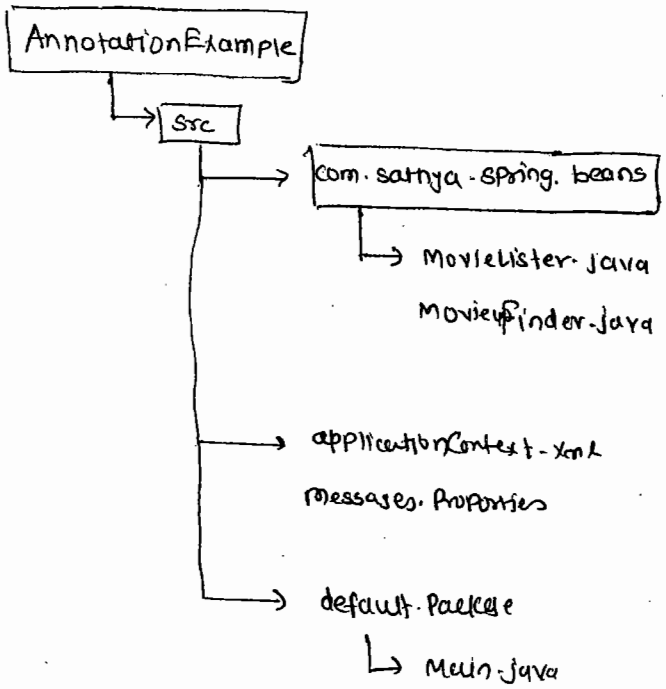
→ when multiple classes are implementing a common interface then, to tell the container give more Priority to one implementation class we use @Primary Annotation on top of the class

Package Pack1;
@Service
P c Travel
{
@Autowired
Private Vehicle v;
⇒ setter
}

Package Pack1;
@Service
→ @Primary
P c car implements Vehicle
{
}

Package Pack1;
@Service
P c Bike implements Vehicle
{
}

Program



① // MovieLister.java

```
Package com.satnya.spring.beans;
import javax.annotation.PostConstruct;
@Service
@Scope ("Prototype")
Public class MovieLister
{
    @Value ("${msg}")
    Private String message;
    @Autowired
    Private MovieFinder finder;
Public void setMovieFinder (MovieFinder movieFinder)
    {
        this.finder = movieFinder;
    }
```

@PostConstruct

```
Public void setUp()
{
   sopln ("I am setup method");
}
```

@PreDestroy

```
Public void tearDown()
{
   sopln ("I am tearDown method");
}

Public void showMessage ()
{
   sopln (message);
}

Public void m1 ()
{
   sopln ("I am m1 of MovieLister");
   finder.m2 ();
}
}
```

(2) // MovieFinder.java

@Component

```
Public class MovieFinder
{
  Public void m2()
  {
     sopln ("I am m2 of MovieLister");
  }
}
```

③ application context.xml

```
<beans>
    http://www.springframework.org/schema/beans/spring-beans-4.0.xsd
    http://www.springframework.org/schema/context/
```

```
<context:component-scan base-package="com.sathya.spring.beans"/>
<context:property-placeholder location="messages.properties"/>
<context:annotation.config/>
```

↓

this tag tells the container execute autowiring & misellenous annotation of Core module.

④ # messages.properties

```
msg = Hello
```

⑤ Main.java

```
Public class main
    {
      main {

ClassPathXmlApplicationContext ctx = new ClassPathXmlApplicationContext
                                     ("applicationContext.xml");

MovieLister lister = (MovieLister) ctx.getBean ("movieLister");

lister.showmessage();

lister.m1();

ctx.close();
 }
}
```

list of jar

1. aopalliance - 1-0.jar → not from spring software
2. commons-logging - 1.0.4.jar
3. spring-aop - 4.0.5. RELEASE
4. spring-beans - 4.0.5 RELEASE
5. spring-context - 4.0.5. RELEASE
6. spring-context-support
7. spring-core
8. spring-expression

Spring Expression Language (SpEL)

12-3-2015

${expression} → JSP

#{expree} → spring

name="a" value = "#{id1.k * 10}"

↑ value assigned at runtime & depends on k

→

→ So far, we know the we can pass the static value to the properties of a bean at compiled time in a xml file.

→ if we want to set a value to ~~the~~ a property, by reading it from another bean property or by calling method of another bean at run time. then we use SpEL in spring configuration file, we can pass expression as value, so that it will be evaluated at run time & its value will be injected to the properties at runtime.

Example

```
P C A
{
  Private int k;
  Public void setk (int k)
  {
    this.k = k;
  }
  Public int getK()
  {
    return k;
  }
```

```
P C B
{
  Private int z;
  P v setZ (int z)
  {
    this.z = z
  }
}
```

```
<bean  id = "id1"   class = "A" >
    <Property  name = "k"  value = "10" />
</bean>
<bean  id = "id2"  class = "B" >
    <Property  name = "z"  value = " #{ id1.k * 10 } " />
</bean>
```

* the Syntax of Spring expression begins with #{ , end with }

→ In the above expression it calls a getter method of Property "A" at runtime, then multiplies with 10 & then injects that value to "z" Property

Example :

```
P  C  Guiter
{
  Private String songName;
  Public void setSongName (String songName)
    {
      this.songName = songName;
    }
}

P  C  Song
  {
    Public String readSong()
      {
        return " xxxxx ";
      }
  }
```

Xml Configuration

```
<beans id = "id1" class = "Guiter">
    <property name = "songName"
              value = "#{id2.readSong()}" />
                          ↳ calls at runtime.
</bean>
<bean id = "id2" class = "Song" />
```

→ at runtime, a String value returned by ~~react Son~~ readSong() method will be set to songName property.

→ When calling a method on a method in a string expression, if first method returns null then when calling the 2nd method NullPointerException will be thrown.

→ To avoid NullPointerException, SpEL has added a ~~NullSafe operator~~ null safe operator (?.)

→ the null safe operator checks wheather a left side value is null or not null before calling the right side method. If left side value is null then it doesn't call right side method.

```
<bean id = "id1" class = "Guiter">
    <property name = "songName"
              value = "#{id2.readSong()?.toUpperCase()}" />
</bean>
<bean id = "id2" class = "Song" />
```

→ if we want to call either static method or static properties of a class in expression language then we use "T" operator

```
<bean id = "id1" class = "A">
    <property name = "K"
    value = "#{T (java.lang.Math).pow(2,3)}" />
</bean>
<bean id = "id2" class = "B">
    <property name = "x"
        value = "#{T (java.lang.Math).PI}" />
</bean>
```

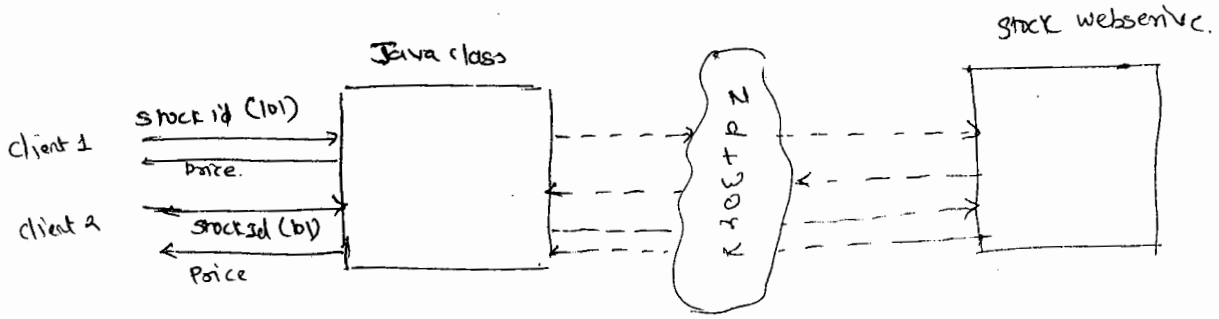
Diff. b/t^n BeanFactory & ApplicationContext Container

BeanFactory (Never used in Real Project)	ApplicationContext (always used.)
① BeanFactory is basic container & it cannot Pre instantiated Singleton beans.	① ApplicationContext is an advanced container and it can Preinstantiate Singleton bean.
② BeanFactory container only supports Singleton & Prototype supports.	② ApplicationContext Container supports all beans scope.
③ SpEL is not supported by BeanFactory.	③ SpEL is supported by ApplicationContext.
④ In BeanFactory we need to explicitly add an object of a BeanPostProcessor class to the BeanfactoryContainer.	④ In ApplicationContext, we need to configure the BeanPostProcessor class in Xml. but we no need to add an object explicitly.
⑤ a BeanFactory container can create a Bean objects and injects dependency. but it cannot provide other additional services like transaction, scheduling, or messaging etc.	⑤ ApplicationContext container will inject dependencies and also provides the additional services to the beans.

18-8-2015

Aspect Oriented Programming (AOP)

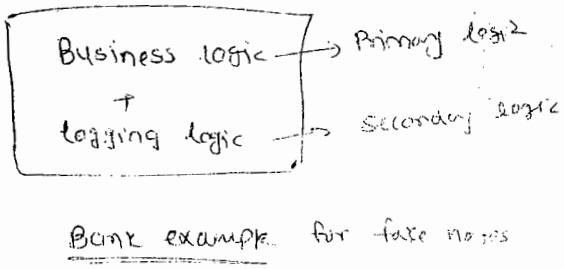
Primary logic & secondary logic



Primary logic ⟶ business logic

Secondary logic ⟶ caching logic → to improve performance.

```
p    c    withdraw
{
  p  v   withdrawMoney (-)
  {
     ≡ logic.
  }
}
```



Bank example for fake notes

→ Aspect oriented Programming

→ when creating Java Appln with busi logic, it consist both Primary and secondary logics. a Primary logic is a business logic and secondary logic is a service logic to a business logic, to make it as efficient.

for Example

① → We have created a Java class, it reads a stockId from client then calls a web service in network to read the price and then returns to the client. this is business logic in the class.

② → If other clients are also sending same stockId then reading the price everytime by calling the webservice will make the Performance of an Application is Poor.

③ → To improve the Performance we can add some catching logic to the business logic. Here business logic is Primary & secondary logic is catching

→ a Primary logic can function on its own but a secondary logic will not function on its own

→ In Spring ~~technology~~ terminology the secondary logic is called cross-cutting concerns.

Ex:1

```
P  C  withdraw
{
   P  V  withdraw ()
   {
      // logging service
      // Authentical service / security service   } → secondary logic or cross-cutting concerns
      // withdraw logic                           } Primary logic.
   }
}
```

Ex.2

```
P  C  Deposit
{
   P  Y  deposit()
   {
      // logging service      } → cross-cutting concerns
      // Authentical service  }
      // deposit logic
   }
}
```

→ Mostly same ~~class~~ cross-cutting logic are required multiple Business logic in an application

→ if Cross-cutting logics are duplicated then we will get redundancy & also code maintainance Problem

→ In order to overcome above problem we separate the cross-cutting logics from business logic of Application.

IS-A → tight coupling
HAS-A → loose coupling

Q. What is backend OS ? → file system.

Q. diff betn AOP & OOP → AOP is way of Achieving OOP
Internally AOP is also uses OOP

16-3-2015

→ By using Object oriented Programming (OOP) relationships like IS-A & HAS-A, we can combine services to the business

→ In IS-A relationship, a business class extends services class.

We find two Problems here.

① If a business class extends services class then it can't extends another business class.

② If we forgot to call a service then a business will loose that service.

~~16-3-2015~~

→ In case of a HAS-A relationship, we need to create an object of services class in each business class, we have a problem in this HAS-A relation

(i) if we forgot to call a service on a business then it will loose that service

→ to resolve the problem of manually applying IS-A & HAS-A relation between business & service classes, spring framework has introduced another style of programming called AOP.

→ In case of AOP, the spring container will generate a proxy class per business class for combining the services to the business, for this we need to tell the container about what services are needed to what business by defining through an XML Configuration file.

→ AOP Terminology:

* **Aspects:** It is a term used to denote a particular cross cutting concern.

For Example: We call login service as logging aspect

Advice:-

→ an advice is an implementation of aspect.

→ an advice will add some new behaviour to the business.

→ The implementation code of a logging aspect is logging

JoinPoint: → nothing but the method

→ It is a point at which services are going to be combined with a business.

→ In spring the services are combined with a business then that business method is accessed. so it is called a method access join point.

PointCut

→ a PointCut is an expression, which is used to put a condition for identifing JointPoints

→ So finally, a PointCut is a group of Joint Points.

Advisor

→ A PointCut expression only identifies JointPoints, but a PointCut doesnot know what advises are required for the jointPoints.

→ an advisor combines an advice to a PointCut. it is also called a PointCut advisor.

advisor = PointCut + advice

Weaving

→ It is a Process of converting a target class into Process class by adding advices/ services.

→ Before adding services through AOP a business class is called a target class & after adding the services through AOP a business class becomes a Proxy class.

→ PreAOP a business class is target

→ PostAOP a business class is Proxy

→ this weaving can be done at runtime by a, Weaver class.

17-3-2015

In Spring framework AOP application can be developed in 3 approaches

① Proxy-style AOP [1.x]
② xml-based AOP [2.0]
③ Annotation-based AOP [2.5]

① Proxy-style AOP

creating advices

→ this style of AOP is introduced with Spring 1.x. the
→ the implementation of a service will be defined in a advice.
→ Services are need to be injected to the business logic at 4 points
① before a business logic starts. it may need a service.
② after a business logic is executed, it may need a service.
③ a service may be needed when an exception is occured in the business logic.
④ a service may be needed before business logic starts & after a business logic is executed.

→ Based on the places in a business for injecting a services, advices are divided into 4 types

① before advice.
② after returning advice.
③ throws advice.
④ around advice.

① before advice

→ a service logic which we want to apply to a business logic, before business logic going to start is defined in before advice by implementing our class from ~~method before~~ MethodBeforeAdvice interface.

→ MethodBeforeAdvice has one abstract method called "before". So we should override this method to implement a service

- During compilation time the services will not be applied to our logic, services will apply only at run time.

for ex: - so ~~is~~ we need to implement our class to MethodBeforeAdvice interface

```
Public class LoggingBefore implements MethodBeforeAdvice
{
    @Override
    Public void before (Method m, Object[] args, Object target)
    {
        ==
    }
}
```

→ by using Method class object, we a service can access a name of business method.

→ by using @Object[], a service can access a business method Parameter.

→ by using target object, a service can access helper methods of Business Class.

(2.) After Returning Advice

→ the services which we want to apply to a method, after a business logic is executed, will be defined as an after returning advice.

→ an after Returning advice can be created by implementing our class from ~~after~~ AfterReturningAdvice interface.

```
Public class LoggingAfter implements AfterReturningAdvice
{
    @Override
    Public void afterReturning (Object returnvalue, Method m,
                                Object[] args, Object target)
    {
        =
    }
}
```

→ a service defined in afterReturning advice can access the returnvalue of the business method.

→ If a business method has return type as void then the returnvalue in the advice will be null.

③ throws advice

→ If ~~you~~ we want to apply a service to a business logic, when an exception is occured in business, then we need to ~~this~~ create ~~a~~ a throws advice

→ To create a throws advice, we need to implement our class from ThrowsAdvice.

→ ThrowsAdvice is a marker, so it doesn't have any abstract methods.

→ We must defined the implementation of service in afterThrowing() method only because a weaver class in spring defined in such a way that it calls after throwing method of a throws advice.

for Ex

```
P C LoggingThrows implements ThrowsAdvice
                                ↑ marker interface
{
    P V afterThrowing ( Method m, Object[] args, Object target,
                        Exception e)
    {
        ≡
    }
}
```

18-3-2015

④ around advice

↓

before advice + after returning advice

→ an around advice is a combination of before advice & after returning advice

→ this around advice type is included by spring framework from another open source group called AOP-alliance group.

→ to create an around advice, we implement our class from an interface MethodInterceptor

→ MethodInterceptor interface has one abstract method called around(), so we need to override around() method, to define before & after returning services or advices.

```
Public class LoggingAround implements MethodInterceptor
{
  @Override
    Public Object around (MethodInvocation mi)
      {
        == } before

      Object o = mi.Proceed();

        == } after-returning

      return o;
    }
}
```

→ In the middle Proceed method is called to invoke the business method.

→ the return value of Business method will be stored in Object. if a business method has return type as a void then null will be stored in an object

Defining PointCuts:

→ In spring framework, JointPoints are methods

→ all business methods of class may not require advices so a Pointcut identifies a collection of jointPoints based on some condition

→ spring framework has already given some predefined Pointcut classes, to configure the conditions. if they are not suitable to our project then we can define custom Pointcut class.

String mappedName;

String[] mappedNames;

↓ configure with <list> tag

→ the 2 Predefine Pointcut classes given by fw

1. NameMatchMethodPointcut
2. JdkRegexpMethodPointcut

→ with NameMatchMethodPointcut class, we need to configure either mappedName or MappedNames Property.

→ To configure a single condition for identifing jointpoints, we configure MappedName Property

→ for multiple conditions, we configure MappedNames Property

Example 1

```
<bean id = "id1" class = "NMmp">
    <property name = "MappedName"
              value = "Say *" />
</bean>
```

```
void sayHi(){
    ==
}
void welcome{
    ==
}
void sayBye{
    ==
}
```

Example 2

```
<bean id = "id1" class = "NMmp">
    <property name = "MappedNames">
        <list>
            <value> Say* </value>
            <value> *e </value>
        </list>
    </property>
</bean>
```

→ In Example 1 a Pointcut makes methods starts with Say as joint points. in example 2 Pointcut makes methods starts with Say and ends with e as joint points.

→ In ~~Name match~~ "NameMatchMethodPoint" we can use only one special character "*" and it can be used at end of values or at begin of value but not in middle

→ MappedNames is a property of type String[] so it is configure in the xml with <list> tag

* → with ~~JDKReg~~ JdkRegularExpressionMethodPointcut we can configure Pointcut conditions effectively, with the help of Regular expression Symbols

→ each ~~Value~~ Regular expression value must begin with ".*"

→ we can use the following special character to define condition.

- `.*` → replaces 0 or more characters
- `.+` → replaces 1 or more characters
- `.?` → replaces 0 or 1 characters.

`[]` → for range of characters

Example 1

```
<bean id = "id1" class = "JRMP">
    <Property name = "pattern"
              value = ".*say.+[0-9]" />
```

↳ Starts with say & ends with (0-9) number

Example 2

```
<bean id = "id2" class = "JRMP" >
  <Property name = "patterns">
    <list>
      <value> .*say.+[0-9] </value>
      <value> .*[a-z].*[a-e] </value>
    </list>
  </Property>
</bean>
```

→ StaticMethodMatcherPointcut
→ DynamicMethodMatcherPointcut

} class

19-3-2015

Creating a Custom Pointcut

→ if a predefined Pointcut classes are not enough then we can defined user defined Pointcut classes. they are called Custom Pointcut.

→ In Proxy style Aop Pointcut classes are two types

① Static Pointcut.

② Dynamic Pointcut.

→ a Static Pointcut identifies a JoinPoint, based on class name & Method name but not on Parameters values of the method.

→ a dynamic Pointcut identifies JoinPoint based on class name, method name and also Parameters values.

→ if we want to create user define Static Pointcut class then we need to extend ~~out~~ our class from StaticMethodMatcherPointcut

```
Public class MyPoint extends StaticMethodMatcherPointcut
                                    (abstract)
{
  @Override
    Public boolean matches (Method m, Class c)
    {
    =
    }
}
```

→ If we want to create a user define dynamic Pointcut class then we need to extend our class from DynamicMethodMatcherPointcut class

Public class MyPointcut extends DynamicMethodMatcherPointcut {
(abstract)

@Override

public boolean matches (Method m, Class c, Object[] args)

{

=

}

}

Configuring advisor

→ an advisor will attach advices to a set of joinpoint identified by Pointcut.

→ an advisor is a combination of Pointcut & advice.

→ framework has given predefined advisor class called DefaultPointcutAdvisor and we need to configure this class in spring configuration file

```
<bean id = "advisor"  class = "DefaultPointcutAdvisor">
    <property  name = "pointcut"
               ref = "pt1" />
    <property  name = "advice"
               ref = "adv1" />
</bean>
```

ProxyFactoryBean → act as Weaver class

Create Proxy class at run time & create proxy class object

Weaving → convert target class to Proxy class

Creat Proxy class at runtime & add business & service logic at runtime

↳ by weaver class

ProxyFactoryBean → is Weaver class

① ~~ProxyInterfaces~~

Configuring Weaver

→ a Weaver class creates a Proxy class at run time for combining the services with a Business logic.

→ when creating a Proxy class at run time, that Proxy class is implemented from an interface i.e implemented by target class (Business class).

→ In Spring a bean which returns an object of another bean is called a FactoryBean.

→ a ProxyFactoryBean is Weaver class given by Spring framework, which created a Proxy class & returns an object of the Proxy class..

→ with ~~Proxy fact~~ ProxyFactoryBean class, we need to configure following 3 Properties.

① ProxyInterfaces → Interface name.

② InterceptorNames → id's of advisor

③ target → id of target class.

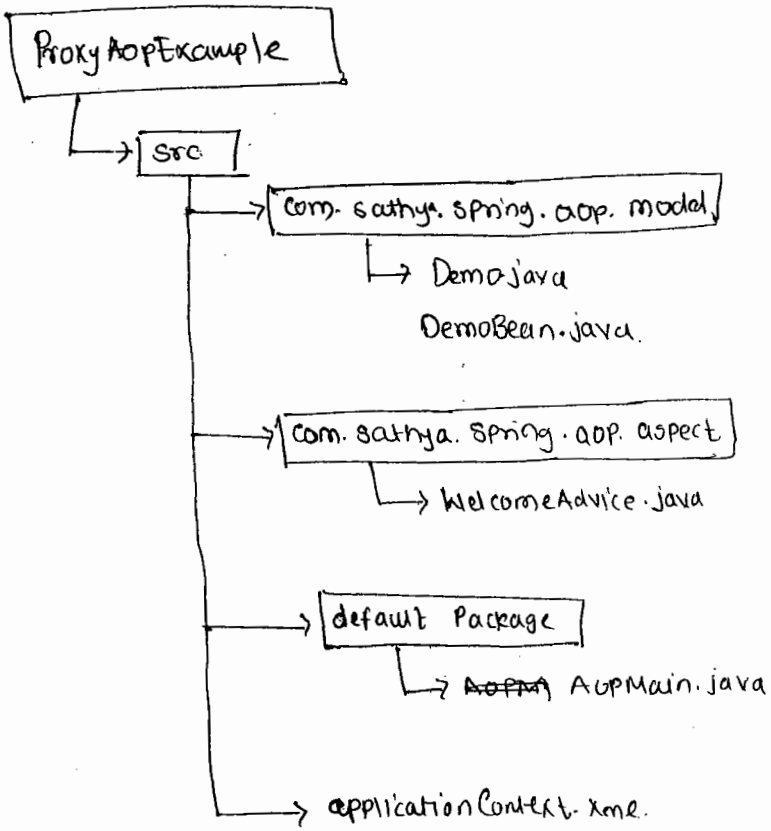
```
<bean id = "pfb" class = "ProxyFactoryBean">
    <p name = "proxyInterfaces" value = "Demo"/>
    <p name = "interceptorNames">
    <list>
        <value> advisor1 </value>
        <value> advisor2 </value>
    </list>
```

</bean>

20-3-2015

Q. What is factoryBean ?

→ or Produces object of another Bean class



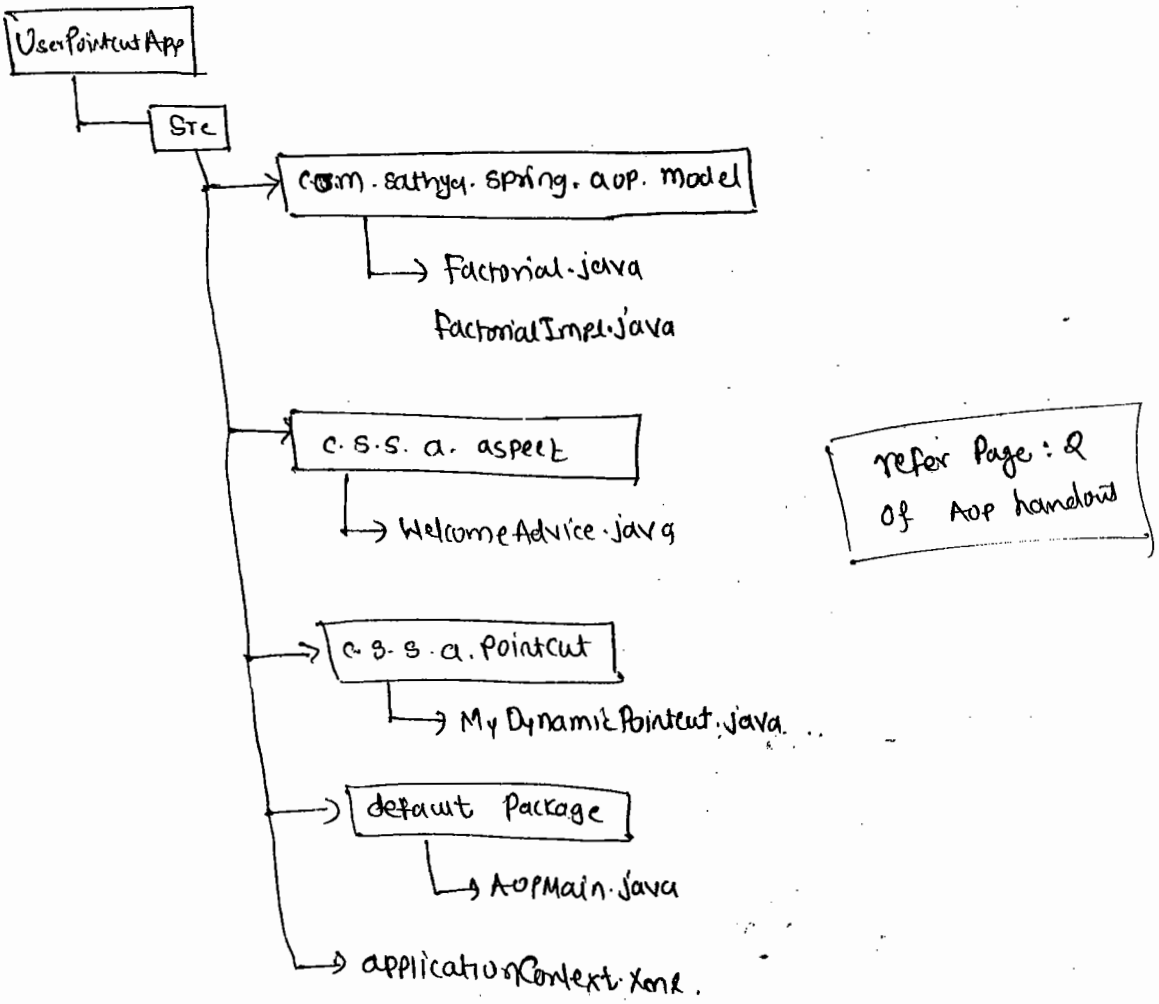
refer Page 1 of AOP Handout

Jar files to build Path

- commons-logging
- spring-expression-4.0.5.RELEASE
- spring-aop-4.0.5.RELEASE
- spring.beans
- spring-context
- spring-context-support
- spring-core
- aopalliance-1.0

Example on User defined pointcut

→ In the below example, we are creating and user defined Pointcut it makes a method as Pointcut if its parameter value greater than 0



21-3-2013

① Proxy-style [1.x]
② xml-based [2.0]
③ annotation-based [2.5]

② * XML-Based AOP *

→ the following are the Problems identified in Proxy-style AOP

① In Proxy-style, to create an advice, our class must implement a Predefined framework given interface. so application is Tightly coupled with framework.

② By creating multiple services, we need to create multiple classes. So the number of classes are increased.

③ All advice classes are configured in xml file. so the number of objects are increased.

→ To overcome the issues raised with the Proxy style AOP, we got xml-Based AOP in Spring 2.0.

→ In xml-Based AOP we can defined all advices in one class, so the number of classes are reduced.

→ a class used to defined advices is a POJO class, so it is not tightly coupled with framework

→ Because of one advice class, the no of objects are decreased.

Creating Advices in XML-Based AOP

→ In xml based AOP a new type of advice is added called after or finally advice

→ after returning advice will be executed if business method is executed or completed without exceptions, similarly a throws advice is executed, when a business method raises with an exception.

→ after advice is executed when a business method is completed without exception or with exception it means this after advice is like a finally block

→ In xml-based AOP we have totally 5 types of advices.

1. befor advice.
2. after-returning advice.
3. after-throwing / throws advice
4. after / finally advice.
5. around advice.

(1, 2, 3, 4, 5) *

JoinPoint → interface

↑ extends

ProceedingJoinPoint → interface

(1)

→ When defining advices, if Business method name and Business method arguments are needed to define the service logic then they are accessible through JoinPoint Parament.

→ JoinPoint is an Interface of AspectJ language

→ In xml Based & Annotation Based AOP our spring framework takes the support of AspectJ language.

```
Public class MyAspect
{
   Public void loggingBefore ( JoinPoint jp)
   {
      Method m = jp.getSignature(); → return method object
      String str = m.getName();
      Object[] args = jp.getArgs();
      ----
      ----
   }
}
```

23-3-2015

(2)

→ when we are creating afterReturning advice, if we want to access a business method return value then we need to take a Parameter of type "Object" in the advice method.

```
Public class MyAspect
{
   Public void loggingAfterReturning(JoinPoint jp, Object o)
   {
      =
   }
}
```

↓
it stores return value of a business method.

(3) → When we are creating "after-throws" advice, if we want access the Exception details occured / raised in business method then we need to take Exception or Throwable as parameter to the throwsAdvice() method.

```
Public class MyAspect
{
    Public void loggingAfterThrows (JoinPoint jp, Exception e)
    {
        ==
    }
}
```

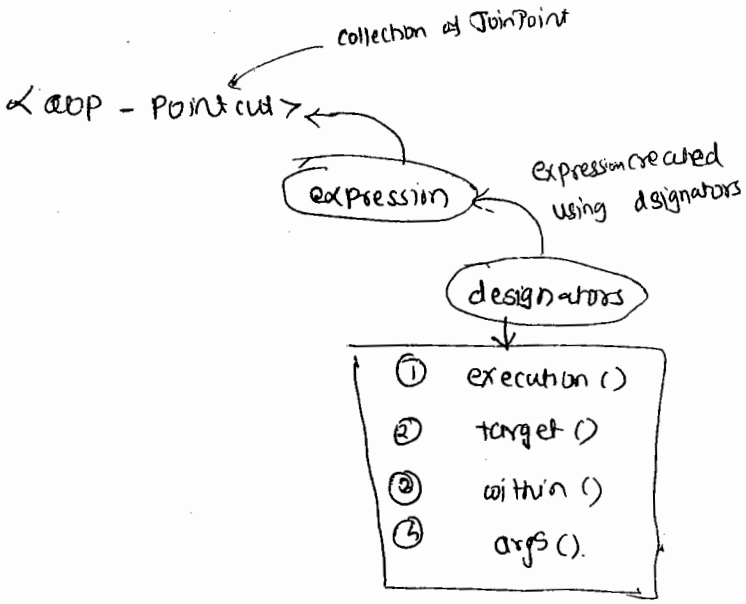
(4) If we want to define a "finally-advice" then it can only access the method name and arguments of the business method, but it cannot access return value of a business method.

```
Public class MyAspect
{
    Public void loggingAfter (JoinPoint jp)
    {
        ==
    }
}
```

(5)

→ When defining a "around" advice, the method must take ~~Proceeding~~ ProceedingJoinPoint object as a Parameter, because in the middle we need to called Proceed() method in order to called the business method.

```
Public class MyAspect
{
    Public Object around (ProceedingJoinPoint PJP)
    {
        =
        =
        Object o = PJP.Proceed();
        =
        =
    }
}
```



advisor = Pointcut + advice

JoinPoint

Configuring Pointcuts :

→ Pointcuts identifies JoinPoints based on condition

→ In an application there are multiple classes and each class can have multiple methods. but all classes and the all methods does not required advices. so a Pointcut identifies JoinPoints methods based on condition.

→ in xml based aop pointcuts are expressions, they are constructed using designators and configure using <aop:pointcut> tag.

```
<aop: Pointcut id = "pt1"
        expression = "pointcut expression" />
```

→ Pointcut expressions are constructed using the following 4 designators

①	execution
②	target
③	within
④	args.

4 designators.

- Pointcut identifies JoinPoint
- JoinPoint is method
- methods declared in expression

① execution () designator :

→ ~~base~~ this designators is used for passing a condition to identify the JoinPoints, as an expression.

→ a condition expression follows the below syntax.

```
[modifier]  returntype  method-name-pattern ( Params - Pattern)
```

optional

Example 1

<aop:Pointcut id = "pt1"
expression = "execution (Public double com.sathya.Spring.Account.*(..))" />

Account → may be Interface or class.

→ According to the above expression methods of account with return type double, with Public modifier and having any Parameters are identified as JoinPoints.

Example 2

<aop:Pointcut id = "pt1"
expression = "execution (* com.sathya.spring.*.*(..))" />

(com.sathya.spring → Package, * → class, * → methods, (..) → any no. of Parameters)

→ ~~below~~ according to the above condition or expression, all the methods of all the classes of com.sathy.spring package are identified as JoinPoint

Example 3

<aop:Pointcut id = "pt1"
expression = "execution (int com.sathya.spring..*.*(int,..))" />

(int → Return type)

→ according to the above condition the methods of classes of com.sathya.spring and its subpackages, return type int and first Parameter is int ~~and~~ are identified as JoinPoints.

Example 4

```
<aop: Pointcut  id = "Pt1"
    expression = " execution ( *  * (..)) " />
```

⇒ according to the above condition, all the methods of of an application are identified as JoinPoints

24-3-2015

(2) Within designator

— this designator is to pass a package name or class name and the methods identified by execution designator will become JoinPoint, if they are called from either a given class or given package.

ex.1

```
<aop : Pointcut  id = " Pt1"
    expression = " execution (* Pack1.A.* (..)) &&
                 within (Pack2.B) " />
```

→ according to the above pointcut configuration methods of Pack1.A will become JoinPoints, if they are called from methods of Pack2.B

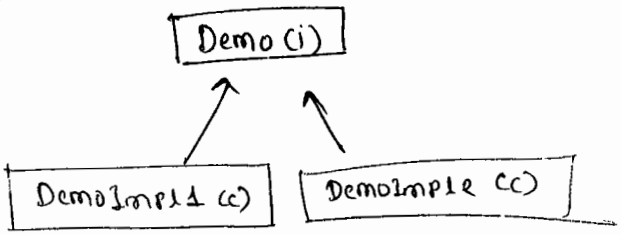
Ex.2

```
<aop: Pointcut  id = "Pt1"
     expression = "execution ( * Pack1.A.* (..)) &&
                   within ( com. sathya.spring. trading)" />
```

→ according to the Pointcut configuration methods of Pack1.A will become Joinpoint, if they are called from the classes of com. sathya. spring. trading Package.

③ Target

POJO | POJO model



→ these designator is used to make methods of an interface as Joinpoint, if target Object to be converted to a proxy. belongs to a given target class type.

Ex 1

```
<aop : Pointcut  id = "Pt1"
     expression = "execution ( * Pack1. Demo. * (. .))
                   && target (Pack1. DemoImpl2)" />
```

→ according to the above pointcut configuration methods of Demo Interface will become Joinpoint, if target object is an Object of DemoImpl2 class.

④ args

```
package Pack1
Public class A
{
    p v m1 (int a, double d)
    { }
    p v m2 (int x, int y, int z)
    { }
}
```

every primitive, collection, & wrapper ~~are~~ all are Serializable.

~~<aop~~

→ this designator makes methods as join Points if they arguments (Parameter) are of a given type

ex:

```
<aop: Pointcut id = "pt1"
    expression = " execution (* Pack1.*.* (..))
                 && args (java.io.Serializable)" />
```

→ according to the above Pointcut configuration, the methods of classes of Pack1 are ~~are~~ going to become Joinpoint if ~~they~~ there argument are of type Serializable.

→ In Java bydefault all primitive type, wrapper type & collection types are Serializable.

→ if we want to make an user defined type as Serializable then we need to implement our class from java.io.Serializable interface.

Configuring Advisor

→ In xml based aop, advisors are tags but not classes

→ for five types of advises, 5 advisors tags are given to combine to ~~advice~~ advise with pointcut.

① <aop : before >
② <aop : after - returning >
③ <aop : after - throwing >
④ <aop : after >
⑤ <aop : around >

ex.1

```
<aop : before  pointcut-ref = "pt1"
               method = "loggingBefore" />
```

→ this advisor tag combines before advice to the pointcut.

ex.2

```
<aop: after-returning pointcut-ref = "pt1"
                      method = "loggingAfterReturning"
                      returning = "o" />
```

↳ Parameter to store the return value of a Business method

ex.3 `<aop: after-throwing pointcut-ref = "pt1"`
`method = "loggingAfterThrowing"`
`throwing = "e" />`
↳ Parameter to store the exception occured in Business method.

ex.4 `<aop: after pointcut-ref = "pt1"`
`method = "loggingAfter" />`

ex.5 `<aop: around pointcut-ref = "pt1"`
`method = "loggingAround" />`

ProxyFactoryBean.

`<aop: aspectj-autoProxy />` ← xml file.

Configuring a Weaver

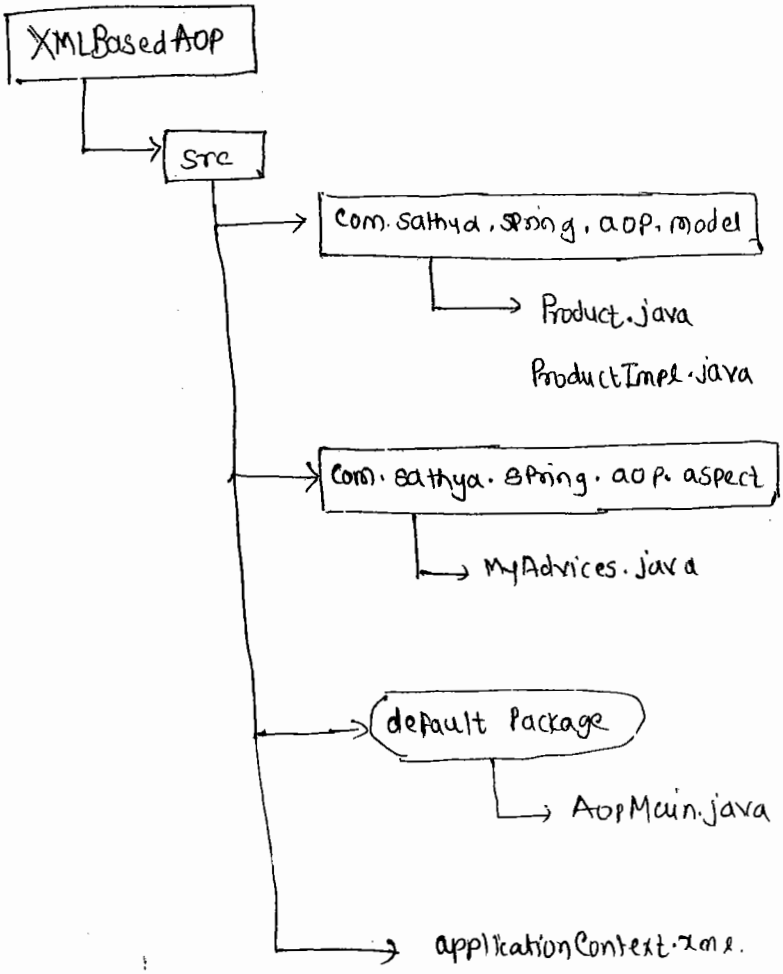
→ In xml base AOP spring framework takes a support of ~~aspect~~ aspectj. So it calls weaver class of aspectj.

→ In order to call the Weaver class of AspectJ, we need to configure the below tage in xml file

`<aop: aspectj-autoProxy />`

→ at run time, Spring Container calls a weaver class aspectj ~~autoProxy create a~~ AspectjAutoProxyCreator to convert a target object into a Proxy object.

25-3-2015



refer. Page 4 of AOP Handout

Jar file

1. commons-logging
2. spring-expression
3. spring-aop
4. spring-beans
5. spring-context
6. spring-context-support
7. spring-core
8. aopalliance-1.0
9. aspectjweaver-1.6.2
10. aspectjrt-1.6.9.jar

} newly added

Annotation Based - AOP

→ annotation are given to cut down the AOP configuration .xml file.

→ Annotation of AOP are given by AspectJ programming language.

→ at run time Spring container will convert the annotation into AOP configuration

→ there are 7 annotations to configure the AOP

1. @Aspect
2. @Pointcut
3. @Before
4. @AfterReturning
5. @AfterThrowing
6. @After
7. @Around

(3 to 7) advisors

→ to tell the Spring container that, read the information to configure AOP from a class, on top of that class we add @Aspect annotation

→ @Pointcut annotation is to add a Pointcut expression to empty method, a Pointcut can be reused by calling that empty method.

→ if you do not want to reuse a Pointcut expression then we no need to create empty method

→ the remaining 5 annotation are advisors

ex: 1

@Aspect

```
Public class MyAspect
{
    @PointCut (" execution (* com.sathya.spring.aop.model.*.*(..))")
    Public void ok () { } → (empty | Pointcut method)

    @Before ("ok()")
    Public void loggingBefore (JoinPoint jp)
    {
        ----
    }

    @Around ("ok()")
    ~~Public Object logging (Proce~~
    Public Object loggingAround (ProceedingJoinPoint p)
    {
        ----
        Object o = PJP.Proceed();
        ----
        return o;
    }
}
```

ex:2

```
@Aspect
Public class MyAspect
{
  @Before("execution(* com.sathya.spring.aop.model.*.*(..))")
  Public void loggingBefore(JoinPoint jp)
    {
      ---
    }

  @Around("execution(* com.sathya.spring.aop.model.*.*(..))")
  Public Object loggingAround(ProceedingJoinPoint p)
   {
     ---
     Object o = p.proceed();
     ---
    return o;
  }
```

diff. b/t^n ex.1 & ex.2

→ is in example 1 Pointcut method is created and it is called for 2 times. but in example 2 Pointcut method is not created so expression is repeated for 2 times.

29-3-2015

@Before ("OR()")
↳ Pointcut method.

~~@After~~

@Before	→	Value
@AfterReturning	→	Pointcut, returning, Value
@AfterThrowing	→	Pointcut, throwing, Value
@After	→	Value
@Around	→	Value.

(1) → In @AfterReturning annotations, we can pass the following 3 as elements as Parameter.

1. Pointcut
2. returning
3. value.

→ if you want to store return value of a business method in a method Parameter of an advice then we need to pass Pointcut, returning elements to the @AfterReturning annotation.

→ If we do not want to store return value of business method in a advice method Parameter then we directly pass value element.

Ex 1

```
@Aspect
Public class MyAspect
{
  @Pointcut ("execution (* com.sathya.spring.model.*.*(..))")
  Public void test(){}
```

@AfterReturning (pointcut = "test()", returning = "o")

Public void loggingAfterReturning (JoinPoint jp, Object o)

{

}

}

② → In @AfterThrowing annotation we can pass the following 3 elements as Parameter.

@) 1) Pointcut 2) throwing 3) value

→ If we want to store exception of business method in advice method parameter, then we need to pass pointcut and throwing elements. If you do not want exception of business method in a advice then we can directly pass value

ex.1

@Aspect

Public class MyAspect

{

@Pointcut ("execution (* c.s.s.a.m.*.*(..))")

Public void test (){}

@AfterThrowing (pointcut = "test()", throwing = "e")

Public void loggingAfterThrowing (JoinPoint jp, Exception e)

{

}

}

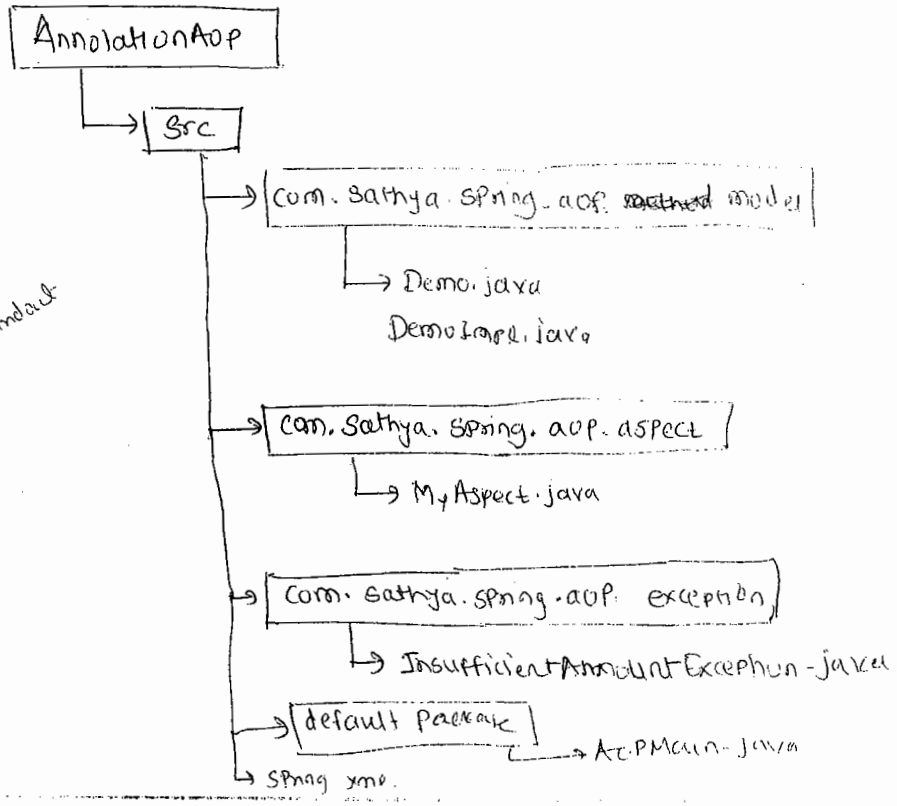
③ → @After & @Around annotation contents only a single element called value.

Q. How do you find the your java program takes how much time to execute program.

```
class A
  {  Main
     {
       long x = System.currentTimeMillis();
       ---
       ---
       ---
       long y = System.currentTimeMillis();
       long z = y - x;
       sop(z);
     }
}
```

this called timer service.

Example 4

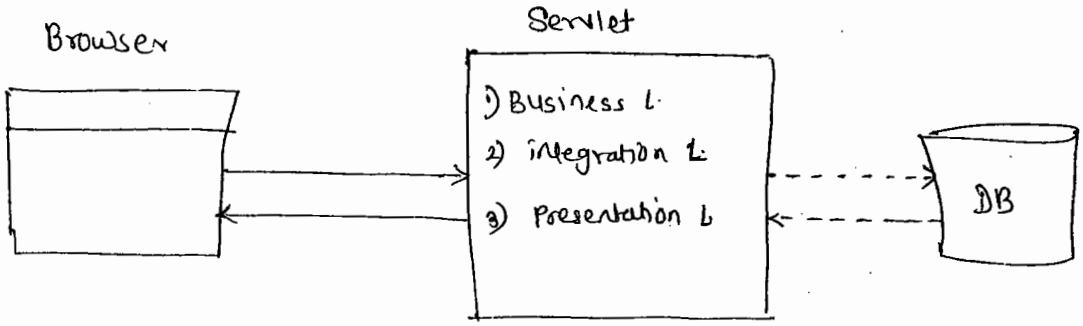


Refer Page. 5 of AOP Handout

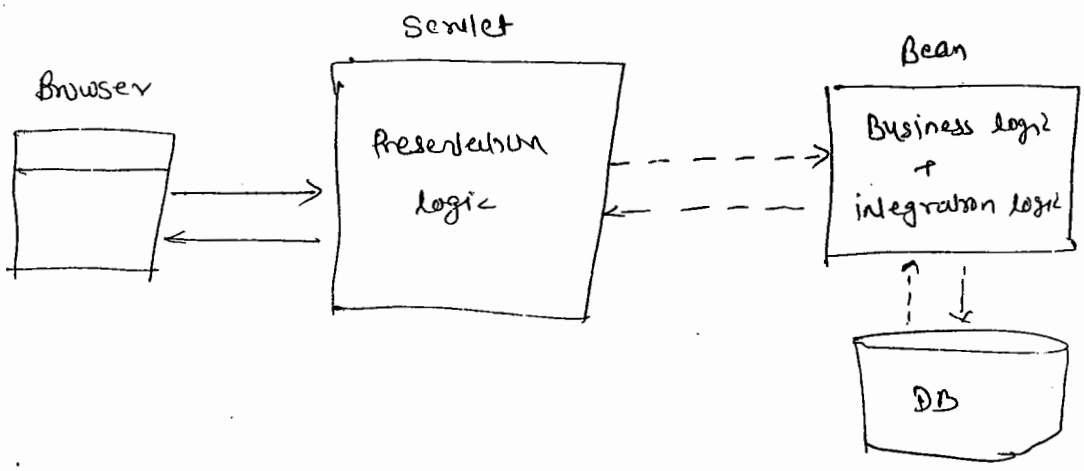
Spring MVC

28-3-2015

→ In Java, web application development started with Servlet technology. In early days of web application development we used to servlet to defined business logic, integration logic & Presentation logic in a servlet only.



→ To reduce the burdon on a servlet business & integration logics are separated from servlet to a Java bean & only presentation logic is developed in servlet.



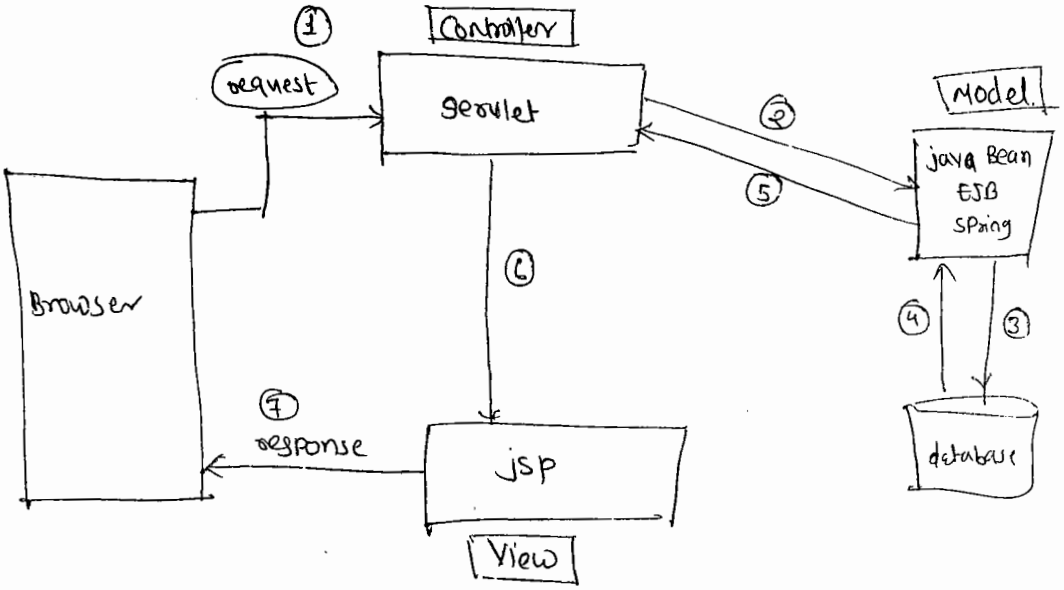
→ The web application which are developed with Servlet or Servlet with Bean are called Model-1 application

→ the applications developed in model-1 are going to have a tightly coupling between presentation logic & Business logic.

→ In order to provide loose coupling between business logic and Presentation logic, we got Model-2.

→ In Java model-2 is introduced with JSP it means model-2 applications are developed by using Servlet, Bean, & jsp component.

→ a bean contains business & Integration logics, a View (jsp) contains Presentation logic & both are controlled by Servlet. this model-2 is also MVC model.



→ According to the MVC model, the control flow logic for all the model & view components will be defined in a servlet

→ defining all control flow logic in a Servlet has increased the burden on Servlet, so some helper classes are created to manage the control flow by a Servlet it is called MVC-2 model.

→ In Java we have only MVC & MVC-2, but there is no MVC-1

→ a difference bet^n MVC & MVC-2 is, in MVC control flow define in a servlet & in MVC-2 control flow of Servlet will divided to its helper class.

Q → why Servlet is a Controller in MVC?

A → We can take a jsp as controller because jsp is aware of http Protocol but for control flow large Amount of Java Code is need to be return in a jsp. according to the Industry Standard, we can't write large amount of code in jsp so jsp is not a suitable for controller.

→ In a Bean we can write large amount of java code but a bean is not aware of http Protocol, so bean is not suitable for controller, so Servlet is used as controller in MVC.

Q why Jsp is a view in MVC?

Ans We can use Servlet as view by defining presentation logic. but if any changes are needed in presentation then we need to recompile a servlet reload application and restart the server. so Servlet is not suitable for view.

→ We can not define presentation logic in java bean because, Java bean is unknown about http protocol. so a bean is not suitable for a view.

→ so, JSP is used as view in MVC.

Q Why Bean is a model in MVC?

→ we can write business logic in a servlet but a servlet container will not offer (or provide) multiple services required for B-logic. So servlet is not suitable as model.

→ In a JSP we cannot write large amounts of the business logic so JSP is not suitable as model. so a bean can have B-logic of MVC.

30-3-2015

Q. Explain flow of MVC? Imp

DispatcherServlet

Adv. Java.

Servlet Receive request & control the flow

MVC.

only receive request control by other helper class or handler class

Controller Responsibility

↓

to find appropriate handler to process the request

Browser

↓

Request

↓

DispatcherServlet ← receive the request & find appropriate

↓

~~AnnotationMapping~~ HandlerMapping ← handle the request based on one

→ DispatcherServlet

→ HandlerMapping

→ ControllerBean

→ ViewResolver

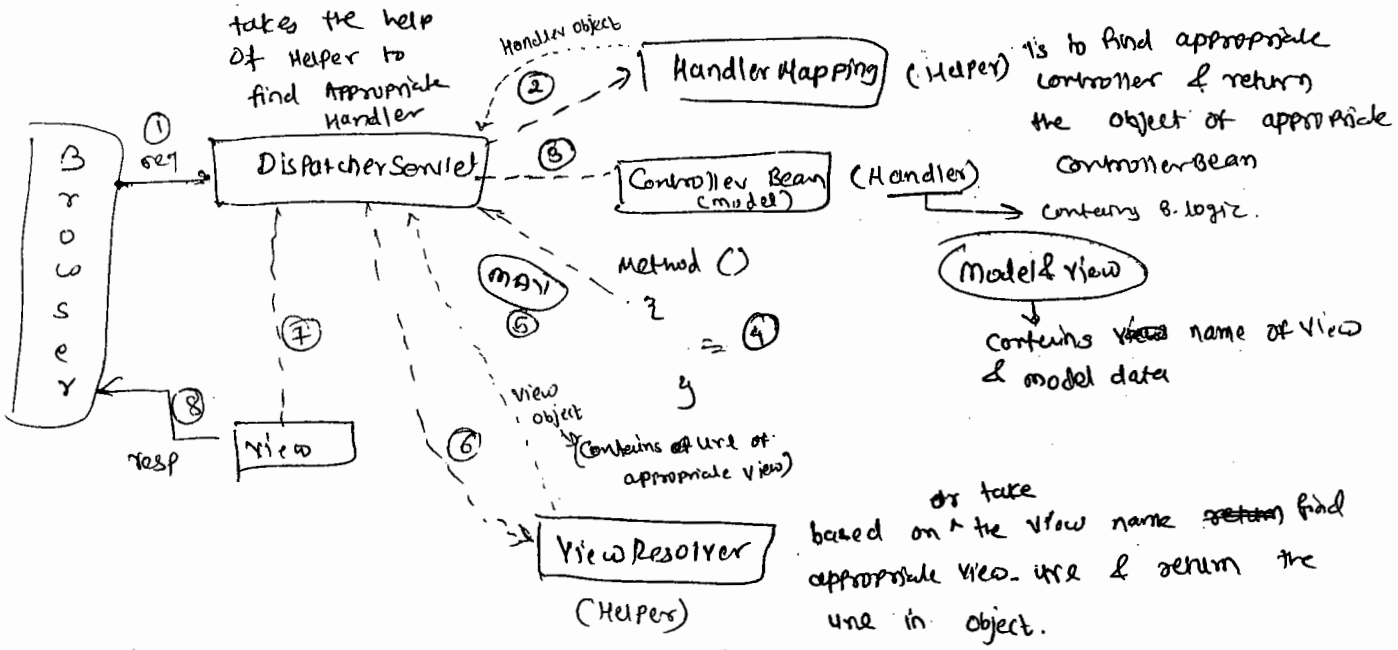
→ View

} Helper classes to DispatcherServlet

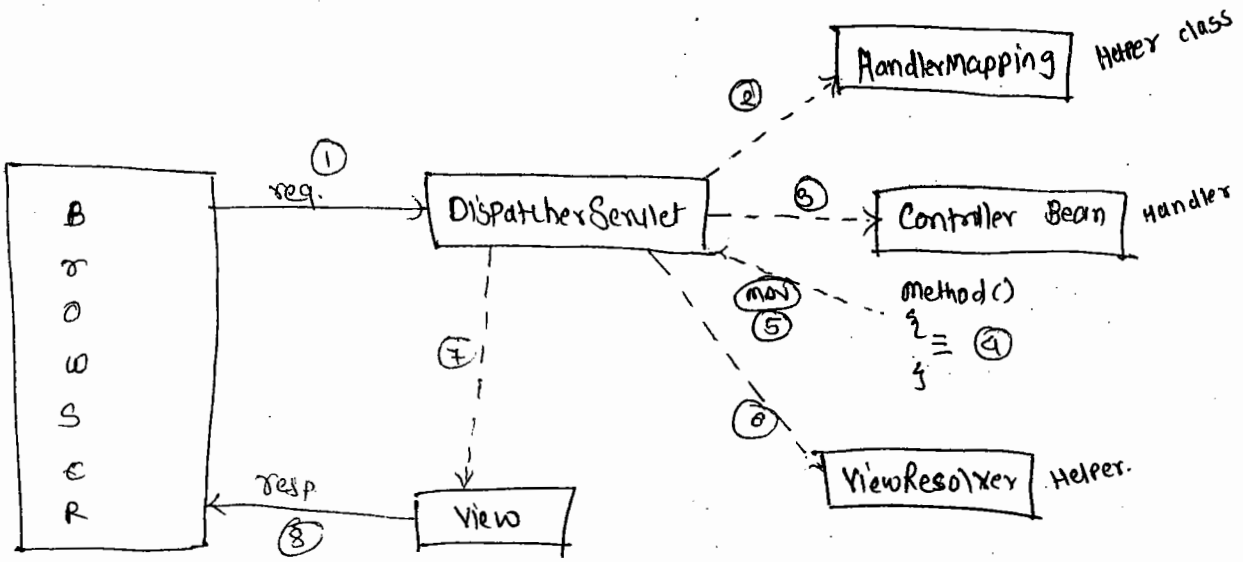
ControllerBean → Handler

HandlerMapping ----------> based on URL Pattern of the request

request ——————> Handler



Spring-MVC flow



Step

① a first step for a request in spring-mvc is at the ~~Reque~~ DispatcherServlet. It is the FrontController of the spring-mvc application

② DispatcherServlet calls its helper class called HandlerMapping. this ~~Contro~~ HandlerMapping components maps a request to a Controller Bean (request handler) based on url pattern of the request.

③ finally HandlerMapping returns an Object of Controller Bean to the DispatcherServlet.

③ & ④ DispatcherServlet calls method of Controller Bean and that method will process the given request

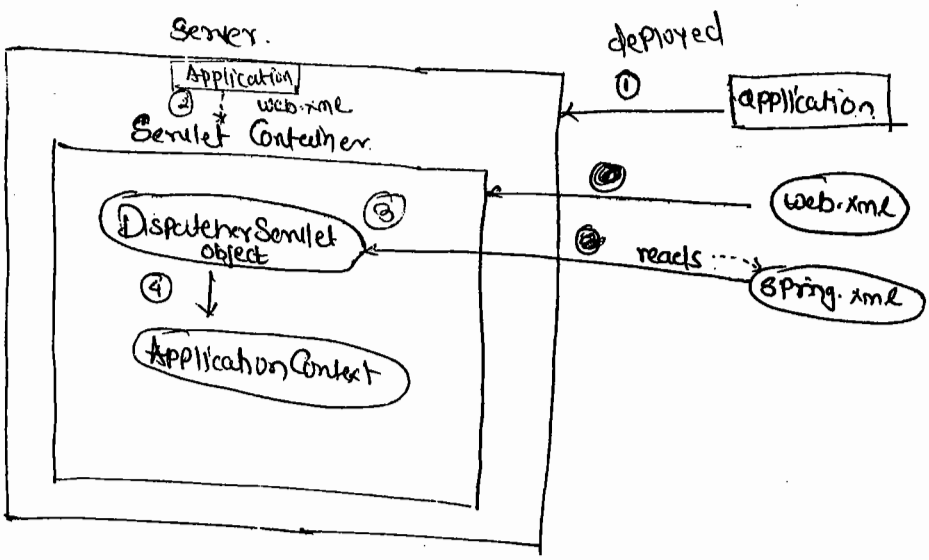
⑤ a Controller Bean returns result in the form ~~ModelView~~ ModelAndView Object. this object contains only view name or view name with model data.

⑥ DispatcherServlet calls another helper class of it called ViewResolver. this ViewResolver will identify the appropriate view based on the view name and returns url of that view in the form of View object.

⑦ DispatcherServlet will forward a request to the view.

⑧ finally a view renders response to browsers.

- HandlerMapping find the appropriate controller depends on url and return the object of controllerBean to the dispatcher
- DispatcherServlet calls the methods of ControllerBean using this object and returns the view name in the form of ModelAndView object



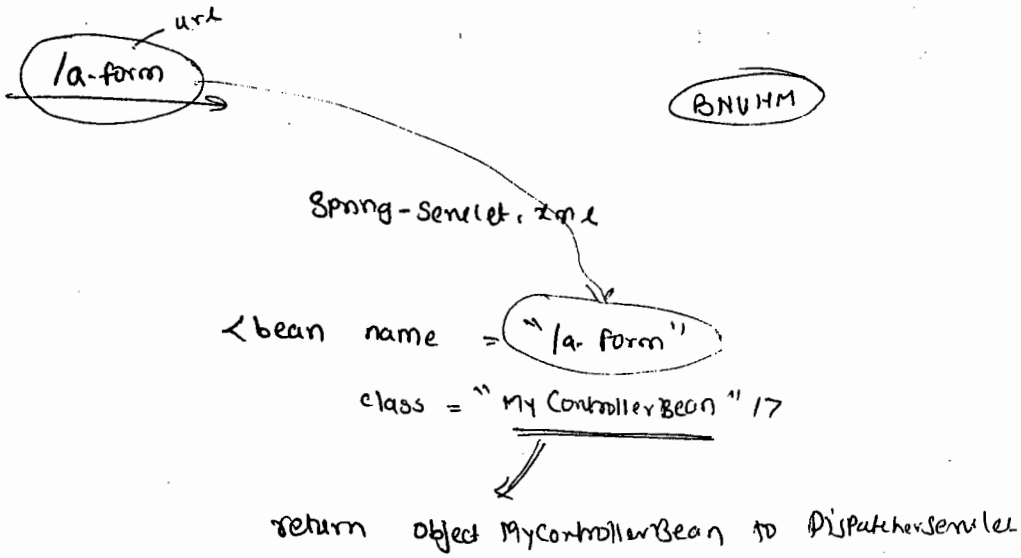
Configuring DispatcherServlet.

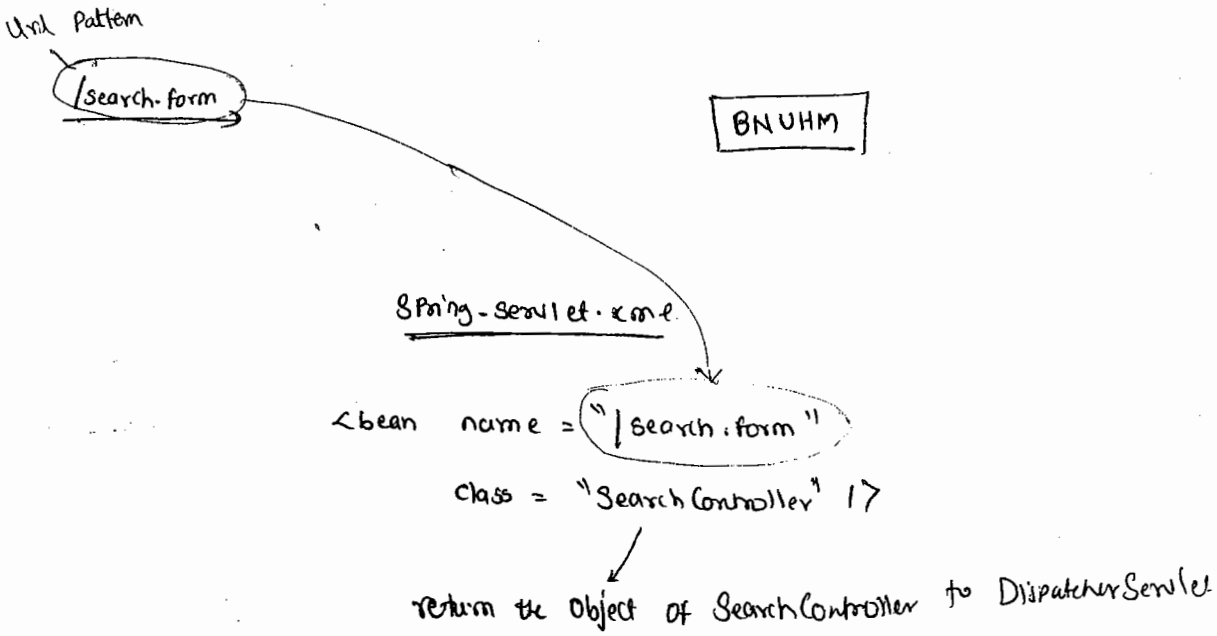
→ In Java MVC framework a framework only supplies a control flow logic in the form of a predefined Servlet class. e.g. in case of Spring framework it is DispatcherServlet.

→ DispatcherServlet is a subclass of HttpServlet, so it can only handle Http Protocol request

→ we need to configure DispatcherServlet class in a web.xml (Deployment descriptor).

→ when a Spring MVC application is deployed in a server, Servlet container will immediately created DispatcherServlet Object at the deployement time only. DispatcherServlet will immediately create a Spring ApplicationContext container Object, by loading Spring Configuration file.





HandlerMapping Component

→ a HandlerMapping class is Helper class to the DispatcherServlet, it will maps a request to the appropriate Controller Bean based on url pattern

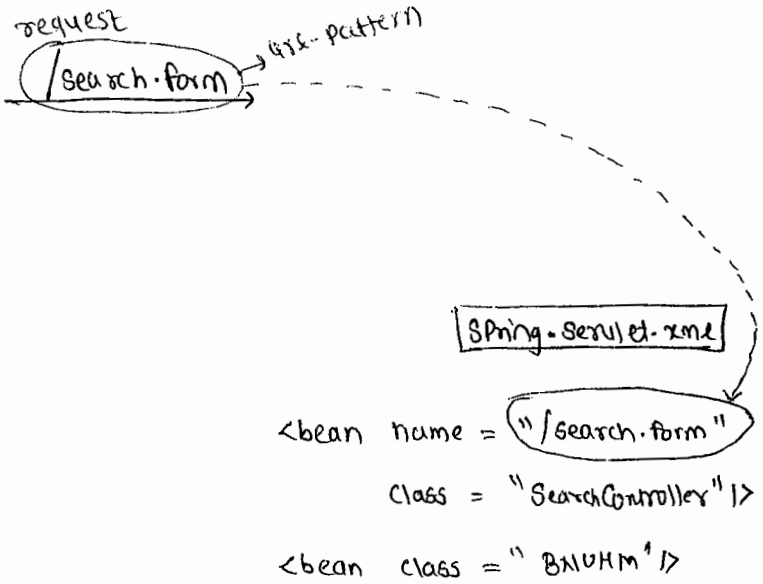
→ Predefined HandlerMapping classes given by framework are

1. BeanNameUrlHandlerMapping
2. SimpleUrlHandlerMapping
3. DefaultAnnotationHandlerMapping

→ if Predefined HandlerMapping classes are not ↓ suitable an application then a Programmer can create a custom Handler mapping class by ~~extending~~ implementing HandlerMapping interface

→ BeanNameUrlHandlerMapping will ~~find~~ Map a request to Controller bean like the following.

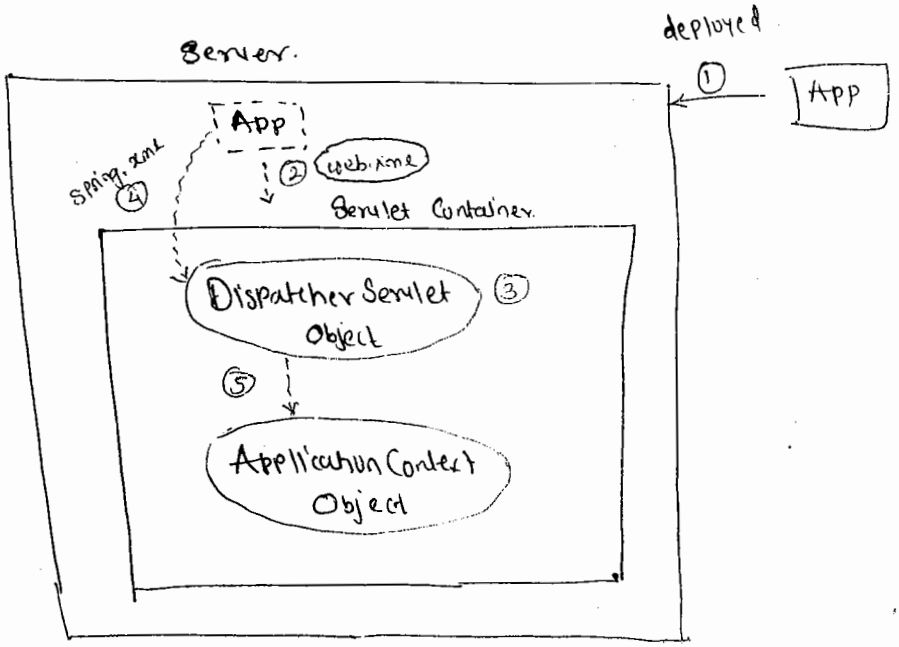
① BNUHM reads url pattern of the request
② It search a bean name in Spring container (spring.xml) ← Spring configuration file.
which is matched with url-patter of the request
③ If matched then return a object of the Controller bean class to the DispatcherServlet.



Creating a Controller bean.

✗ ① P C SearchController implements Controller.
↑
given by framework. by using this it tightly coupled with f/w class

✓ ② @Controller
P C SearchController
{ } ← Intermediate logic which calls the business logic which is ~~writter~~



31-3-2015

one request to one servlet
for one servlet one url-pattern

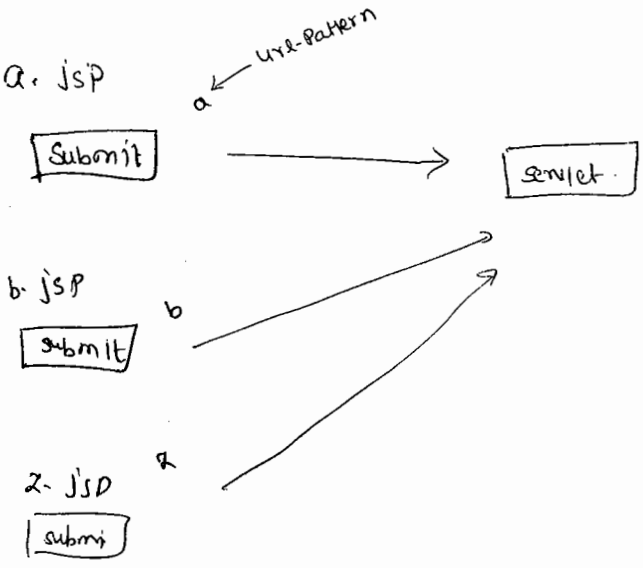
exact match
directory match
extension match
} Types of url-pattern in servlet mapping

for one servlet we can configure ~~there are~~ n numbers of url-pattern

Ex <s-m>
<s-n>
<u-p>/a</u-p>
<u-p>/b</u-p>
</s-m>

web.xml

/a
/b



→ DispatcherServlet should be configure in deployment descriptor like the following.

web.xml

```
<web-app>
    <servlet>
        <servlet-name> spring </servlet-name>
        <servlet-class> org.springframework.web.servlet.DispatcherServlet </servlet-class>
        <load-on-startup> 1 </load-on-startup>
        <init-param>
            <param-name> contextConfigLocation </param-name>
            <param-value> /WEB-INF/applicationContext.xml </param-value>
        </init-param>
    </servlet>
    <servlet-mapping>
        <servlet-name> spring </servlet-name>
        <url-pattern> *.form </url-pattern>
    </servlet-mapping>
</web-app>
```

↳ to stop each request at DispatchServlet use extension-mapping Match which is type of url-pattern

→ If spring configuration file name is given by following <servlet-name>-servlet.xml format then there is no need to configure init-parameter for DispatcherServlet in web.xml.

BeanNameUrlHandlerMapping

→ reads url-pattern of request.

~~→ Search for a~~

1-4-2015

- Controller → interface
  - AbstractController → class
  - AbstractCommandController → class
  - SimpleFormController → class
  - MultiActionController → class

Stereotype Annotation

- @Component
- @Service
- @Repository
- @Controller.

(tells the role of the class)

→ a Controller bean in a spring mvc application contains business logic for processing the req or intermediate logic for processing a request.

→ One spring-mvc Appln can have multiple Controller Bean classes.

→ We can create a Controller Bean in 2 approaches

① by implementing our class from Controller interface or by extending our class from one of the subclasses of Controller interface.

② by adding "@Controller" annotation on top of the class

1st approach

```
Public class MyController implements Controller
{
  @Override
  Public ModeAndView handleRequest (request, response)
  {
    ≡
  }
}
```

② Approach

```
@Controller
Public class MyController
{
  =
  =
}
```

ModelAndView

↓

Contains either View name or
View name + model data
↳ data in form of key value.

```
ModelAndView mav = new ModelAndView ("success");
```
↳ view name. it is logical name

```
ModelAndView mav = new ModelAndView ("success", "msg", "Hello");
```
"success" → View name
"msg", "Hello" → model data (key, value)

Key must be string
value can be anything

ModelAndView Object

→ a Controller Bean returns ModelAndView object after processing a request

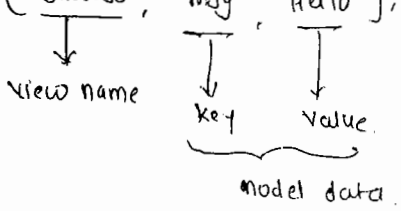
→ a ModelAndView object contains either [view name] or [view name + model data]

→ a Model data will be in key/value pair and this data will displayed on the browser by a view with some presentation.

→ we can create a ModelAndView object like the following.

① `ModelAndView mav = new ModelAndView ("success");`
↳ view name also called logical name

② `ModelAndView mav = new ModelAndView("success", "msg", "Hello");`



→ We can add a multiple key/value Pairs of model data by calling addObject() method

```
ModelAndView mav = new ModelAndView("success");
        mav.addObject("msg", "Hello");
        mav.addObject("k1", 1000);
        mav.addObject("e1", emp1);
```

Note: When adding the Model data, key must be a String and value can be any object.

View resolver bean

→ a view resolver in Spring mvc is a helper class to Dispatcher Servlet. a view resolver finds appropriate view based on view name return by controller bean and then it returns the url of the view in the form of View object.

→ Spring framework has given 4 Predefined view Resolver classes.

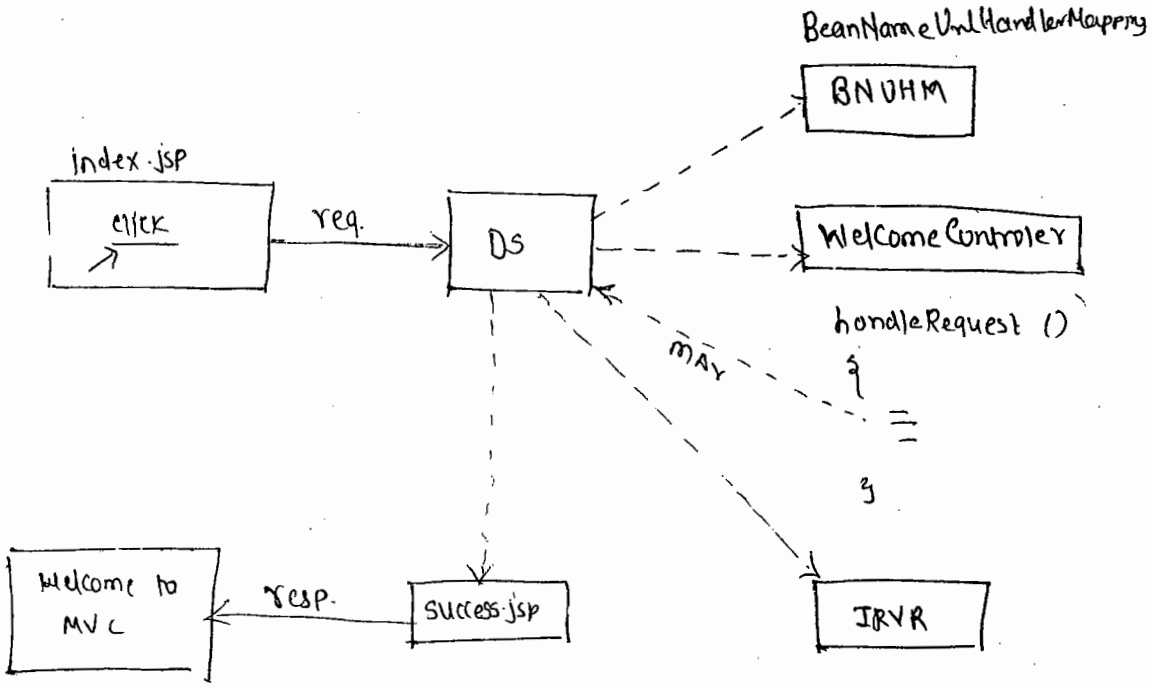
① InternalResourceViewResolver
② BeanNameViewResolver.
③ ResourceBundleViewResolver.
④ XmlViewResolver

→ if predefined view resolver classes are not suitable to a project then a programmer can create custom view resolver class by implementing ViewResolver interface.

→ InternalResourceViewResolver will find appropriate url of a view, by adding a prefix and suffix to the view name.

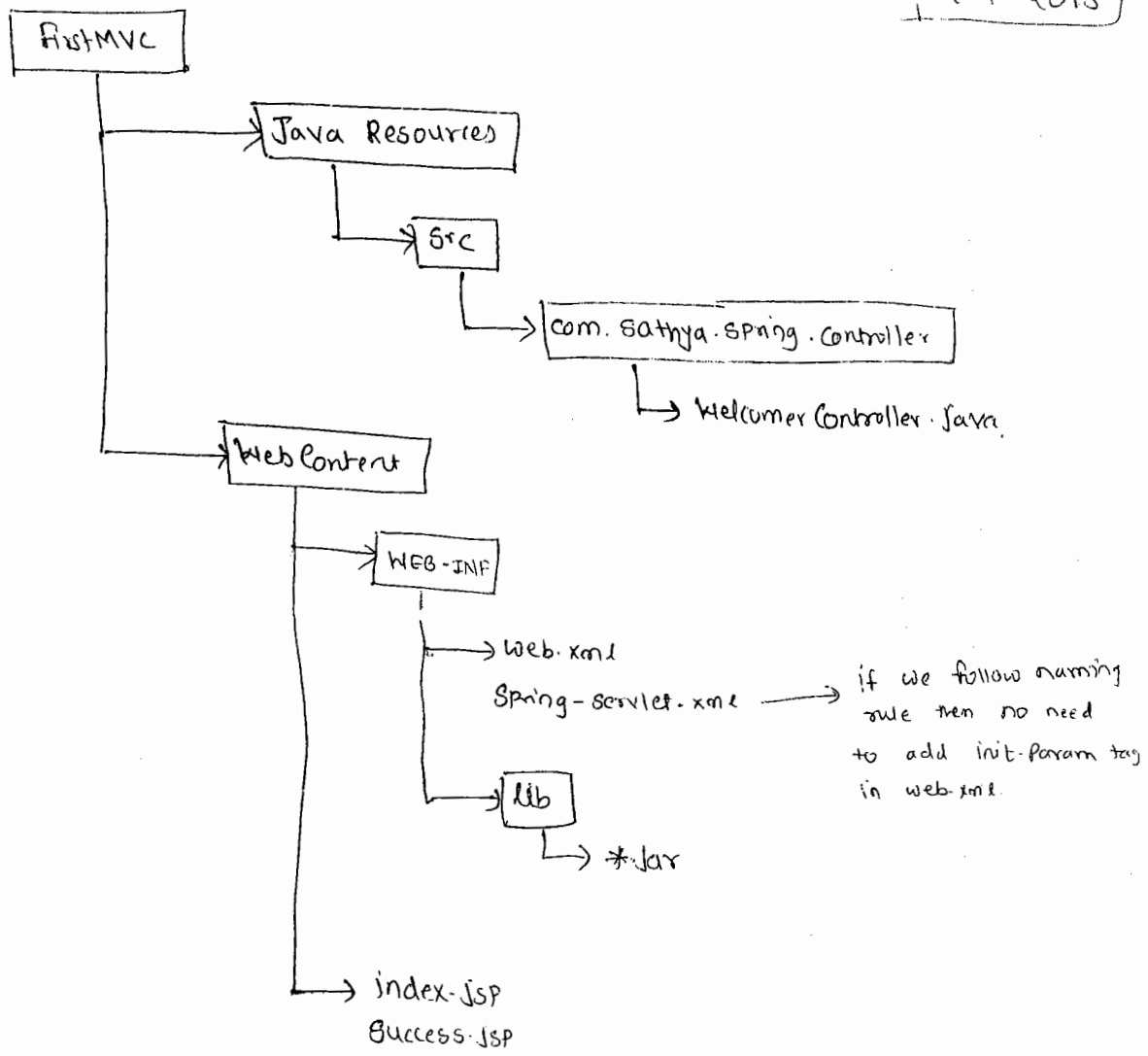
→ with InternalResourceViewResolver class we need to configure prefix and suffix properties.

```
<bean id = "resolver" class = "InternalResourceViewResolver" >
    <property name = "prefix" value = "/" />
    <property name = "suffix" value = ".jsp" />
</bean>
```



① index.jsp
② success.jsp
③ WelcomeController.java
④ DS → DispatcherServlet → web.xml
⑤ BNUHM
⑥ IRVR
} → Spring Configuration file.

2-4-2015



→ Add the following list of jar to the lib

Spring-web-4.1.2.RELEASE.jar
Spring-webmvc-4.1.2.RELEASE.jar
} newly added.

Commons-logging
Servlet-api
Spring-beans-
Spring-context.
Spring-context-support
Spring-core
Spring-expression.

index.jsp

```
<center>
    <a href = " Welcome.form " > click here </a>
</center>
```

web.xml

```
<web-app

    <display-name> FirstMvc </display_name>
    <welcome-file-list>
        <welcome-file> index.jsp </welcome-file>
    </welcome-file-list>
    <servlet>
        <servlet-name> spring </servlet-name>
        <servlet-class> org.springframework.web.servlet.DispatcherServlet
                                                        </servlet-class>
        <load-on-startup> 1 </load-on-startup>
    </servlet>
    <servlet-mapping>
        <servlet-name> spring </servlet-name>
        <url-pattern> *.form </url-pattern>
    </servlet-mapping>

</web-app>
```

// WelcomeController.java

Package com.sathya.spring.controller

```
Public class WelcomeController implements Controller
{
    return type        method Name
  Public ModelAndView handleRequest (HttpServletRequest request, HSR response)
                                        throws Exception
  {
    ModelAndView mav = new ModelAndView ("success");
    return mav;
  }
}
```

spring-servlet.xml

```
<beans>
  <bean id = "handlerMapping"
        class = "org.springframework.web.servlet.handler.BeanNameUrlHandlerMapping" />

  <bean name = "/welcome.form"
        class = 'com.sathya.spring.controller.WelcomeController' />

  <bean id = "ViewResolver"
        class = 'org.springframework.web.servlet.view.InternalResourceViewResolver' >
    <Property name = "Prefix" value = "/" />
    <Property name = "suffix" value = ".jsp" />
  </bean>
</beans>
```

success.jsp

```
<h1>
    welcome to spring MVC
</h1>
```

Creating a Controller Bean with annotations.

→ the 2 min. annotations of spring flw used in for creating a controller bean is

@Controller
@RequestMapping

→ If Controller bean is created with annotations then DispatcherServlet calls DefaultAnnotationHandlerMapping, this handler mapping class reads url pattern of a request without extension and it match ~~the value with value~~ url pattern with value of @RequestMapping annotation.

→ to create Controller bean WelcomeController with annotation for in the above example then we need to to the following changes.

① Create WelcomeController.java like the following

```
Package
import o.s.stereotype.Controller
;

@Controller
Public class WelcomeController
{
  @RequestMapping (value = "/welcome")
    Public ModelAndView test()
      {
        return new ModelAndView ("success");
      }
}
```

(or)

```
ModelAndView mav
= new ModelAndView
("success");
return mav;
```

(2) In spring-servlet.xml, configure DefaultAnnotationHandlerMapping & the enable Component scanning like the following.

```
<bean id = "handlerMapping"
    class = "org.springframework.web.servlet.mvc.annotation.
                                    DefaultAnnotationHandlerMapping"/>

<context:component-scan  base-package = "com.sathya.spring.controller"/>

<bean id = "viewResolver"
    class =
    <property
    <property
</bean>
```

(3) add spring-aop & aopalliance jar to lib folder

3-4-2015

Login Application with database Support

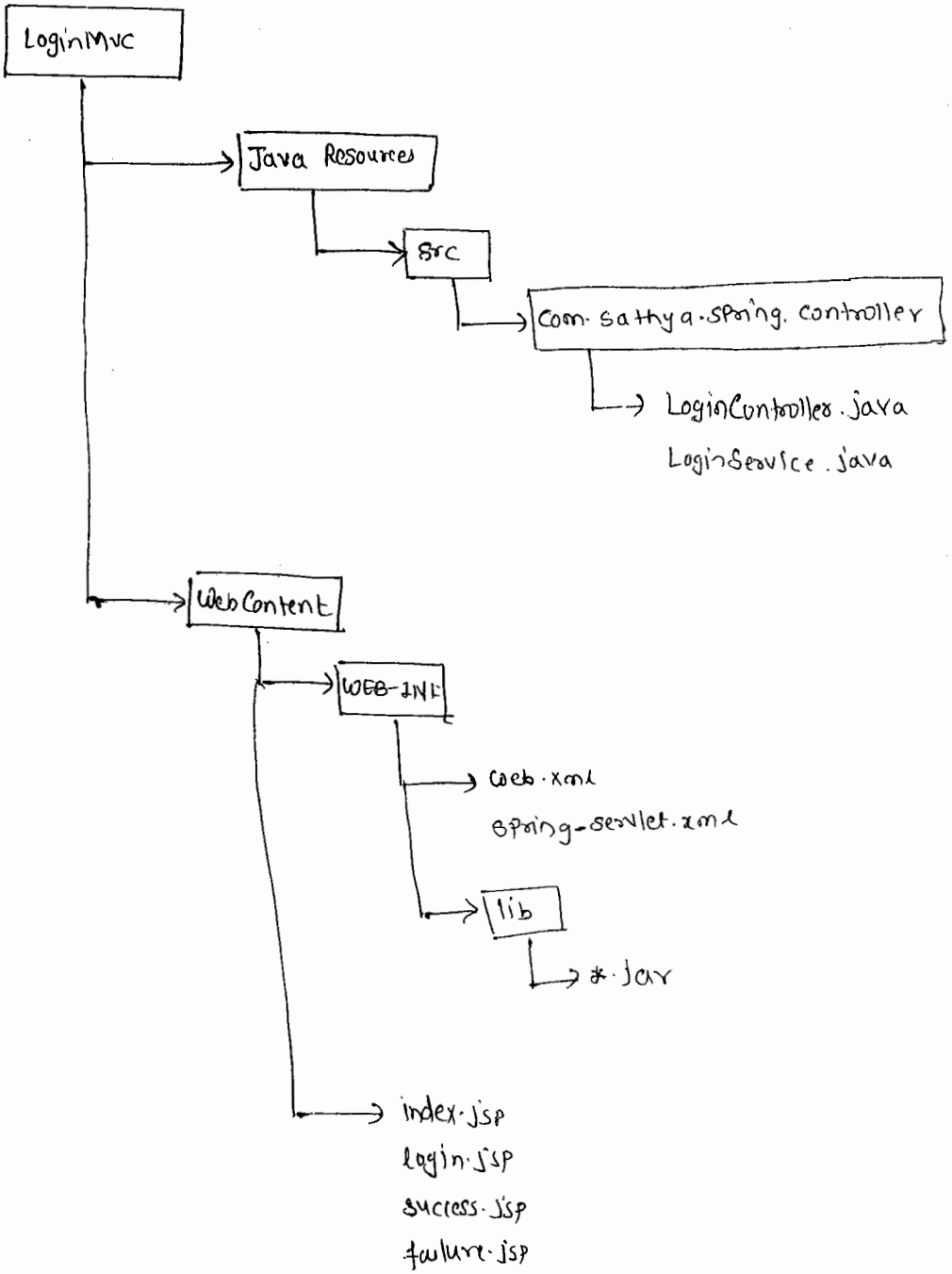
→ In the following example we are creating a login page to submit a username & password

→ the controller bean of this appl" collaborates with another bean to find wheather user name & password entered by user are valid or not.

→ In spring-jdbc layer, JdbcTemplate class is given to perform the database operations. this template class will depend on Datasource object open a Connection with database.

→ when developing spring-mvc appl" it is not a recommended way to directly send a request from browser to the jsp page, which contains form design. so we need to create an index page and the through a flow of mvc we need to get the

jsp page on to the Browser, in which we design the form.



jar files

- Ojdbc14.jar
- spring-jdbc-4.0.5.RELEASE
- spring-tx-4.0.5.RELEASE

newly added.

aopalliance-1.0.jar
commons-logging
Servlet-cupi
Spring-AOP - 4.0.5.Release
Spring-beans - 4.0.5 Release
Spring-context
Spring-context-support
Spring-Core
Spring-expression
Spring-web
Spring-web.mvc - 4.0.5.RELEASE

4-4-2015

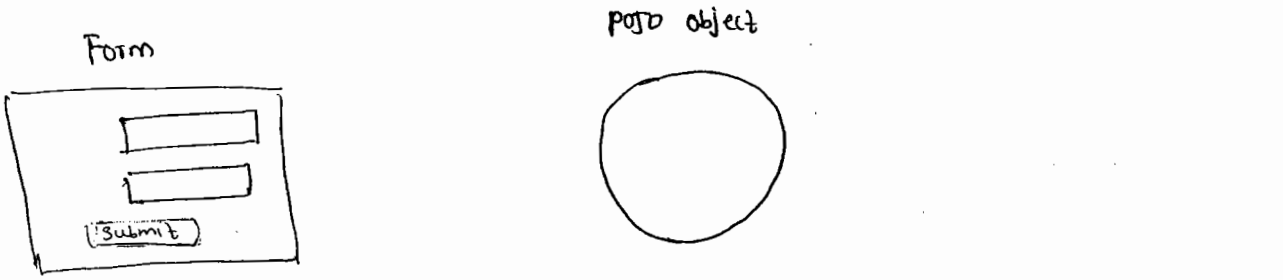
* Using a POJO or command class or form backing das:

→ In Spring MVC, we use a POJO class for storing the form data submitted by end user there are two reasons for storing a form data in a POJO class object

① To reuse a form data in multiple methods or classes of the MVC application

② To apply validations on the input values

URI → Uniform Resource Identified



Key i.e command name to connect POJO class object & form design

(view, key, value)
↑ JSP Page
↓ POJO class object

"http://www.springframework.org/tags/form"
uri

→ When we want to store the form data in a POJO class object, before sending the form to the Browser, we should put a link bet" form and the POJO class object.

→ a link can be put bet" a POJO class object & a form by making key of a POJO class object as command name

in form design.

→ Before a form is going to be send to the browser, we need to create a POJO class object and we need to set a key in controller bean.

→ If we want to do store a form data in POJO class object when it is submitted then we need to design the form using framework given form tags, but not with ordinary Html form tags.

→ Spring framework has given tag library for a group of tags to design the forms.

→ When creating a jsp page we need to import the uri of tag library using taglib directive of jsp like the following

```
<%@ taglib uri = "http://www.springframework.org/tags/form"
Prefix = "form" %>
```

↓

it can be anyname.

5-4-2015

applicable to all POJO class → JEE6 → Bean validation api → given by Sun

↓

javax.validation.annotations

Hibernate validator API (Hibernate)

↓

set of annotation parser class — contain some logic to read annotations

↳ necessary to read annotations

Apply validations on POJO

→ as Part of JAVA EE 6, Sun microsystem released bean Validation API with set of annotations. for applying validation on properties of a POJO class.

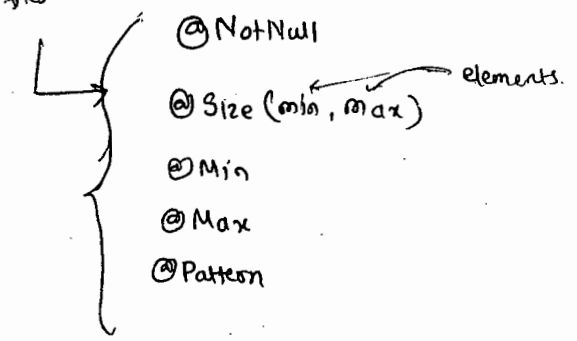
→ bean validation api contains annotations and the implementation of annotation (Parsers) are provided by vendors.

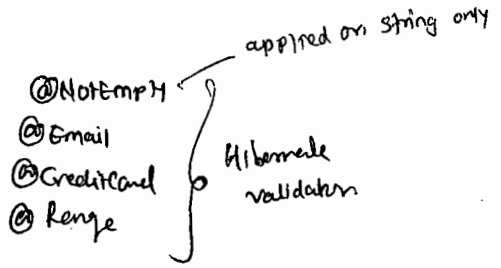
→ for ex: Hibernate has provided implementation for annotations for bean validation API. and released in the form of Hibernate validator API

→ In Spring framework, we use annotations of bean validation API with Hibernate validator implementation to apply the validations on POJO class

```
Public class Login ⇒ POJO
{
   @NotNull
    Private String uname;
                    ↳ Property
   @Size (min=5, max=8)
    Private int pwd;
                ↳ Property
   @Min (18)
   @Max (30)
    Private int age;
                ↳ Property
```

these are bean validator api






```
@NotEmpty
Private String uname;
```

Sathya.Properties.

NotEmpty.Login.uname = NotEmpty.k.t.uname = xxxxxx
↓ key of the object of POJO class
↳ error message

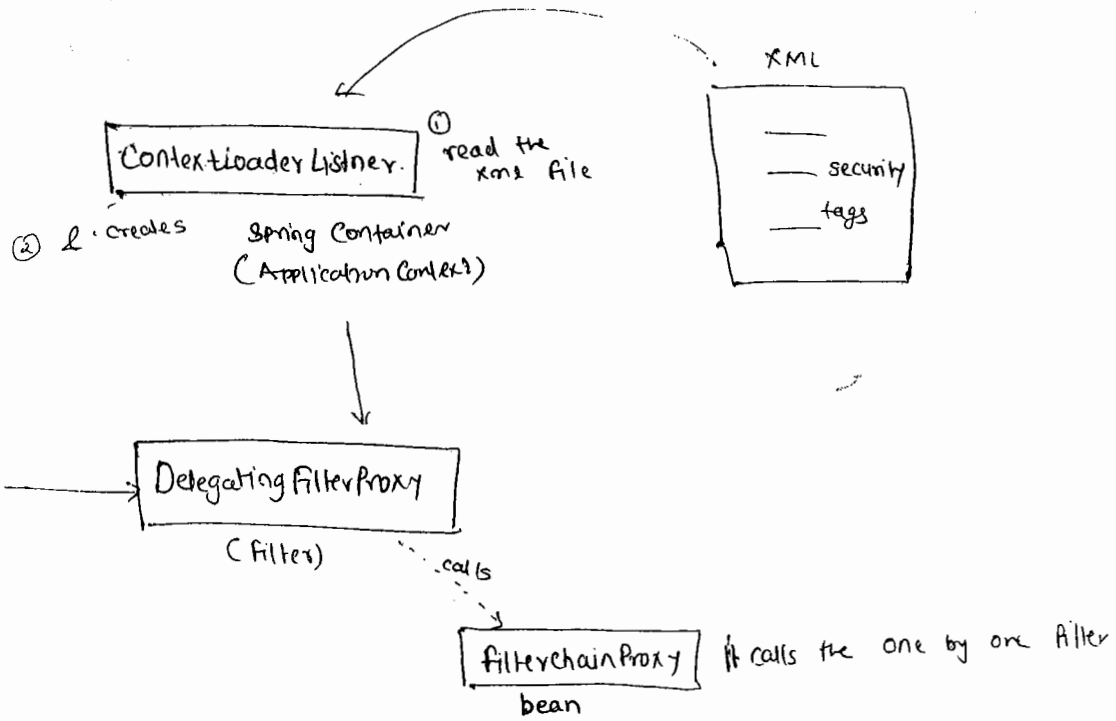
4-4-2015

Providing Security to MVC application

→ an mvc appln contains multiple resources like HTML's, JSP's, Controller beans etc.

→ In a web application security may not be required for each resource of the application, it means we make some resources as Protected.

→ In Spring framework to configure the security framework has provided a set of tags under security name space



→ In Spring framework, the authentication logic to provide security in multiple filters but configuration of multiple filters in web.xml will increase the burdon on developers.

→ In order to reduce the developer burdon, Spring f/w only has given Predefine bean class FilterChainProxy and this class will

security related filters.

→ So instead of writing all the filter configuration, we need to configure just ~~filter chain proxy~~ FilterChainProxy bean in a spring configuration file.

→ To load the security configuration and to construct the a spring container object at application deployment ~~time~~ time flw has provided a Predefined Listner class called a ContextLoaderListner. this Listner class will read the spring configuration file from Context Parameter called ContextConfigLocation.

→ When a request comes from browser, to trap the request and a execute the security filter of the spring, flw has given a filter ~~de~~ DelegatingFilterProxy and we need to configure this filter in web.xml.

→ In a web.xml file, apart from DispatcherServlet configuration, we also need to configure the following 3

① Context Parameter
② Listner.
③ Filter.

extension match → url starts with * ex → *.form

directory match → * Starts with / ex → /*

1)

```
<context-Param>
    <Param-name> contextConfigLocation </Param-name>
    <Param-value> /WEB-INF/ security-config.xml </Param-value>
<context-Param>
```

name can be anyone

(2)

```
<listner>
    <listner-class> org.springframework.web.context.ContextLoaderListner
                                                    </listner-class>
</listner>
```

(3)

```
<filter>
    <filter-name> springSecurityFilterChain </filter-name>
    <filter-class> org.springframework.web.filter.DelegatingFilterProxy
                                                    </filter-class>
</filter>
<filter-mapping>
    <filter-name> springSecurityFilterChain </filter-name>
    <url-pattern> /* </url-pattern>
</filter-mapping>
```

Q. Why we are using same name as "springSecurityFilterChain"

A → When configuring the filter name (alias name) must be springSecurityFilterChain only because when a request comes then this filter class will read its alias name and it will search in the spring ApplicationContext container object for filterChainProxy with this alias name as id

→ In spring configuration, FilterChainProxy is also be configured with the ~~spring security~~ springSecurityFilterChain as id the bean. So the link bern DelegatingFilterProxy and FilterChainProxy bean is established using this alias name only.

5-4-2015

hasRole → security expression.

↓

to use this

<security:http use-expression="true">

one Role

↓

group of username & password

<security: http: basic> → by default dialog box comes for login.

repo.spring.io | release |

↓

select org

↓

spring framework

↓

security

<security: form-login />

↳ by default gives login page

↳ also we can use user defined login page.

<security: user-service> → Inmemory authentication.

and

<security = jdbc-user-service> database authentication.

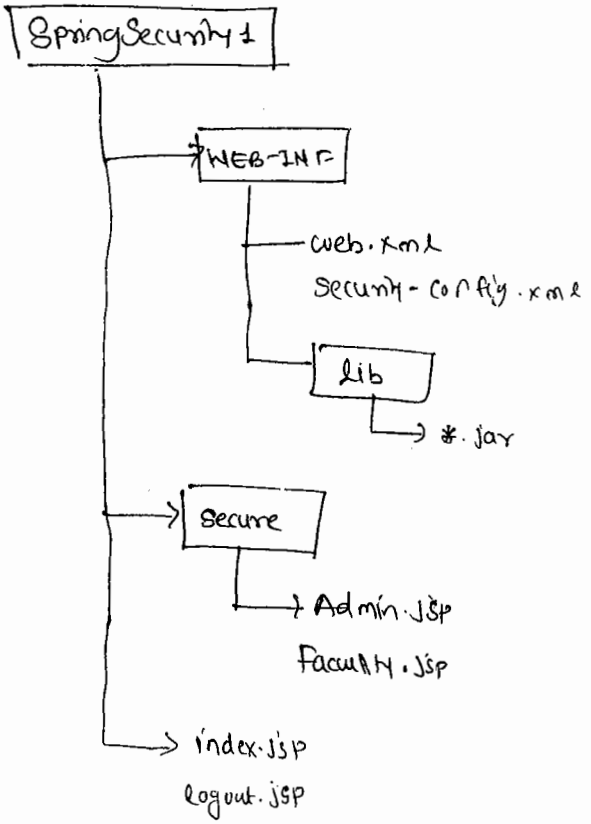
Example

→ the following example contains two secured resources admin.jsp & faculty.jsp

→ we are going to configure the admin & faculty related username & password in spring xml file only. it means we are using Inmemory authentication

→ a problem with Inmemory authentication is, if we want to add new users or if we want to delete existing users then we need to reload the application again

→ In the following ex. we are not defining controller beans with logic. so we are not configure DispatcherServlet, HandlerMapping, ViewResolver in the application.



Refer. P.N. 12 for JEE Handout

List of jar

aopalliance-1-0
common-logging
Spring-aop-4.0.5
Spring-beans-4.0.5
Spring-context-4.0.5
Spring-context-support-4.0.5
Spring-core
Spring-expression
Spring-security-config-3.2.7.RELEASE
Spring-security-core-3.2.7.RELEASE
Spring-security-web-3.2.7.RELEASE
Spring-tx-4.0.5.RELEASE
Spring-web-4.0.5.RELEASE
Spring-webmvc-4.0.5

Note

We can download spring-security-api from the below website.

http://repo.spring.io/release/org/springframework/security/spring-security/

→ download dist.zip

Example 2

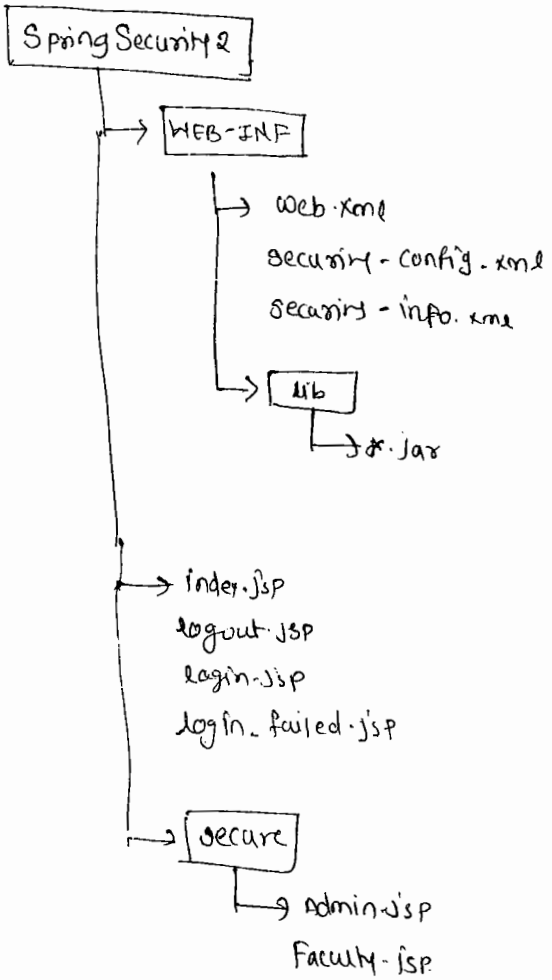
Example 2

→ this example is also for securing admin & faculty pages, but using jdbc authentication.

→ for jdbc authentication we need to create two tables in my-sql like the following.

```
create table users ( user_id int primary key, username varchar(10),
                     password varchar(10), enabled tinyint);

create table user_roles (authority varchar(10), user_id int references users(user_id));
```



Ref. P.N. 13 of JEE Handout

Note : Add 2 additional jar to lib folder.

1. mysql-connector-java.jar
2. spring-jdbc.jar

← End of MVC Security →

→ the list of annotations used for applying the validation are

1) @NotNull
2) @NotEmpty
3) @Size (min=5, max=8) → applied only on String
4) @Email
5) @Min (value=18)
6) @Max (value = 25)
7) @Pattern (regex = "^[A-Z][a-z]*") → this Pattern allows first letter as Uppercase & remaining lowercase.
8) @Range (min=10, max=20) → applicable for numeric type (int, byte, short, long, float, byte)
9) @CreditCardNumber
10) @AssertTrue } both are applicable for only boolean
11) @AssertFalse.

Example @AssertTrue
Private boolean b;

→ In Spring framework we need to defined the error messages called Validation in resource bundle.

→ in a resource bundle we need to follow the naming rule for a key to defined the error message.

annotationname. PojoObjectkey. Propertyname = error message.

for ex:
NotEmpty. k1. uname = Username is required.

→ to configure the resource bundle in a spring, we use ResourceBundleMessageSource bean in Spring configuration file.

<bean id = "messageResource" class = "ResourceBundleMessageSource">
<Property name = "basename" value = "sathya" />
↳ bundle name from (sathya. Properties)
</bean>

7-4-2015

POJO

"annotations"

Bean Validation api ← used by any fw

Hibernate Validation api

<context: annotation-config> ⇒ to enable the annotation used in java class

<form: errors /> → to display error message collecting from ResourceBundle

→ When a form is submitted then the I/p values of the form are stored in a POJO class object by DispatcherServlet when calling a Controller bean method to tell the DispatchServlet that inject the POJO class object to the method Parameter, we use @ModelAttribute annotation.

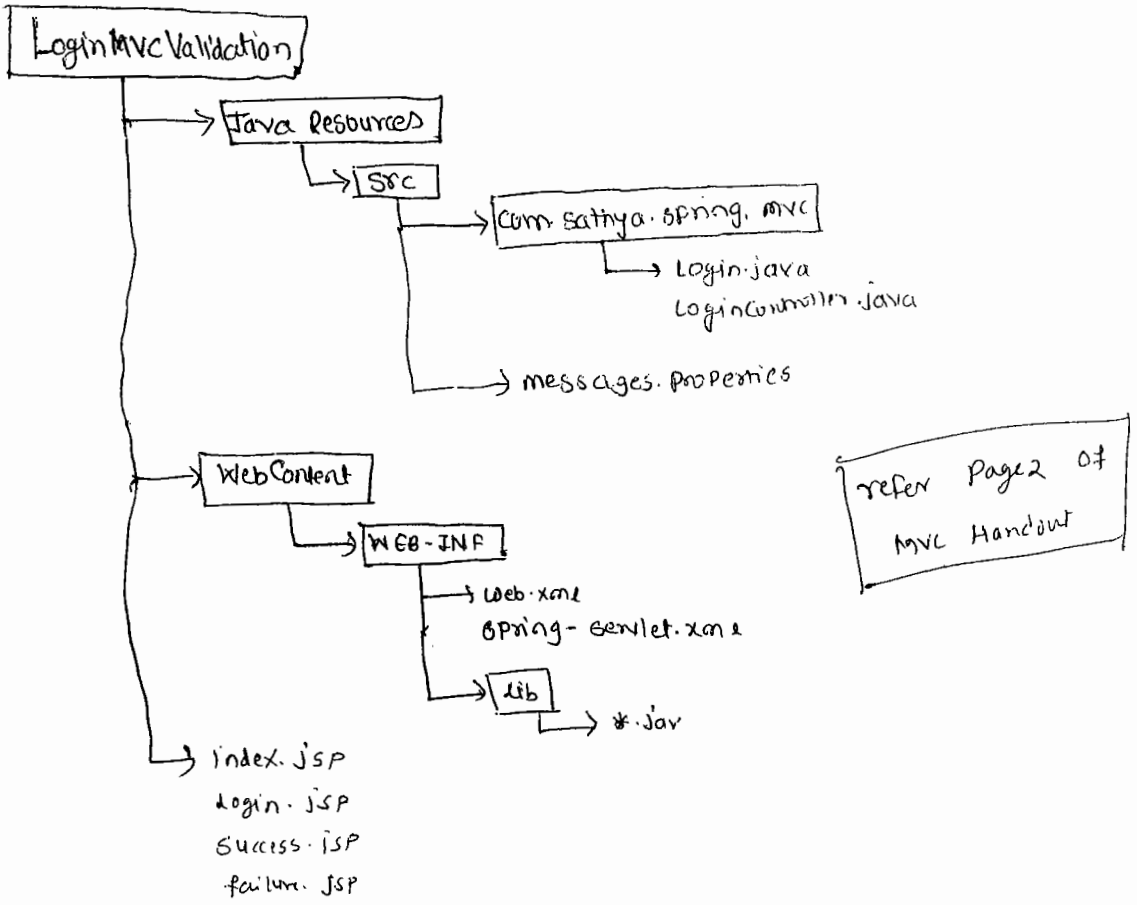
→ before injecting a POJO class object, to tell the DispatcherServlet that to execute a validation, then we add @Valid annotation before @ModelAttribute annotation.

→ while executing a validations, if any errors are occured then those errors are stored BindingResult class object in the form of Error Objects.

→ In a Controller bean method, we need to take a Parameter of type BindingResult then DispatcherResult will pass BindingResult object to that method at the time of calling it.

```
@Controller
Public class LoginController
{
    @RequestMapping (value = "check", Method = RequestMethod.POST)
    Public ModelAndView ProcessRequest ( @Valid @ModelAttribute
                ("login") Login log, BindingResult errors)
    {
        =
        =
    }
}
```

→ To show the error messages on browser we need to use a tag <form:errors/> in the form design.



refer Page2 of MVC Handout

aopalliance

commons-logging

hibernate-validator-4.0.2.GA.jar

servlet-api

slaj-api-1.6.1.jar

Spring-aop

Spring-beans

Spring-context

Spring-context-support

Spring-core

Spring-expression

Spring-web

Spring-webmvc

validation-api-1.0.0.GA.jar

→ newly added

8-4-2015

file to upload "get" method not supported

<form method="POST"

enctype="multipart/form-data">

↳ don't apply any encryption on file data.

by default browser add some encryption when sending data over server.

Common-fileupload api

CommonsMultipartResolver

predefined class given by spring fw

→ Internally used to read file from request

→ Internally uses Commons-fileupload api to read file from request object

→ convert file to multipart-file object

→ return this object to DispatcherServlet

```
@Controller
Public class MyController
{
    RequestMapping
  p ModelAndView Method (MultipartFile file)
   {
      read & store object
    }
}
```

File Uploading in Spring MVC

→ file uploading is nothing but transfering the content of the file from client system to servers.

→ when a file uploaded then the uploaded file will be stored on a server either in HardDisk or Database.

→ when designing a form per enduser to allow a enduser for selecting a file we need to provide a browse button. It is possible by using <input type = file /> tag of html.

→ file uploading will not be supported by the http protocol get method, so we need to change method as POST in the form design.

→ by default a browser will apply some encryption, before it is sending data to the server. but in file uploading we need to disable browser encryption. for this we need to add enctype attribute to the form

→ by designing a form we need to make a 3 changes

① <form action = "xxx" ① method = "POST"

② enctype = "multipart/form-data" >

select file : <input type = file > ③

<input type = submit>

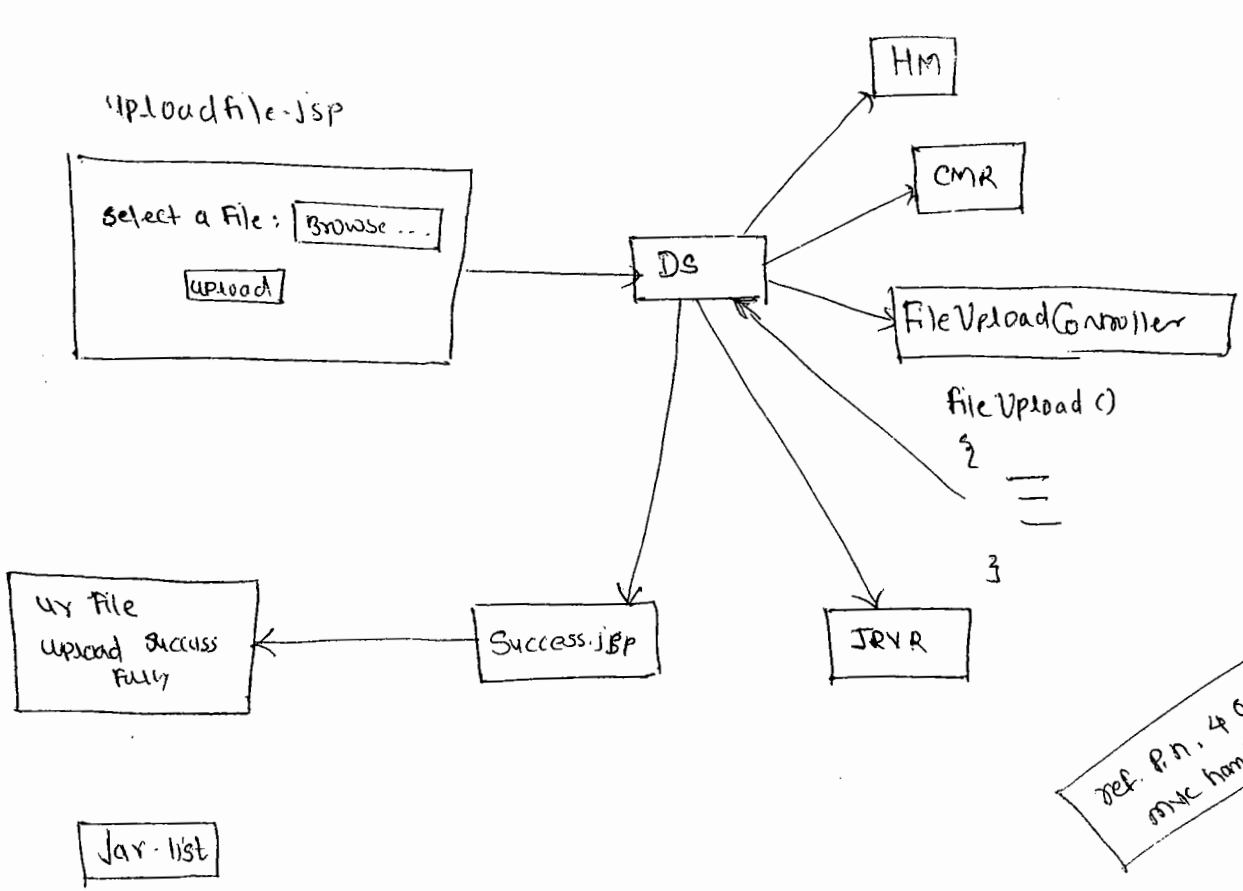
</form>

→ In spring framework a predefined class for CommonsMultipartResolver is given for converting an uploaded file into a MultipartFile object

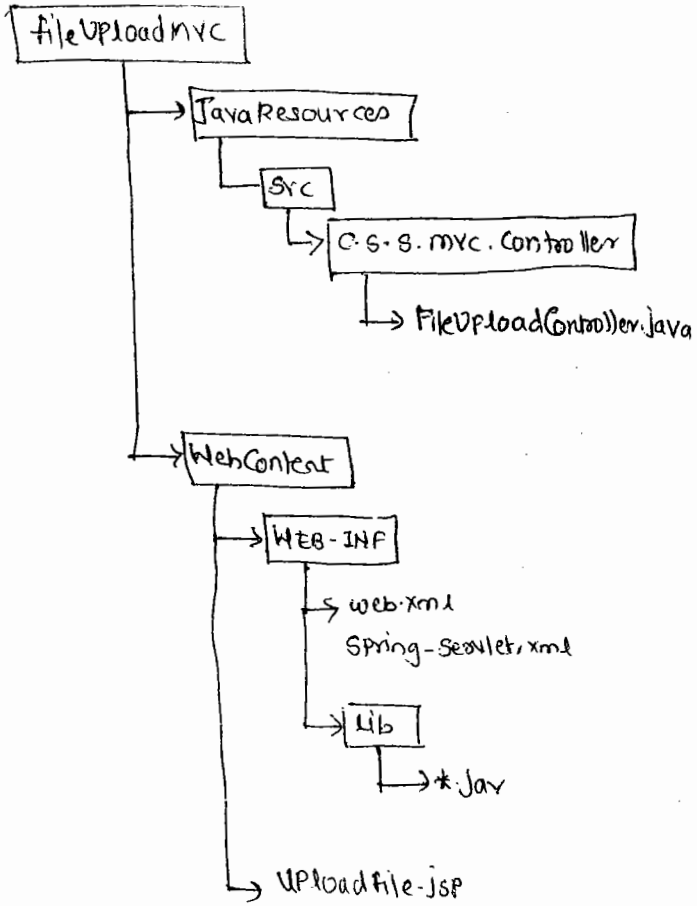
→ CommonsMultipartResolver uses commons-fileupload api of apache for converting an uploaded file into MultipartFile object.

→ when calling a the method of Controller Bean DispatcherServlet will inject MultipartFile object to Parameter of the method and the we can defined the logic for either storing the file on Server HardDisk or database in that method.

commons-fileupload-1.1.1 is depends on commons-io-1.1.jar



ref. P.n. 4 of mvc handout

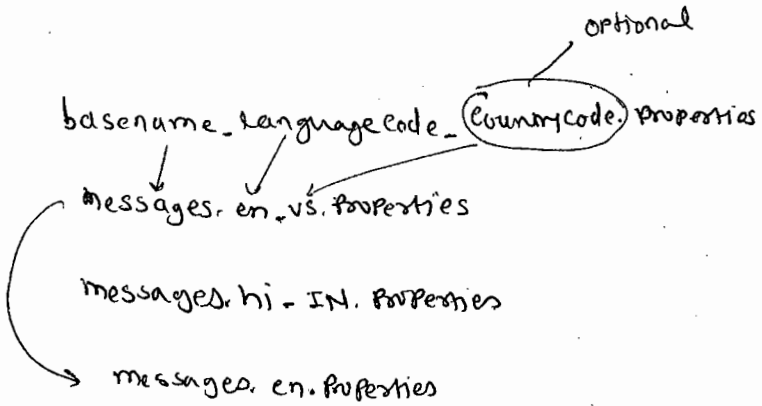


Jar list

aopalliance
commons-fileupload-1.1.1.jar
commons-io-1.1.jar
commons-logging.jar
javax.servlet.jsp.jstl-1.2.1.jar
javax.servlet.jsp.jstl-api-1.2.1.jar
Servlet api
Spring-aop
Spring-beans
Spring-context
Spring-context-support
Spring-core
Spring-expression
Spring-web
Spring-webmvc-3.0.5

14-4-2015

I18N with MVC



→ I18N :- Internationalization.

→ I18N is a process of displaying the content on browser in a selected language by the end user.

→ by depending on the langauges that are offered to the end user we need to prepared that many number of Resource Bundle.

→ When Preparing the Resource Bundle the Bundle name should follow a Syntax **basename_languagecode_Countrycode.properties**

→ In the above languagecode & countrycodes are ISO given codes. and the countrycode is optional.

⇒ for ex: if we want to offer two languages english & hindi to the end user we need to create two resource bundle with the following name.

① messages_en_VS.Properties
② messages_hi_IN.Properties

OR

messagen_en.properties
messages_hi.properties

→ When defining the Resource Bundles, In all bundles keys must be same & there are values depending on the langauge.

→ IN I18N application, an application consists multiple resource Bundle, but only 1 bundle should be loaded based on the language code for a selected language. In

→ In order to load 1 Bundle amoung multiple, we need to use the following 2 classes or ~~java.lang~~ java.util. Package.

① Locale
② ResourceBundle

→ Locale is a class and ResourceBundle is an abstract class

```
String s = request.getParameter("lc");
Locale l = new Locale(s);
ResourceBundle rb = ResourceBundle.getBundle("messages", l);
```

→ If the value of requestParameter is en then the above code loads messages_en.properties file

→ When working with frameworks we no need of writing the above code In an application for loading a resource Bundle. In place of code, we need to configured a predefined framework classes in configuration file.

→ In case of the spring framework if we want to apply I18N support to MVC application then we need to configure the following 2 predefined bean classes in spring configuration file.

① LocaleChangeInterceptor
② SessionLocaleResolver.

→ LocaleChangeInterceptor class reads the value request Parameter then creates a Locale object and then stores Locale object in Session object.

→ SessionLocaleResolver reads Locale object from Session, then loads a Resource bundle for the given Locale.

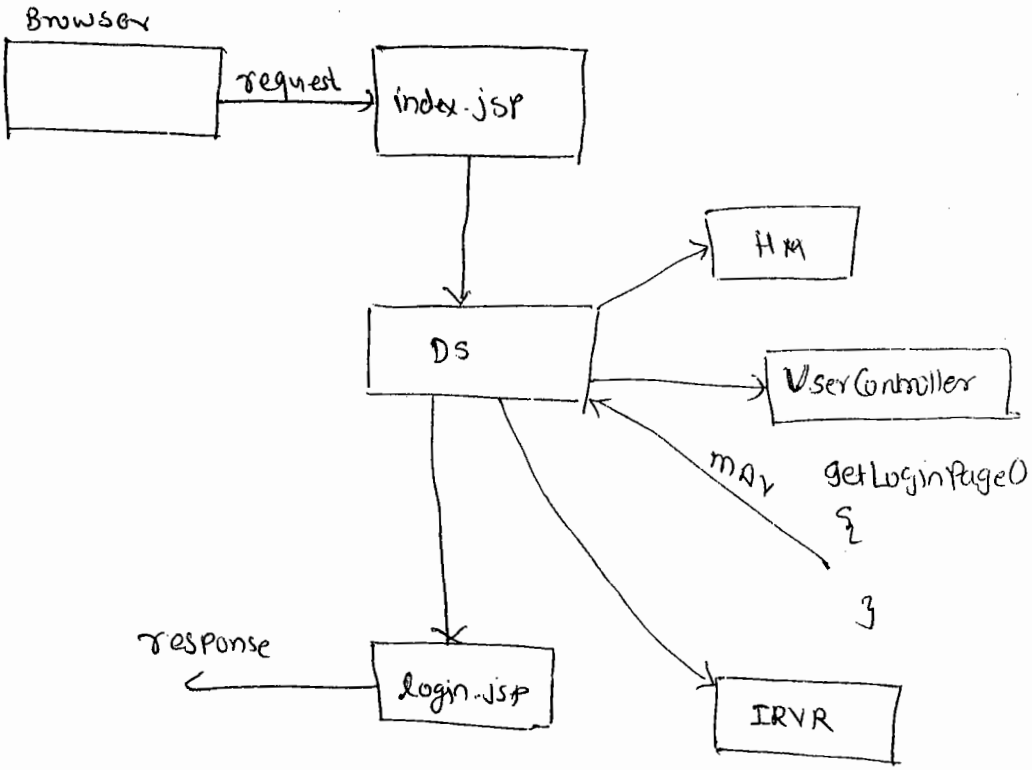
```
<bean id = "change"
      class = "LocaleChangeInterceptor">
    <Property name = "paramname"
              value = "lc" />
</bean>
```

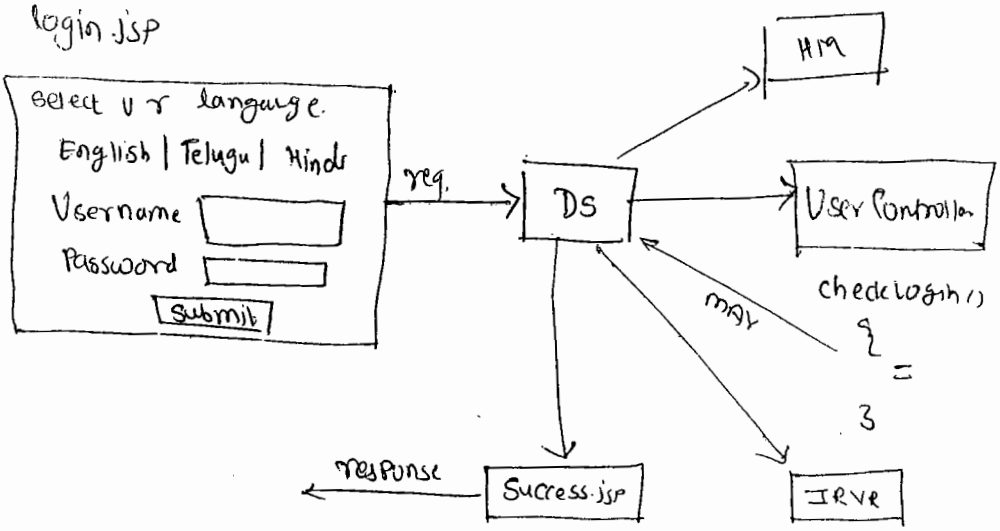
→ here lc is a name request Parameter through which a language code is sent from browser with request.

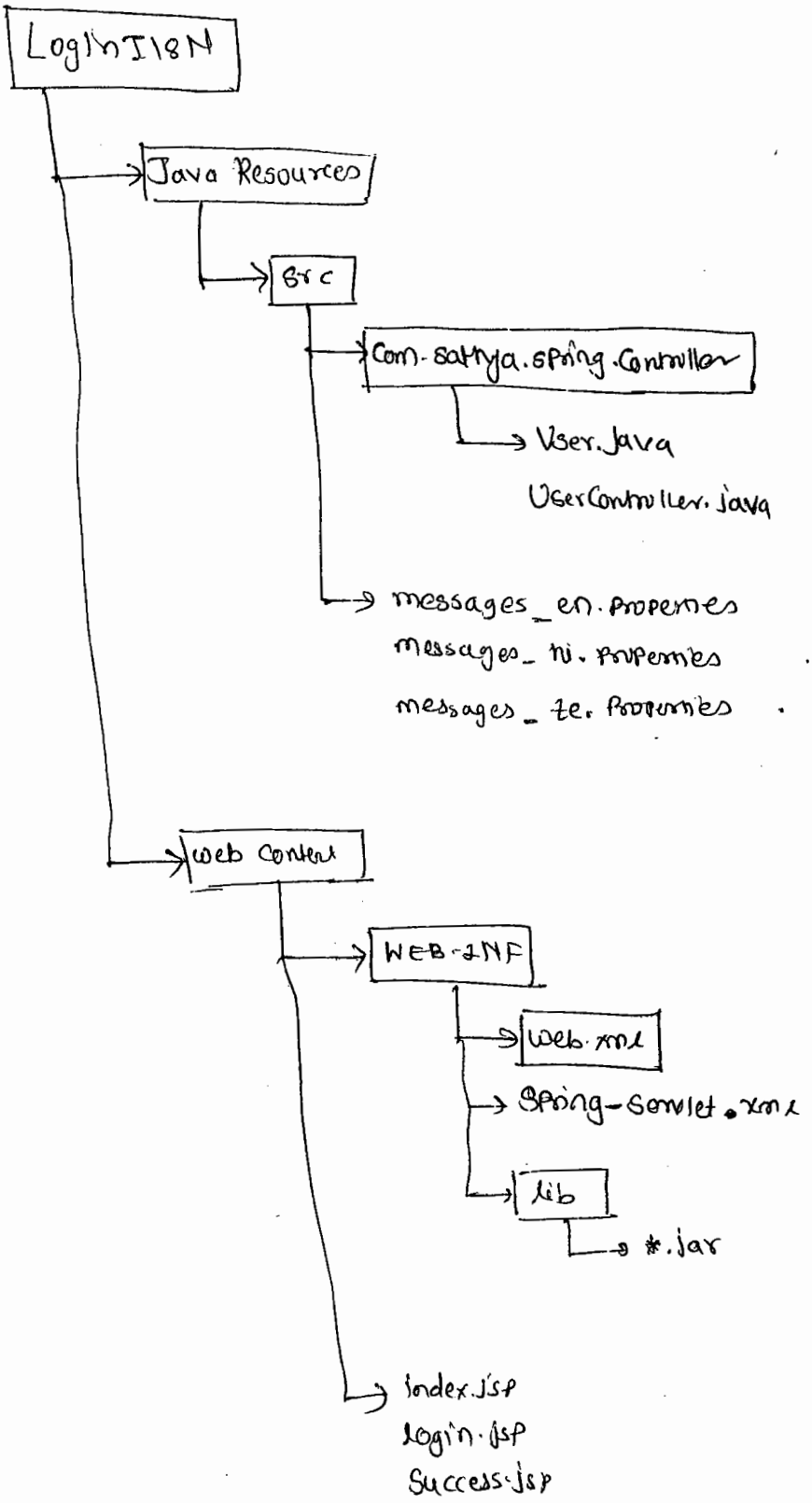
15-4-2015

I18N Example

→ In the following ex we are creating a login Appl^n with validations. & we are offering login Page in Hindi, English & Telugu, by applying I18N.


Browser
request
index.jsp
HM
DS
UserController
MAV
getLoginPage()
{
}
response
login.jsp
IRVR


login.jsp
Select ur language
English | Telugu | Hindi
Username
Password
Submit
req.
DS
HM
UserController
checkLogin()
MAV
{
=
}
response
Success.jsp
IRVR



→ aopalliance-1.0
commons-logging
depends on → hibernate-validator-4.0.2.GA
SLF4J-api-1.6.1.jar
servlet-api.jar
spring-aop
spring-beans

Spring Context
spring Context-support
spring-core
spring-expression
spring-web
spring-webmvc
Validation-api

16-4-2015

User.java

```
Public class User
{
    @NotEmpty
    Private String uname;
    @NotEmpty
    Private String pwd;
    // setters & getters
}
```

UserController.java

```
@Controller
Public class UserController
{
    @RequestMapping (value = "/login", method = RequestMethod.GET)
    Public ModelAndView getLoginPage ()
    {
        return new ModelAndView ("login", "command", new User());
    }

    @RequestMapping (value = "/check", method = RequestMethod.POST)
    Public ModelAndView checklogin (@Valid ~~@ModelAttribute ("command")~~
                @ModelAttribute ("command") User user, BindingResult br)
```

key — POJO class object

```
{
  if (br.hasErrors())
   {
    return new ModelAndView("login");
   }
  else
    {
      return new ModelAndView("Success");
    }
 }
}
```

spring-servlet.xml

```
<context:component-scan base-package="com.satya.spring.Controller" />

<!-- handler mapping -->

<bean class="org.s DefaultAnnotationHandlerMapping" />
<!-- View resolver -->
<bean class="InternalResourceViewResolver" >
    <Property name="Prefix" value="/" />
    <Property name="suffix" value=".jsp" />
</bean>
<bean id="messageSource" class="ResourceBundle
    <property name="basename" value="messages" />
</bean>

<mvc:annotation-driven />
```

index.jsp

```
<jsp:forward  Page = "login.form"></jsp:forward>
```

login.jsp

```
<%@ taglib  uri = "http://www.springframework.org/tags/form"
            Prefix = "form"%>

<%@ taglib  uri = "http://www.springframework.org/tags"
            prefix = "spring"%>

Select V r language ? <a href = "login.form?p1=en"> English </a>
                      || <a href = "          = te"> Telugu </a>
                      || <a href = "          = hi"> Hindi </a>

<br><br>

<center>
    <form:form action = "check.form" method = "POST" commandName
                                                = "command">
        <spring:message code = "label.username" /> : <form:input Path =
        "uname" /> <form:errors Path = "uname"> <spring:message code =
        <spring:message code = "label.Password" /> "NotEmpty.command.Uname"
        /> </form:errors> <br>

        <spring:message code = "label.Password" /> : <form:Password Path = "Pwd" />
            <form:errors Path = "Pwd" /> <spring:message Code = "NotEmpty.command
                .Pwd" /> </form:errors> <br>

        <input type = submit Value = "submit" />
    </form:form>
</center>
```

messages_en.properties.

messages_en.properties

label.username = Username

label.password = password

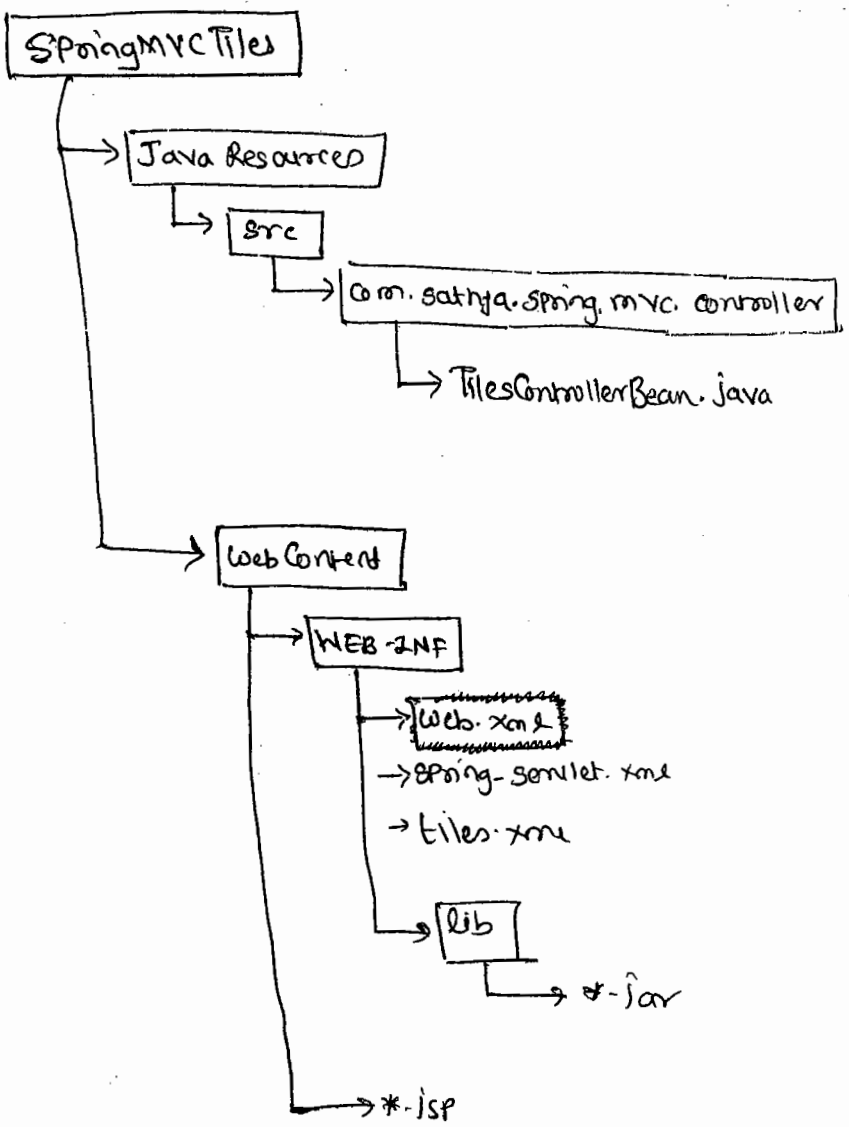
NotEmpty.command.uname = Username is required

NotEmpty.command.pwd = Password is required

Note → In the remaining 2 bundles also, we need put a same keys but the values are html text and it can be generated or ~~collected~~ collected from below website

http://vikku.info/indian-language-unicode-converter/

18-4-2015



Refer. Page-9
MVC Handout.

Jar file list

aopalliance-1.0
commons-beanutils-1.8.0
commons-digester-2.0.jar
commons-logging
Javax.servlet.jsp.jstl-1.2.1
Javax.servlet.jsp.jstl-api-1.2.1
slf4j-api-1.6.1
spring-aop-4.1.2.RELEASE
spring-beans-4.1.2.RELEASE
spring-context-4.1.2 RELEASE
spring-context-support-4.1.2 RELEASE
spring-core-4.1.2 RELEASE
spring-expression-4.1.2 RELEASE
spring web-4.1.2.REL
spring-webmvc-4.1.2.REL
tiles-api-3.0.5
tiles-autotag-core-runtime-1.1.0.jar
tiles-compat-3.0.5.jar
tiles-core-3.0.5.jar
tiles-el-3.0.5
tiles-extras-3.0.5
tiles-jsp-3.0.5.jar
tiles-request-api-1.0.6
tiles request.jsp 1.0.6.
tiles request.servlet-1.0.6
tiles-request-servlet wildcard-1.0.6
tiles-servlet 3.0.5
tiles.template-3.0.5

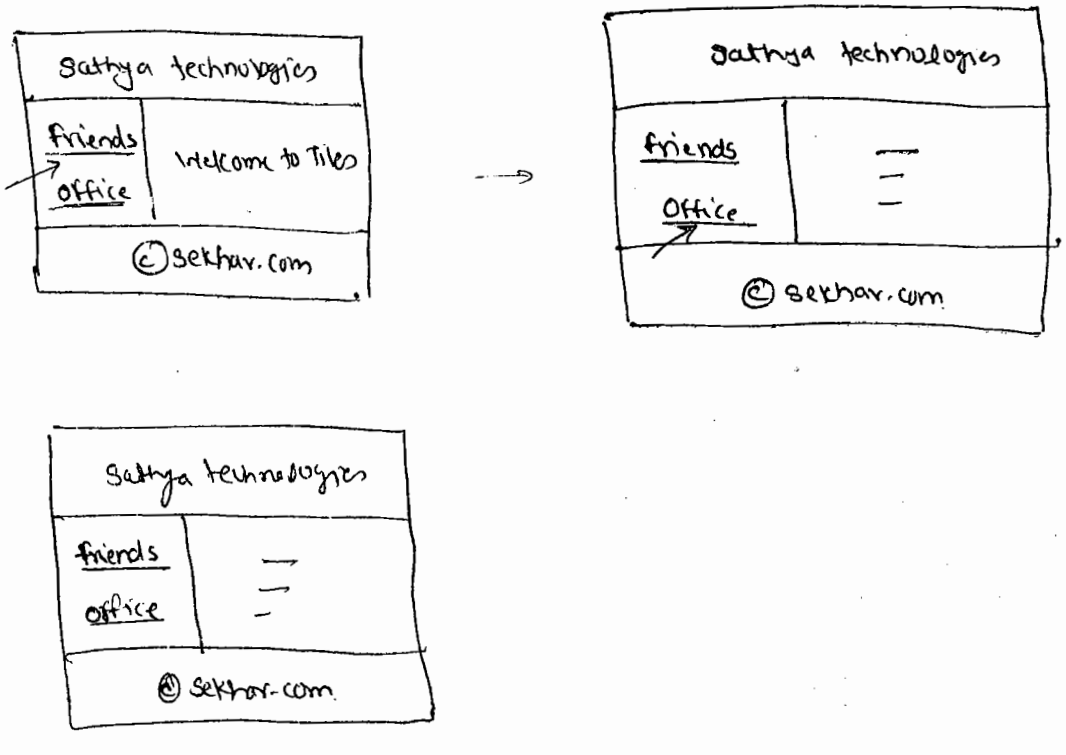
```
<bean id = "xxx" class = "TilesConfigurer">
    <property name = "definitions">
        <list>
            <value> /WEB-INF/tiles.xml </value>
        </list>
    </property>
</bean>
```

→ When working with tiles application, we need to configure ~~tiles~~ TilesViewResolver in spring configuration file. this ViewResolver will read appropriate view from tiles container and returns a tilesView object to the DispatcherServlet.

→ the following example contains 3 composite views.

→ to create a composite we are using spring MVC with tiles flw of apache.

→ the composite views are following a common structure with 4 regions.



tiles.xml

```
<tiles-definitions>
    <definition name = "def1" template = "/layout.jsp">
      ---
    </definition>
    <definition name = "def2" extends = "def1">
      ---
    </definition>
    <definition name = "def3" extends = "def1">
      ---
    </definition>
</tiles-definitions>
```

→ if a large number of definitions are needed then we can define multiple tiles configuration file.

Spring beans to work with tiles:

→ Spring fw has given a bean class for loading the one or more tiles configuration file called TilesConfigurer.

→ TilesConfigurer internally uses tiles API for reading the definitions of tiles configuration file and finally prepared the tiles container object.

→ ~~Tiles~~ TilesConfigurer class has a ~~Reference~~ Property "definitions" & we need to configure one or more TilesConfiguration file as a list of value

17-4-2015

* Creating a layout Page

→ a layout Page is to defined the structure of the composite view Pages into a seperate Page.

→ In order to work with tiles framework a 1st step is creating layout Page

→ a layout Page divides a view into a multiple regions & assigns a name for each layer

→ while creating a layout Page to create a different Regions we use <table> tags of Html.

→ to assign a name to each region we use a tiles tag of apache called <tiles: insertAttribute>

→ In a layout Page, to use a tiles tag of apache we need to import a tag library Uri into our jsp page using a taglib directive.

```
<%@ taglib uri = "http://tiles.apache.org/tags-tiles"
           prefix = "tiles" %>
```

* tiles Configuration file.

→ the Composite views of a web application which follows a layout are Configured in a tiles Configuration file.

→ a tiles-configuration file contains one or more definition where each definition indicates a composite view of the application if multiple composite views contains common content in some regions then to get reusability we can create a base defination and we can extend it into other definations of configuration file.

```
<mvc:interceptors>
    <bean id="localeChangeInterceptor"
        class="org.springframework.web.servlet.i18n.LocaleChangeInterceptor">
        <Property name="ParamName" value="pj" />
    </bean>
</mvc:interceptors>

<bean id="localeResolver"
    class="org.springfw.web.servlet.i18n.SessionLocaleResolver" />

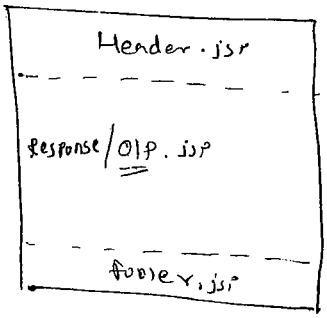
</beans>
```

web.xml

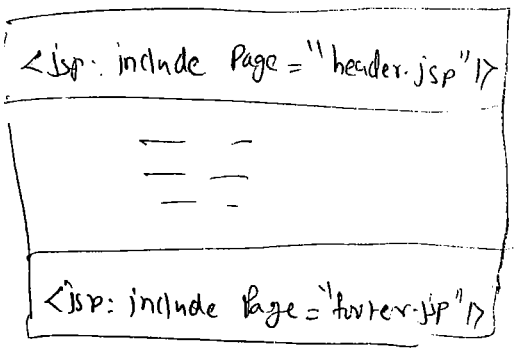
Copy paste

Spring MVC with Tiles

Response of multiple page or single page.



3 .jsp o/p in 1 single page is CompositeView



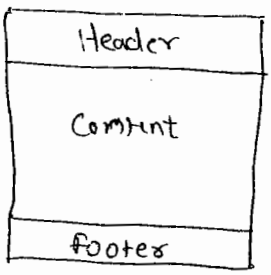
→ In web application, if response to the browser is constructed by a single jsp Page then it is called simple view. If response is constructed by including response of other jsp pages including then it is called Composite view.

→ to create Composite views, we include the response of other jsp Pages. with the help of <jsp:include> action tag

→ when an web application content a large number of Composite view Pages then we use the support of Tiles framework given by apache.

→ by without using support of tiles framework also we can develop Composite views, but when change is required in the layout of Pages then we need to open and modify each Composite Page.

→ for eg: ① we are creating a web application with Composite view Pages, by displaying a Header on top, a footer on bottom. and content in a middle of the Page.



② if our web application containing 50 Composite views and a if we want to change a look after composite view like a Header as footer & footer as Header. then we need to open & modify all the 50 composite view Pages. it will be burdon on developer and a it is a time consuming process.

⑧ In order to reduce the above problem apache introduce tiles flw.

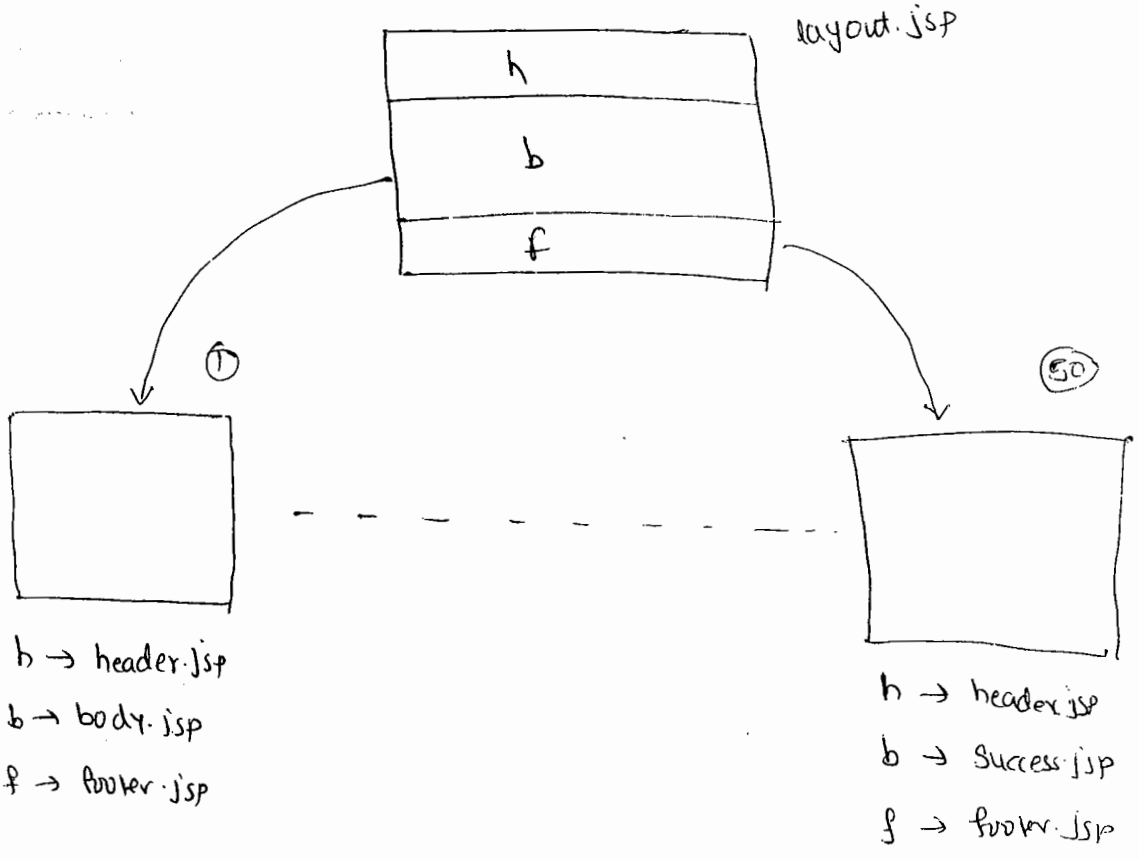
→ Tiles flw of apache can be used by any coeb application framework which wants to create a composive view pages in a web application

→ In spring-mvc applications also, we use Tiles flw of apache.

→ In tiles programming jsp we need to create a layout page to define the structure of the composite views.

→ When creating composite views, we insert the struture to all the composite views.

→ the benefit of using tiles flw is when we want to change the structure or look of the composite views then we can ~~just~~ only ~~just~~ just ~~modified~~ modify the layout page automatically the changes are affected on all the composite views.



⇒ Suppose if want to modify header as footer & footer as header in a 50 composite pages then we need to just open the layout page & then we need to modify regeion name h as f & f as h. automatically all the 50 composite pages are modified.

Generating Response in Pdf & Excel formats.

→ In Spring MVC, to generate response like a Pdf Page or an Excel Page then there are 2 options

Option ① in jsp Page, we can set MIME type as Pdf or Excel & we can define the logic

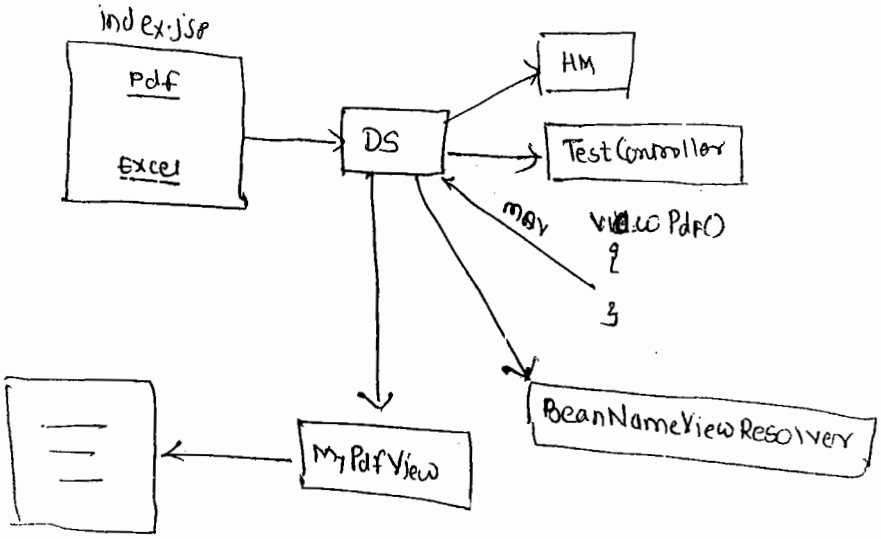
Option ② we can create a Bean class and we can extend from abstract base classes ~~provided~~ given by the framework.

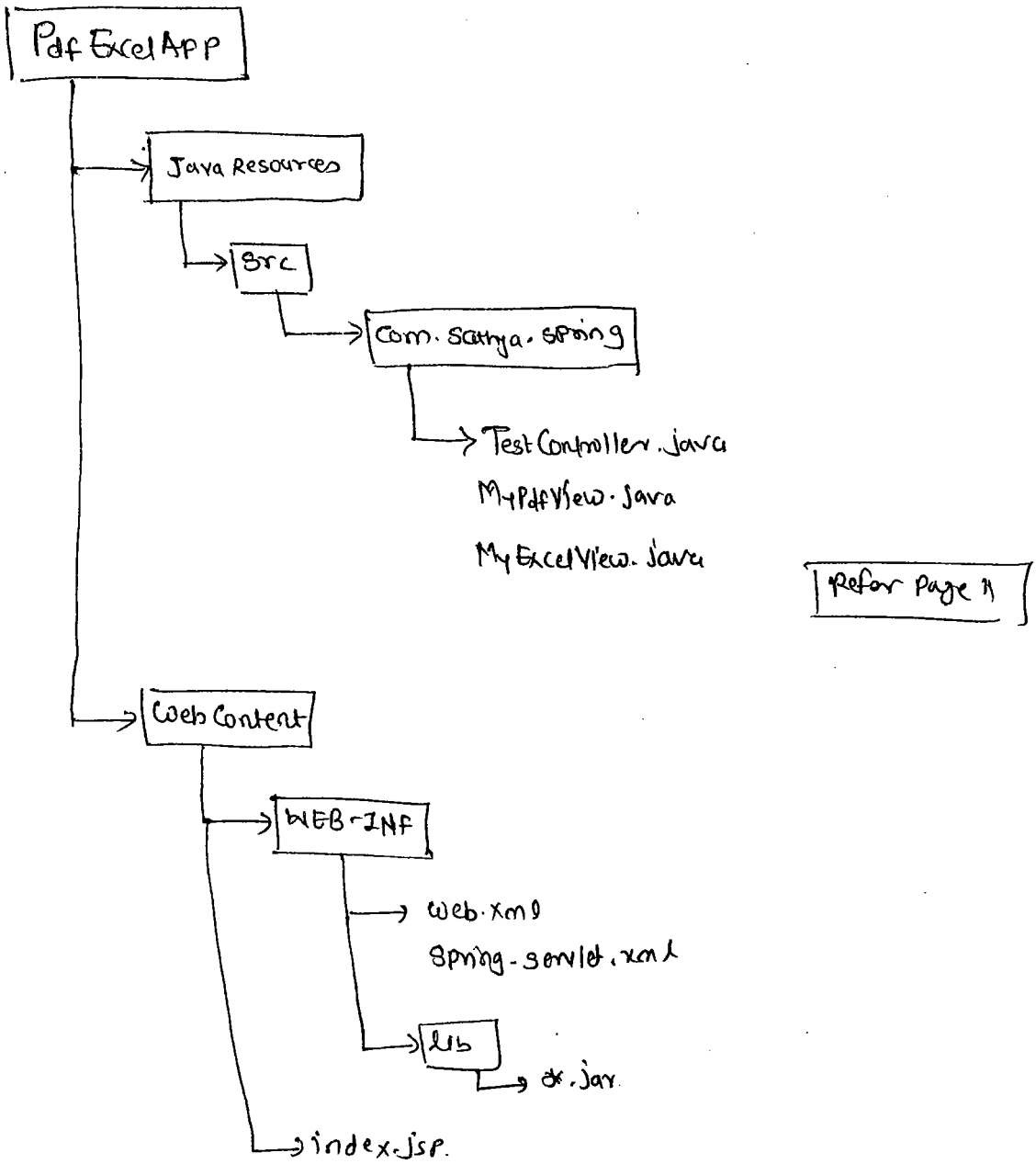
→ Spring framework has provided 2 abstract base classes called ~~abstract pdf view and abstr~~ AbstractPdfView & AbstractExcelView.

→ AbstractPdfView is an abstract class with one abstract method buildPdfDocument(). So we need to override this abstract method

→ AbstractExcelView is also an abstract class with one abstract method buildExcelDocument(). So we need to override this abstract method.

Example





jar

1. aopalliance-1.0
2. commons-logging
3. iText-2.1.7
4. poi-3.9

newly added

5. servlet-api
6. spring aop
7. spring beans
8. spring context
9. spring context-support
10. spring-core
11. spring-expression
12. spring web
13. spring webmvc.

```
Public class MyExcelView extends AbstractExcelView
{
  Protected void buildExcelDocument (Map m, HssfWorkbook wb, HttpServletRequest req,
  HttpServletResponse res) throws Exception
  {
      Sheet Sheet = wb.createSheet ("Sheet1");

        Row r1 = Sheet.createRow (0);
        Cell c1 = r1.createCell(0);
          c1.SetCellValue ("company name");
          Cell c2 = r1.createCell(1);
          c2.setCellValue ("website");

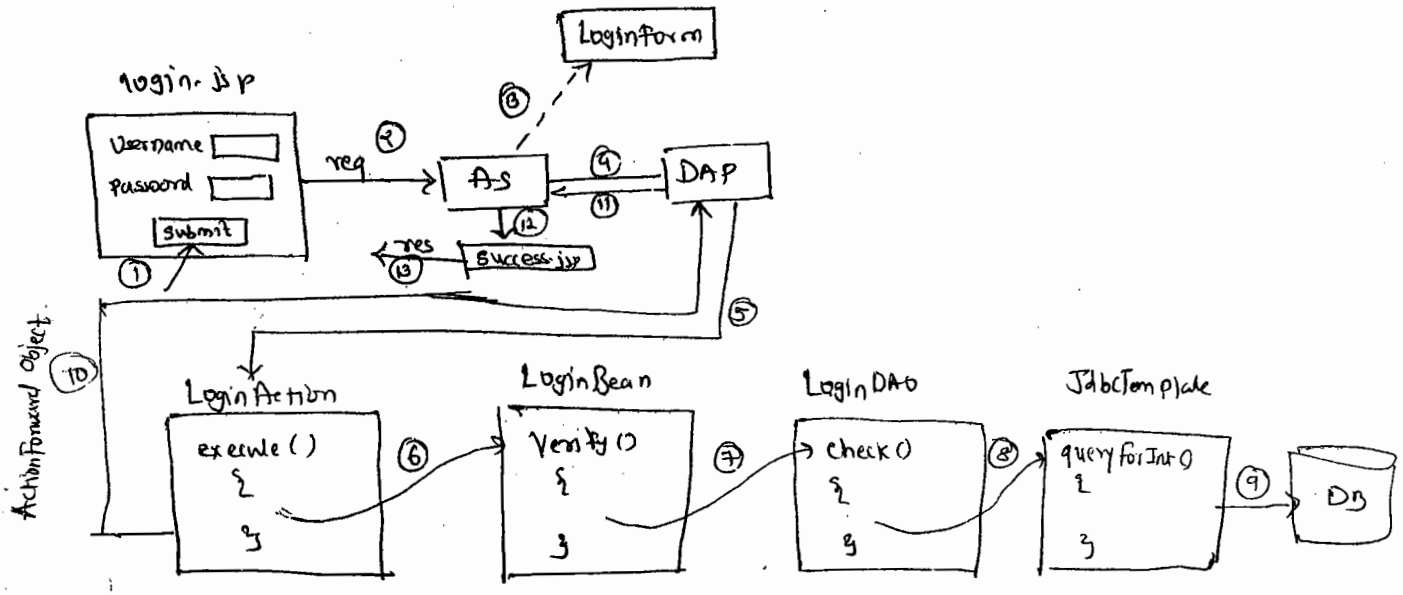
      Row r2 = Sheet.createRow (1);
      Cell c3 = r2.createCell (0);
        c3.setCellValue ("Oracle");
        Cell c4 = r2.createCell(1);
        c4.setCellValue ("http://www.oracle.com");

      Row r3 = Sheet.createRow (2);
        Cell c5 = r3.createCell(0);
        c5.SetCellValue ("IBM");
        Cell c6 = r3.createCell(1);
        c6.setCellValue ("http://www.ibm.com");
      }
  }
```

20-4-2015

Q. Plugin for integrating struts & spring

Context loader Plugin given by ~~spring~~ spring fw



VIMP

Q. Diff. bet^n beanFactory & ~~ApplicationFactory~~ ApplicationContext container

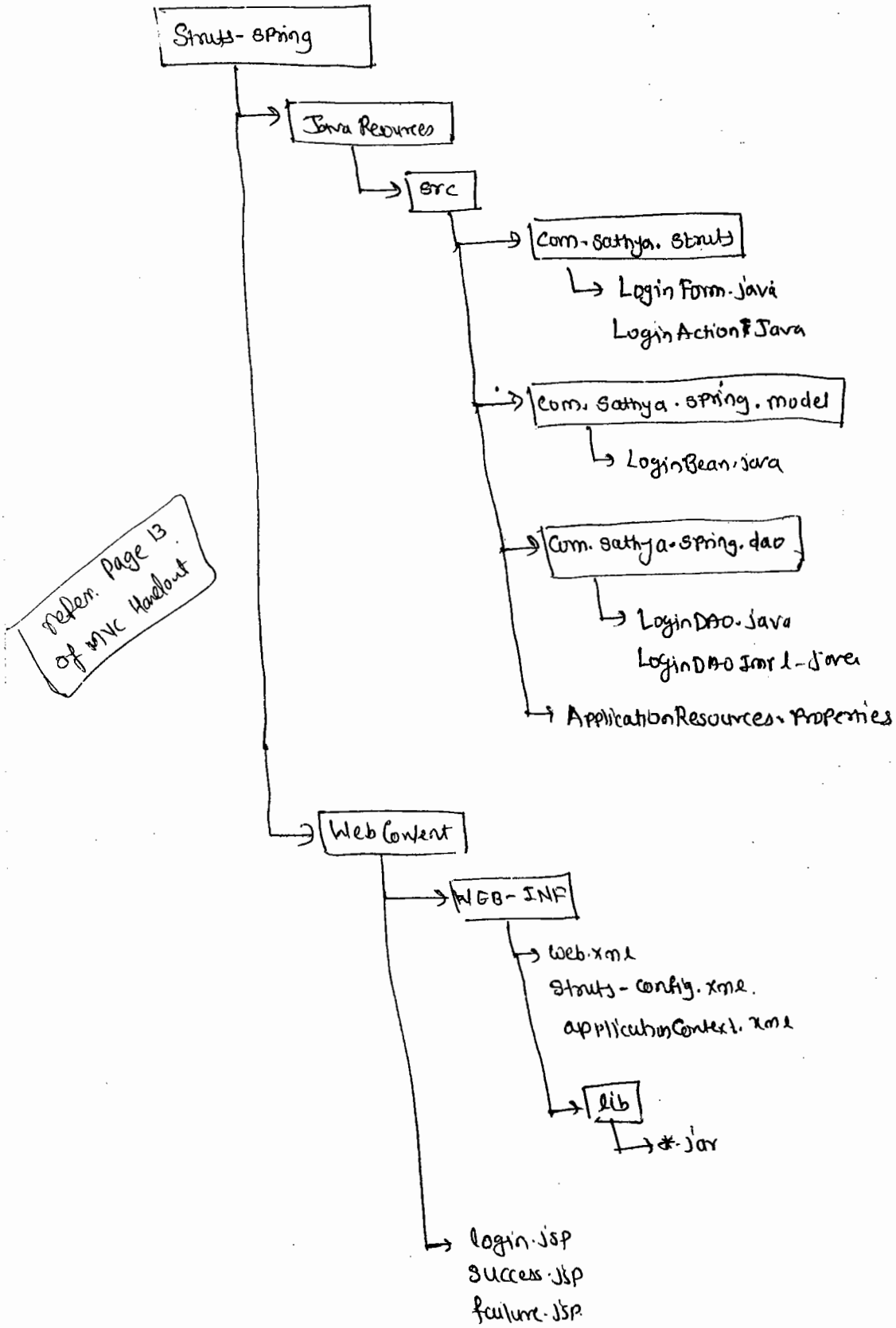
Example

→ In the following example, we are creating login appln for integrating struts & spring with database.

→ LoginAction class of struts will call the LoginBean class of spring.

→ LoginBean class of calls a DAO calls & then it calls JdbcTemplate to execute the sql command / query on database.

↳ Next Page.


Struts-spring
Java Resources
src
com.sathya.struts
LoginForm.java
LoginAction.java
com.sathya.spring.model
LoginBean.java
com.sathya.spring.dao
LoginDAO.java
LoginDAOImpl.java
ApplicationResources.properties
refer. Page 13 of MVC Handout
WebContent
WEB-INF
web.xml
struts-config.xml
applicationContext.xml
lib
*.jar
login.jsp
success.jsp
failure.jsp

Jars

1. antlr - 2.7.2
2. commons-beanutils-1.7.0
3. commons-chain-1.1
4. commons-digester-1.8
5. commons-logging-1.0.4
6. commons-validator-1.3.1
7. ojdbc14.jar
8. ORO - 2.0.8.
9. Spring-beans-4.1.2.REL
10. Spring-Context-4.1.2.REL
11. Spring-Context-support-4.1.2 REL
12. Spring-core-4.1.2
13. Spring-expression
14. Spring-jdbc
15. Spring-struts-3.2.4.RELEASE.jar. → not spring fw download separately
16. Spring-tx
17. Spring-web
18. Spring-webmvc
19. Struts-core 1.3.8
20. struts-taglib-1.3.8

* ResourceBundleViewResolver

→ If url's of views are defined in a ResourceBundle then to read the url and to return a view object, ResourceBundleViewResolver is used.

→ In a ResourceBundle, for each view we need to define two keys

`viewname.class & viewname.url`.

→ ResourceBundleViewResolver loads a ResourceBundle and returns a url in the form view class object.

→ In spring configuration file we need to configure ResourceBundle name as a value for base name property.

In spring-servlet.xml:

```
<bean id = "viewResolver"
      class = "org.springframework.web.servlet.view.ResourceBundleViewResolver">
  <Property name = "basename"
            value = "spring-view" />
</bean>
```

In InternalResourceViewResolver view name & page name (jsp) must be same, but in ResourceBundleViewResolver not necessary may be differed

spring-views.properties

```
success.(class) = org.springframework.web.servlet.view.JstlView
success.url = /ok.jsp
failure.(class) = org.springframework.web.servlet.view.JstlView
failure.url = /sorry.jsp
```

---

* SimpleUrlHandlerMapping

→ this UrlHandlerMapping class will identified appropriate controller bean like the following

→ ① It reads the url pattern of Request
  ② It searches for a key in a mappings property matched with url pattern.
  ③ It reads the value of key and then it will search for bean whose id is matched with that value.
  ④ finally returns & an object of Controller Bean to the DispatcherServlet.

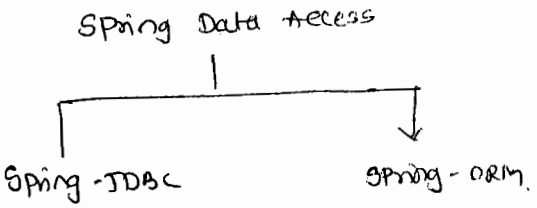
→ with SimpleUrlHandlerMapping, we need to configure mappings property. this mappings is a collection of type ~~properties~~. Property.

for ex:

```
<bean id = "xx" class = "SimpleUrlHandlerMapping">
    <Property name = "mappings">
      <Props>
          <Prop key = "/login.form"> id1 </Prop>
          <Prop key = "/register.form"> id2 </Prop>
      </Props>
    </Property>
</bean>
```

Spring Data Access Module

9-4-2015



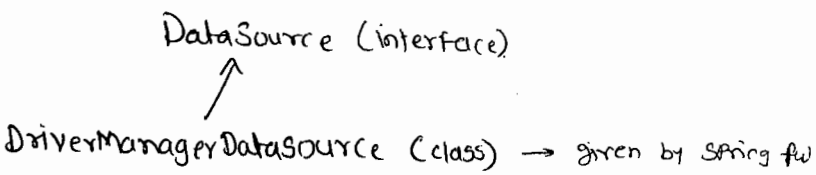
Spring-JDBC

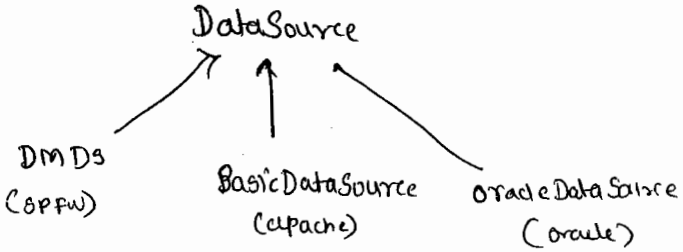
→ Spring-JDBC is an abstraction layer on top of JDBC technology when directly coorking with JDBC technology the following 3 Problems identified by the Programmers

1. Boiler plate code (repetative code)
2. checked Exceptions
3. Memory leak

→ In order to reduce the above Problems Spring framework has Provided a layer on top JDBC & named it as Spring-JDBC

JdbcTemplate → given by fw





~~JDBC Template~~

JdbcTemplate

→ Spring fw has provided a central class for executing "CURD" operations ~~on database~~ called JdbcTemplate.

→ this JdbcTemplate class depends on a DataSource object.

→ JdbcTemplate class has provided convenient method for executing SQL commands on database.

→ JdbcTemplate class will internally take care about Boiler Plate code i.e required for executing the sql commands.

→ In Spring, the ~~exception~~ checked exception of jdbc are Translated as unchecked exception of Spring framework. so a Programmer is no need handle the exception by adding try & catch block.

→ When working with JdbcTemplate, a Programmer only concentrates on sending sql queries to database and Processing results of the query

~~Public class JdbcTemplate~~

→ In JdbcTemplate class a constructor and also a setter method are defined for injecting a DataSource object. so Spring Configuration file we can Configure JdbcTemplate class like the following

```
<bean id = "jt"  class = "JdbcTemplate">
    <constructor-arg  ref = "ds" />
</bean>
```

OR

```
<bean id = "jt" class = "JdbcTemplate">
    <Property name = "dataSource"
              ref = "ds" />
</bean>
```

```
Public class JdbcTemplate {

    Private DataSource dataSource;

    Public JdbcTemplate (DataSource ds) {      → dataSource
        this.ds = ds;
    }

    Public void SetDs (DataSource ds)
    {
        this.ds = ds;
    }
}
```

constructor

better method

→ DataSource is an interface that framework has provided one implementation class of DataSource interface called a DriverManagerDataSource.

→ In Spring Configuration file, we can configure either framework provided implementation class or Vendor provided implementation class.

for example : BasicDataSource is an implementation class of DataSource interface of an apache

→ So we can configured BasicDataSource class also in spring configuration file

Methods of JdbcTemplate:

- execute & update are for non-select
- query method for select operation on database

① execute(): ".create / alter / truncate / drop"
(DDL command)

```
Scanner s = new Scanner(System.in);
→ sid
→ sname
→ marks
Object arr[] = {sid, sname, marks};
int i = jTemplate.update("insert into student values (?,?,?)", arr);
```

OR

```
int i = jTemplate.update("insert into student values (?,?,?)", sid, sname, marks);
                                                              Varargs
```

DDL Commands index Parameter (?) not allowed

① **execute()**: this method is only for executing DDL Commands
(Create, alter, truncate, drop)

```
jt.execute("create table xyz (id number(3), name varchar(10))");
```

Note: In DDL command, "index parameter (?)" are not allowed.

② **update();** this method is to execute DML commands (insert, update, delete)

→ this method return integer value which indicates the number of rows updated or affected.

→ we can use update method in 3 ways

① update (command) } → static command

② update (command, Object[])

~~③ update (command, Object ...varargs~~ } dynamic command

③ update (command, Object...)

↓

varargs

Example: 1

```
int i = jt.update("insert into student values (101,'A',500)");
```

Example 2

```
Scanner s = new Scanner (System.in);
SOP ("enter sid");
int sid = s.nextInt ();
SOP ("enter sname");
String sname = s.next ();
SOP ("enter marks");
int marks = s.nextInt ();

Object[] arr = { sid, sname, marks};
int i = jt.update ("insert into student values (?,?,?)", arr);
```

Pt. B

int i = jt.update("insert into Student values(?,?,?)",
sid, sname, marks);

④ ~~queryMap();~~

10-4-2015

⑤ queryForMap

→ It is used for reading a single row from a database

→ this method returns a Map, which contains columns name as key and ~~Cotton~~ column values as value.

→ to make the query as dynamic we can use index parameter & we can set the values by passing an Object[] or varargs

a) queryForMap(command)

b) queryForMap(Command, Object[])

c) queryForMap(Command, Object...)

ex:

Map m = jt.queryForMap("select * from emp where empno=7788");

for Map object it is not possible to use Iterator. so convert it into set

Set s = m.entrySet();

Iterator it = s.iterator();

while(it.hasNext())

{

Map.Entry me = (Map.Entry) it.next();

SOP(me.getKey() + " " + me.getValue());

}

④ **queryForList();** → this Method is used to read the Multiple rows from database

~~List list = ;~~

→ Internally, Spring stores each row data in a Map & then stores Map Objects in a list & then returns list object

→ to make the query as dynamic, we can pass index parameters and the values can be set by creating an Object[] or varargs.

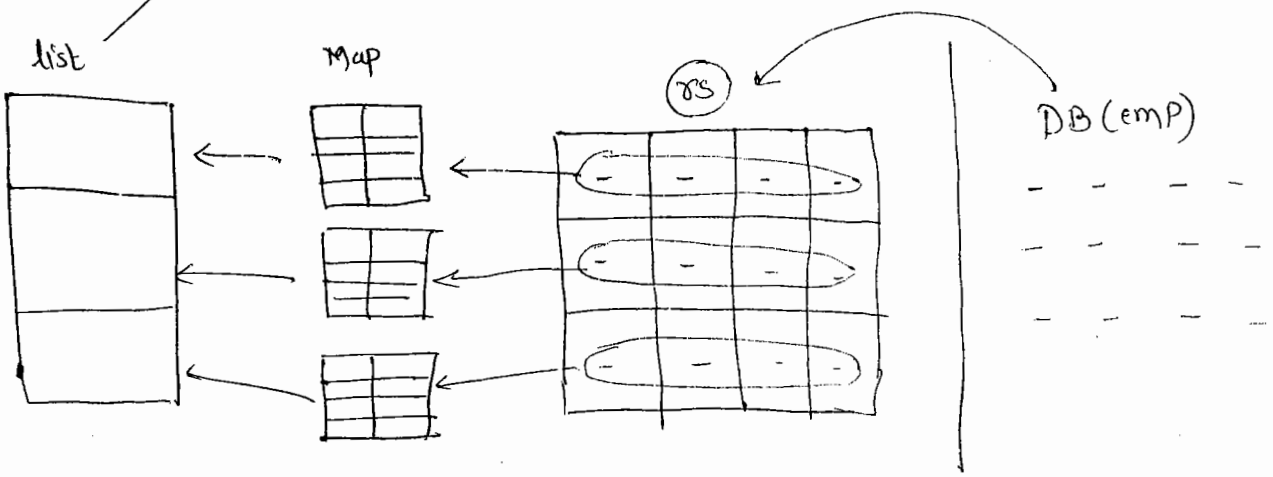
a) queryForList(command)

b) queryForList(command, Object[])

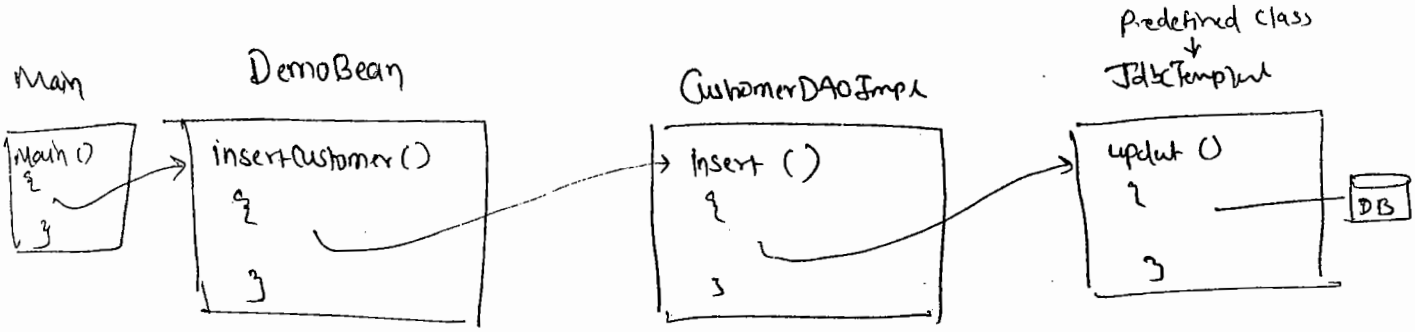
c) queryForList(command, ~~Object...~~) called Varargs

Ex:

```
List list = jt.queryForList("select * from emp");
```




```
Iterator it = list.iterator();
while (it.hasNext())
{
    Map m = (Map) it.next();
    Set s = m.entrySet();
```



DemoBean Depends on CustomerDAO Impl.

CustomerDAOImpl depend on JdbcTemplate

JdbcTemplate depends on DataSource

DataSource
(Interface)

~~DriverManager~~
DriverManagerDataSource
(Implemented class)

Xml

~~business class Configuration~~

~~id = db class DemoBean~~

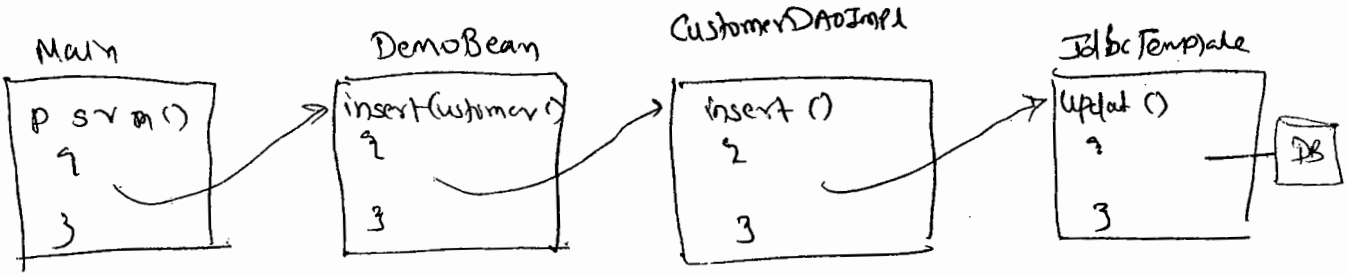
jar

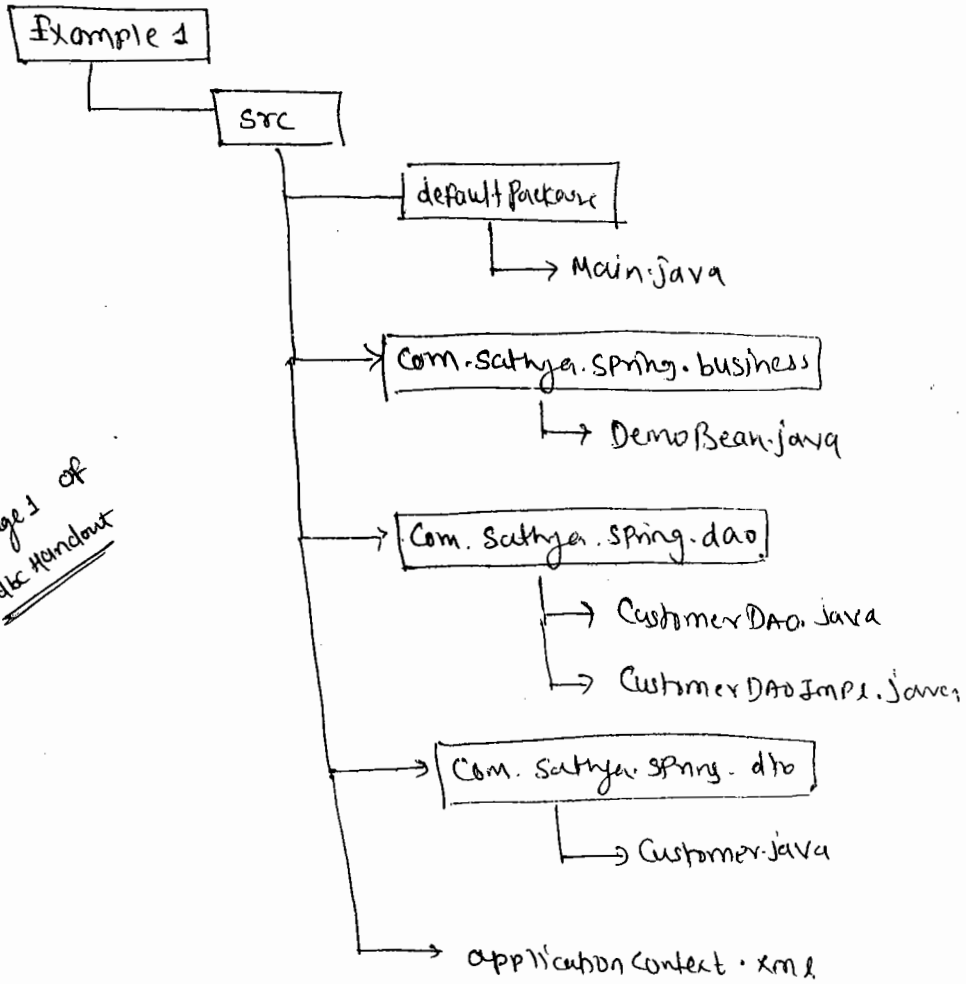
Spring jdbc depends on
spring-tx

11-4-2015

Example)

this example is to insert the values by calling update()
method of JdbcTemplate





Ref Page 1 of Jdbc Handout

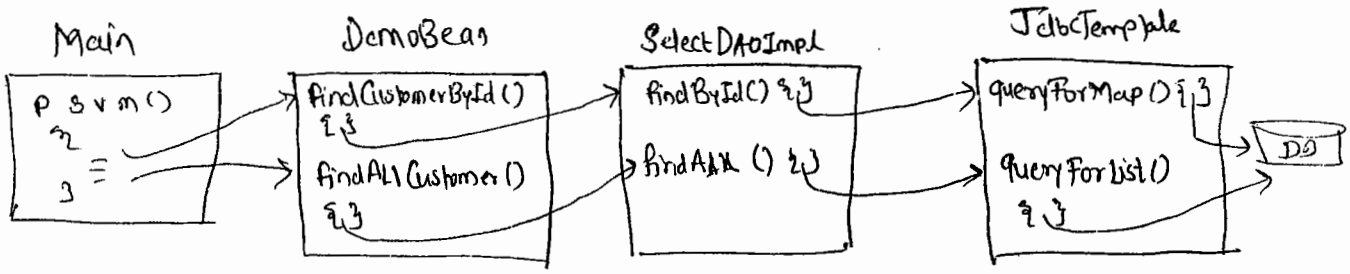
list of Jar

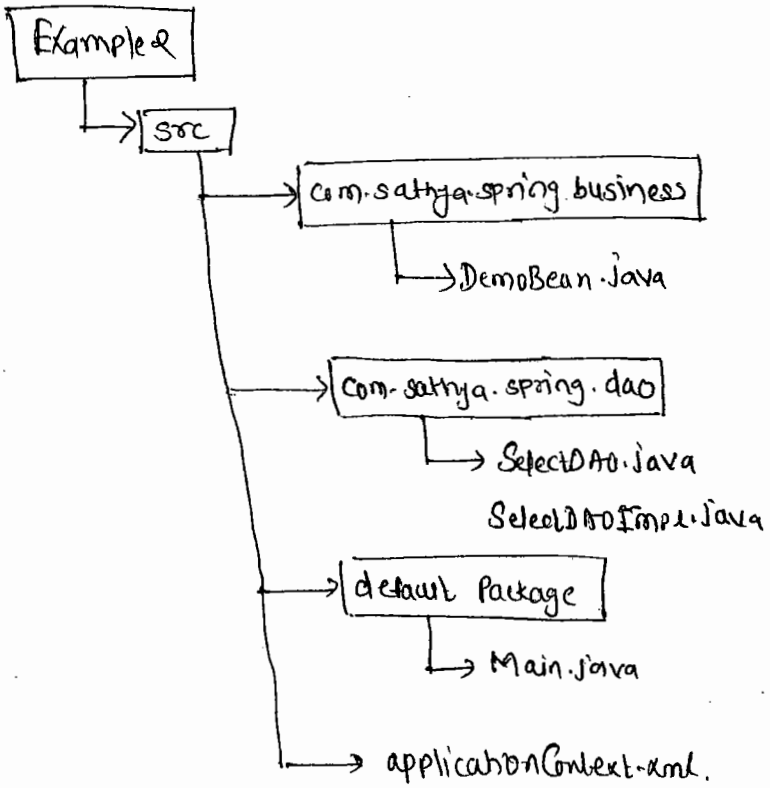
newly added
- Spring tx
- Spring beans
- Spring-Context
- Spring-context-support
- Spring-core
- Spring-expression

newly added
- Spring-Jdbc
- Commons-logging
- mysql-connector / Ojdbc14.

Example 2

the following example is for reading customers by calling query methods of JdbcTemplate





Ref Page.2
Jdbc Handout

List of Jar

commons-logging
mysql-connector
spring tx
spring beans
spring-context
spring-context support
spring-core
spring expression

spring-jdbc
tomcat-dbcp
spring-aop → depends on ~~spring-aop alliance~~ aopalliance

~~queryForInt()~~
~~queryForLong()~~
} ⇒ deprecated in 4.x
(means if we called this method automatically they ~~stricte~~ striked.

Note

queryForInt() and queryForLong() methods are deprecated from Spring 4.x instead we use queryForObject

Q What is callback ~~method~~ interface in JdbcTemplate.

RowMapper is a callback interface.

* Query with RowMapper. *

→ In Spring-Jdbc the framework has provided a feature to the programmer that a programmer can pass some code into predefined methods of JdbcTemplate class.

→ To defined the code a programmer should implement a class from callback interfaces.

→ RowMapper is one of the callback interfaces that a programmer can implement this interface when a programmer want to convert each row of a ResultSet object into a POJO class object.

→ Public class EmployeeMapper implements RowMapper

```
{
    @Override
    Public Object mapRow (ResultSet rs, int index) throws SQLException
    {
        Employee e= new Employee();
        e.setXxx (rs.getXxx());
        return e;
    }
}
```

↳ throws exception because we are using ResultSet object

We pass an Object of EmployeeMapper to the query() method of JdbcTemplate. that query() methods calls ~~Mapper~~ mapRow() method of the class for each row of ResultSet.

Predefined method in JdbcTemplate.

```
query (String sql, RM rm)
{
    Opening connect
    create statement
    Rs rs = stmt.executeQuery(sql);
    while (rs.next())
    {
        Object o = rm.mapRow(rs, i);
        arrayList.add(o);
    }
    return arrayList;
}
```

for each row of the ResultSet it will call MapRow method

→ returns Pojo class Object.

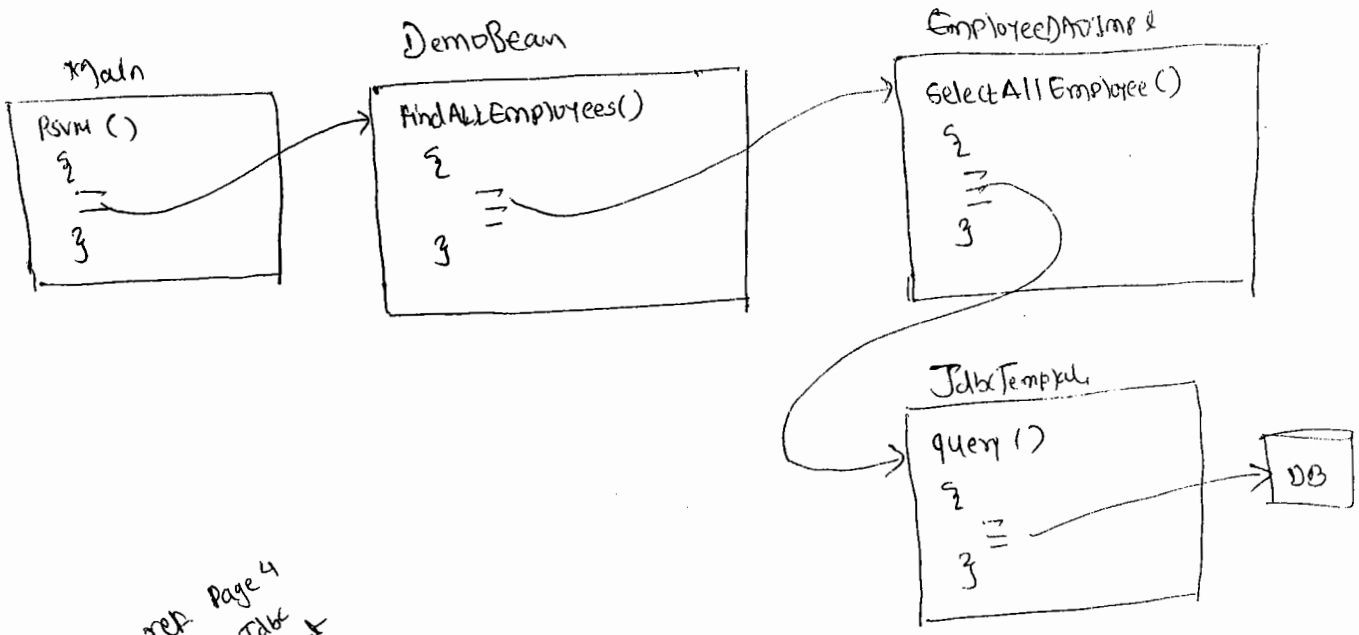
→ query() method JdbcTemplate stores all the Objects in a list and finally returns list Object.

```
List list = jt.query("select * from emp", new EmployeeMapper());
```

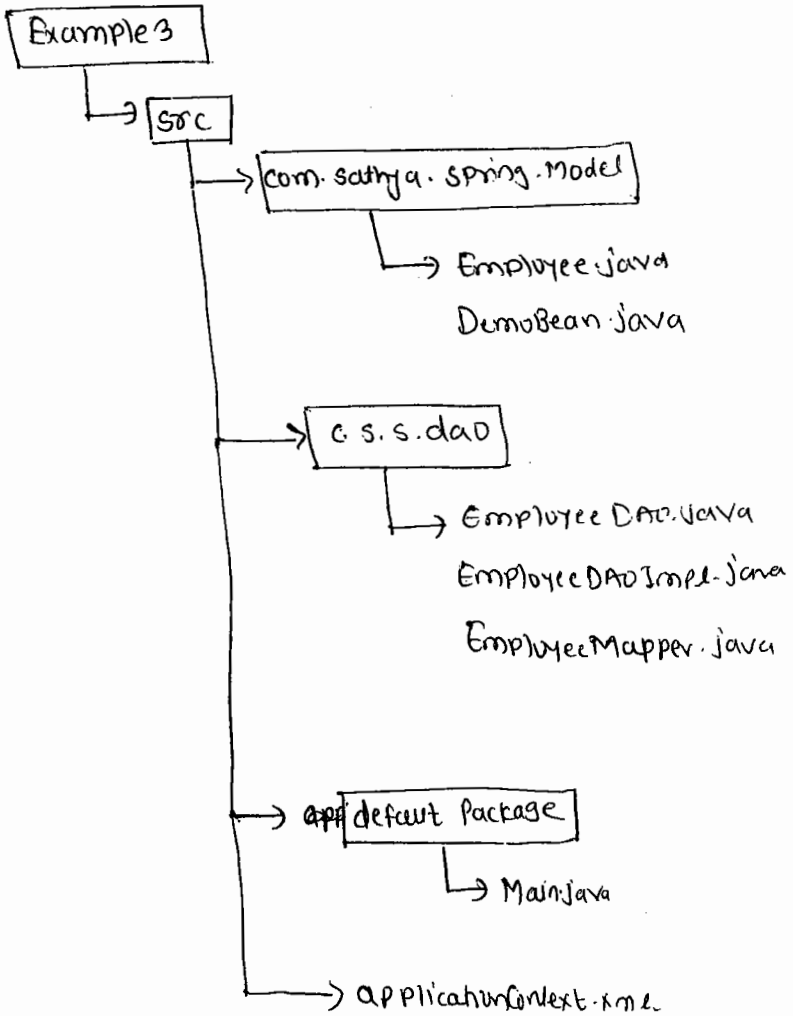
Example 3

→ In the following Example we are converting each row of ResultSet ~~as~~ into an Object of Employee Pojo class, by Implementing our class from a call back interface called RowMapper.

Example 3



ref. Page 4 for Jdbc Handout



VIMP

Q. RowMapper & ResultSetExtractor diff

Query with ResultSetExtractor

Public class

Query with ResultSetExtractor

- ResultSetExtractor is a another call back Interface like a RowMapper.

→ query() method of JdbcTemplate class calls & extractData() method of ResultSetExtractor one time by passing complete ResultSet object

→ In extractData() method a Programmer can decide which type of Collection object is required and also a Programmer can decide what rows of ResultSet need to be converted to the POJO class objects.

→ the difference bet^n RowMapper & ResultSetExtractor is in a RowMapper Collection is a list and a there is a guarantee that for each row of ResultSet a POJO class object is created. but in ResultSetExtractor a Collection type can be decided by Programmer and a Programmer can decide for what rows of ResultSet, POJO class object are required.

```
Public class EmployeeExtractor implements ResultSetExtractor
{
    @Override
    Public Object extractData(ResultSet rs) throws SQLException
    {
        Set s = new HashSet();
        int count = 1;
        While (rs.next())
        {
            If (count==1 || count==10)
            {
                Employee e = new Employee();
                e.setxxx(rs.getxxx());
                s.add(e);
            }
            count++;
        }
        return s;
    }
```

→ according to the above code collection type is decided by the Programmer as Set and only 1st and 10th rows is are converted into Pojo class object.

→ we can call query() method of JdbcTemplate like the following

```
Object o = jt.query("select * from emp", new EmployeeExtractor());
Set s = (Set) o;
Iterator it = s.iterator();
While (it.hasNext())
{
```

```
Employee e = (Employee) it.next();
System.out.println(e.getXxx());
}
```

batch Processing in Spring-Jdbc:

means multiple trips converted into single trips

Statement → 1) addBatch
2) executeBatch()

several command combine called batch

- select command not allowed in batch.

JdbcTemplate → batchUpdate()

→ batch Processing means sending multiple commands to database in one ~~trip~~ trip.

→ In JdbcTemplate class, batchUpdate() method is given for batch Processing.

→ batch Processing can be done by static command & dynamic Commands.

→ for static command, first we need to store the static command in a String[] array and then we need to pass that String[] array to as a Parameter to batchUpdate() method.

```
String str[] = {"insert into student values (1,'a',400)",
                "update emp set sal=9000 where empno=7788",
                "delete from emp where empno = 7901"};
int i[] = jt.batchUpdate(str);
```

12-4-2015

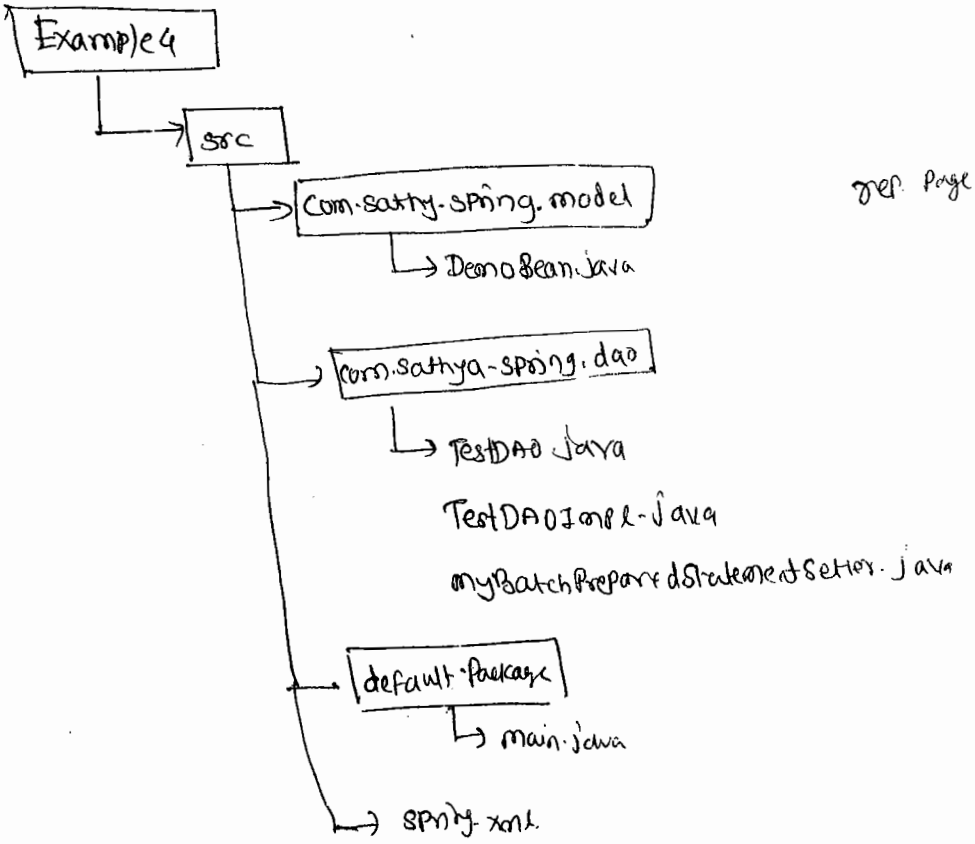
→ for batch processing using a dynamic query, to define the batch size & and also to set the values to the query for multiple times we need to implement our class from a callback interface called BatchPreparedStatementSetter.

→ BatchPreparedStatementSetter interface has two abstract method

① getBatchSize()

② setValues()

→ first getBatchSize() method called & based on the int value returned setValues() method called that many number of time.



Ref Page. 8 of JDBC Handout

Note:→ before executing the application, we need to create table in database like the following.

create table Test(id number(15) primary key, name varchar2(10));

Calling a Procedure or function

→ In Spring Jdbc also we need to create logic using callable statement for calling a procedure or function of database

→ In Jdbc Template class, there is a method called call() for calling procedure or function, it returns a map object. which contains the output values return by the procedure or function

→ In order to create a callable statement object, we need to implement our class from callback interface CallableStatementCreator.

for ex:

```
Public class MyCallableStatementCreator implements CallableStatementCreator
{
  @Override
  Public CallableStatement createCallableStatement (Connection con) throws
  SQLException
    {
      CallableStatement cstmt = con.prepareCall ("{ xxx }");
      ----
      return cstmt;
    }
}
```

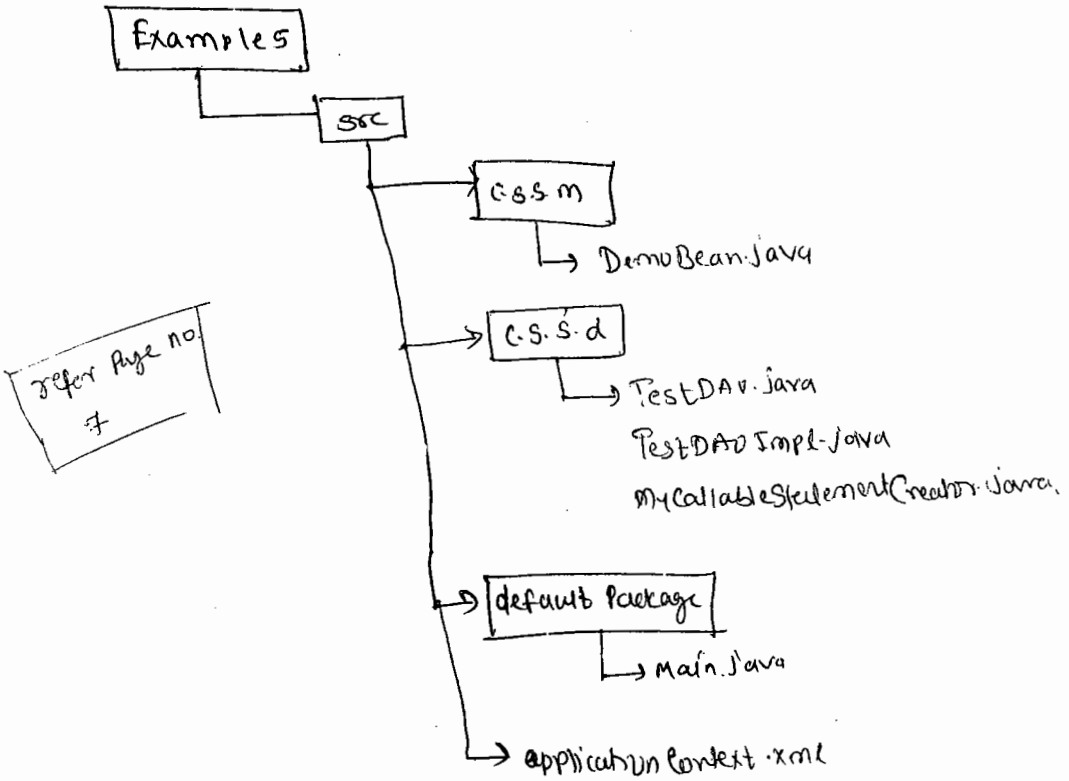
→ for each Parameter used in ~~Repo~~ ~~Callable Statement~~ CallableStatement we need to ~~use SQL~~ create a SqlParameter class object for representing it. we create SQL out Parameter class object for representing OutParameter of an CallableStatement.

→ we need to add SQL Parameter object to the array list & then we need to pass that array list as Parameter to the

call() method of JdbcTemplate.

→ the following Example is going to call a function of database
* using JdbcTemplate

→ A function of database reads i/p as an Employee Number & returns bonus as an output.



refer Page no. 7

→ before executing the above we need create a function in oracle like the following

eg:) ed f1 → enter it will open f1 file

```
create or replace function fun1 (eno number)
return number is
    bonus number(5);
    temp number(6);
begin
    select sal into temp from emp where
    empno=eno;
```

```
if temp between 1 and 2000 then
    bonus := temp * 0.1;
else
    bonus := temp * 0.2;
end if;
return bonus;
end;
```

```
SQL> @f1
IS
/
function created.
```

Spring - ORM

→ this spring - ORM is an abstraction layer for ORM tools like a Hibernate, jdo, & ibatis

→ this spring-ORM layer is introduced for the following 3 reasons.

① to reduce Boiler Plate code used when working with ORM tools.

② to Provide common exception hirarchy to make exception handling as easy.

③ To use an ORM tool as part global transaction.

→ Spring framework has Provided common Exceptions Hierarchy for the multiple data access technology. it means spring throws same exceptions for any data access technology like jdbc or hibernate or jdo ... etc.

→ the ORM tool we used Popularly is Hibernate and the Spring-ORM has Provided a template class for reducing Boiler Plate or code of hibernate as "HibernateTemplate".

HibernateTemplate starts with Session

depends on ↓

LocalSessionFactoryBean → Session factory object

→ HibernateTemplate class need a SessionFactory and the it will be produced by LocalSessionfactoryBean

→ LocalSessionFactoryBean is a factoryBean which Produces Sessionfactory of Hibernate.

→ Spring container injects Sessionfactory Produced by LocalSessionFactoryBean to the HibernateTemplate

```
<bean id = "ht" class = "HibernateTemplate" >
    <property name = "sessionFactory" ref = "lsfb" />
</bean>
<bean id = "lsfb" class = "LocalSessionfactoryBean" >
    ---
</bean>
```

Spring- Services Module

21-4-2015

⑤ Scheduling Service

24-4-2015

@Schedule. Annotation

```
Public class xxa
{
   @Scheduled (xxxxx)
   Public void doWork()
   {
      =
      =
   }
}
```

fixedRate / fixedDelay / cron

@Scheduled (fixedRate = 5000) → for every 5 sec doWork() method called

```
Public void doWork()
{
   =
   =
}
```

@Scheduled (fixedDelay = 5000)

```
Public void doWork()
{
   Thread.sleep(10000);
}
```

Previous Task then 5 sec Delay next task complete

→ Scheduling is a service used for automatic a job.

→ When we want to execute a job (work) automatically by without explicity calling then we need to apply this scheduling

→ a scheduler will automatically invoke a method in which the logic is defined, at a specific point of time.

→ Ex In a mobile we can scheduled an alaram and the it will executed automatically according to the schedule.

→ In Spring frameworks to defined the schedule we have a method level annotation @Scheduled.

→ @Scheduled annotation can have one of the following three elements

① fixedRate.
② fixedDelay
③ cron.

→ FixedRate is a time interval between the two ~~invocation~~ invocations of a method.

→ for Ex:

```
@Scheduled(fixedRate = 5000)
Public void doWorK()
{
    =
}
```

→ doWorK() method called by scheduler automatically for every 5 seconds.

→ fixedDelay is a fixed time period between completion of last invokation to the start of the next invocation of the job

Main.java

WorkerBean.java

```
import org.spfw.scheduling.annotation.Scheduled;

@Service
class ()
{
  @Scheduled
  method
  {
  }
}
```

applicationContext.xml

↓

task schema.

task: annotation-driven.

cron = "cron expression"

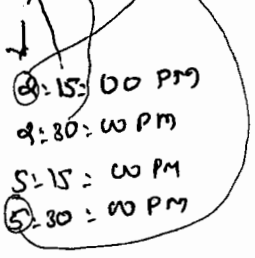
↓

contain 6 parts separated with whitespace.

cron = " 0 0 16 1 * ? " → 4:0:0 PM

cron = " 0 0 16 1 JAN-APR ? " → 4:0:0 PM

cron = " 0 15,30 14,17 * * ? "



cron = " 0 0 18 ? * FRI "

↙ month

6:0:0 PM

FRIL → Last friday of month

FRI #2 → 2nd friday of month

cron = " 0 0 0 1 JAN ? 2016 "

0:0:0 AM

Cron

→ to define a schedule for a job efficiently we need to pass cron element to the @Schedule annotation.

→ to the cron element we need to pass a cron expression as a value.

→ a cron ~~element~~ expression contains 7 parts & each part will be separated with white-space.

Cron = "cron expression"

1. Seconds (0-59)
2. Minutes (0-59)
3. hours (0-23)
4. Day (1-31)
5. Month (1-12) (JAN-DEC)
6. day of Week (1-7 | SUN-SAT)
7. year (optional)

Ex:1

→ Cron = " 0 0 16 1 JAN-APR ? "

4:0:0 PM

Scheduler will run the Job 4PM on 1st of the Month from JAN to APR.

Ex:2 cron = " 0 15,30 14,17 * * ? "

2:15:00 PM
2:30:00 PM
5:15:00 PM
5:30:00 PM.

→ Scheduler will run the job at the above ~~before~~ timers. on every day.

Main.java

```
{
ApplicationContext ctx = new ClassPathXmlApplicationContext ("applicationContext.xml");
}
```

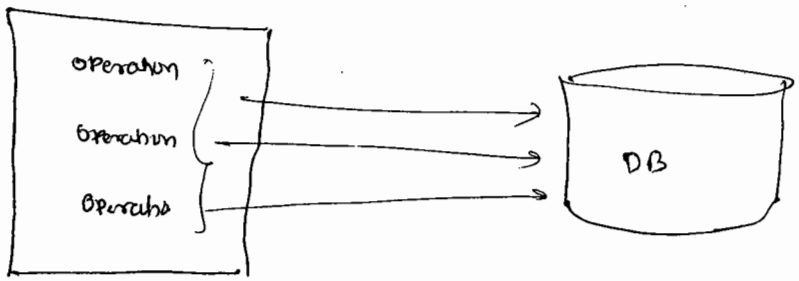
jar

① aopalliance-1.0.jar
② commons-logging
③ spring-aop
④ spring-beans
⑤ spring-context
⑥ spring-context support
⑦ spring-core
⑧ spring-expression
⑨ spring-tx

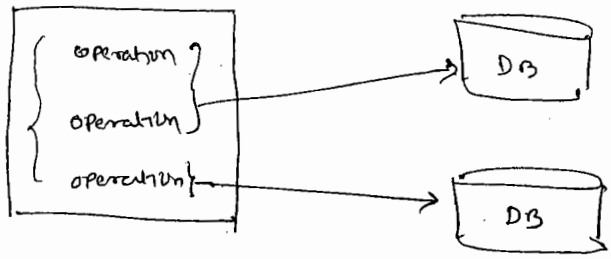
25-4-2014

Transaction Management

① local transaction
② global | distributed | XA transaction.



local — one database used for transaction.



global transaction.

more than one DB used for transaction.

transaction bet^n same bank is local transaction. One ~~el~~ database. used.

transaction bet^n two diff. bank is global transaction as ~~diff~~ more than one DB

→ A transaction is a unit of a coork

→ To complete a unit of work it may have one or more operations.

→ we can define a transaction as a group of operations that are used to perform a unit of a work.

→ Transaction may reach to either a success or a failure point.

→ If all operations are executed successfully then a transaction will become a success.

→ if any operation is fail then a transaction will become fail.

→ If a transaction reached to success then operations are committed on the database. and the if it is reach to failure. then the operations are rolled back on database.

→ In Java we can execute two types of transaction.

① Local transaction

② global | distributed | XA transaction.

→ a local transaction means, a one database will be used for executing the operation of a transaction.

→ In a Global transaction, more than one database will be used for executing the operations of ~~the~~ a transaction.

→ In Java with Jdbc API & the hibernate API we can perform only local transaction.

→ with EJB API & Spring, we can perform both local and the global transaction.

Q Why Jdbc does not support a global transaction?

→ In Jdbc, to manage a transaction, the methods are given in Connection interface.

→ To execute a global transaction, in a Jdbc Application multiple Connections are required.

→ if one connection committed, and while committing another connection, if any Exception occurs then already committed connection cannot rolledback its operations.

→ so a Transactions is not going to follow its authencity property.

Ex:

```
Connection con1 = ......;
Connection con2 = ......;

Con1.SetAutoCommit(false);
Con2.SetAutoCommit(false);

Statement stmt1 = con1.createStatement();
Statement stmt2 = con2.createStatement();
try
 {
    stmt1.xxxx("......");
    stmt2.xxxx("......");
```

```
        con1.commit();
        con2.commit();
    }
    catch (Exception e)
    {
        con1.rollback();
        con2.rollback();
    }
```

→ In the above code if an Exception is occured while commiting the con2, then control enters into catch block but con1 cannot be rollback because it is already committed. So with Jdbc it is not possible to run global transaction.

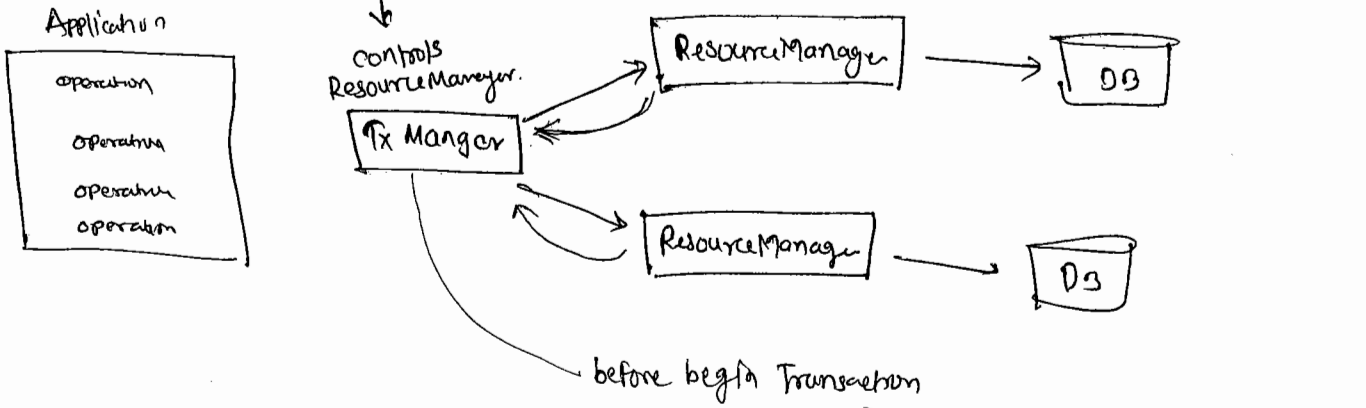
JEE (JEE)

JTA (Java Transaction API)

UserTransaction (i)

Transactionmanager.

every dB comes with ResourceManager



before begin Transaction

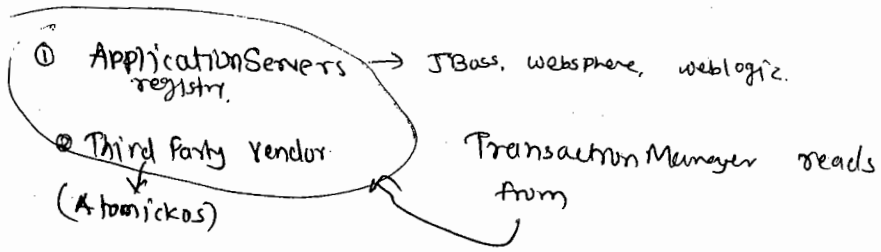
2-pc → 2-Phase Commit

① Propogation
② Isolation
③ Timeout

} Transaction Attribute will set behaviour of transaction

26-4-2015

How Global transactions work ?



→ To execute Global Transaction, as part of J2EE SUN microsystem has provided JTA (Java transaction Api)

→ As part of JTA, an interface is given called UserTransaction. an object of its implementation class is called a TransactionManager.

→ implementation for JTA will be provided by Application Servers & Third Party vendors.

→ A TransactionManager manages one or more ResourceManagers, to execute a the Global Transaction.

→ every database will have its own ResourceManager & the a TransactionManager controls ResourceManagers.

→ the below are the steps that are executed, in a Global Transaction

① an Application read the TransactionManager object or (TransactionManager Object) from JNDI registry of application server or an application creates an object of implementation class of UserTransaction.

② When a Transaction begin then TransactionManager tells ResourceManagers to execute the operations temporary on data base.

③ When application asks a TransactionManager to commit then TransactionManager will ask a ResourceManagers Wheather they are

ready to commit or not.

④ if ResourceManager returns "yes" then TransactionManager will issue Commit to the ResourceManager. so that ResourceManager will commit the operations.

⑤ if ResourceManager returns "no" then TransactionManager issues Rollback to the ResourceManagers.

Note

→ In Java all application servers comes with a Global Transaction Manager

→ when application server started then the server will create TransactionManager object and then it will store the object in registry with key as "java:comp/userTransaction".

→ In a java Application, before a transaction begins the application should read TransactionManager object from JNDI Registry, by calling lookup() method.

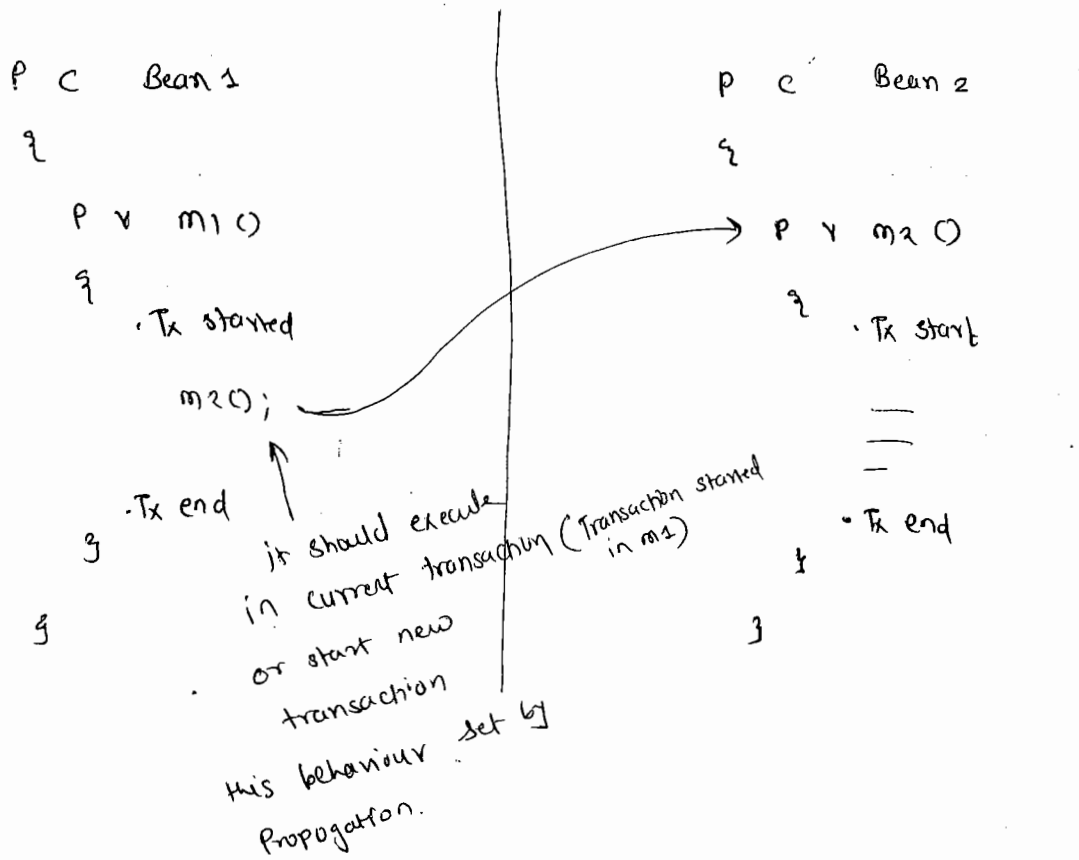
* Transaction Attribute

→ In a Global Transactions, a Program instructs TransactionManager to begin transaction but a program cannot directly begin a transaction.

→ before a transaction begins we need to define the behaviour of a transaction in the form of Transaction Attributes.

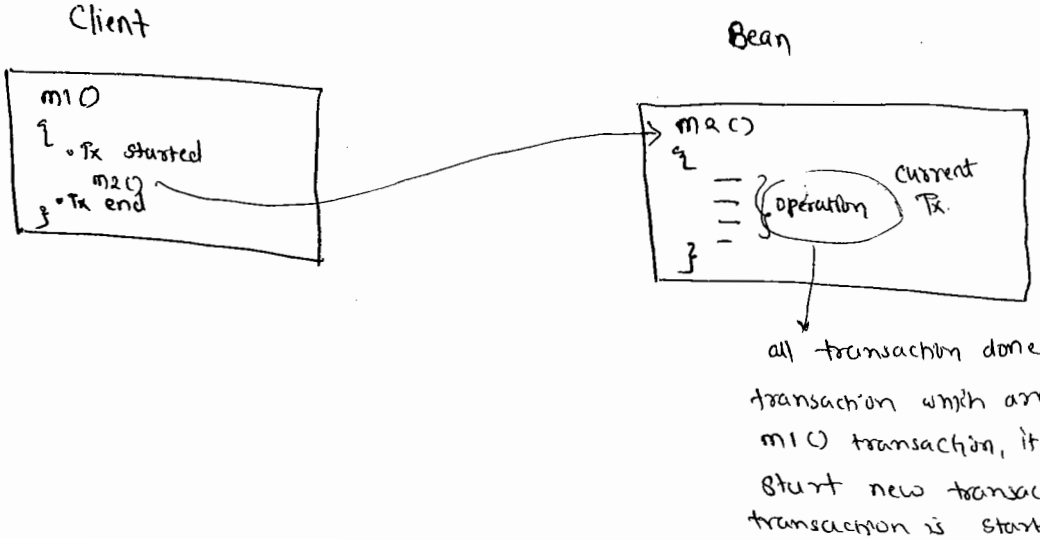
→ Transaction Attributes are.

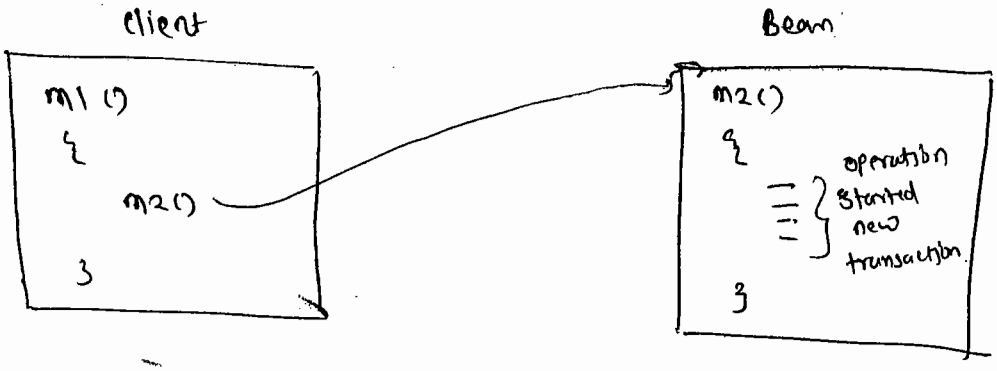
① Propogation
② Isolation
③ timeout



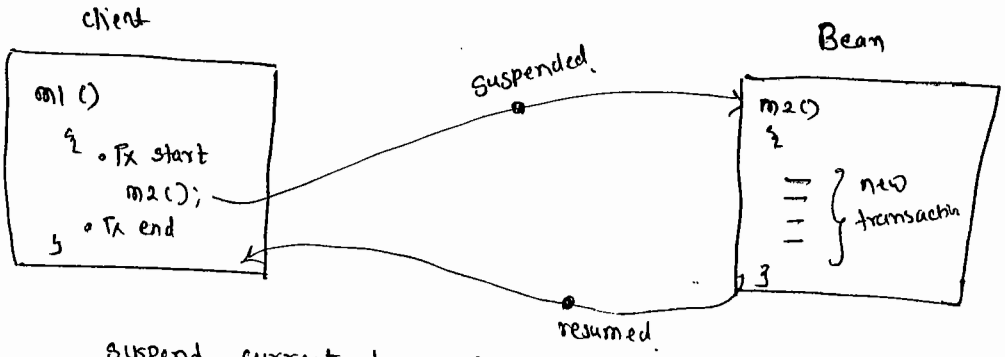
Propogation:

① required:

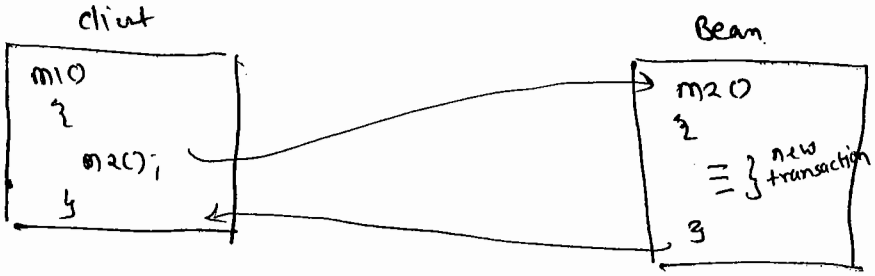




(2) **requires new:**



Suspend current transaction & started new transaction & after that current transaction resumed.



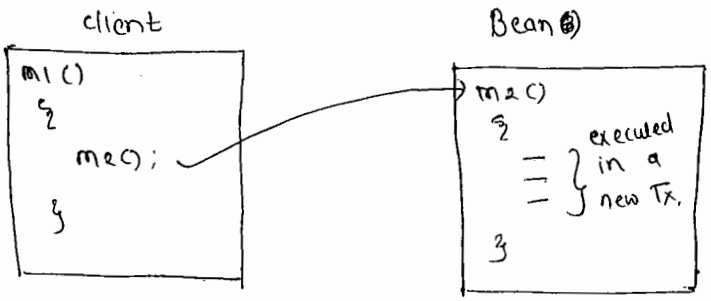
27-4-2015

① Required:

→ a Transaction manager finds wheather a transaction started at client or not, if started then a transactional method of bean also executed in same transaction.

→ if there is no transaction started at client then Transaction manager begin a new transaction for the transaction method.

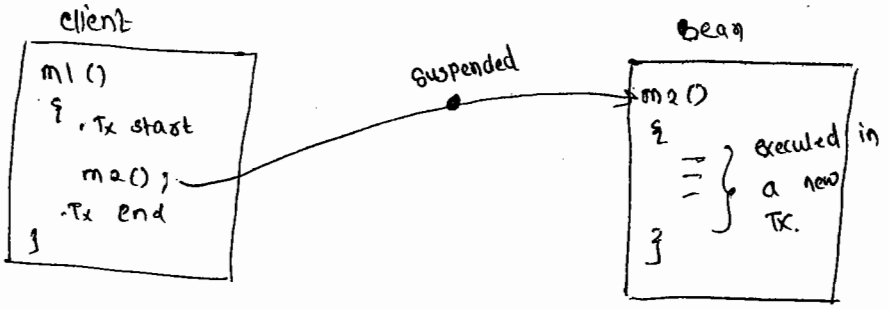


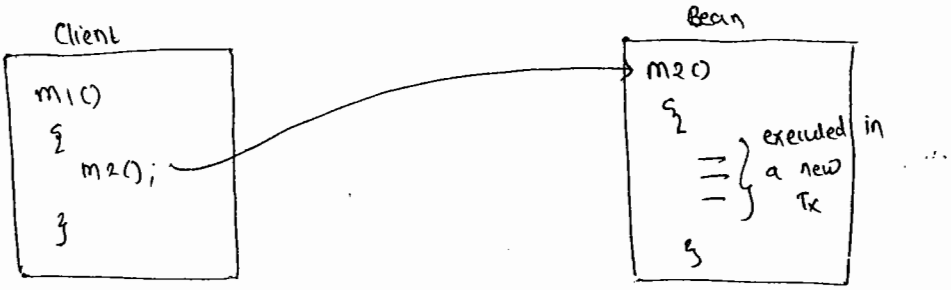


② requires new:

→ a transaction manager verifies whether transaction started at client side or not, if started then it suspends it & starts a new transaction for the transactional method of Bean.

→ if there is no transaction started in client then a transaction manager begins a new transaction for the transactional method of Bean.

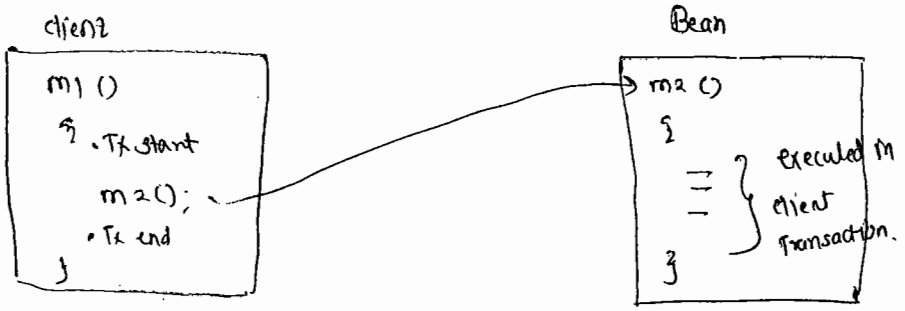


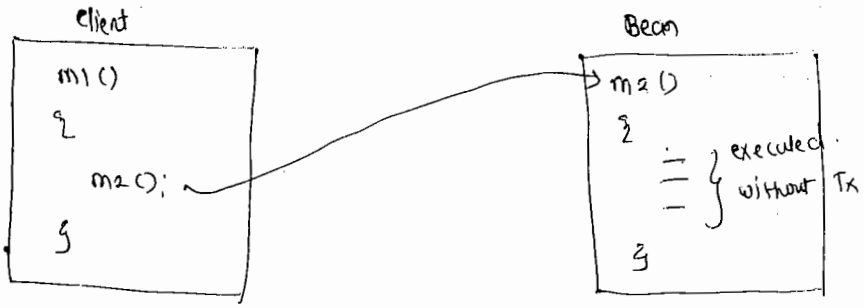


(3) supports:

→ a transaction manager verifies whether at client side a transaction is started or not, if started then transactional method of Bean also executed on the client transaction.

→ if no Transaction started at client then a transactional method of a Bean also executed without a transaction.

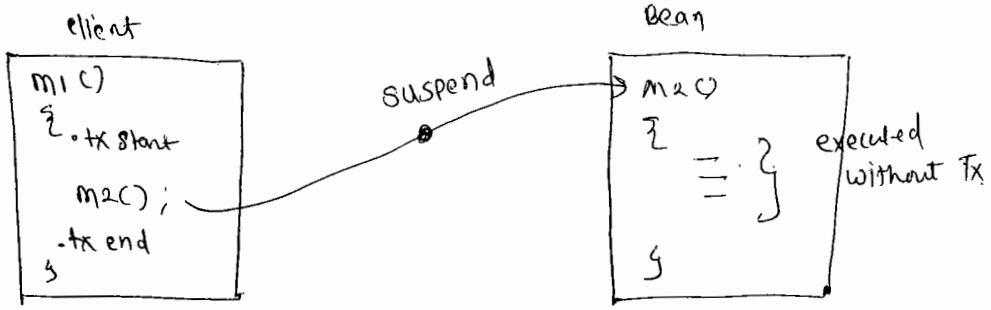


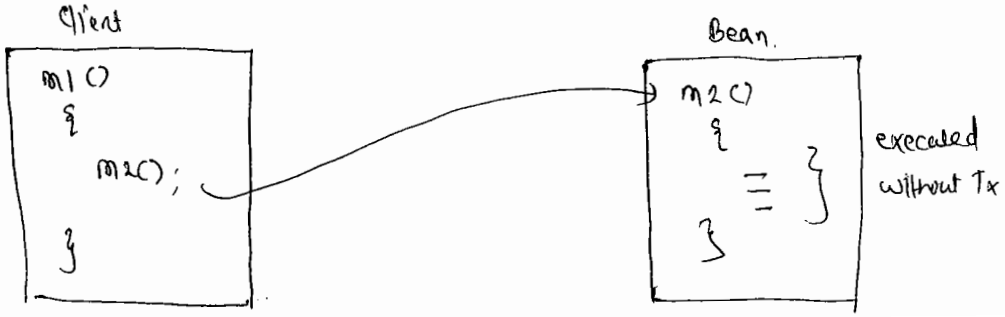


④ not Supported

→ a transaction manager verifies wheather a transaction started at client side or not, if started then suspends it and execules transactional method of a Bean without a transaction.

→ If there is no transaction at client side then a transactional method of a Bean also executed without a transaction.

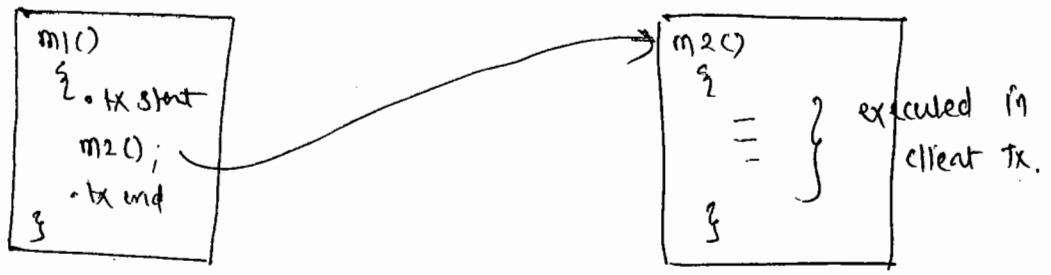


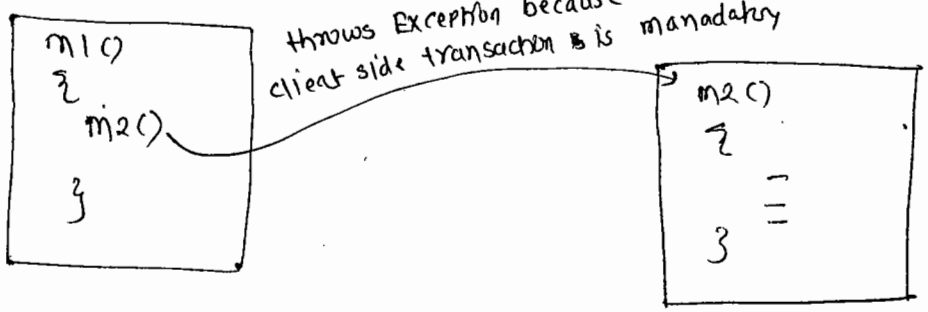


⑤ manadatory:

→ a transaction manager verifies wheather a transaction started at client side or not, if started then executes a transachonal method also in current transaction. or same transachon.

→ if no transachon started at client side then throws an excepton, because at client side transachon is manadatory.

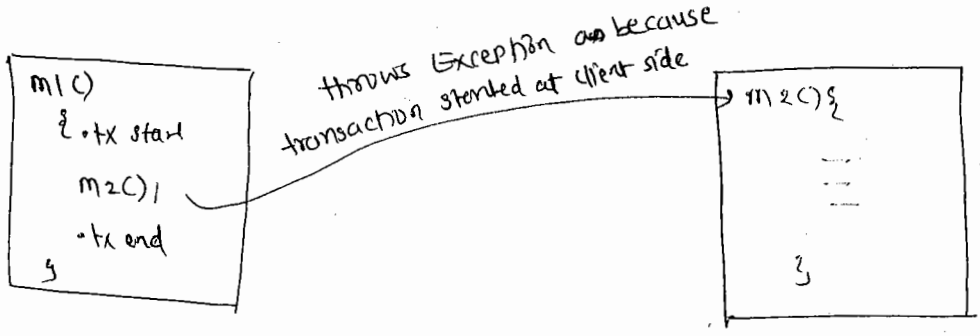


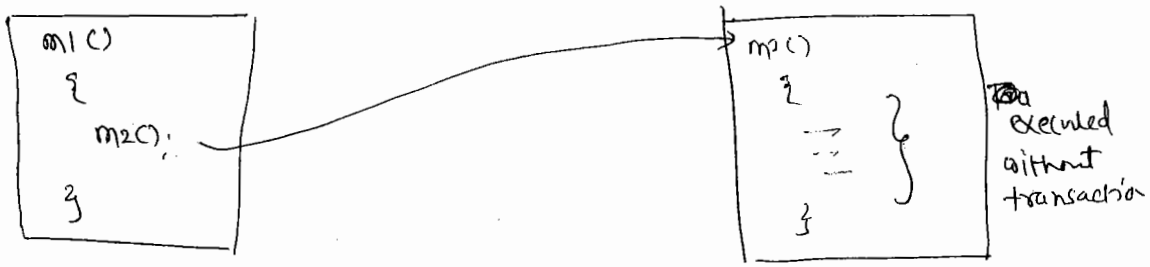


⑥ never:

→ a transachon manager verifies wheather a transachon started at client side or not if started then throws an execeptiom

→ if no transaction started at client side then a transactional method of Bean also executed without a transachon.

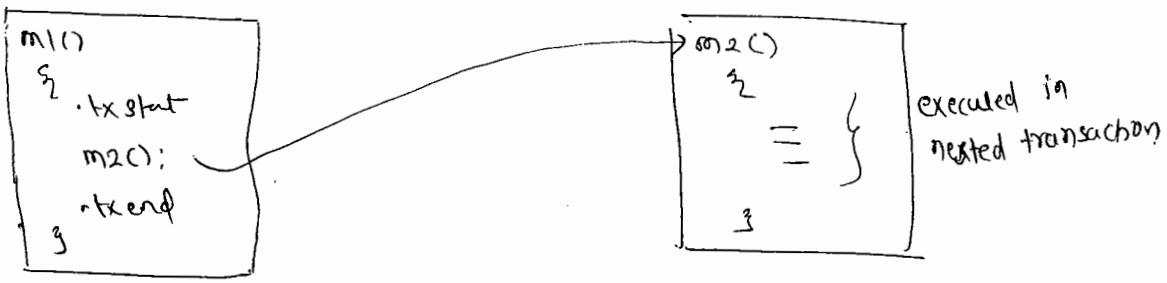


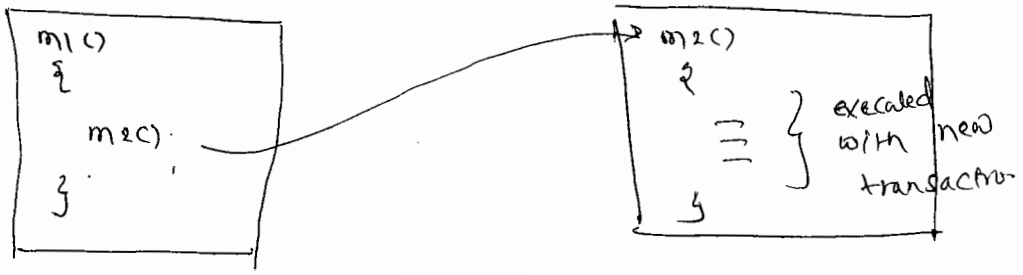


⑦ nested

→ a transaction manager verifies wheather a transaction started at client side or not, if started then a transaction manager starts a nested transaction for the transactional method.

→ If no transaction started at client, then a new transaction will be started for the transactional method of a bean by transactional manager.





Concurrency Problems in Transactions:

USERS

uid	uname	age
1	A	30
2	B	28
3	C	25

dirty read Problem

Tx 1

⇒ started

—
—
—
—

Select age from USERS where uid = 1 ;

// age = 32

—
—

Tx 2

⇒ started

—
—

Update USERS set age=32 where uid = 1;

—
—
—
—
—

Roll back

Concurrency Problems in Transactions:

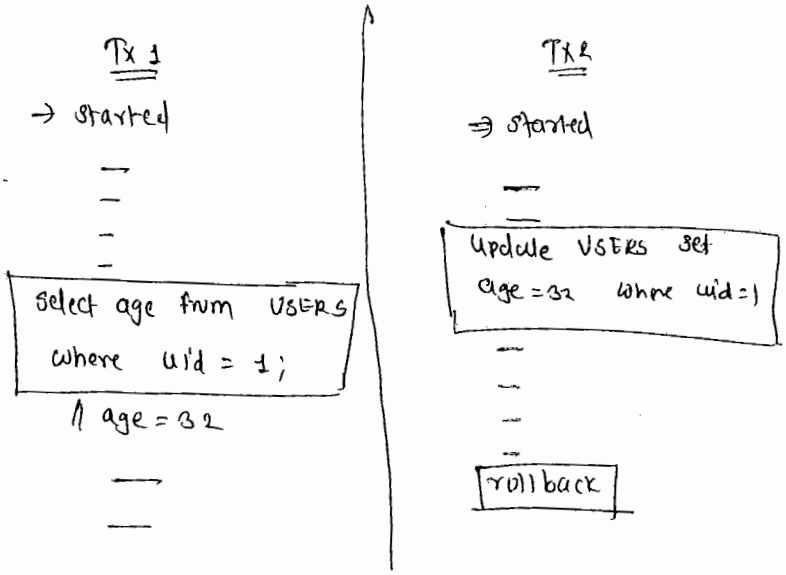
→ by default changes made on data by one transaction will be visible to other transaction when both are concurrently working or executing on the same data.

→ the Problems that are going to occur, when multiple transaction ~~work~~ are working simultaneously on the same data of the same table are called concurrency Problem.

① **Dirty read Problem:**

USERS

uid	uname	age
1	A	30
2	B	28
3	C	25



→ In the above Tx2 updated age as 32 and after that Tx1 has read.

→ later, tx2 has rollback so data read by tx1 is not a valid data its a wrong data. this is dirty read Problem.

→ dirty read Problem occurs, when transaction is reading a data which is updated by another transaction before the another transaction is committed.

→ if another transaction is rollback then the data read by first transaction becomes dirty data.

28-4-2015

② Non-repeatable read

→ this concurrency problem occurs in transactions, when a transaction is reading a same data for multiple times and the if any changes are observed in data from previous read to the current read.

Tx 1	Tx 2
→ started	→ started.
`select age from USER where uid = 1;`	—
// age = 30	—
—	—
—	`Update USERS set age = 32 where uid = 1;`
—	`commit`
`select age from USERS Where uid = 1`	—
// age = 32	—
	—

→ In the above, Tx1 has selected age for twice and the changes are observed in age from 1st read to 2nd read so it is called a non-repeatable read problem.

③ Phantom read Problem.

→ This Concurrency Problem occurs when a transaction is reading a data within a range for two times. and if any extra rows are identified at the 2nd time than 1st time.

→ the extra rows identified are called Phantom rows.

Tx1	Tx2
→ Started	→ Started
Select * from USERS where age between 20 and 28;	—
// 2 rows	—
—	—
—	—
—	insert into USERS values (4, 'D', 26);
Select * from USERS where age between 20 and 28;	Commit
// 3 rows	—
	—
	—

→ In the above, tx1 is selecting the rows for 2 times betn age 20 & 28, and the 2nd time 1 extra row is identified this is happen because another concurrent transaction tx2 inserted a row matching the condition.

→ Here the Problem is called a Phantom read Problem.

Isolation level

Isolation level	dirty read	non-repeatable read	Phantom read
read uncommitted	✓	✓	✓
read committed	×	✓	✓
repeatable read	×	×	✓
Serializable	×	×	×

→ transaction concurrency Problem are resolved by setting an Isolation level to a transaction.

→ In Java there are 4 Isolation level defined

① read uncommitted (default)
② read committed
③ repeatable read
④ Serializable

→ read uncommitted Isolation level allows all the 3 concurrency Problem in a transaction.

→ read committed Prevents dirty read Problem, but allows non-repeatable problem & Phantom read Problem.

→ repeatable reads ~~Problem~~ Prevents ~~direct~~ dirty read & non-repeatable read, but allows Phantom read Problem.

→ In case of repeatable read a transaction will lock the row while selecting the data, so other transaction can not update or delete that row.

→ Serializable Prevents all concurrency problem.

→ In case of Serializable, a transaction will lock the table while selecting the data, so other transactions can not insert the any rows to the table, until the lock is released.

* Spring Transaction Management

→ In spring fw a transaction manager is required for both local and global transaction.

→ the framework has given multiple predefined transaction manager classes & we need to configure a suitable transaction manager class in a configuration file based on the persistance technology used in the transaction.

① jdbc → DataSourceTransactionManager

② hibernate → HibernateTransactionManager

③ jdo → JdoTransactionManager.

④ jta → JtaTransactionManager.

→ In spring framework a transaction service can be add to a method in 2 ways.

① declaratively (using xml configuration)

② using Annotations.

Declarative Approach

→ Spring framework has provided a special advice called transaction advice to configured the transaction attributes.

→ Aop configuration is used for adding transaction advice to a methods of Bean, by identifing the method through a pointcut

29-4-2015

Spring transaction.

① local
② global - } Transaction Manager.

① Declarative → <tx: advice>
↳ Aop configuration
② annotation
↓
@Transactional.

→ ~~Spring for~~ In Declarative Approach a transaction advice can be added to business method through Aop configuration.

→ ↳ Next Page

```
<tx: advice id = "advice1" transaction-manager = "txm"
                                  ↳ id TransactionManager class
  <tx: attributes>
    <tx: method name = "method1"
                propagation = "REQUIRES_NEW"
                isolation = "READ_COMMITTED"
                timeout = "18"/>
  </tx: attributes>
</tx: advice>

<aop: config>
    <aop: pointcut id = "pointcut1"
                expression = "execution (* com.sathy.spring.transaction.*.*
                                (..))"/>
    <aop: advisor advice-ref = "advice1"
                pointcut-ref = "pointcut1" />
</aop: config>
</beans>
```

Annotation Approach

→ In order to reduce the xml configuration in spring configuration file, the framework has given @Transactional annotation

→ @Transactional is a method level annotation.

→ this annotation has 3 elements to define the Transaction attribute

① ~~Propagation~~ ① Propagation
② ~~Isolation~~ ② isolation
③ timeout.

→ Propogation value defined in an Enum called Propogation.

→ * isolation value defined in an enum called Isolation.

→ ~~timeout value defined in xxx enum called~~

@Transactional (Propogation = Propogation.REQUIRED,
Isolation = Isolation.SERILIZABLE,
timeout = 5)

```
Public void method1 ()
{
  —
  =
  —
}
```

30-4-2015

JTA Transaction Manager
↓ Depends
UserTransactionManager (atomikos)

```
DataSource ds = new xxxx ();
ds.setXxx ();
Connection con = ds.getConnection ();
```

```
ConnectionPoolDataSource cpds = new xxxx ();
cpds.setXxx ();
PooledConnection pc = cpds.getPooledConnection ();
```

Global Transaction In Spring

→ a Global Transaction means multiple resources are involved in a transaction's

→ In spring fw, JtaTransactionManager class is given to manage the distributed transaction.

→ JtaTransactionManager depends on a global transaction manager Provided by Appln server or a third party to manage a global transaction.

→ Atomikos is a 3rd party which provided implementation for Jta.

→ Atomikos has given UserTransactionManager, it is an implementation class of UserTransaction Interface as a global Transaction manager. so if Atomikos API is used then Jta transaction manager of spring depends on UserTransactionManager of Atomikos to manage a global transaction.

→ a Connection i.e open by using an implementation class of XADataSource interface can only Partipate in global transaction.

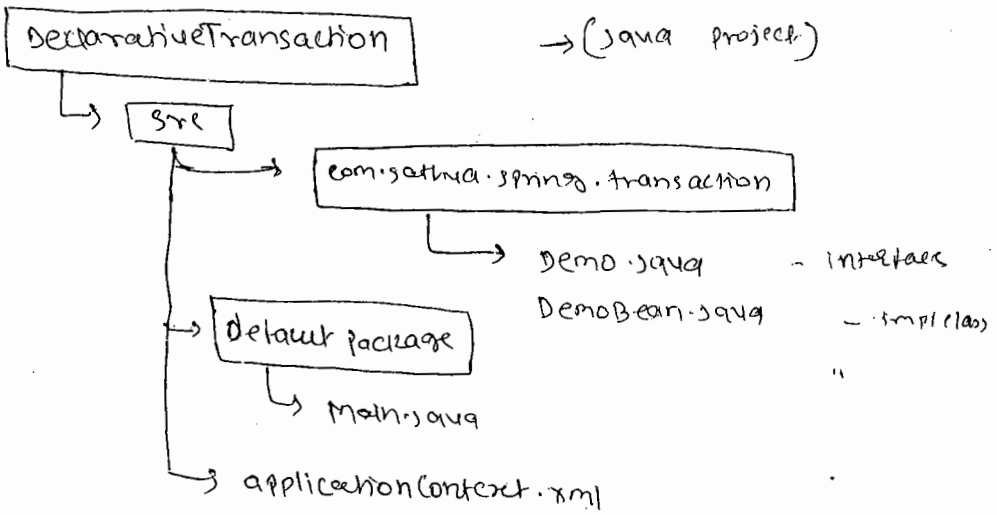
→ Atomikos API has given AtomikosDataSourceBean, which uses implementation class of XADataSource, Creates a connection with database & stores them in a Connection Pool.

→ when developing global-transaction Appln in spring, JdbcTemplate of Spring reads a connection from a connection Pool created by AtomikosDataSourceBean.

11/5/15

* Directory structure for handout of spring-service [Local Transaction]

① Declarative Transaction [Page 13].

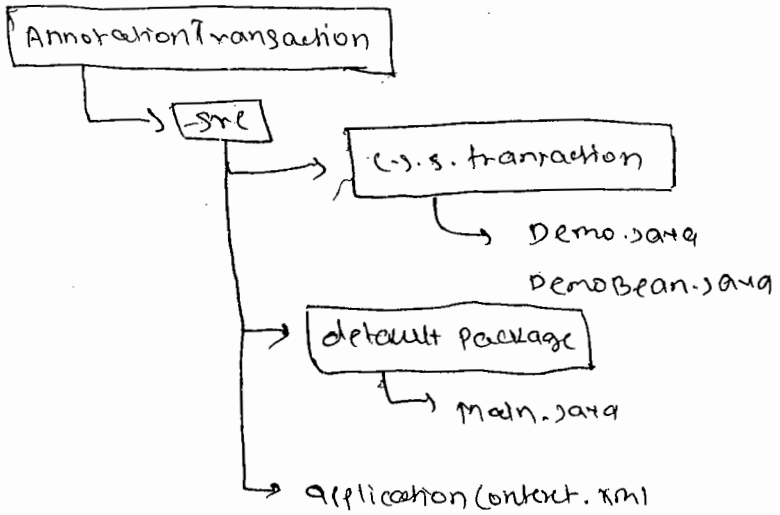


* jar's LOT

① odbc14.jar ② spring-aop-3.2.4 ③ spring-beans-3.2.4 ④ spring-context ⑤ s-context-support ⑥ s-core ⑦ s-expression-3.2.4 ⑧ spring-~~jdb~~ jdbc 3.2.4 ⑨ spring-tx-3.2.4 ⑩ aspectjweaver-1.6.2 ⑪ aspectjrt-1.6.9 ⑫ commons-logging-1.0.4 ⑬ aopalliance-1.0.jar.

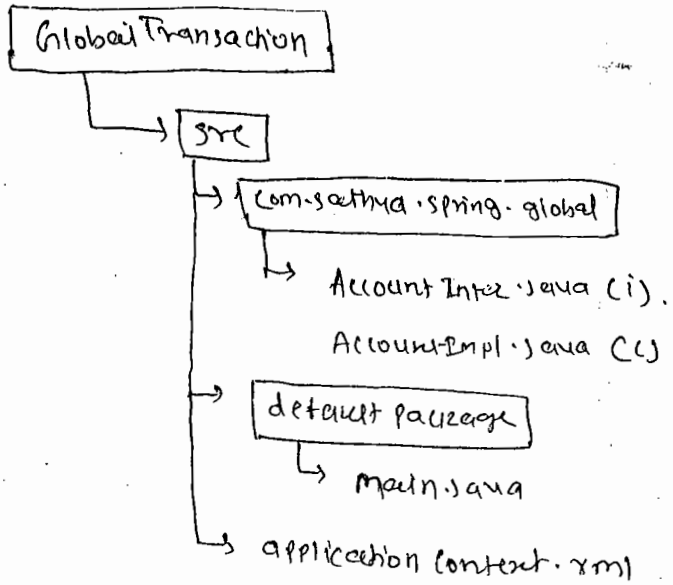
⑪ Annotation Transaction

// pre-req spring-services handout. ⑪.



* follow the same jar of above ex ①.

(11) Global Transaction application, refer page (11) ex: (11)



jar's (buildpath)

1. commons-logging-1.0.4.
2. spring-aop-3.2.4
3. aspectjrt-1.6.9
4. aspectjweaver-1.6.2.
5. ojdbc14.jar
6. transaction-jsr250-1.0
7. automikos-transaction-api.jar
8. automikos-util.jar
9. transaction-3.0.0
10. jta.jar
11. spring-tx-3.2.4.
12. s-bean
13. s-context
14. s-c-support
15. spring-core
16. spring-expression-3.2.4.
17. spring-jdbc-3.2.4
18. aopalliance 1.0.jar
19. mysql-connector-java-5-1.0-bin.jar
20. automikos-transaction-jta.jar

Note:- Before executing global transaction application we need to create account1 table in oracle and account2 table in mysql as ready.

(i) In Oracle d/b

SQL> create table Account1 (accno number(5) primary key, bal number(5));
SQL> insert into Account1 values (101, 5000);
SQL> commit;

(ii) In mysql DB

SQL> create table Account2 (accno int primary key, bal int);
SQL> insert into Account2 values (102, 1000);
SQL> commit;

11/5/15

* Spring mail :-

As a part of J2EE, sun-microsystem provided a service api called java mail api for sending an e-mail from a java application and for reading a mail from a java application.

→ Java mail api consists a set of interfaces and classes and they are given under two packages..

① javax.mail

② javax.mail.internet

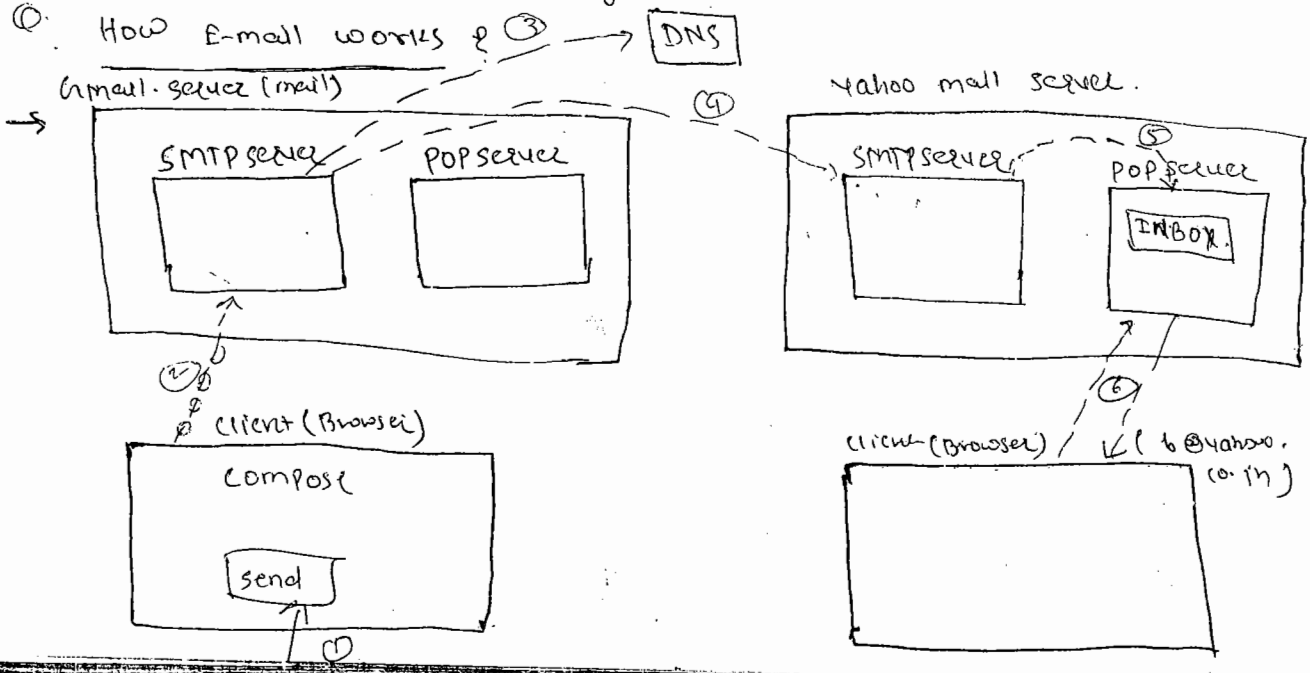
→ Using java mail api, to send an e-mail or to read a mail, there is a need to write boiler plate code.

→ Spring provided an abstraction layer to avoid the boilerplate code and given spring mail api.

→ Spring mail api is only given for sending an e-mail from a spring application, but not for reading an e-mail.

→ When developing java projects, there is a need to send the mail to a customer mail account after processing the request. But there is no need for an api to read a mail. Because a customer can read the mail directly through browser.

ⓒ How E-mail works ?



(1) A user compose the mail and then click as on send button.

(ii) A mail will be created as a message object and then it will be send to SMTP server (outgoing).

(iii) SMTP server checks wheater reciver as mail account belongs to same mail server or another. If another then finds the location throw DNS. (Domain Name server).

(4) SMTP server will send the message to SMTP server of other mail server.

(5) SMTP server will send the message to pop server [incoming server] pop server stores message like a file in INBOX folder.

(6) A reciver connects with POP server and reads a mail from inbox.

2/5/15

→ In spring mail api, a class JavaMailSenderImpl is given to send an email. so, our spring bean class depends on JavaMail -SenderImpl class.

→ JavaMailSenderImpl class has 4 properties --

(i) Host (ii) username (iii) password (iv) JavaMailProperties.

→ In spring configuration file we need to configure JavaMailSend -er Impl class within its properties.

→ In spring framework, SimpleMailMessage class is given to create a message without attachments.

→ We need to create an object of SimpleMailMessage & then we need to set the properties like -- from, to, subject and text, to prepare a message, for sending it as an email.

* E-mail without attachment ... jars. 2/5/15

(10) MailTest2

- src
  - default Package
    - TestBean.java
    - client.java
  - spring-config.xml

(i) mail.jar (ii) activation.jar
(iii) s-expression 3.2.4 (iv) s-beans
(v) s-context (vi) s-context-support
(vii) core (viii) commons-logging-jar.

→ Refer page (5) of spring-services handout.

* Mail with Attachment :-

To send an email with text and with attachment, we need an object of MimeMessage class.

→ MimeMessage class is from java mail api.

→ To set the text and also to add attachment to the MimeMessage class object, we need to use MimeMessageHelper class given by spring framework.

(11) MailTest2

- src
  - default package
    - TestBean.java
    - client.java
  - spring-config.xml

jars are same

(i) TestBean.java

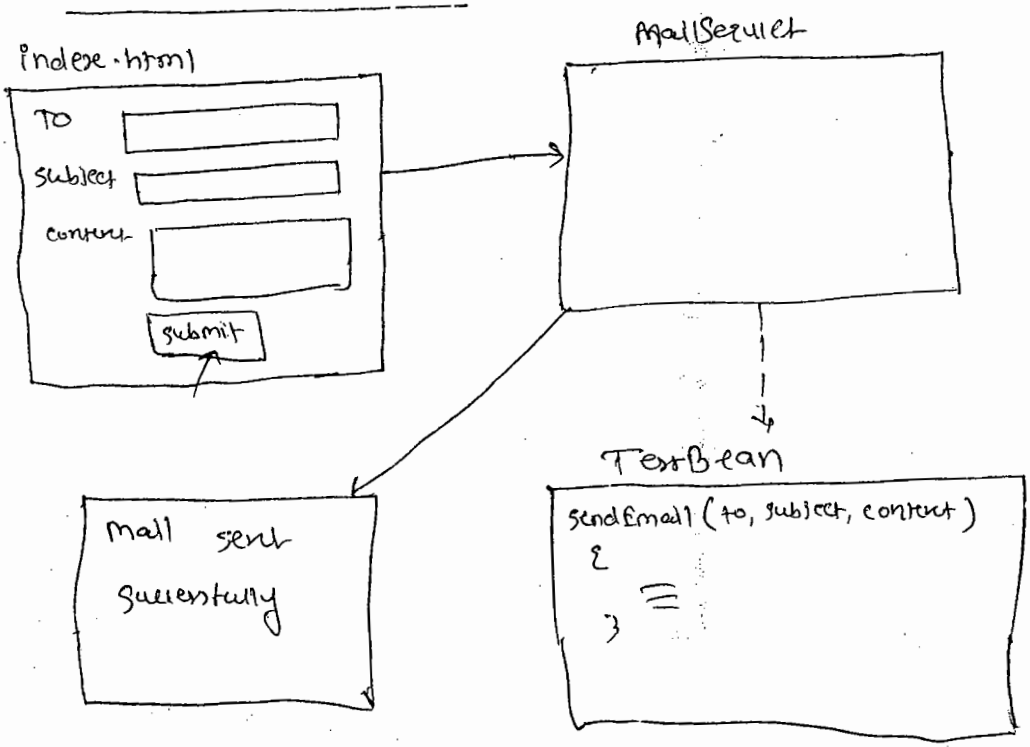
```
public class TestBean
{
    private JavaMailSenderImpl mailSender;
    public void setMailSender(JavaMailSenderImpl mailSender)
    {
        this.mailSender = mailSender;
    }
    public void sendEmail()
    {
        try
        {
```

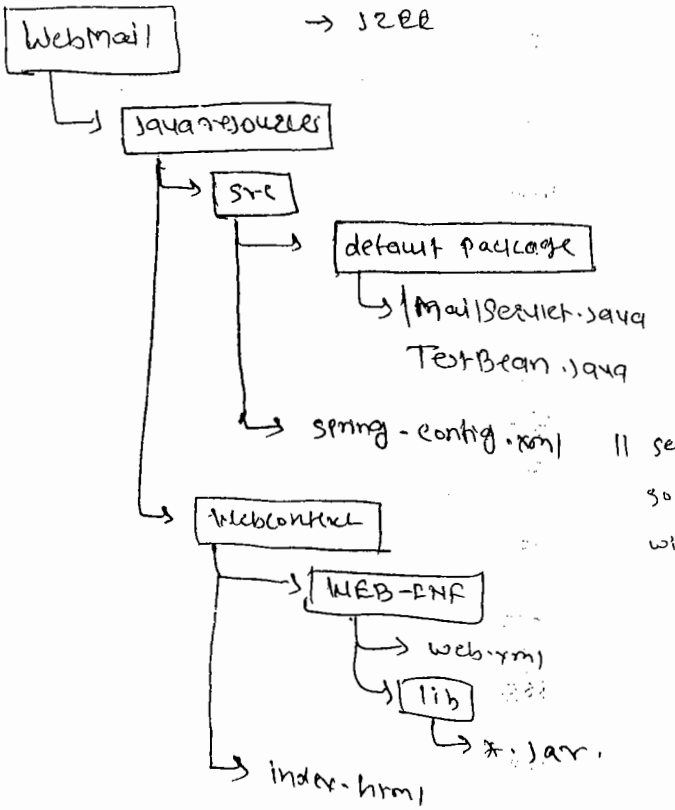
```
MimeMessage msg = mailSender.createMimeMessage();
MimeMessageHelper helper = new MimeMessageHelper(msg, true);
  helper.setFrom("bokarenitish@gmail.com");
  helper.setTo("bokarenitish43@gmil.com"); // or yahoo or anything...
  helper.setSubject("mail from spring application");
  helper.setText("This is a test mail with attachment");
FileSystemResource fsr = new FileSystemResource("location of file");
                                                  // D: hibernate/Insert.java /
  helper.addAttachment("Demo.java", fsr);
                       ("Insert.java", fsr);
  mailSender.send(msg);
    SOP("mail successfully sent");
}
catch (Exception e)
    {
      sop(e);
    }
}
```

→ Copy remaining file from previous example of mail.

→ If you want to add more attachment then use more FileSystemResource

(iii) Spring - servlet integration [J2EE]





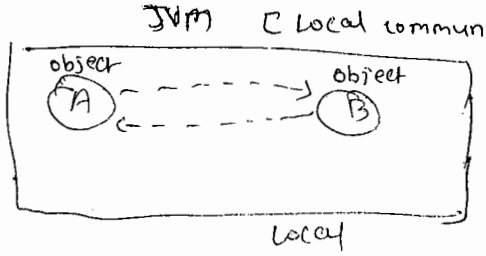
// servlet want the object of spring container so thats why we should store spring xml file will be stored in [src] folder

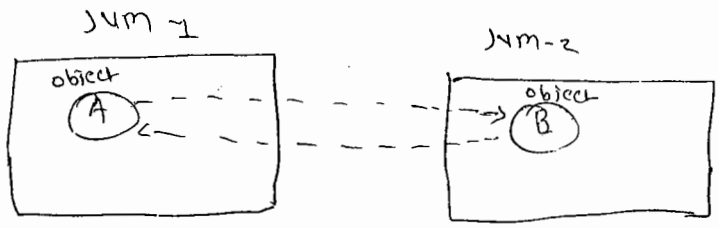
4/5/15

① Spring - Remoting

```
class A
{
    B b = new B();
    void m()
    {
        b.m2();
    }
}
```

class A & class B are working on same machine or JVM. Local.



② 

Remote communication

(A) Spring - RMI

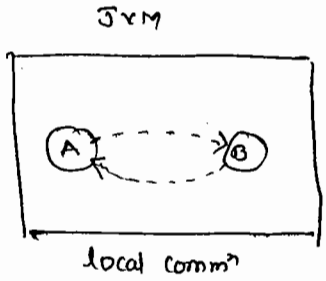
(B) Spring - HttpInvoker

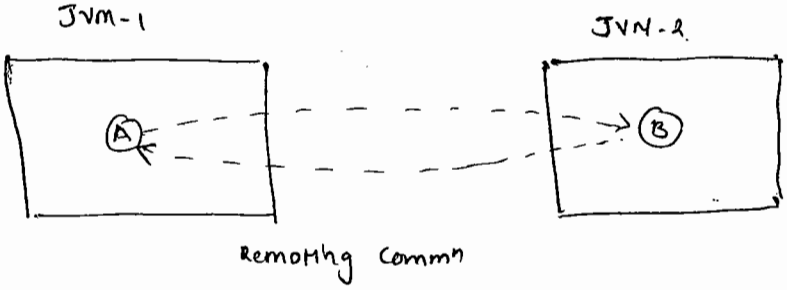
Stub object = proxy object - [mediator b/w client & server and to translate this stub registry required]

4-5-2015

* Spring - Remoting Service *

```
class A
{
   B b = new B();
   void m1()
    {
      b.m2();
    }
}
```





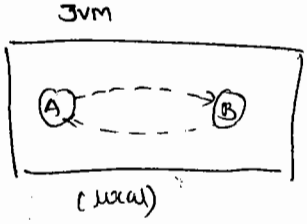
2 types

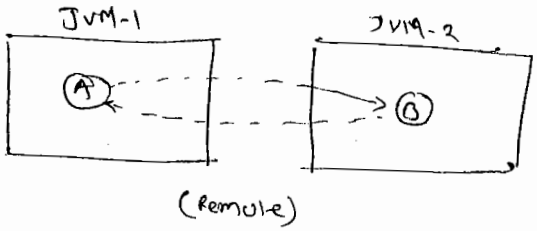
- Spring - RMI
- Spring - HttpInvoker

→ In a java Application, if two objects are communicating with each other within a same JVM, then we call it as local comm^n.

→ if two objects running on 2 different JVM's are communicating with each other then it is called as remote comm^n.

```
class A
{
   B b = new B();
   void m1()
    {
      b.m2();
    }
}
```



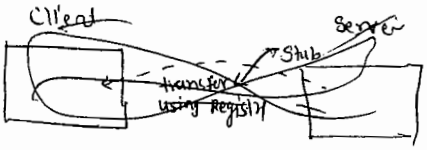


→ In spring framework, we use below two technique to get remote comm^n bet^n two spring beans.

① Spring - RMI
② Spring - HttpInvoker

Spring - RMI

→ RMI is a distributed technology given as Part of Java SE for communicating 2 objects that are running in two different JVM.



→ In Remoting, a client object can not directly called Service, because Service object is running on another JVM.

→ To Provide the communication bet^n client & Service object In the middle Helper objects are required & they are called Stubs

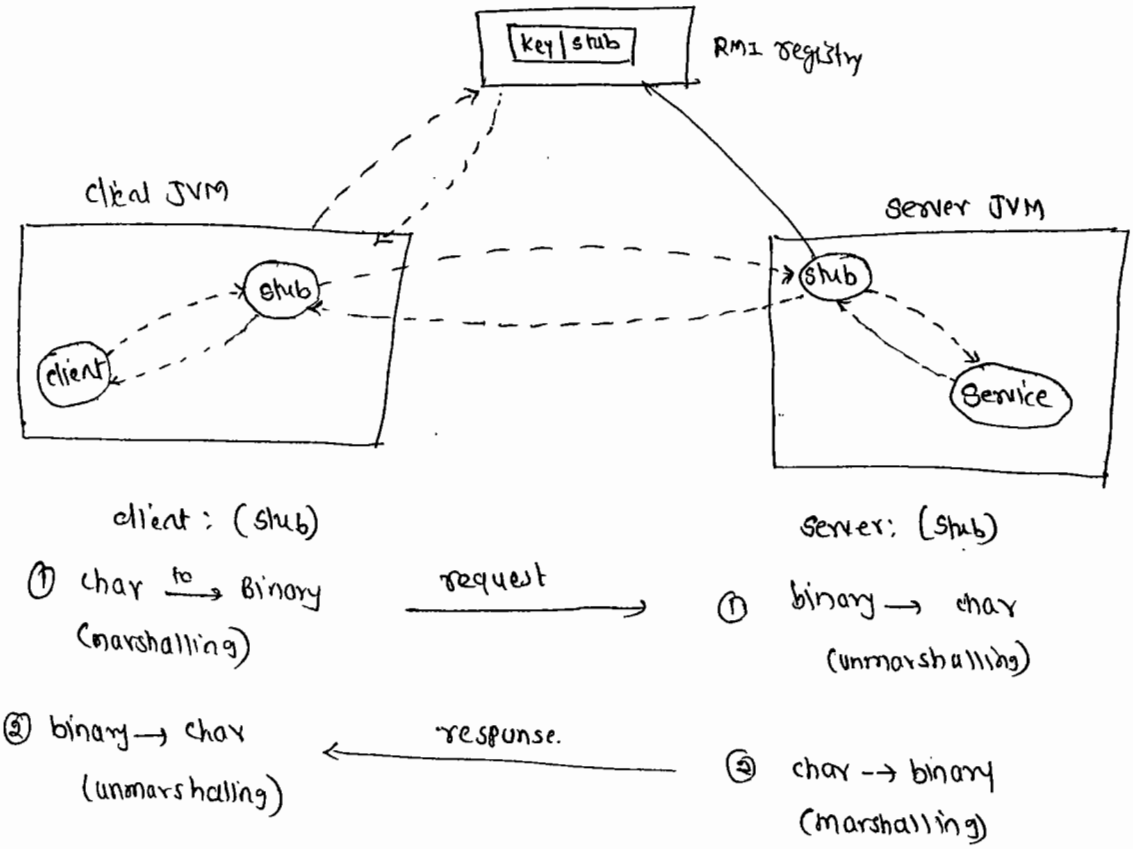
→ a Stub object will be created at Server & then it will be stored in RMI Registry with some key.

→ at client side, the application reads the Stub object from registry

→ the stub object at client & server will do the marshalling & unmarshalling.

→ marshalling is a process of converting character data to binary & unmarshalling is a process of converting binary data to character data.

→ a typical RMI comm^{n} diagram will be like the following.



→ Spring-RMI is an abstraction layer on top of RMI technology.

→ If we directly work with RMI technology to create a service we need to follow the below rules.

→ (1) we need to to extend the Service interface from java.rmi.Remote (It is a marker interface).

② we need to extend Service class from a Super class UnicastRemoteObject.

③ we need to generate a Stub using RMI compiler (rmic)

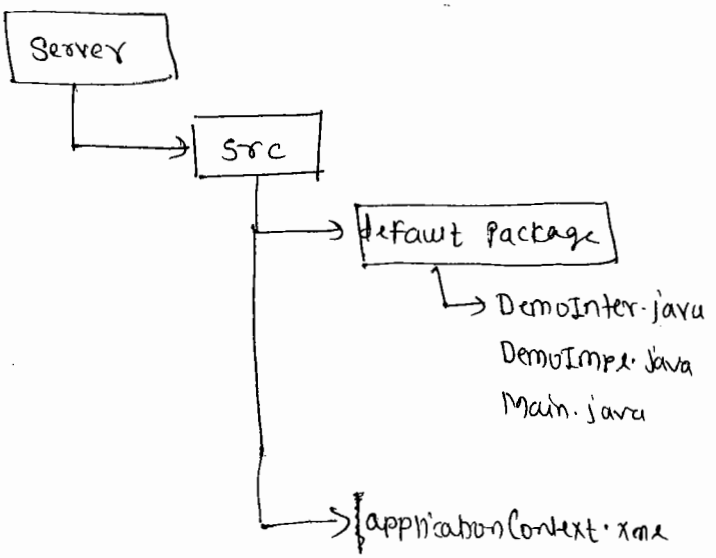
④ we need to bind the Stub object with RMI registry.

⑤ we need to start RMI registry.

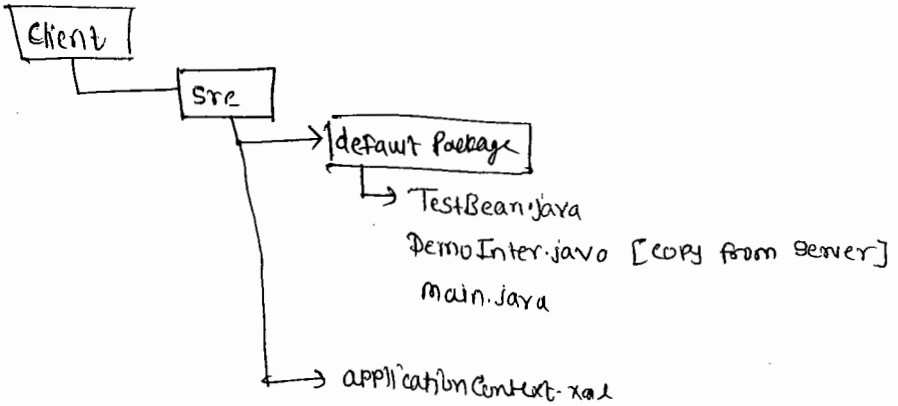
→ In Spring-RMI layer, ~~RMI Service~~ RmiServiceExporter class is given and a it will do all the above steps automatically so there will be no burden on a server side programmer.

→ In a spring configuration file, we need to configure ~~RMI S~~ RmiServiceExporter with required properties.

5-5-15



refer Pageno. 9 of Spring_Services.txt



refer page 9 Spring-Services.txt

Jar (add in both client & server)

1. aopalliance
2. commons-logging
3. spring-expression
4. spring-aop
5. spring-beans
6. spring-context
7. spring-context-support
8. spring-core

Spring HttpInvoker:

→ In Spring-Rmi Rmi Protocol is used for connecting with RMI registry and to read the stub object from RMI Registry

→ RMI Protocol is not a firewall friendly it means if firewall is enable in system in which RMI Registry is running then RMI protocol cannot connect to registry so connectivity fails.

→ To overcome the above problem framework has given its own ~~remoting~~ remoting technology as Spring-HttpInvoker.

→ In Spring-HttpInvoker, Http protocol is used to read a stub object from server.. Http is a firewall friendly protocol so the comm^n is possible always.

→ To make a server side bean accessible to all the client application through http protocol, we need to deploy server side bean in a web or application server.

→ To deploy a server side bean in web|appl^n server, we need to store serverside bean in a web appl^n & then we need to deploy

that server side bean in a server.

→ In this spring Http. Invoker, DispatcherServlet of spring mvc is used to receive a request coming from client application.

→ to create a stub class and to return a stub object for a service bean, spring framework has given a predefined class HttpInvokerServiceExporter.

→ When a request comes from client application then the below steps are executed :

① DispatcherServlet stops the request.

② DispatcherServlet calls HandlerMapping Bean.

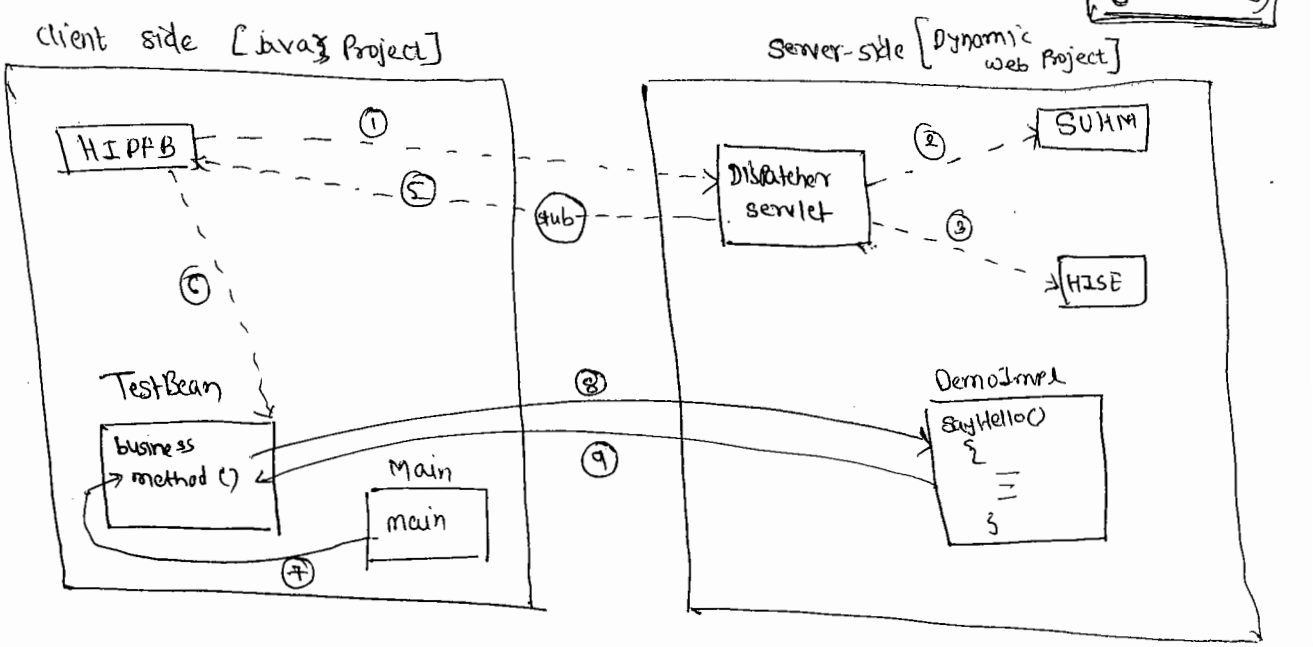
③ HandlerMapping bean finds appropriate bean to handle !

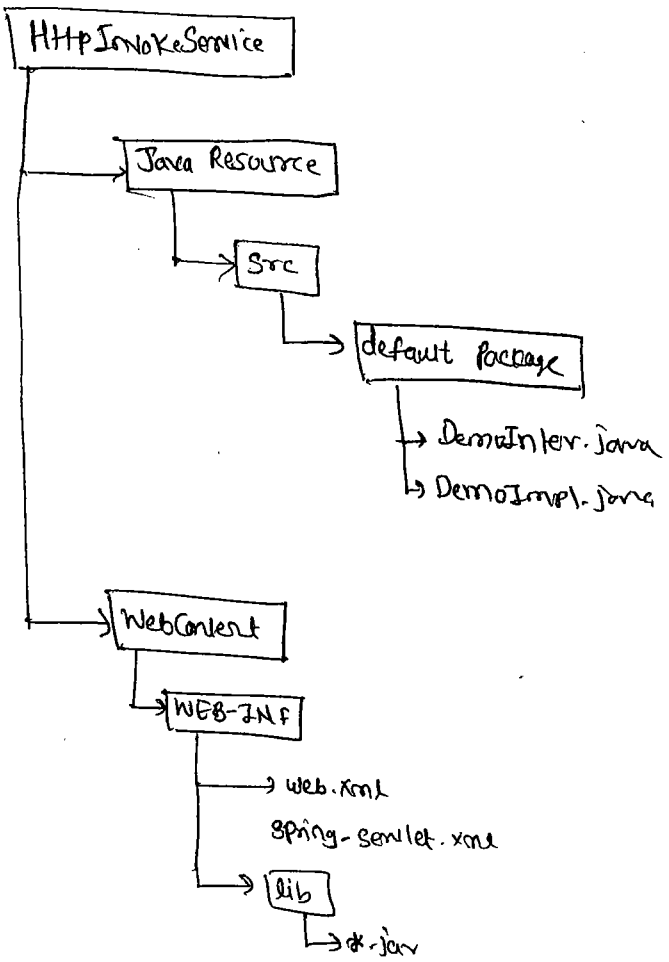
④ DispatcherServlet calls the ~~bean~~ HttpInvokerServiceExporter bean

⑤ HttpInvokerServiceExporter returns a stub object & the stub object will return to client Application by DispatcherServlet.

→ To call a DispatcherServlet & to read stub object, a client application uses a predefined bean class HttpInvokerProxyFactory ~~Bean~~ bean

6-5-2015

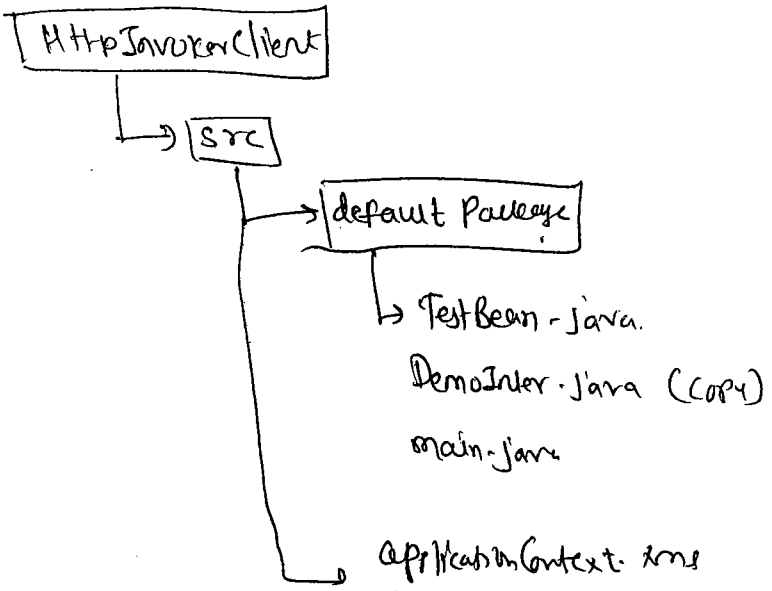




Jars

1. aopalliance
2. spring-aop
3. commons-logging
4. spring-context
5. spring-context-support
6. spring-beans
7. spring-core
8. spring-expression
9. spring-web-4.0.5
10. spring-web-mvc

(2) Client application in Java



Refer Page (11)

7-11-2015

JMS Terminology

① **JMS Provider :-** JMS Provider is a vendor who provides implementation for interfaces. the implementation given by vendor is called a JMS Server

→ all Java application server are jms providers.

②

② **JMS Client :**

→ It is a Java application used for either sending a message or for receiving a message

③ **JMS administered objects :**

→ a ConnectionFactory and the destination object are called administered object of JMS.

→ a JMS Provider will provide a tool to configure administered objects of JMS.

→ an administered ~~object~~ of JMS configure administered objects & stores them in JNDI Registry.

→ In JMS there are two types of destination object

① Queue ② Topic.

→ Queue is a destination object to store the object in Point to Point model

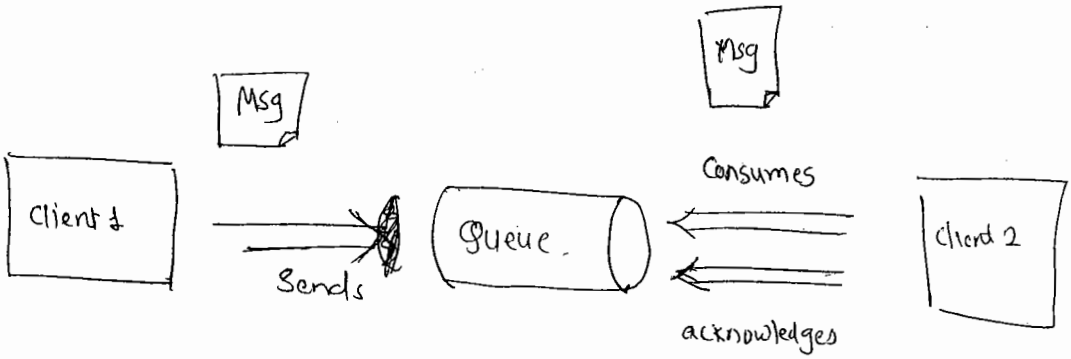
→ Topic is a destination object to store ~~the~~ messages in Publish Subscribe model.

→ JMS support 2 types of messaging model

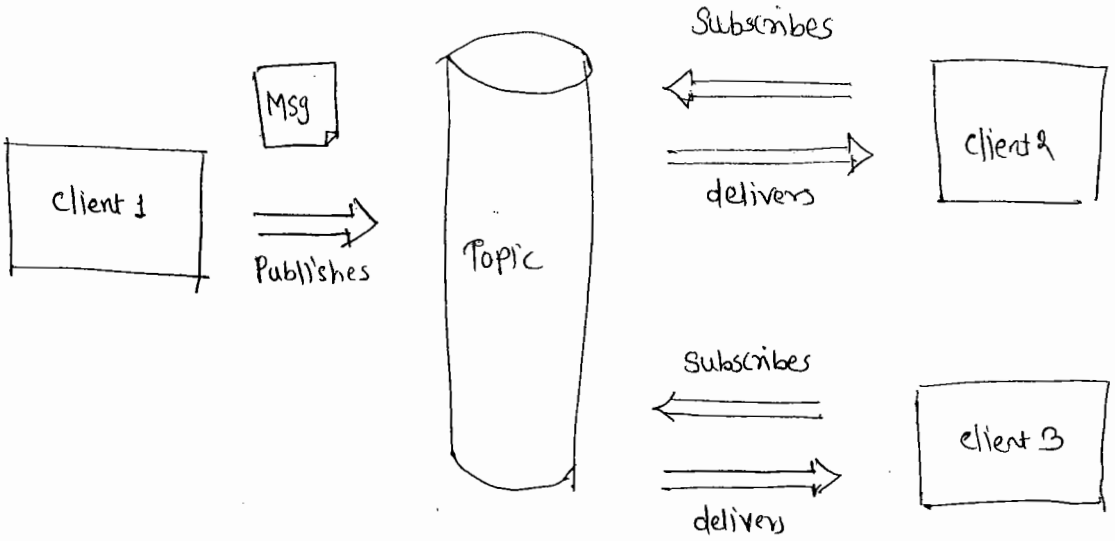
① Point to Point or one to one.

② Publish-subscribe or one to many

Point-to-Point Messaging



Publish/Subscribe Messaging



→ In JMS a client application which want to create a message and to send it to destination it should follow the below steps

① A client application should read a ConnectionFactory object from JNDI Registry.

② Using ConnectionFactory creates a Connection to the JMS Server

③ Using Connection a session is created

④ Using a session a Message & Producer object are created.

⑤ a client appln reads dest^n object from JNDI ^ registry.

⑥ a Producer object will send message to destination object.

→ a JMS Client Appln who wants to receive a message should follow the below steps.

① a client application should read a ConnectionFactory object from JNDI registry

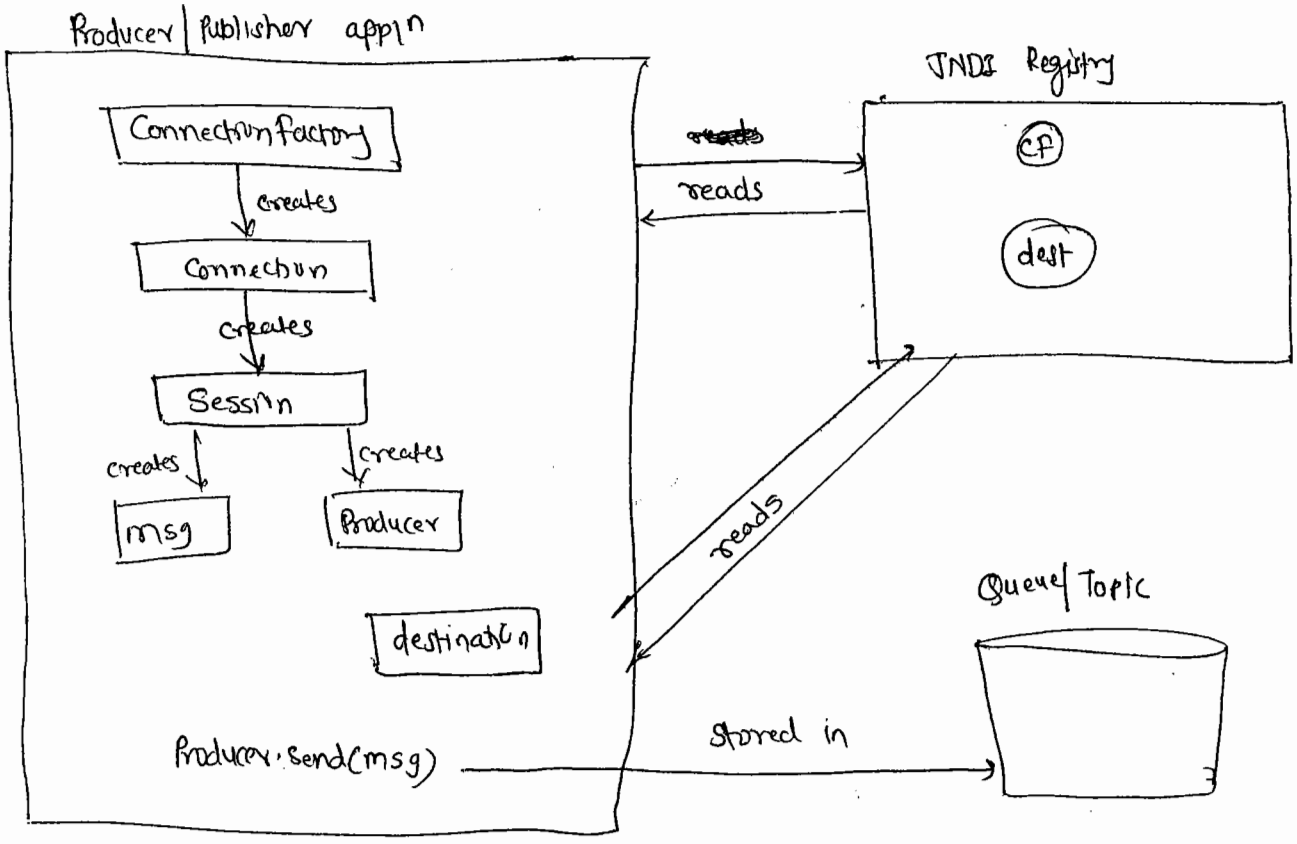
② Using ConnectionFactory a Connection is created to JMS server

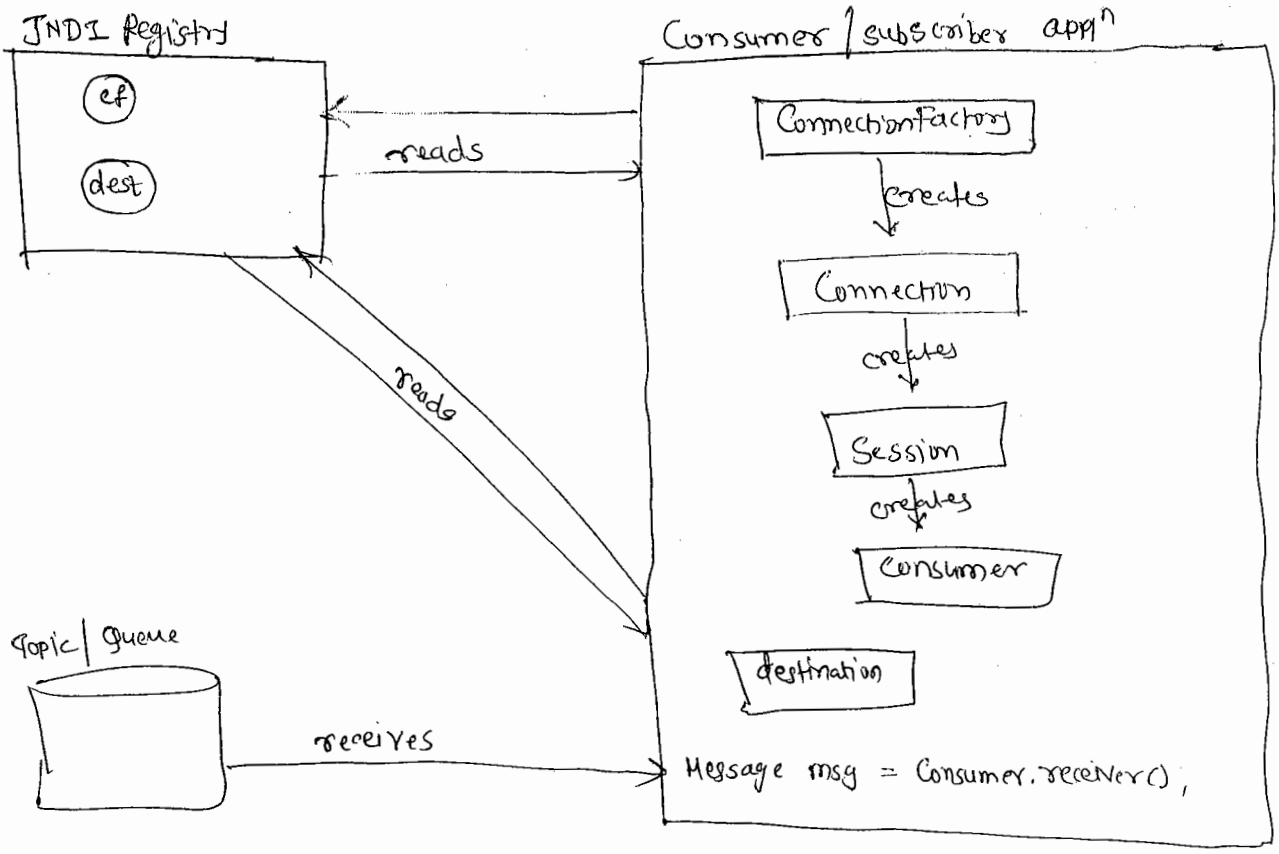
③ Using Connection a session is created

④ Using Session a Consumer object is created.

⑤ a client Application reads destination object from registry

⑥ a Consumer object receive ~~de~~ message from destination object.





→ When developing a JMS appln in Point to point model or in Publish Subscribe model, there is a BoilerPlateCode at both Producer & Consumer application.

→ In order to avoid a BoilerPlateCode Spring fw has given Spring-jms as an abstraction layer on top java jms API

→ In a Spring-Jms, JmsTemplate class is given & the using this template class a Spring bean can send a message to a destination message or it can receive a message from destination object

→ In JMS 4 types of messages can be send from a Producer Appln to a Consumer Appln.

① TextMessage. → To send a strip as data
② MapMessage. → To send a data as key/value pair
③ BytesMessage. → To send an image as data
④ ObjectMessage → To send a serialized object of Java as message

→ the above 4 classes are subclasses of an abstract class Message.

→ In spring to create a Message Object & to set the data framework has given an interface MessageCreator & we need to implement this interface into a class

→ Configuring JMS administrative object in Glassfish server

Steps

① Start the glassfish Server

C:\Program Files\glassfish-4.0\glassfish\bin> asadmin start-domain domain1

② Open the browser & type the following request.

http://localhost:4848

③ Expand resources → select JMS Resources → Click on Connectionfactories Click on new button

Enter JNDI Name → qcfactory

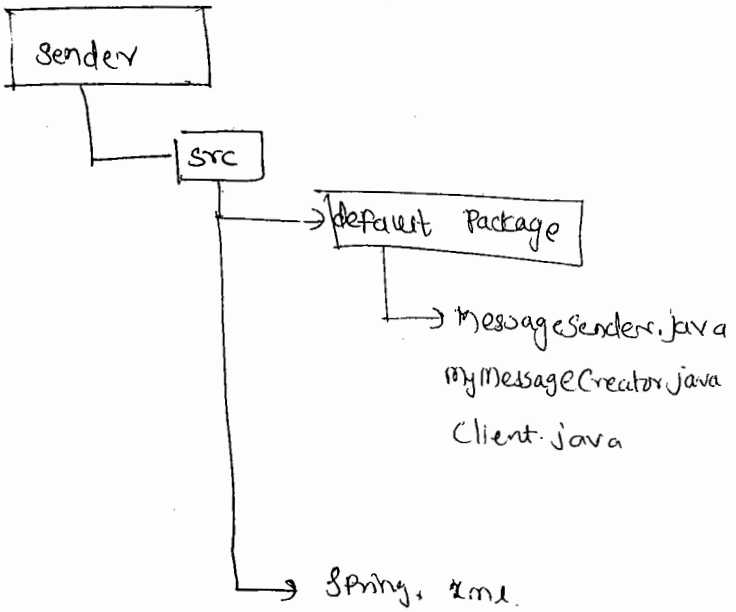
Resource Type → javax.jms.QueueConnectionFactory → OK

④ Click on jms Resources → (select) Destination Resources → new button →

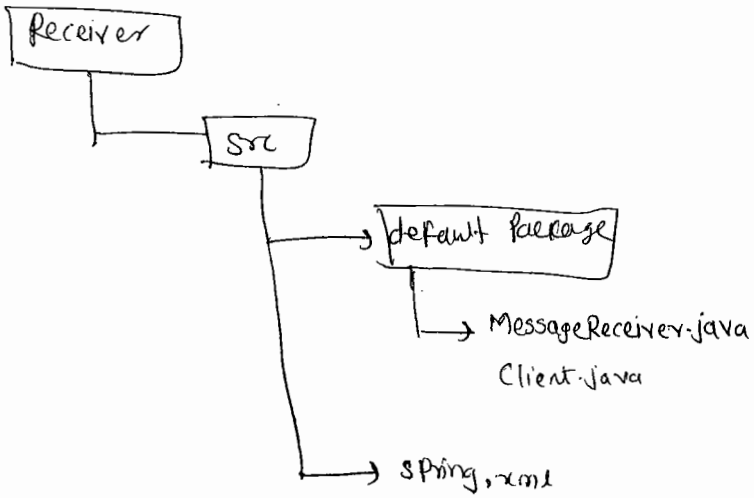
Enter JNDI Name → que

Physical Destination Name → que

Resource Type → javax.jms.Queue → OK



Refer Page. 6 of Spring Services handout



refer Page. 7 of Spring Services

1. aopalliance-1.0
2. appserv-rt.jar → C:\Program files\glassfish-4.0\glassfish\lib
3. Common-logging
4. jms-1.1
5. Spring aop
6. Spring beans
7. Spring Context
8. Spring-Context-support
9. spring-core-4.0.5
10. spring-expression
11. Spring-jms
12. Spring-tx.

AOP

Title: AOP Application with pointcut (1)

Demo.java

```
package com.sathya.spring.aop.model;
public interface Demo {
        void sayHello();
        void sayBye();
        String  getMessage();
        void welcome();
}
```

POJO

DemoBean.java

POJO (Business class) | target class

```
package com.sathya.spring.aop.model;
public class DemoBean implements Demo {
        @Override
        public String getMessage() {
                return "I am getMessage";
        }
        @Override
        public void sayBye() {
                System.out.println("I am sayBye");
        }
        @Override
        public void sayHello() {
                System.out.println("I am sayHello");
        }
        @Override
        public void welcome() {
                System.out.println("I am welcome");
        }
} //end of the class
```

output

```
I am BeforeAdvice
Welcome to : sayHello
I am sayHello
=========
I am BeforAdvice
Welcome to : sayBye
I am sayBye
---------
I am getMessage
---------
I am welcome
```

WelcomeAdvice.java

```
package com.sathya.spring.aop.aspect;
import java.lang.reflect.Method;
import org.springframework.aop.MethodBeforeAdvice;
public class WelcomeAdvice implements MethodBeforeAdvice
{
        @Override
        public void before(Method arg0, Object[ ] arg1, Object arg2)
                        throws Throwable {
                String  methodName=arg0.getName();
                System.out.println("I am BeforeAdvice");
                System.out.println("Welcome to : "+methodName);
        }
}
```

applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">

 <!-- Business ciass configuration -->
  <bean id="db"    class="com.sathya.spring.aop.model.DemoBean"/>
  <!-- advice configuration -->
  <bean id="before"  class="com.sathya.spring.aop.aspect.WelcomeAdvice"/>
  <!-- pointcut configuration -->
  <bean id="pointcut1" class="org.springframework.aop.support.JdkRegexpMethodPointcut">
     <property  name="patterns">
      <list>
        <value>.*say.*</value>
      </list>
     </property>
   </bean>
   <!--   advisor configuration -->
   <bean id="advisor1" class="org.springframework.aop.support.DefaultPointcutAdvisor">
     <property name="pointcut"  ref="pointcut1"/>
      <property  name="advice"    ref="before"/>
   </bean>
```

the method which satisfies this condition called JoinPoint

Advisor = Pointcut + advice

```
    <!-- proxy factory bean configuration -->
    <bean id="pfb1"
    class="org.springframework.aop.framework.ProxyFactoryBean">
      <property name="target"    ref="db"/>
      <property name="proxyInterfaces"
      value="com.sathya.spring.aop.model.Demo"/>
       <property name="interceptorNames">
        <list>
           <value>advisor1</value>
         </list>
       </property>
    </bean>
</beans>
------------------AOPMain.java----------------------
//AOPMain.java
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import com.sathya.spring.aop.model.Demo;
public class AOPMain {
      public static void main(String[ ] args) {
           ApplicationContext  ctx =
           new ClassPathXmlApplicationContext("applicationContext.xml");
           Object  o =ctx.getBean("pfb1");
           Demo   demo=(Demo)o;
           demo.sayHello();
           System.out.println("========");
           demo.sayBye();
           System.out.println("========");
           String  str =demo.getMessage();
           System.out.println(str);
           System.out.println("=========");
           demo.welcome();
      }
}
=============================================================:
      Title: AOP Application  with  user-defined Dynamic pointcut
----------------Factorial.java--------------------
package com.sathya.spring.aop.model;
public interface Factorial {
      void findFactorial(int x);
}
----------------FactorialImpl.java-----------------------
package com.sathya.spring.aop.model;
public class FactorialImpl implements Factorial
{
      @Override
      public void findFactorial(int x)
      {
           if(x<0)
           {
                System.out.println("parameter is -ve");
                return;
           }
           int fact=1;
           for(int i=1; i<=x; i++)
           {
                fact=fact * i;
           }
           System.out.println("factorial of  : "+x +" is  :"+fact);
      }
}
-------------WelcomeAdvice.java---------------------
package com.sathya.spring.aop.aspect;
import java.lang.reflect.Method;
import org.springframework.aop.MethodBeforeAdvice;
public class WelcomeAdvice implements MethodBeforeAdvice
```

Weaver class

we need to pass created advisor class id

Container created or sto

ApplicationContext container can do Preinstantiation of Singleton class, but not BeanFactory Container.

(2)

O/P

factorial of : 0 is : 1

I am BeforAdvice
welcome to : findFactorial
factorial of : 5 is : 120

Parameter is -ve

```
{
    @Override
    public void before(Method arg0, Object[ ] arg1, Object arg2)
            throws Throwable {
        String  methodName=arg0.getName();
        System.out.println("I am BeforeAdvice");
        System.out.println("Welcome to : "+methodName);
    }
}
----------MyDynamicPointcut.java----------
package com.sathya.spring.aop.pointcut;
import java.lang.reflect.Method;
import org.springframework.aop.support.DynamicMethodMatcherPointcut;
import com.sathya.spring.aop.model.FactorialImpl;

public class MyDynamicPointcut extends DynamicMethodMatcherPointcut
{
    @Override
    public boolean matches(Method method, Class clazz, Object[ ] args)
    {
        boolean  flag=false;
        int k=(Integer)args[0];
        if(clazz == FactorialImpl.class)
        {
            if(method.getName().startsWith("f") && k>0)
            {
                flag=true;
            }

        }
        return flag;
    }
}
----------applicationContext.xml----------
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">

 <!-- business class  configuration -->
 <bean  id="fact"
 class="com.sathya.spring.aop.model.FactorialImpl"/>
 <!--  advice configuration -->
 <bean  id="before"
 class="com.sathya.spring.aop.aspect.WelcomeAdvice"/>
 <!-- pointcut configuration -->
 <bean  id="pointcut1"
 class="com.sathya.spring.aop.pointcut.MyDynamicPointcut"/>
 <!-- advisor configuration -->
 <bean  id="advisor1"  class="org.springframework.aop.support.DefaultPointcutAdvisor">
   <property  name="pointcut"  ref="pointcut1"/>
   <property  name="advice"    ref="before"/>
 </bean>
 <!--  proxy factory bean configuration  -->
 <bean  id="pfb"  class="org.springframework.aop.framework.ProxyFactoryBean">
   <property  name="target"   ref="fact"/>
   <property  name="proxyInterfaces"  value="com.sathya.spring.aop.model.Factorial"/>
   <property  name="interceptorNames">
    <list>
      <value>advisor1</value>
    </list>
   </property>
 </bean>
</beans>
----------AOPMain.java----------
import org.springframework.context.ApplicationContext;
```

```
import org.springframework.context.support.ClassPathXmlApplicationContext;
import com.sathya.spring.aop.model.Factorial;
public class AOPMain {
      public static void main(String args[ ])
      {
            ApplicationContext  ctx=
            new ClassPathXmlApplicationContext("applicationContext.xml");
             Object  o = ctx.getBean("pfb");
             Factorial   f = (Factorial)o;
             f.findFactorial(0);
             System.out.println("==========");
             f.findFactorial(5);
             System.out.println("==========");
             f.findFactorial(-1);
      }
}
```

==========

Title: AOP Application with XML based AOP (3)

```
----------Product.java----------
package com.sathya.spring.aop.model;
public interface Product {
      void add(int x,int y);
      int   multiply(int x,int y);
}
----------ProductImpl.java----------
package com.sathya.spring.aop.model;
public class ProductImpl implements Product {
      @Override
      public void add(int x, int y) {
                  System.out.println("Addition = "+(x+y));
      }
      @Override
      public int multiply(int x, int y) {
            return  (x*y);
      }
}
----------MyAdvices.java----------
package com.sathya.spring.aop.aspect;
import org.aspectj.lang.JoinPoint;
public class MyAdvices {
      public  void  before(JoinPoint jp)
      {
            String  methodName=jp.getSignature().getName();
            Object  args[ ]=jp.getArgs();
            System.out.println("I am before advice to : "+methodName);
            System.out.println("It's arguments values are : "+args[0]+" , "+args[1]);
      }
      public  void  afterReturning(JoinPoint jp,Object  result)
      {
            String  methodName=jp.getSignature().getName();
            Object  args[ ]=jp.getArgs();
            System.out.println("I am after returning advice to : "+methodName);
            System.out.println("It's arguments values are : "+args[0]+" , "+args[1]);
            System.out.println("The result of this method is : "+result);
      }
}
----------applicationContext.xml----------
      <!-- applicationContext.xml -->
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:aop="http://www.springframework.org/schema/aop"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-3.0.xsd
http://www.springframework.org/schema/aop
http://www.springframework.org/schema/aop/spring-aop-3.0.xsd">
      <!-- business class  configuration  -->
```

O/P

I am before advice to : add
It's arguments values are : 10, 20
Addition = 30
I am after returning advice to : multiply
It's arguments values are : 10, 20
The result of this method is : 200
200

xmlns → tags for configuring beans.

```
265       <bean id="pimpl"  class="com.sathya.spring.aop.model.ProductImpl"/>
266       <!--  advices class configuration -->
267       <bean id="ma"   class="com.sathya.spring.aop.aspect.MyAdvices"/>
268       <!--  xml based aop configuration -->
269       <aop:config>
270        <aop:aspect   ref="ma">
271         <aop:pointcut  id="pt1"
272          expression="execution(*  com.sathya.spring.aop.model.Product.add(..))"/>
273         <aop:pointcut  id="pt2"
274          expression="execution(* com.sathya.spring.aop.model.Product.multiply(..))"/>
275         <aop:before  pointcut-ref="pt1"  method="before"/>
276         <aop:after-returning  pointcut-ref="pt2"
277               method="afterReturning"  returning="result"/>
278        </aop:aspect>
279     </aop:config>
280     <aop:aspectj-autoproxy/>
281 </beans>
282 ----------------AOPMain.java--------------------------
283 import org.springframework.context.ApplicationContext;
284 import org.springframework.context.support.ClassPathXmlApplicationContext;
285 import com.sathya.spring.aop.model.Product;
286 public class AopMain {
287           public static void main(String[ ] args) {
288           ApplicationContext  ctx=
289           new  ClassPathXmlApplicationContext("applicationContext.xml");
290           Object  obj=ctx.getBean("pimpl");
291           Product  p=(Product)obj;
292           p.add(10,20);
293           System.out.println("===========");
294           p.multiply(10, 20);
295       }
296 }
297 ==========================================================================
298       Title: AOP Application with  Annotations based AOP  (4)
299 ----------------Demo.java----------------
300 package com.sathya.spring.aop.model;
301 public interface Demo {
302       void businessMethod1(int amount) throws Exception;
303       void businessMethod2();
304 }
305 ----------------DemoImpl.java----------------
306 package com.sathya.spring.aop.model;
307 import com.sathya.spring.aop.exception.InsufficientAmountException;
308 public class DemoImpl implements Demo
309 {
310       @Override
311       public void businessMethod1(int amount) throws Exception
312       {
313            if(amount <=0)
314                 throw new InsufficientAmountException("U have given Amount as : "+
315                 amount+" it is not valid");
316            else
317                 System.out.println("Amount is  : "+amount+" it is valid");
318       }
319       @Override
320       public void businessMethod2() {
321            try
322            {
323                 Thread.sleep(5000);
324            }
325            catch(Exception e)
326            { }
327       }
328 }
329 ----------InsufficientAmountException.java----------
330 package com.sathya.spring.aop.exception;
```

```
331 public class InsufficientAmountException extends
332                     RuntimeException
333 {
334       public  InsufficientAmountException(String message)
335       {
336             super(message);
337       }
338 }
339 ----------MyAspect.java----------
340 package com.sathya.spring.aop.aspect;
341 import org.aspectj.lang.JoinPoint;
342 import org.aspectj.lang.ProceedingJoinPoint;
343 import org.aspectj.lang.annotation.After;
344 import org.aspectj.lang.annotation.Around;
345 import org.aspectj.lang.annotation.Aspect;
346 @Aspect
347 public class MyAspect {
348       @After("execution(* com.sathya.spring.aop.model.Demo.businessMethod1(..))")
349       public void  after(JoinPoint jp)
350       {
351             String str = jp.getSignature().getName();
352             System.out.println("I am after advice to : "+str);
353       }
354       @Around("execution(*  com.sathya.spring.aop.model.Demo.businessMethod2(..))")
355       public Object around(ProceedingJoinPoint  pjp)
356       {
357             Object o=null;
358             try{
359             long x=System.currentTimeMillis();
360              o = pjp.proceed();
361             long y=System.currentTimeMillis();
362             long z = y-x;
363             String  str=pjp.getSignature().getName();
364             System.out.println("I am around advice to : "+str);
365             System.out.println("Time taken to run business is : "+z +" milliseconds");
366             }catch(Throwable  t)
367             { }
368             return o;
369       }
370 }
371 ----------spring.xml----------
372 <beans xmlns="http://www.springframework.org/schema/beans"
373 xmlns:aop="http://www.springframework.org/schema/aop"
374 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
375 xsi:schemaLocation="http://www.springframework.org/schema/beans
376 http://www.springframework.org/schema/beans/spring-beans-3.0.xsd
377 http://www.springframework.org/schema/aop
378 http://www.springframework.org/schema/aop/spring-aop-3.0.xsd">
379   <!--  business class configuration -->
380   <bean  id="di"  class="com.sathya.spring.aop.model.DemoImpl"/>
381   <!--  aspect class configuration -->
382   <bean  id="myaspect"  class="com.sathya.spring.aop.aspect.MyAspect"/>
383   <aop:aspectj-autoproxy/>
384 </beans>
385 ----------AOPMain.java----------
386 import org.springframework.context.ApplicationContext;
387 import org.springframework.context.support.ClassPathXmlApplicationContext;
388 import com.sathya.spring.aop.model.Demo;
389 public class AOPMain
390 {
391       public static void main(String...  args)
392       {
393             ApplicationContext  ctx=
394             new ClassPathXmlApplicationContext("spring.xml");
395             Object  o = ctx.getBean("di");
396             Demo  d=(Demo)o;
```

Unchecked exception as extends from RuntimeException

Ans.

Q How do you find the your Java Program takes this much time to execute Program.

```
397             try{
398                 d.businessMethod1(1000);
399             }
400             catch(Exception e)
401             {
402                 System.out.println(e);
403             }
404             System.out.println("==========");
405             d.businessMethod2();
406             System.out.println("==========");
407             try{
408             d.businessMethod1(-100);
409             }catch(Exception e)
410             {
411                 System.out.println(e);
412             }
413     }
414 }
415 ================================================================
```

O/P

Amount is : 1000 it is valid

I am after advice to : businessMethod1

I am around advice to : businessMethod2

Time taken to run business is : 5000 millisecond

I am after advice to : businessMethod1

com.satnya.spring.aop.exception.InsufficientAmountException: U have given

A given Amount as -100 It is not valid.

```
1       Title: Login MVC with Database support using Spring-JDBC
2  ----------index.jsp----------
3  <jsp:forward page="login.form"></jsp:forward>
4  ----------login.jsp----------
5  <form action="check.form" method="post">
6   Username : <input type=text name="t1"> <br>
7   Password : <input type=password name="t2"><br>
8   <input type=submit value="submit">
9  </form>
10 ----------spring-servlet.xml----------
11 <beans xmlns="http://www.springframework.org/schema/beans"
12 xmlns:context="http://www.springframework.org/schema/context"
13 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
14 xsi:schemaLocation="http://www.springframework.org/schema/beans
15 http://www.springframework.org/schema/beans/spring-beans-4.0.xsd
16 http://www.springframework.org/schema/context
17 http://www.springframework.org/schema/context/spring-context-4.0.xsd">
18
19  <context:component-scan base-package="com.sathya.spring.controller"/>
20  <context:annotation-config/>
21  <!-- handler mapping -->
22  <bean class="org.springframework.web.servlet.mvc.annotation.DefaultAnnotationHandlerMapping"/>
23  <!-- view resolver -->
24  <bean class="org.springframework.web.servlet.view.InternalResourceViewResolver">
25   <property name="prefix" value="/"/>
26   <property name="suffix" value=".jsp"/>
27  </bean>
28
29  <bean id="jt" class="org.springframework.jdbc.core.JdbcTemplate">
30     <property name="dataSource" ref="ds"/>
31  </bean>
32  <bean id="ds" class="org.springframework.jdbc.datasource.DriverManagerDataSource">
33     <property name="driverClassName" value="oracle.jdbc.driver.OracleDriver"/>
34     <property name="url" value="jdbc:oracle:thin:@localhost:1521:XE"/>
35     <property name="username" value="system"/>
36     <property name="password" value="root"/>
37  </bean>
38 </beans>
39 ----------LoginController.java----------
40 package com.sathya.spring.controller;
41 import org.springframework.beans.factory.annotation.Autowired;
42 import org.springframework.stereotype.Controller;
43 import org.springframework.web.bind.annotation.RequestMapping;
44 import org.springframework.web.bind.annotation.RequestMethod;
45 import org.springframework.web.bind.annotation.RequestParam;
46 import org.springframework.web.servlet.ModelAndView;
47 @Controller
48 public class LoginController
49 {
50     private LoginService loginService;
51     @Autowired
52     public void setLoginService(LoginService loginService)
53     {
54         this.loginService=loginService;
55     }
56     @RequestMapping(value="/login", method=RequestMethod.GET)
57     public ModelAndView getLoginPage()
58     {
59         return new ModelAndView("login");
60     }
61
62     @RequestMapping(value="check", method=RequestMethod.POST)
63     public ModelAndView checkLogin(@RequestParam("t1")String s1,@RequestParam("t2")
64     {                                                                    String s2)
65         boolean b = loginService.check(s1,s2);
66         if(b)
```

```
67                  return  new ModelAndView("success");
68              else
69                  return  new  ModelAndView("failure");
70          }
71  }
72  ------------------LoginService.java-------------------------
73  package com.sathya.spring.controller;
74  import org.springframework.beans.factory.annotation.Autowired;
75  import org.springframework.beans.factory.annotation.Qualifier;
76  import org.springframework.jdbc.core.JdbcTemplate;
77  import org.springframework.stereotype.Repository;
78  @Repository
79  public class LoginService
80  {
81      private JdbcTemplate  jt;
82      @Autowired
83      @Qualifier("jt")
84      public void setJt(JdbcTemplate jt)
85      {
86          this.jt=jt;
87      }
88      public boolean  check(String  s1, String s2)
89      {
90          int  i= jt.queryForInt("select  count(*) from users  where  uname=? and  pwd=?",s1,s2);
91          if(i==1)
92              return  true;
93          else
94              return false;
95      }
96  }
97  ------------------success.jsp-----------------------
98  <h1> Login Success </h1>
99  ------------------failure.jsp---------------
100 <h1> Login Failed </h1>
101 ============================================================
102         Title: Login MVC with annotation validations
103 ------------index.jsp-------------------------
104 <jsp:forward page="loginpage.form"></jsp:forward>
105 ----------login.jsp--------------------
106 <%@taglib uri="http://www.springframework.org/tags/form"  prefix="form"%>
107  <form:form  method="post"  action="check.form"  commandName="loginFormBackingObject">
108    Username: <form:input path="uname"/> <form:errors path="uname"/><br>
109    Password : <form:password path="pwd"/><form:errors path="pwd"/> <br>
110    <input type=submit value="click"/>
111  </form:form>
112 --------------spring-servlet.xml---------------
113 <beans xmlns="http://www.springframework.org/schema/beans"
114    xmlns:mvc="http://www.springframework.org/schema/mvc"
115    xmlns:context="http://www.springframework.org/schema/context"
116 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
117 xsi:schemaLocation="http://www.springframework.org/schema/beans
118 http://www.springframework.org/schema/beans/spring-beans-4.0.xsd
119 http://www.springframework.org/schema/context
120 http://www.springframework.org/schema/context/spring-context-4.0.xsd
121 http://www.springframework.org/schema/mvc
122 http://www.springframework.org/schema/mvc/spring-mvc-4.0.xsd">
123
124   <context:component-scan base-package="com.sathya.spring.mvc"/>
125    <!--  view resolver -->
126   <bean id="viewResolver"  class="org.springframework.web.servlet.view.InternalResourceViewResolver">
127      <property  name="prefix"  value="/"/>
128      <property  name="suffix"  value=".jsp"/>
129   </bean>
130   <bean id="messageSource"  class="org.springframework.context.support.ResourceBundleMessageSource">
131     <property  name="basename"  value="messages"/>
132   </bean>
```

returns new ModelAndView("sarat", "name", s1);

My name: ${name} or Hi : ${name}

(2) from DispatcherServlet- forward request to Login Page

tag starts with form

Ports is empty - Spring (2)

bind pwd element to POJO class property pwd

binding - It is a property name of POJO class

key of POJO class object

Resource bundle name

this tag

```
133   <mvc:annotation-driven/>
134  </beans>
```

→ to enable the validation of annotations

```
135  ----------Login.java----------
136  //Login.java  (condmma class)
137  package com.sathya.spring.mvc;
138
139  import javax.validation.constraints.Size;
140
141  import org.hibernate.validator.constraints.NotEmpty;
142
143  public class Login
144  {
145       @Size(min=4,max=8)
146       private String uname;
147       @NotEmpty
148       private String  pwd;
149
150       //setters & getters
151       public String getUname() {
152            return uname;
153       }
154       public void setUname(String uname) {
155            this.uname = uname;
156       }
157
158       public String getPwd() {
159            return pwd;
160       }
161
162       public void setPwd(String pwd) {
163            this.pwd = pwd;
164       }
165  }
166  ----------LoginController.java----------
167  package com.sathya.spring.mvc;
168  import javax.validation.Valid;
169  import org.springframework.stereotype.Controller;
170  import org.springframework.validation.BindingResult;
171  import org.springframework.web.bind.annotation.ModelAttribute;
172  import org.springframework.web.bind.annotation.RequestMapping;
173  import org.springframework.web.bind.annotation.RequestMethod;
174  import org.springframework.web.servlet.ModelAndView;
175  @Controller
176  public class LoginController
177  {
178       @RequestMapping(value="loginpage",method=RequestMethod.GET)
179       public ModelAndView  getLoginPage()
180       {
181            return  new ModelAndView("login","loginFormBackingObject",new Login());
182       }
183
184       @RequestMapping(value="check",method=RequestMethod.POST)
185       public ModelAndView  processRequest( @Valid @ModelAttribute("loginFormBackingObject")
186       Login login, BindingResult result)
187       {
188            if(result.hasErrors())
189            {
190                 return new ModelAndView("login");
191            }
192            else
193            {
194            String username=login.getUname();
195            String password=login.getPwd();
196            if(username.equals("sathya") && password.equals("java"))
197                 return new ModelAndView("success");
198            else
```

"/" is optional

POJO object

View name

key to link betn Login form & object of POJO class

It stores error object

to check error are there or not & apply if else cond" in result object

It returns true if there are errors

key of POJO class object

@Valid → It is for executing the validations

@ModelAttribute → to inject POJO class object

```
199                 return new ModelAndView("failure");
200             }
201         }
202 }
203 ----------messages.properties----------
204 #messages.properties
205 Size.loginFormBackingObject.uname=Username should be from {2} to {1} length
206 NotEmpty.loginFormBackingObject.pwd=Password is required
207 ----------failure.jsp----------
208 <h1>
209 <%
210  out.println("Login failed");
211 %>
212 </h1>
213 ----------success.jsp----------
214 <h1>
215 <%
216   out.println("Login  Success");
217 %>
218 </h1>
219 ==========
220
221       Title: FileUpload Application
222 ==========
223 ----------FileUploadController.java----------
224 package com.sathya.spring.mvc.controller;
225 import java.io.FileNotFoundException;
226 import java.io.FileOutputStream;
227 import java.io.IOException;
228 import javax.servlet.ServletContext;
229 import javax.servlet.http.HttpServletRequest;
230 import javax.servlet.http.HttpSession;
231 import org.springframework.stereotype.Controller;
232 import org.springframework.web.bind.annotation.RequestMapping;
233 import org.springframework.web.bind.annotation.RequestMethod;
234 import org.springframework.web.bind.annotation.RequestParam;
235 import org.springframework.web.multipart.MultipartFile;
236 import org.springframework.web.servlet.ModelAndView;
237 @Controller
238 public class FileUploadControlier
239 {
240 @RequestMapping(value="upload", method=RequestMethod.POST)
241 public ModelAndView fileUpload(@RequestParam("file1")
242 MultipartFile file,HttpServletRequest request)
243 {
244 String fail="";
245 String success="";
246 if(file.getSize()!=0)
247 {
248 if(file.getContentType().equals("text/xml"))
249 {
250 try {
251 FileOutputStream fos=new FileOutputStream("E:/"+file.getOriginalFilename());
252 fos.write(file.getBytes());
253 success="U r file : "+file.getOriginalFilename()+" Uploaded Successfully";
254 } catch (FileNotFoundException e) {
255 // TODO Auto-generated catch block
256 e.printStackTrace();
257 } catch (IOException e) {
258 // TODO Auto-generated catch block
259 e.printStackTrace();
260 }
261 }
262 else
263 {
264 fail="The file should be XML only!!!";
```

the Parameters are replaced in alphabetical order so {1} is Max & {2} is Min

Min 4 Max value 8

Size → name of annotation

```
 265  }
 266  } //outer if
 267  else
 268  {
 269       fail="The file should not be empty!!!";
 270  }
 271  if(success.length()>1)
 272  {                                  viewname   key      value
 273       return new ModelAndView("success","message",success);
 274  }
 275  else
 276  {
 277       return new ModelAndView("uploadfile","message",fail);
 278  }
 279  } //end of fileUpload() method
 280  }//end of class
 281  ----------success.jsp-------------------
 282  <h1>
 283  ${message}
 284  </h1>
 285  ----------uploadfile.jsp-------------------
 286  <%@taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c"%>
 287  <html>
 288   <body>                    Not Equal to
 289   <center>                      ↑
 290     <c:if test="${message ne null}">  ---> if (message != null){
 291       <c:out value="${message}"/>              out.println(message);
 292     </c:if>                                  }
 293     <br>
 294     <form action="upload.html" method="post" enctype="multipart/form-data">
 295      Select a File : <input type=file name="file1"> <br>
 296      <input type=submit value="UPLOAD">
 297     </form>
 298   </center>
 299   </body>
 300   </html>
 301  ----------spring-servlet.xml-------------------
 302  <beans xmlns="http://www.springframework.org/schema/beans"
 303   xmlns:context="http://www.springframework.org/schema/context"
 304  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 305  xsi:schemaLocation="http://www.springframework.org/schema/beans
 306  http://www.springframework.org/schema/beans/spring-beans-3.0.xsd
 307  http://www.springframework.org/schema/context
 308  http://www.springframework.org/schema/context/spring-context-3.0.xsd">
 309    <context:component-scan base-package="com.sathya.spring.mvc.controller"/>
 310    <bean id="viewResolver"
 311    class="org.springframework.web.servlet.view.InternalResourceViewResolver">
 312      <property name="prefix" value="/"/>
 313      <property name="suffix" value=".jsp"/>
 314    </bean>
 315    <bean id="multipartResolver"
 316      class="org.springframework.web.multipart.commons.CommonsMultipartResolver"/>
 317  </beans>
 318  ----------web.xml-------------------
 319  <?xml version="1.0" encoding="UTF-8"?>
 320  <web-app id="WebApp_ID" version="2.4" xmlns="http://java.sun.com/xml/ns/j2ee"
 321  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 322  xsi:schemaLocation="http://java.sun.com/xml/ns/j2ee
 323  http://java.sun.com/xml/ns/j2ee/web-app_2_4.xsd">
 324        <display-name>
 325        SpringMVC</display-name>
 326        <servlet>
 327         <servlet-name>spring</servlet-name>
 328         <servlet-class>
 329         org.springframework.web.servlet.DispatcherServlet</servlet-class>
 330         <load-on-startup>1</load-on-startup>
```

```
    </servlet>
    <servlet-mapping>
      <servlet-name>spring</servlet-name>
      <url-pattern>*.html</url-pattern>
    </servlet-mapping>
</web-app>
      Title: SpringI18N application
==========================================================
------------MyController.java--------------------
package com.sathya.spring.i18n;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.servlet.ModelAndView;
@Controller
public class MyController
{
      @RequestMapping("i18ntest")
      public  ModelAndView  welcome()
      {
            return  new  ModelAndView("index");
      }
}
------------index.jsp--------------------
<!-- index.jsp -->
<%@ taglib prefix="spring" uri="http://www.springframework.org/tags" %>
<%@page session="false" %>
<html>
<body>
<a href="i18ntest.form?lang=en">English</a> |
<a href="i18ntest.form?lang=te">Telugu</a> |
<a href="i18ntest.form?lang=hi">Hindi</a>
<br>
 <br>
 <center>
    <spring:message code="welcome.msg"  text="This is default text" />
 </center>
</body>
</html>
------------messages_en.properties--------------------
#english
welcome.msg=This is a message written in English
------------messages_te.properties--------------------
# telugu
welcome.msg=&#3080; &#3128;&#3074;&#3110;&#3143;&#3126;&#3074;&#3108;&#3142;&
&#3137;&#3095;&#3137;&#3122;&#3146;&#3125;&#3149;&#3120;&#3134;&#3128;&#3135
&#3110;&#3135;
------------messages_hi.properties--------------------
# hindi
welcome.msg=&#2351;&#2361; &#2319;&#2325;&#2381; &#2360;&#2344;&#2381;&#2342;
&#2358;&#2381; &#2361;&#2367;&#2344;&#2381;&#2342;&#2381; &#2350;&#2375; &#23
&#2367;&#2326; &#2361;&#2375;
------------spring-servlet.xml--------------------
<beans xmlns="http://www.springframework.org/schema/beans"
      xmlns:mvc="http://www.springframework.org/schema/mvc"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xmlns:context="http://www.springframework.org/schema/context"
      xsi:schemaLocation="http://www.springframework.org/schema/beans
            http://www.springframework.org/schema/beans/spring-beans-4.0.xsd
            http://www.springframework.org/schema/context
      http://www.springframework.org/schema/context/spring-context-4.0.xsd
      http://www.springframework.org/schema/mvc
      http://www.springframework.org/schema/mvc/spring-mvc-4.0.xsd">
      <context:component-scan
            base-package="com.sathya.spring.i18n" />
      <!-- Application Message Bundle -->
      <bean id="messageSource"
```

```
397       class="org.springframework.context.support.ResourceBundleMessageSource">
398            <property name="basename" value="messages" />
399       </bean>
400       <mvc:interceptors>
401       <bean id="localeChange"
402       class="org.springframework.web.servlet.i18n.LocaleChangeInterceptor">
403            <property name="paramName" value="lang" />
404       </bean>
405       </mvc:interceptors>
406       <bean id="localeResolver"
407            class="org.springframework.web.servlet.i18n.SessionLocaleResolver">
408    </bean>
409    <bean id="viewResolver"
410       class="org.springframework.web.servlet.view.InternalResourceViewResolver" >
411       <property name="prefix" value="/"/>
412       <property name="suffix" value=".jsp"/>
413    </bean>
414 </beans>
415 ----------web.xml----------
416 <web-app>
417       <display-name>Spring3MVC</display-name>
418       <welcome-file-list>
419            <welcome-file>index.jsp</welcome-file>
420       </welcome-file-list>
421       <servlet>
422            <servlet-name>spring</servlet-name>
423            <servlet-class>
424                 org.springframework.web.servlet.DispatcherServlet
425            </servlet-class>
426            <load-on-startup>1</load-on-startup>
427       </servlet>
428       <servlet-mapping>
429            <servlet-name>spring</servlet-name>
430            <url-pattern>*.form</url-pattern>
431       </servlet-mapping>
432       <listener>
433       <listener-class>org.springframework.web.context.ContextLoaderListener
434       </listener-class>
435       </listener>
436       <context-param>
437        <param-name>contextConfigLocation</param-name>
438        <param-value>/WEB-INF/spring-servlet.xml</param-value>
439       </context-param>
440 </web-app>
441 ----------
(5) 442       Title: SpringLoginI18N application
443 ==========
444 ----------Login.java----------
445 //command class
446 //Login.java
447 package com.sathya.spring.mvc.controller;
448 import javax.validation.constraints.Size;
449 import org.hibernate.validator.constraints.NotEmpty;
450 public class Login {
451       @NotEmpty
452       private  String uname;
453       @NotEmpty
454       private  String pwd;
455       public String getUname() {
456            return uname;
457       }
458       public void setUname(String uname) {
459            this.uname = uname;
460       }
461       public String getPwd() {
462            return pwd;
```

```
463         }
464         public void setPwd(String pwd) {
465             this.pwd = pwd;
466         }
467 }
468 ------------LoginController.java-------------------
469 package com.sathya.spring.mvc.controller;
470 import javax.validation.Valid;
471 import org.springframework.stereotype.Controller;
472 import org.springframework.validation.BindingResult;
473 import org.springframework.web.bind.annotation.ModelAttribute;
474 import org.springframework.web.bind.annotation.RequestMapping;
475 import org.springframework.web.bind.annotation.RequestMethod;
476 import org.springframework.web.bind.annotation.SessionAttributes;
477 import org.springframework.web.servlet.ModelAndView;
478 @Controller
479 public class LoginController {
480       @RequestMapping("loginpage")
481       public ModelAndView  getLoginPage()
482       {
483             return  new ModelAndView("login","command",new Login());
484       }
485       @RequestMapping(value="logincheck",method=RequestMethod.POST)
486       public  ModelAndView  verifyLogin(@Valid @ModelAttribute("command")
487       Login login, BindingResult  result)
488       {
489             if(result.hasErrors())
490                   return  new  ModelAndView("login");
491             else
492                   return  new  ModelAndView("success","msg","Login Success");
493       }
494 }
495 ------------messages_de.properties------------------   de → german lang. code
496 label.uname=User
497 label.pwd=Psword
498 NotEmpty.command.uname=User is mandatory
499 NotEmpty.command.pwd=Psword is mandatory
500 ------------messages_en.properties------------------
501 label.uname=Username
502 label.pwd=Password
503 NotEmpty.command.uname=Username is mandatory
504 NotEmpty.command.pwd=Password is mandatory
505 ------------spring-servlet.xml-------------------
506 <beans xmlns="http://www.springframework.org/schema/beans"
507  xmlns:context="http://www.springframework.org/schema/context"
508  xmlns:mvc="http://www.springframework.org/schema/mvc"
509 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
510 xsi:schemaLocation="http://www.springframework.org/schema/beans
511 http://www.springframework.org/schema/beans/spring-beans-3.0.xsd
512 http://www.springframework.org/schema/context
513 http://www.springframework.org/schema/context/spring-context-3.0.xsd
514 http://www.springframework.org/schema/mvc
515 http://www.springframework.org/schema/mvc/spring-mvc-3.0.xsd">
516   <mvc:annotation-driven/>
517   <context:component-scan  base-package="com.sathya.spring.mvc.controller"/>
518   <bean id="viewResolver"
519   class="org.springframework.web.servlet.view.InternalResourceViewResolver">
520     <property  name="prefix"  value="/"/>
521     <property  name="suffix"   value=".jsp"/>
522   </bean>
523   <bean  id="messageSource"
524   class="org.springframework.context.support.ResourceBundleMessageSource">
525     <property  name="basename"   value="messages"/>
526   </bean>
527   <mvc:interceptors>
528   <bean id="localeChangeInterceptor"
```

```
        class="org.springframework.web.servlet.i18n.LocaleChangeInterceptor">
            <property name="paramName" value="language" />
    </bean>
    </mvc:interceptors>
    <bean id="localeResolver"
        class="org.springframework.web.servlet.i18n.SessionLocaleResolver"/>
</beans>
-----------index.jsp--------------------
<jsp:forward page="loginpage.html"/>
-----------login.jsp--------------------
<%@ page session="false" %>
<%@taglib uri="http://www.springframework.org/tags/form"  prefix="form"%>
<%@taglib uri="http://www.springframework.org/tags" prefix="spring"%>
<html>
  <body>
    <center>
      <a  href="loginpage.html?language=en">English</a>
      <a href="loginpage.html?language=de">German</a> <br>
   <form:form  action="logincheck.html"  method="post">
    <spring:message code="label.uname"/> :
    <form:input  path="uname"/> <form:errors  path="uname"/><br>
    <spring:message  code="label.pwd"/> :
    <form:password  path="pwd"/><form:errors  path="pwd"/> <br>
    <input type=submit  value="submit">
   </form:form>
  </body>
</html>
-----------success.jsp--------------------
<%-- success.jsp --%>
<h1>
  ${msg}
</h1>
-----------failure.jsp--------------------
<!-- failure.jsp -->
<h1>
  ${msg}
</h1>
      Title:  SpringMVCTiles application
==========================================
-----------TilesController.java--------------------
package com.sathya.spring.mvc.controller;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.servlet.ModelAndView;
@Controller
public class TilesController {
      @RequestMapping("welcome")
      public  ModelAndView  m1()
      {
           return  new  ModelAndView("welcome");
      }
      @RequestMapping("friends")
      public  ModelAndView  m2()
      {
           return new ModelAndView("friends");
      }
      @RequestMapping("office")
      public  ModelAndView  m3()
      {
           return  new  ModelAndView("office");
      }
}
-----------index.jsp--------------------
<%-- index.jsp --%>
<jsp:forward  page="welcome.html"/>
-----------welcome.jsp--------------------
```

```
<h2>
Welcome to Spring with Tiles
</h2>
----------header.jsp--------------------
<div align="center" style="font-weight:bold">
Sathya Technologies
</div>
----------footer.jsp--------------------
<div align="center">&copy; Sekhar.com</div>
----------menu.jsp--------------------
 <br><br>
 <a  href="friends.html">Friends</a> <br><br>
 <a  href="office.html"> Office </a>
----------layout.jsp--------------------
      <!--  layout.jsp -->
<%@taglib uri="http://tiles.apache.org/tags-tiles" prefix="tiles"%>
<html>
  <body>
   <table border="1" cellpadding="2" cellspacing="2" align="center">
    <tr>
     <td height="30" colspan="2">
       <tiles:insertAttribute name="header" />
     </td>
    </tr>
    <tr>
     <td height="250">
       <tiles:insertAttribute name="menu" />
     </td>
     <td width="350">
       <tiles:insertAttribute name="body" />
     </td>
    </tr>
    <tr>
     <td height="30" colspan="2">
       <tiles:insertAttribute name="footer" />
     </td>
    </tr>
   </table>
  </body>
</html>
----------office.jsp--------------------
<html>
   <body>
    <center>
       SATHYA TECHNOLOGIES<br>
       AMEERPET <br>
       HYD
     </center>
    </body>
</html>
----------friends.jsp--------------------
<html>
  <body>
   <center>
      STRUTS<br>
      SPRING<br>
      HIBERNATE<br>
    </center>
  </body>
</html>
----------spring-servlet.xml--------------------
<beans xmlns="http://www.springframework.org/schema/beans"
 xmlns:context="http://www.springframework.org/schema/context"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-3.0.xsd
```

```
661  http://www.springframework.org/schema/context
662  http://www.springframework.org/schema/context/spring-context-3.0.xsd">
663    <context:component-scan  base-package="com.sathya.spring.mvc.controller"/>
664    <bean id="viewResolver"
665    class="org.springframework.web.servlet.view.UrlBasedViewResolver">
666     <property  name="viewClass"
667     value="org.springframework.web.servlet.view.tiles2.TilesView"/>
668    </bean>
669    <bean id="configurer"
670    class="org.springframework.web.servlet.view.tiles2.TilesConfigurer">
671     <property  name="definitions">
672       <list>
673         <value>/WEB-INF/tiles.xml</value>
674       </list>
675     </property>
676    </bean>
677  </beans>
678  ------------tiles.xml--------------------
679  <!DOCTYPE tiles-definitions PUBLIC
680        "-//Apache Software Foundation//DTD Tiles Configuration 3.0//EN"
681        "http://tiles.apache.org/dtds/tiles-config_3_0.dtd">
682              <!--  tiles.xml  -->
683  <tiles-definitions>
684    <definition name="baseLayout" template="/layout.jsp">
685      <put-attribute name="header" value="/header.jsp"/>
686      <put-attribute name="menu"   value="/menu.jsp"/>
687      <put-attribute name="body"   value=""/>
688      <put-attribute name="footer"   value="/footer.jsp"/>
689    </definition>
690
691    <definition name="welcome" extends="baseLayout">
692      <put-attribute name="body"   value="/welcome.jsp"/>
693    </definition>
694
695    <definition name="friends" extends="baseLayout">
696      <put-attribute name="body"   value="/friends.jsp"/>
697    </definition>
698
699    <definition name="office" extends="baseLayout">
700      <put-attribute name="body"   value="/office.jsp"/>
701    </definition>
702  </tiles-definitions>
703  ------------web.xml--------------------
704  <?xml version="1.0" encoding="UTF-8"?>
705  <web-app id="WebApp_ID" version="2.4" xmlns="http://java.sun.com/xml/ns/j2ee"
706  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
707  xsi:schemaLocation="http://java.sun.com/xml/ns/j2ee
708  http://java.sun.com/xml/ns/j2ee/web-app_2_4.xsd">
709        <display-name>
710        SpringMVC</display-name>
711        <servlet>
712          <servlet-name>spring</servlet-name>
713          <servlet-class>org.springframework.web.servlet.DispatcherServlet
714          </servlet-class>
715          <load-on-startup>1</load-on-startup>
716     </servlet>
717     <servlet-mapping>
718       <servlet-name>spring</servlet-name>
719       <url-pattern>*.html</url-pattern>
720     </servlet-mapping>
721  </web-app>
722
723  Title:  PdfExcelApp  application
724  ================================================================:
725  ------------MyController.java-------------------
726  package com.sathya.spring;
```

```
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.servlet.ModelAndView;
@Controller
public class MyController
{
	@RequestMapping("pdf")
	public ModelAndView pdfResponse()
	{
		return new ModelAndView("pdf");
	}
	@RequestMapping("excel")
	public ModelAndView excelResponse()
	{
		return new ModelAndView("excel");
	}

}
----------MyExcelView.java--------------------
package com.sathya.spring;
import java.util.Map;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import org.apache.poi.hssf.usermodel.HSSFRichTextString;
import org.apache.poi.hssf.usermodel.HSSFSheet;
import org.apache.poi.hssf.usermodel.HSSFWorkbook;
import org.springframework.web.servlet.view.document.AbstractExcelView;
public class MyExcelView extends AbstractExcelView {                    HSSF (Horrible SPreadsheet format)
	@Override
	protected void buildExcelDocument(Map arg0,HSSFWorkbook arg1,
	HttpServletRequest arg2, HttpServletResponse arg3)
			throws Exception {
	HSSFSheet sheet=arg1.createSheet("Names");
	getCell(sheet,0,0).setCellValue(new HSSFRichTextString("User Names"));
	getCell(sheet,1,0).setCellValue(new HSSFRichTextString("Sathya"));
	getCell(sheet,2,0).setCellValue(new HSSFRichTextString("Ram"));
	getCell(sheet,3,0).setCellValue(new HSSFRichTextString("Rom"));

	}
}
----------MyPdfView.java--------------------
package com.sathya.spring;
import java.util.Map;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import org.springframework.web.servlet.view.document.AbstractPdfView;
import com.lowagie.text.Document;
import com.lowagie.text.Paragraph;
import com.lowagie.text.Table;
import com.lowagie.text.pdf.PdfWriter;
public class MyPdfView extends AbstractPdfView
{
	@Override
	protected void buildPdfDocument(Map arg0, Document arg1,
	PdfWriter arg2, HttpServletRequest arg3, HttpServletResponse arg4)
		throws Exception {
		Paragraph p = new Paragraph("User details");
		p.setAlignment("center");
		arg1.add(p);
		Table t = new Table(1);
		t.addCell("Sathya");
		t.addCell("Ram");
		t.addCell("Rom");
		arg1.add(t);
	}
}
```

```
793 ----------index.html--------------------
794 <center>
795   <a  href="pdf.form">PDF</a> <br>
796   <a  href="excel.form">Excel</a>
797 </center>
798
799 ----------spring-servlet.xml--------------------
800 <beans xmlns="http://www.springframework.org/schema/beans"
801       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
802       xmlns:context="http://www.springframework.org/schema/context"
803       xsi:schemaLocation="http://www.springframework.org/schema/beans
804       http://www.springframework.org/schema/beans/spring-beans-3.0.xsd
805       http://www.springframework.org/schema/context
806       http://www.springframework.org/schema/context/spring-context-3.0.xsd">
807       <context:component-scan
808             base-package="com.sathya.spring" />
809    <bean id="id"
810    class="org.springframework.web.servlet.view.BeanNameViewResolver"/>
811    <bean  name="pdf"  class="com.sathya.spring.MyPdfView"/>
812    <bean  name="excel"  class="com.sathya.spring.MyExcelView"/>
813       </beans>
814
815 ----------web.xml--------------------
816 <web-app>
817       <display-name>Spring3MVC</display-name>
818       <welcome-file-list>
819             <welcome-file>index.html</welcome-file>
820       </welcome-file-list>
821       <servlet>
822             <servlet-name>spring</servlet-name>
823             <servlet-class>
824                   org.springframework.web.servlet.DispatcherServlet
825             </servlet-class>
826             <load-on-startup>1</load-on-startup>
827       </servlet>
828       <servlet-mapping>
829             <servlet-name>spring</servlet-name>
830             <url-pattern>*.form</url-pattern>
831       </servlet-mapping>
832
833 </web-app>
834
835 Title: struts and spring integration application
836 ==========
837 ----------login.jsp--------------------
838 <%@ taglib uri="http://struts.apache.org/tags-html"  prefix="html"%>
839 <center>
840   <html:errors/> --> to display the error messages
841   <html:form  action="login.do">
842       Username : <html:text  property="uname"/>
843         <br>
844       Password: <html:password  property="pwd"/>
845         <br>
846        <html:submit  value="click"/>
847  </html:form>
848 </center>
849 ----------web.xml--------------------
850 <web-app>                                  It is front controller of Struts
851   <servlet>                                          ↑
852     <servlet-name>action</servlet-name>
853     <servlet-class>org.apache.struts.action.ActionServlet</servlet-class>
854     <load-on-startup> 1 </load-on-startup>
855     <init-param>
856        <param-name>config</param-name>
857        <param-value>/WEB-INF/struts-config.xml</param-value>
858     </init-param>
                                   It is a Struts configuration file
```

```
  </servlet>
  <servlet-mapping>
     <servlet-name>action</servlet-name>
       <url-pattern>*.do</url-pattern>
  </servlet-mapping>
</web-app>
------------struts-config.xml-------------
<!DOCTYPE struts-config PUBLIC
      "-//Apache Software Foundation//DTD Struts Configuration 1.3//EN"
       "http://struts.apache.org/dtds/struts-config_1_3.dtd">
<struts-config>
  <form-beans>
     <form-bean name="frm1" type="com.sathya.struts.LoginForm"/>
  </form-beans>
  <action-mappings>
     <action path="/login" name="frm1"
type="org.springframework.web.struts.DelegatingActionProxy" validate="true"
     input="/login.jsp">
       <forward name="success" path="/success.jsp"/>
       <forward name="failure" path="/failure.jsp"/>
     </action>
  </action-mappings>
  <message-resources parameter="ApplicationResources"/>
 <plug-in className="org.springframework.web.struts.ContextLoaderPlugIn">
    <set-property property="contextConfigLocation"
            value="/WEB-INF/applicationContext.xml"/>
  </plug-in>
</struts-config>
--------------LoginForm.java----------------
package com.sathya.struts;
import org.apache.struts.action.*;
import javax.servlet.http.*;
public class LoginForm extends ActionForm
{
     private String uname,pwd;

     public void setUname(String uname)
     {
          this.uname=uname;
     }
     public String getUname()
     {
          return uname;
     }

     public void setPwd(String pwd)
     {
          this.pwd=pwd;
     }
     public String getPwd()
     {
          return pwd;
     }
     public ActionErrors validate(ActionMapping mapping,
               HttpServletRequest request)
     {
          ActionErrors aes = new ActionErrors();
          if(uname.equals("")||uname.length()==0)
          {
          ActionMessage am1 = new ActionMessage("user.wrong");
          aes.add("uname",am1);
          }
          if(pwd.equals("")||pwd.length()==0)
          {
          ActionMessage am2 = new ActionMessage("pwd.wrong");
               aes.add("pwd",am2);
```

both are same

→ url pattern of request

→ this page is forwarded if validation fails

↳ It is to enable to valida

→ it is resource Bundle name

→ spring-configuration file.

It is given by spring fw

It is key in the Resource Bundle.

```
925             }
926             return aes;
927       }
928 }
929 ----------LoginAction.java----------
930 package com.sathya.struts;
931 import javax.servlet.http.*;
932 import org.apache.struts.action.*;
933 import com.sathya.spring.model.LoginBean;
934 public class LoginAction extends Action
935 {
936       //Here LoginBean is a spring bean. LoginAction
937       //depends on LoginBean
938       private LoginBean loginBean;
939       //setter injection
940       public void setLoginBean(LoginBean loginBean)
941       {
942             this.loginBean=loginBean;
943       }
944 public ActionForward execute(ActionMapping mapping,ActionForm form,
945 HttpServletRequest req,HttpServletResponse res) throws Exception
946       {
947             LoginForm lf=(LoginForm)form;
948             String s1=lf.getUname();
949             String s2=lf.getPwd();
950             //call business method of spring bean
951             int k = loginBean.verify(s1,s2);
952             if(k==1)
953                   return mapping.findForward("success");
954             else
955                   return mapping.findForward("failure");
956
957       }//execute
958 }//class
959 ----------LoginBean.java----------
960 package com.sathya.spring.model;
961 import com.sathya.spring.dao.LoginDAO;
962 public class LoginBean {
963       private LoginDAO loginDAO;
964       public void setLoginDAO(LoginDAO loginDAO)
965       {
966             this.loginDAO=loginDAO;
967       }
968       public int verify(String str1,String str2)
969       {
970             int k=loginDAO.check(str1, str2);
971             return k;
972       }
973 }
974 ----------LoginDAO.java----------
975 package com.sathya.spring.dao;
976 public interface LoginDAO {
977       int check(String str1,String str2);
978 }
979 ----------LoginDAOImpl.java----------
980 package com.sathya.spring.dao;
981 import org.springframework.jdbc.core.JdbcTemplate;
982 public class LoginDAOImpl implements LoginDAO {
983       private JdbcTemplate jt;
984       public void setJt(JdbcTemplate jt)
985       {
986             this.jt=jt;
987       }
988       @Override
989       public int check(String str1, String str2)
990       {
```

```
            int k=jt.queryForInt("select count(*)  from login_table
            where username=? and password=?",str1,str2);
            return k;
        }
}
------------applicationContext.xml--------------
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
  <!-- struts action class configuration -->
  <bean name="/login"  class="com.sathya.struts.LoginAction">
    <property name="loginBean"  ref="logBean"/>
  </bean>
   <!-- LoginBean configuration -->
  <bean id="logBean" class="com.sathya.spring.model.LoginBean">
    <property name="loginDAO"  ref="logDAO"/>
  </bean>
   <!-- dao configuration -->
  <bean id="logDAO" class="com.sathya.spring.dao.LoginDAOImpl">
   <property name="jt" ref="jt"/>
  </bean>
  <!-- JdbcTemplate configuration -->
  <bean id="jt" class="org.springframework.jdbc.core.JdbcTemplate">
    <property name="dataSource" ref="ds"/>
  </bean>
   <!-- DataSource configuration -->
  <bean id="ds"
  class="org.springframework.jdbc.datasource.DriverManagerDataSource">
    <property name="driverClassName" value="oracle.jdbc.OracleDriver"/>
    <property name="url" value="jdbc:oracle:thin:@localhost:1521:orcl3"/>
    <property name="username" value="system"/>
    <property name="password" value="tiger"/>
   </bean>
</beans>
---------------success.jsp----------------------
<%@ taglib uri="http://struts.apache.org/tags-bean"  prefix="bean"%>
<h2>
U entered  Username as : <bean:write  name="frm1" property="uname"/><br>
U entered Password  as : <bean:write  name="frm1" property="pwd"/> <br>
</h2>
<%
   out.println("<h2> login success </h2>");
%>
------------failure.jsp-------------------
<%@ taglib uri="http://struts.apache.org/tags-bean"  prefix="bean"%>
<h2>
U entered  Username as : <bean:write  name="frm1" property="uname"/><br>
U entered Password  as : <bean:write  name="frm1" property="pwd"/> <br>
</h2>
 <%
   out.println("<h2> login  failed </h2>");
 %>
 ----------ApplicationResources.properties------------
user.wrong=<font color='red'>Username is required</font> <br>
pwd.wrong=<font  color='red'>Password is required</font>
```

```
1  ============================================================
2            Title:  Spring-JDBC-Example1
3  ------------Customer.java------------------
4  package com.sathya.spring.dto;
5  public class Customer {
6        private  int  customerId;
7        private String customerName;
8        private  String customerAddress;
9
10       //  --> setters & getters <--
11       public int getCustomerId() {
12            return customerId;
13       }
14       public void setCustomerId(int customerId) {
15            this.customerId = customerId;
16       }
17       public String getCustomerName() {
18            return customerName;
19       }
20       public void setCustomerName(String customerName) {
21            this.customerName = customerName;
22       }
23       public String getCustomerAddress() {
24            return customerAddress;
25       }
26       public void setCustomerAddress(String customerAddress) {
27            this.customerAddress = customerAddress;
28       }
29 }
30 ------------CustomerDAO.java----------------
31 //CustomerDAO.java
32 //DAO interface
33 package com.sathya.spring.dao;
34 import com.sathya.spring.dto.Customer;
35 public interface CustomerDAO {
36   int  insert(Customer c);
37 }
38 ------------CustomerDAOImpl.java-------------
39 //DAO class
40 package com.sathya.spring.dao;
41 import org.springframework.jdbc.core.JdbcTemplate;
42 import com.sathya.spring.dto.Customer;
43 public class CustomerDAOImpl implements CustomerDAO {
44       private  JdbcTemplate  jt;
45       //setter injection
46       public  void setJt(JdbcTemplate jt)
47       {
48            this.jt=jt;
49       }
50       @Override
51       public int insert(Customer c)  {
52            int cid=c.getCustomerId();
53            String cname=c.getCustomerName();
54            String caddr=c.getCustomerAddress();
55       int k=jt.update("insert  into  customer_table  values(?,?,?)",
56            cid,cname,caddr);
57            return k;
58       }
59 }
60 ---------------DemoBean.java---------------
61 //Business class
62 package com.sathya.spring.business;
63 import com.sathya.spring.dao.CustomerDAO;
64 import com.sathya.spring.dto.Customer;
65 public class DemoBean {
66       private CustomerDAO  customerDAO;
```

```
67        //setter injection
68        public void setCustomerDAO(CustomerDAO customerDAO)
69        {
70              this.customerDAO=customerDAO;
71        }
72        public void insertCustomer(int cid,String cname,String caddr)
73        {
74              Customer c=new Customer();
75              c.setCustomerId(cid);
76              c.setCustomerName(cname);
77              c.setCustomerAddress(caddr);
78              int k=customerDAO.insert(c);
79              System.out.println(k+" row inserted");
80        }
81  }
82  ----------applicationContext.xml----------
83              <!--  applicationContext.xml -->
84  <beans xmlns="http://www.springframework.org/schema/beans"
85  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
86  xsi:schemaLocation="http://www.springframework.org/schema/beans
87  http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
88     <!-- business class configuration -->
89   <bean id="db" class="com.sathya.spring.business.DemoBean">
90    <property name="customerDAO" ref="customerDAO"/>
91   </bean>
92        <!--  DAO class configuration -->
93   <bean id="customerDAO" class="com.sathya.spring.dao.CustomerDAOImpl">
94    <property name="jt" ref="jt"/>
95   </bean>
96        <!-- JdbcTemplate class configuration -->
97    <bean id="jt" class="org.springframework.jdbc.core.JdbcTemplate">
98     <constructor-arg ref="ds"/>
99    </bean>
100       <!-- DataSource bean configuration -->
101   <bean id="ds"
102   class="org.springframework.jdbc.datasource.DriverManagerDataSource">
103     <property name="driverClassName" value="oracle.jdbc.OracleDriver"/>
104     <property name="url" value="jdbc:oracle:thin:@localhost:1521:orcl"/>
105     <property name="username" value="system/>
106     <property name="password" value="tiger"/>
107   </bean>
108 </beans>
109 ----------Main.java----------
110 import org.springframework.context.ApplicationContext;
111 import org.springframework.context.support.ClassPathXmlApplicationContext;
112 import com.sathya.spring.business.DemoBean;
113 public class Main {
114   public static void main(String[] args) {
115     ApplicationContext ctx=new ClassPathXmlApplicationContext
116                           ("applicationContext.xml");
117       Object o =ctx.getBean("db");
118       DemoBean db=(DemoBean)o;
119       db.insertCustomer(4,"D", "Hyd");
120       }
121 }
122 ================================================
123       Title:  Spring-JDBC-Example2
124 ----------DemoBean.java----------
125 package com.sathya.spring.business;
126 import java.util.Iterator;
127 import java.util.List;
128 import java.util.Map;
129 import java.util.Set;
130 import org.springframework.beans.factory.annotation.Autowired;
131 import org.springframework.beans.factory.annotation.Qualifier;
132 import com.sathya.spring.dao.SelectDAO;
```

```
public class DemoBean
{
    private SelectDAO selectDAO;
    //setter injection
    @Autowired
    @Qualifier("dao")
    public void setSelectDAO(SelectDAO selectDAO)
    {
        this.selectDAO=selectDAO;
    }
    //business method-1
    public void findCustomerById(int customerId)
    {
        Map map=selectDAO.findById(customerId);
        Set set=map.entrySet();
        Iterator it=set.iterator();
        while(it.hasNext())
        {
            Map.Entry me=(Map.Entry)it.next();
        System.out.println(me.getKey()+" : "+me.getValue());
        }
    } //end of findCustomerById
    //Business method-2
    public void findAllCustomers()
    {
        List list=selectDAO.findAll();
        Iterator it =list.iterator();
        while(it.hasNext())
        {
            Map map=(Map)it.next();
            Set set=map.entrySet();
            Iterator it1=set.iterator();
            while(it1.hasNext())
            {
                Map.Entry me=(Map.Entry)it1.next();
        System.out.println(me.getKey()+" : "+me.getValue());
            }//end of inner while loop
            System.out.println("==========");
        } //end of outer while loop

    }//end of findAllCustomers
}
----------------SelectDAO.java--------------------
//SelectDAO
//DAO interface
package com.sathya.spring.dao;
import java.util.List;
import java.util.Map;
public interface SelectDAO {
    Map findById(int custid);
    List findAll();
}
------------SelectDAOImpl.java-----------------
//SelectDAOImpl.java
//DAO class
package com.sathya.spring.dao;
import java.util.List;
import java.util.Map;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.jdbc.core.JdbcTemplate;
public class SelectDAOImpl implements SelectDAO {
    private JdbcTemplate jt;
    //setter injection
    @Autowired
    @Qualifier("jt")
```

```
199       public void setJt(JdbcTemplate jt)
200       {
201            this.jt=jt;
202       }
203       @Override
204       public List findAll() {
205       List list=jt.queryForList("select * from customer_table");
206            return list;
207       }
208       @Override
209       public Map findById(int custid) {
210       Map map=jt.queryForMap("select * from customer_table where cid=?",custid);
211            return map;
212       }
213  }
214  ----------applicationContext.xml----------
215  <beans xmlns="http://www.springframework.org/schema/beans"
216  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
217  xsi:schemaLocation="http://www.springframework.org/schema/beans
218  http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
219    <!-- business class configuration -->
220    <bean id="db" class="com.sathya.spring.business.DemoBean"/>
221    <!-- dao class configuration -->
222    <bean id="dao" class="com.sathya.spring.dao.SelectDAOImpl"/>
223     <!-- JdbcTemplate class configuration -->
224     <bean id="jt" class="org.springframework.jdbc.core.JdbcTemplate">
225      <property name="dataSource" ref="ds"/>
226     </bean>
227     <!-- DataSource class configuration -->
228     <bean id="ds" class="org.apache.tomcat.dbcp.dbcp.BasicDataSource">
229      <property name="driverClassName" value="oracle.jdbc.OracleDriver"/>
230     <property name="url" value="jdbc:oracle:thin:@localhost:1521:orcl3"/>
231      <property name="username" value="system"/>
232      <property name="password" value="tiger"/>
233    </bean>
234    <context:annotation-config/><-- to enable the autowired annotation.
235  </beans>
236  ----------Main.java----------
237  import org.springframework.context.ApplicationContext;
238  import org.springframework.context.support.ClassPathXmlApplicationContext;
239  import com.sathya.spring.business.DemoBean;
240  public class Main {
241       public static void main(String args[ ])
242       {
243       ApplicationContext ctx=new ClassPathXmlApplicationContext
244                              ("applicationContext.xml");
245            Object o=ctx.getBean("db");
246            DemoBean db=(DemoBean)o;
247            db.findCustomerById(1);
248            System.out.println("=========");
249            db.findAllCustomers();
250       }
251  }
252  ==========================================
253  Title: Spring-JDBC-Example3 (with RowMapper)
254  ----------DemoBean.java----------
255  //DemoBean.java(business class)
256  package com.sathya.spring.model;
257  import java.util.Iterator;
258  import java.util.List;
259  import com.sathya.spring.dao.EmployeeDAO;
260  public class DemoBean {
261       private EmployeeDAO employeeDAO;
262       //setter injection
263       public void setEmployeeDAO(EmployeeDAO employeeDAO)
264       {
```

```
265             this.employeeDAO=employeeDAO;
266         }
267         public  void  findAllEmployees()
268         {
269             List  list=employeeDAO.selectAllEmployees();
270             Iterator  it= iist.iterator();
271             while(it.hasNext())
272             {
273                 Employee  e=(Employee)it.next();
274                 System.out.println("empno = "+e.getEmpno());
275                 System.out.println("ename = "+e.getEname());
276                 System.out.println("sal = "+e.getSal());
277                 System.out.println("deptno = "+e.getDeptno());
278                 System.out.println("==============");
279             }
280         }
281 }
282 ------------Employee.java------------------
283 //Employee.java (pojo)
284 package com.sathya.spring.model;
285 public class Employee {
286         private int empno;
287         private String ename;
288         private  int sal;
289         private int deptno;
290
291         //setters & getters
292         public int getEmpno() {
293             return empno;
294         }
295         public void setEmpno(int empno) {
296             this.empno = empno;
297         }
298         public String getEname() {
299             return ename;
300         }
301         public void setEname(String ename) {
302             this.ename = ename;
303         }
304         public int getSal() {
305             return sal;
306         }
307         public void setSal(int sal) {
308             this.sal = sal;
309         }
310         public int getDeptno() {
311             return deptno;
312         }
313         public void setDeptno(int deptno) {
314             this.deptno = deptno;
315         }
316 }
317 ------------EmployeeDAO.java----------------
318 //EmployeeDAO.java (Dao interface)
319 package com.sathya.spring.dao;
320 import java.util.List;
321 public interface EmployeeDAO {
322         List  selectAllEmployees();
323 }
324 ------------EmployeeDAOImpl.java--------------
325 //EmployeeDAOImpl.java(dao class)
326 package com.sathya.spring.dao;
327 import java.util.List;
328 import org.springframework.jdbc.core.JdbcTemplate;
329 public class EmployeeDAOImpl implements EmployeeDAO {
330         private  JdbcTemplate jt;
```

```
331       //setter injection
332       public void setJt(JdbcTemplate jt)
333       {
334            this.jt=jt;
335       }
336       @Override
337       public List selectAllEmployees() {
338       List list=jt.query("select * from emp",new EmployeeMapper());
339       return list;
340       }
341  }
342  ---------------EmployeeMapper.java---------------
343  //EmployeeMapper.java(callback class)
344  package com.sathya.spring.dao;
345  import java.sql.ResultSet;
346  import java.sql.SQLException;
347  import org.springframework.jdbc.core.RowMapper;
348  import com.sathya.spring.model.Employee;
349  public class EmployeeMapper implements RowMapper
350  {
351       @Override
352  public Object mapRow(ResultSet rs, int index) throws SQLException
353       {
354            Employee e=new Employee();
355            e.setEmpno(rs.getInt(1));
356            e.setEname(rs.getString(2));
357            e.setSal(rs.getInt(3));
358            e.setDeptno(rs.getInt(4));
359            return e;
360       }
361  }
362  ---------------applicationContext.xml--------------------
363  <beans xmlns="http://www.springframework.org/schema/beans"
364  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
365  xsi:schemaLocation="http://www.springframework.org/schema/beans
366  http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
367       <!-- business class configuration -->
368     <bean id="db" class="com.sathya.spring.model.DemoBean">
369        <property name="employeeDAO" ref="dao"/>
370     </bean>
371     <!-- dao class configuration -->
372     <bean id="dao" class="com.sathya.spring.dao.EmployeeDAOImpl">
373       <property name="jt" ref="jt"/>
374     </bean>
375     <!-- JdbcTemplate class configuration -->
376     <bean id="jt" class="org.springframework.jdbc.core.JdbcTemplate">
377      <property name="dataSource" ref="ds"/>
378     </bean>
379     <!-- DataSource configuration -->
380     <bean id="ds"
381     class="org.springframework.jdbc.datasource.DriverManagerDataSource">
382       <property name="driverClassName" value="oracle.jdbc.OracleDriver"/>
383      <property name="url" value="jdbc:oracle:thin:@localhost:1521:orcl3"/>
384       <property name="username" value="system"/>
385       <property name="password" value="tiger"/>
386     </bean>
387  </beans>
388  ---------------Main.java----------------------
389  import org.springframework.context.ApplicationContext;
390  import org.springframework.context.support.ClassPathXmlApplicationContext;
391  import com.sathya.spring.model.DemoBean;
392  public class Main {
393       public static void main(String[ ] args) {
394       ApplicationContext ctx = new ClassPathXmlApplicationContext
395                              ("applicationContext.xml");
396       Object o = ctx.getBean("db");
```

List list=jt.query("select * from emp",new EmployeeMapper());
- "select * from emp" → Command
- new EmployeeMapper() → mapRow();

EmployeeMapper implements RowMapper → Callback interface

Callback interface used to define a Part of code or logic to JdbcTemplate by Programmer.

DemoBean depends on EmployeeDAOImpl.

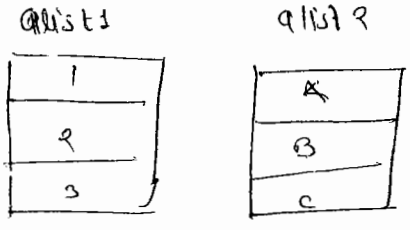
EmployeeDAOImpl depends on JdbcTemplate

JdbcTemplate depends on DataSource.

```
397         DemoBean db=(DemoBean)o;
398         db.findAllEmployees();
399         }
400  }
401  ==========================================
402     Title: Spring-JDBC-Example4(for calling a function)
403  ----------DemoBean.java--------------------
404  package com.sathya.spring.model;
405  import java.util.Map;
406  import org.springframework.beans.factory.annotation.Autowired;
407  import org.springframework.beans.factory.annotation.Qualifier;
408  import com.sathya.spring.dao.TestDAO;
409  public class DemoBean
410  {
411       private TestDAO  testDAO;
412       @Autowired
413       @Qualifier("dao")
414       public void setTestDAO(TestDAO testDAO)
415       {
416            this.testDAO=testDAO;
417       }
418       public void  bm()
419       {
420            Map map=testDAO.callFunction();
421            int bonus=(Integer)map.get("b");
422            System.out.println("Bonus = "+bonus);
423       }
424  }
425  ----------TestDAO.java--------------------
426  //TestDAO.java(dao interface)
427  package com.sathya.spring.dao;
428  import java.util.Map;
429  public interface TestDAO {
430       Map callFunction();
431  }
432  ----------TestDAOImpl.java----------------
433  //TestDAOImpl.java (dao class)
434  package com.sathya.spring.dao;
435  import java.sql.Types;
436  import java.util.ArrayList;
437  import java.util.Map;
438  import org.springframework.beans.factory.annotation.Autowired;
439  import org.springframework.beans.factory.annotation.Qualifier;
440  import org.springframework.jdbc.core.JdbcTemplate;
441  import org.springframework.jdbc.core.SqlOutParameter;
442  import org.springframework.jdbc.core.SqlParameter;
443  public class TestDAOImpl implements TestDAO
444  {
445       private JdbcTemplate jt;
446       @Autowired
447       @Qualifier("jtemplate")
448       public void setJt(JdbcTemplate jt)
449       {
450            this.jt=jt;
451       }
452       @Override
453       public Map callFunction()
454       {
455            ArrayList alist=new ArrayList();
456            alist.add(new SqlOutParameter("b", Types.INTEGER));
457            alist.add(new SqlParameter(Types.INTEGER));
458            Map map=jt.call(new MyCallableStatementCreator(),alist);
459            return map;
460       }
461  }
462  ----------MyCallableStatementCreator.java----------
```

```
package com.sathya.spring.dao;
import java.sql.CallableStatement;
import java.sql.Connection;
import java.sql.SQLException;
import java.sql.Types;
import org.springframework.jdbc.core.CallableStatementCreator;
public class MyCallableStatementCreator implements CallableStatementCreator
{                                                  ↳ call back interface
    @Override
    public CallableStatement createCallableStatement(Connection con)
            throws SQLException
    {
    CallableStatement cstmt= con.prepareCall("{?=call fun1(?)}");
                                                  Procedure :- not start with ?
        cstmt.setInt(2,7965);                     function → starts with "?"
        cstmt.registerOutParameter(1,Types.INTEGER);
        return cstmt;
    }
}
------------applicationContext.xml--------------------
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-3.0.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context-3.0.xsd">
   <!-- business class configuration -->
 <bean id="db" class="com.sathya.spring.model.DemoBean"/>
   <!-- dao class configuration -->
 <bean id="dao" class="com.sathya.spring.dao.TestDAOImpl"/>
   <!-- JdbcTemplate class configuration -->
 <bean id="jtemplate"
 class="org.springframework.jdbc.core.JdbcTemplate">
   <property name="dataSource" ref="ds"/>
 </bean>
 <!-- DataSource configuration -->
  <bean id="ds"
  class="org.springframework.jdbc.datasource.DriverManagerDataSource">
  <property name="driverClassName" value="oracle.jdbc.OracleDriver"/>
  <property name="url" value="jdbc:oracle:thin:@localhost:1521:orcl3"/>
  <property name="username" value="system"/>
    <property name="password" value="tiger"/>
  </bean>
  <context:annotation-config/>
</beans>
---------------Main.java-----------------------
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import com.sathya.spring.model.DemoBean;
public class Main
{
    public static void main(String[ ] args)
    {
    ApplicationContext ctx=new ClassPathXmlApplicationContext
                        ("applicationContext.xml");
    Object o = ctx.getBean("db");
    DemoBean db=(DemoBean)o;
    db.bm();
    }
}
(5) ================================================================= (5)
    Title: Spring-JDBC-Batch processing
---------------DemoBean.java-----------------------
package com.sathya.spring.model;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Qualifier;
```

```
import com.sathya.spring.dao.TestDAO;
public class DemoBean
{
	private TestDAO  testDAO;
	@Autowired
	@Qualifier("dao")
	public void  setTestDAO(TestDAO  testDAO)
	{
		this.testDAO=testDAO;
	}
	public  void  bm()
	{
		testDAO.insertBatch();
	}
}
----------TestDAO.java-------------------------
package com.sathya.spring.dao;
public interface TestDAO {
	void  insertBatch();
}
----------TestDAOImpl.java----------------------
package com.sathya.spring.dao;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.jdbc.core.JdbcTemplate;
public class TestDAOImpl implements TestDAO {
	private  JdbcTemplate  jt;
	@Autowired
	@Qualifier("jt")
	public  void setJt(JdbcTemplate jt)
	{
		this.jt=jt;
	}
	@Override
	public void insertBatch()
	{
		int k[ ]=jt.batchUpdate("insert into Test values(?,?)",
		new MyBatchPreparedStatementSetter());
		System.out.println("Batch Executed");
	}
}
----------MyBatchPreparedStatementSetter.java----------------------
package com.sathya.spring.dao;
import java.sql.PreparedStatement;
import java.sql.SQLException;
import java.util.ArrayList;
import org.springframework.jdbc.core.BatchPreparedStatementSetter;
public class MyBatchPreparedStatementSetter implements
		BatchPreparedStatementSetter {
	ArrayList  alist1 = new  ArrayList();
	ArrayList  alist2 = new  ArrayList();
	public MyBatchPreparedStatementSetter()
	{
		alist1.add(1);
		alist1.add(2);
		alist1.add(3);
		alist2.add("A");
		alist2.add("B");
		alist2.add("C");
	}
	@Override
	public int getBatchSize() {
		return 3;
	}
	@Override
	public void setValues(PreparedStatement pstmt, int i) throws SQLException
```



```
595       {
596            pstmt.setInt(1,Integer.parseInt(alist1.get(i).toString()));
597            pstmt.setString(2,alist2.get(i).toString());
598       }
599  }
600  ----------spring.xml----------
601  <beans xmlns="http://www.springframework.org/schema/beans"
602  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
603  xmlns:context="http://www.springframework.org/schema/context"
604  xsi:schemaLocation="http://www.springframework.org/schema/beans
605  http://www.springframework.org/schema/beans/spring-beans-3.0.xsd
606  http://www.springframework.org/schema/context
607  http://www.springframework.org/schema/context/spring-context-3.0.xsd">
608     <!-- business class configuration -->
609   <bean id="db" class="com.sathya.spring.model.DemoBean"/>
610     <!-- dao class configuration -->
611   <bean id="dao" class="com.sathya.spring.dao.TestDAOImpl"/>
612     <!-- JdbcTemplate class configuration -->
613   <bean id="jt"   class="org.springframework.jdbc.core.JdbcTemplate">
614     <property name="dataSource"  ref="jndi"/>
615   </bean>
616   <bean id="jndi"  class="org.springframework.jndi.JndiObjectFactoryBean">
617     <property name="jndiName" value="oraclejndi"/>
618     <property name="jndiEnvironment">
619       <props>
620         <prop key="java.naming.factory.initial">
621         com.sun.enterprise.naming.impl.SerialInitContextFactory</prop>
622         <prop key="java.naming.provider.url">iiop://localhost:4848</prop>
623       </props>
624     </property>
625   </bean>
626   <context:annotation-config/>
627  </beans>
628  ----------Main.java----------
629  import org.springframework.context.ApplicationContext;
630  import org.springframework.context.support.ClassPathXmlApplicationContext;
631  import com.sathya.spring.model.DemoBean;
632  public class Main
633  {
634       public static  void main(String args[ ])
635       {
636       ApplicationContext  ctx=new  ClassPathXmlApplicationContext("spring.xml");
637       Object  o = ctx.getBean("db");
638       DemoBean db=(DemoBean)o;
639       db.bm();
640       }
641  }
642  ==========
643       Title: Spring-Hibernate  integration(with Spring-ORM)
644  ----------DemoBean.java----------
645  package com.sathya.spring.model;
646  import java.util.Iterator;
647  import java.util.List;
648  import com.sathya.spring.dao.CurdDAO;
649  public class DemoBean
650  {
651       private CurdDAO curdDAO;
652       //setter injection
653       public void setCurdDAO(CurdDAO  curdDAO)
654       {
655            this.curdDAO=curdDAO;
656       }
657       public  void  insertStudent(int sid,String sname,int marks)
658       {
659            Student s =new Student();
660            s.setStudentId(sid);
```

```
661             s.setStudentName(sname);
662             s.setMarks(marks);
663             curdDAO.save(s);
664         }
665         public void updateStudent(int sid,String sname,int marks)
666         {
667             Student s =new Student();
668             s.setStudentId(sid);
669             s.setStudentName(sname);
670             s.setMarks(marks);
671             curdDAO.update(s);
672         }
673         public  void selectStudents()
674         {
675             List  list=curdDAO.findStudents();
676             Iterator it =list.iterator();
677             while(it.hasNext())
678             {
679                 Student s=(Student)it.next();
680                 System.out.println(s.getStudentId()+" "+s.getStudentName()
681                 +" "+s.getMarks());
682             } //end of while
683         } //end of method
684 } //end of class
685 ----------------Student.java---------------------
686 //Student.java(pojo class)
687 package com.sathya.spring.model;
688 public class Student
689 {
690         private int studentId;
691         private String  studentName;
692         private int  marks;
693         //setters & getters
694         public int getStudentId() {
695             return studentId;
696         }
697         public void setStudentId(int studentId) {
698             this.studentId = studentId;
699         }
700         public String getStudentName() {
701             return studentName;
702         }
703         public void setStudentName(String studentName) {
704             this.studentName = studentName;
705         }
706         public int getMarks() {
707             return marks;
708         }
709         public void setMarks(int marks) {
710             this.marks = marks;
711         }
712 }
713 ----------------student.hbm.xml---------------------
714 <!DOCTYPE hibernate-mapping PUBLIC "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
715   "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
716   <!-- student.hbm.xml -->
717 <hibernate-mapping>
718  <class name="com.sathya.spring.model.Student" table="student">
719   <id name="studentId" column="sid"/>
720   <property name="studentName" column="sname"/>
721   <property name="marks" column="marks"/>
722  </class>
723 </hibernate-mapping>
724 -------------applicationContext.xml--------------------
725 <beans xmlns="http://www.springframework.org/schema/beans"
726 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
  <!-- business class configuration -->
 <bean id="db" class="com.sathya.spring.model.DemoBean">
  <property name="curdDAO" ref="dao"/>
 </bean>
     <!-- dao class configuration -->
 <bean id="dao" class="com.sathya.spring.dao.CurdDAOImpl">
  <property name="ht" ref="ht"/>
 </bean>
  <!-- HibernateTemplate class configuration -->
 <bean id="ht" class="org.springframework.orm.hibernate3.HibernateTemplate">
  <property name="sessionFactory" ref="lsfb"/>
 </bean>
  <!-- LocalSessionFactoryBean configuration -->
<bean id="lsfb" class="org.springframework.orm.hibernate3.LocalSessionFactoryBean">
  <property name="dataSource" ref="ds"/>
  <property name="hibernateProperties">
   <props>
    <prop key="dialect">org.hibernate.dialect.Oracle10gDialect</prop>
    <prop key="show_sql">true</prop>
   </props>
  </property>
  <property name="mappingResources">
   <list>
    <value> com/sathya/spring/config/student.hbm.xml </value>
   </list>
  </property>
 </bean>
  <!-- DataSource configuration -->
<bean id="ds" class="org.springframework.jdbc.datasource.DriverManagerDataSource">
  <property name="driverClassName" value="oracle.jdbc.OracleDriver"/>
  <property name="url" value="jdbc:oracle:thin:@localhost:1521:orcl3"/>
  <property name="username" value="system"/>
  <property name="password" value="tiger"/>
 </bean>
</beans>
----------------CurdDAO.java--------------------
//CurdDAO.java
//dao interface
package com.sathya.spring.dao;
import java.util.List;
public interface CurdDAO
{
     void save(Object o);
     void update(Object o);
     List findStudents();
}
----------------CurdDAOImpl.java----------------
//CurdDAOImpl.java
//dao class
package com.sathya.spring.dao;
import java.util.List;
import org.springframework.orm.hibernate3.HibernateTemplate;
public class CurdDAOImpl implements CurdDAO
{
     private HibernateTemplate ht;
     //setter injection
     public void setHt(HibernateTemplate ht)
     {
          this.ht=ht;
     }

     @Override
     public List findStudents() {
          List list = ht.find("from Student s");
```

```
793             return list;
794         }
795         @Override
796         public void save(Object o) {
797             ht.save(o);
798         }
799         @Override
800         public void update(Object o) {
801             ht.update(o);
802         }
803 }
804 ----------------Main.java---------------------
805 //Main.java
806 import org.springframework.context.ApplicationContext;
807 import org.springframework.context.support.ClassPathXmlApplicationContext;
808 import com.sathya.spring.model.DemoBean;
809 public class Main
810 {
811     public static void main(String[] args)
812     {
813         ApplicationContext ctx=
814         new ClassPathXmlApplicationContext(
815             "com/sathya/spring/config/applicationContext.xml");
816         Object o = ctx.getBean("db");
817         DemoBean db=(DemoBean)o;
818         db.insertStudent(111, "AAA", 600);
819         db.updateStudent(111,"BBB", 900);
820         db.selectStudents();
821     }
822 }
823 ============================================================
824     Title: Spring-Hibernate integration(without Spring-ORM)
825 --------------DemoBean.java----------------------
826 //DemoBean.java
827 //business class
828 package com.sathya.spring.model;
829 import java.util.Iterator;
830 import java.util.List;
831 import com.sathya.spring.dao.CurdDAO;
832 public class DemoBean
833 {
834     private CurdDAO curdDAO;
835     //setter injection
836     public void setCurdDAO(CurdDAO curdDAO)
837     {
838         this.curdDAO=curdDAO;
839     }
840     public void insertStudent(int sid,String sname,int marks)
841     {
842         Student s =new Student();
843         s.setStudentId(sid);
844         s.setStudentName(sname);
845         s.setMarks(marks);
846         curdDAO.save(s);
847     }
848     public void updateStudent(int sid,String sname,int marks)
849     {
850         Student s =new Student();
851         s.setStudentId(sid);
852         s.setStudentName(sname);
853         s.setMarks(marks);
854         curdDAO.update(s);
855     }
856     public void selectStudents()
857     {
858         List list=curdDAO.findStudents();
```

```
859             Iterator it =list.iterator();
860             while(it.hasNext())
861             {
862                  Student s=(Student)it.next();
863                  System.out.println(s.getStudentId()+" "+s.getStudentName()
864                  +" "+s.getMarks());
865             } //end of while
866        } //end of method
867   } //end of class
868   ----------------Student.java---------------------
869   //Student.java(pojo class)
870   package com.sathya.spring.model;
871   public class Student
872   {
873        private int studentId;
874        private String  studentName;
875        private int  marks;
876        //setters & getters
877        public int getStudentId() {
878             return studentId;
879        }
880        public void setStudentId(int studentId) {
881             this.studentId = studentId;
882        }
883        public String getStudentName() {
884             return studentName;
885        }
886        public void setStudentName(String studentName) {
887             this.studentName = studentName;
888        }
889        public int getMarks() {
890             return marks;
891        }
892        public void setMarks(int marks) {
893             this.marks = marks;
894        }
895   }
896   ----------------student.hbm.xml----------------------
897   <!DOCTYPE hibernate-mapping PUBLIC "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
898     "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
899    <!-- student.hbm.xml -->
900   <hibernate-mapping>
901    <class name="com.sathya.spring.model.Student" table="student">
902     <id name="studentId" column="sid"/>
903     <property name="studentName" column="sname"/>
904     <property name="marks" column="marks"/>
905    </class>
906   </hibernate-mapping>
907   -------------applicationContext.xml---------------------
908   <beans xmlns="http://www.springframework.org/schema/beans"
909   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
910   xsi:schemaLocation="http://www.springframework.org/schema/beans
911   http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
912    <!-- business class configuration -->
913    <bean id="db" class="com.sathya.spring.model.DemoBean">
914     <property name="curdDAO"  ref="dao"/>
915    </bean>
916        <!-- dao class configuration -->
917    <bean id="dao" class="com.sathya.spring.dao.CurdDAOImpl"/>
918    </beans>
919    ---------------hibernate.cfg.xml-----------------------
920        <!-- hibernate.cfg.xml -->
921   <!DOCTYPE hibernate-configuration PUBLIC
922        "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
923        "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
924
```

```
<hibernate-configuration>
 <session-factory>
        <!-- connection properties -->
  <property name="hibernate.connection.driver_class">oracle.jdbc.OracleDriver
    </property>
  <property name="hibernate.connection.url">
    jdbc:oracle:thin:@localhost:1521:ORCL3</property>
  <property name="hibernate.connection.username"> system</property>
  <property name="hibernate.connection.password"> tiger</property>

        <!-- hibernate properties -->
  <property name="hibernate.dialect"> org.hibernate.dialect.Oracle10gDialect
    </property>
  <property name="hibernate.show_sql">true </property>
  <property name="hibernate.hbm2ddl.auto"> update </property>

        <!-- mapping resources -->
  <mapping resource="com/sathya/spring/config/student.hbm.xml"/>
 </session-factory>
</hibernate-configuration>
----------CurdDAO.java----------
//CurdDAO.java
//dao interface
package com.sathya.spring.dao;
import java.util.List;
public interface CurdDAO
{
    void save(Object o);
    void update(Object o);
    List findStudents();
}
----------CurdDAOImpl.java----------
//CurdDAOImpl.java
//dao class
package com.sathya.spring.dao;
import java.util.List;
import org.hibernate.Query;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.cfg.Configuration;
public class CurdDAOImpl implements CurdDAO
{
    private static SessionFactory factory=null;
    static
    {
        factory=new Configuration().
        configure("com/sathya/spring/config/hibernate.cfg.xml").
        buildSessionFactory();
    }
    @Override
    public List findStudents() {
        Session session=factory.openSession();
        Query qry=session.createQuery("from Student s");
        List list = qry.list();
        return list;
    }
    @Override
    public void save(Object o) {
        Session session = factory.openSession();
        Transaction tx=session.beginTransaction();
        session.save(o);
        tx.commit();
        session.close();
    }
    @Override
```

```
991         public void update(Object o) {
992             Session session = factory.openSession();
993             Transaction tx=session.beginTransaction();
994             session.update(o);
995             tx.commit();
996             session.close();
997         }
998     }
999     ----------Main.java-------------------------
1000    //Main.java
1001    import org.springframework.context.ApplicationContext;
1002    import org.springframework.context.support.ClassPathXmlApplicationContext;
1003    import com.sathya.spring.model.DemoBean;
1004    public class Main
1005    {
1006        public static void main(String[] args)
1007        {
1008            ApplicationContext ctx=new ClassPathXmlApplicationContext(
1009            "com/sathya/spring/config/applicationContext.xml");
1010            Object o = ctx.getBean("db");
1011            DemoBean db=(DemoBean)o;
1012            db.insertStudent(111, "AAA", 600);
1013            db.updateStudent(111,"BBB", 900);
1014            db.selectStudents();
1015        }
1016    }
1017
```

Nitin

```
------Declarative Transaction ------------
//Demo.java
package com.sathya.spring.transaction;
public interface Demo {
  void  method1();
}
//DemoBean.java
package com.sathya.spring.transaction;
import org.springframework.jdbc.core.JdbcTemplate;
public class DemoBean implements Demo
{
  private JdbcTemplate jt;
   public void setJt(JdbcTemplate jt)
   {
    this.jt=jt;
   }
   public void method1()
   {
     jt.update("insert into dept values(60,'EDU','HYD')");
     jt.update("update emp set sal=6000  where empno=7788");
      try {
         Thread.sleep(10000);
       } catch (InterruptedException e) {
        e.printStackTrace();
       }
     jt.update("delete from sathya  where stuid=111");
  }
}
--------------------applicationContext.xml---------------------------------------
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:aop="http://www.springframework.org/schema/aop"
xmlns:tx="http://www.springframework.org/schema/tx"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-3.0.xsd
http://www.springframework.org/schema/aop
http://www.springframework.org/schema/aop/spring-aop-3.0.xsd
http://www.springframework.org/schema/tx
http://www.springframework.org/schema/tx/spring-tx-3.0.xsd">
 <!-- business bean configuration -->
 <bean id="db"  class="com.sathya.spring.transaction.DemoBean">
   <property name="jt"   ref="jt"/>
 </bean>
   <!--  transaction manager  -->
 <bean id="txm"
 class="org.springframework.jdbc.datasource.DataSourceTransactionManager">
   <property name="dataSource"  ref="ds"/>
 </bean>
    <!--  JdbcTemplate  -->
 <bean id="jt"
  class="org.springframework.jdbc.core.JdbcTemplate">
   <property name="dataSource" ref="ds"/>
 </bean>
    <!--  DriverManagerDataSource -->
 <bean id="ds"
 class="org.springframework.jdbc.datasource.DriverManagerDataSource">
   <property name="driverClassName" value="oracle.jdbc.OracleDriver"/>
   <property name="url" value="jdbc:oracle:thin:@localhost:1521:xe"/>
   <property name="username" value="system"/>
   <property name="password" value="root"/>
 </bean>
 <!--  transaction configuration -->
<tx:advice id="advice1"  transaction-manager="txm">
 <tx:attributes>
   <tx:method name="method1"
                        propagation="REQUIRES_NEW"
```

```
                                isolation="READ_COMMITTED"
                                  timeout="18"/>
    </tx:attributes>
   </tx:advice>
   <aop:config>
     <aop:pointcut  id="pointcut1"
                expression="execution(*  com.sathya.spring.transaction.*.*(..))"/>
     <aop:advisor  advice-ref="advice1"
                 pointcut-ref="pointcut1"/>
   </aop:config>
  </beans>
  ------------------------Main.java-----------------------------
  import org.springframework.context.ApplicationContext;
 import org.springframework.context.support.ClassPathXmlApplicationContext;
 import com.sathya.spring.transaction.Demo;
 public class Main
 {
      public static void main(String[] args)
      {
           ApplicationContext  ctx =new  ClassPathXmlApplicationContext
                                                  ("applicationContext.xml");
           Demo  db=(Demo)ctx.getBean("db");
           try{
           db.method1();
           }catch(Exception e)
           {
                System.out.println(e);
           }
      }
 }
 ==============Annotation Transaction=====================
 //Demo.java
 package com.sathya.spring.transaction;
 public interface Demo {
   void method1();
 }
 //DemoBean.java
 package com.sathya.spring.transaction;
 import org.springframework.jdbc.core.JdbcTemplate;
 import org.springframework.transaction.annotation.Isolation;
 import org.springframework.transaction.annotation.Propagation;
 import org.springframework.transaction.annotation.Transactional;
 public class DemoBean implements Demo
 {
      private  JdbcTemplate  jt;
      public  void  setJt(JdbcTemplate  jt)
      {
          this.jt=jt;
      }

      @Transactional(propagation=Propagation.REQUIRES_NEW,
      isolation=Isolation.SERIALIZABLE, timeout=5)
      public  void  method1()
      {
                jt.update("insert  into  dept values(20,'RESEARCH','HYD')");
                jt.update("update emp set sal=9900  where empno=7150");
                try {
                     Thread.sleep(4000);
                } catch (InterruptedException e) {
                     e.printStackTrace();
                }
                jt.update("delete from  sathya  where stuid=111");
     }
 }
 ------------ --------applicationContext.xml--------------------------------
 <beans xmlns="http://www.springframework.org/schema/beans"
```

```
133
134  xmlns:tx="http://www.springframework.org/schema/tx"
135  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
136  xsi:schemaLocation="http://www.springframework.org/schema/beans
137  http://www.springframework.org/schema/beans/spring-beans-3.0.xsd
138  http://www.springframework.org/schema/tx
139  http://www.springframework.org/schema/tx/spring-tx-3.0.xsd">
140
141   <!-- business bean configuration -->
142   <bean id="db"  class="com.sathya.spring.transaction.DemoBean">
143     <property name="jt"   ref="jt"/>
144   </bean>
145     <!--  transaction manager -->
146   <bean id="txm"
147   class="org.springframework.jdbc.datasource.DataSourceTransactionManager">
148     <property name="dataSource"  ref="ds"/>
149   </bean>
150      <!--  JdbcTemplate -->
151   <bean id="jt" class="org.springframework.jdbc.core.JdbcTemplate">
152     <property name="dataSource" ref="ds"/>
153   </bean>
154      <!--  DriverManagerDataSource -->
155   <bean id="ds" class="org.springframework.jdbc.datasource.DriverManagerDataSource">
156     <property name="driverClassName" value="oracle.jdbc.OracleDriver"/>
157     <property name="url" value="jdbc:oracle:thin:@localhost:1521:xe"/>
158     <property name="username" value="system"/>
159     <property name="password" value="root"/>
160   </bean>
161    <tx:annotation-driven transaction-manager="txm"/>
162  </beans>
163  -------------------------Main.java-------------------------------
164  import org.springframework.context.ApplicationContext;
165  import org.springframework.context.support.ClassPathXmlApplicationContext;
166  import com.sathya.spring.transaction.Demo;
167  public class Main {
168       public static void main(String[] args) {
169            ApplicationContext ctx =new  ClassPathXmlApplicationContext
170                                          ("applicationContext.xml");
171            Demo db=(Demo)ctx.getBean("db");
172            db.method1();
173       }
174  }
175  ==============Global Transaction==============================
176  //AccountInter.java
177  package com.sathya.spring.global;
178  public interface AccountInter {
179       void transferMoney(int accno1,int accno2,int amount) throws Exception;
180  }
181  //AccountImpl.java
182  package com.sathya.spring.global;
183  import org.springframework.jdbc.core.JdbcTemplate;
184  import org.springframework.transaction.annotation.Transactional;
185  public class AccountImpl implements AccountInter
186  {
187       private JdbcTemplate jt1;
188       private JdbcTemplate jt2;
189       public void setJt1(JdbcTemplate jt1)
190       {
191            this.jt1=jt1;
192       }
193       public void setJt2(JdbcTemplate jt2)
194       {
195            this.jt2=jt2;
196       }
197       @SuppressWarnings("deprecation")
198       @Override
```

```
199       @Transactional(timeout=8)
200       public void transferMoney(int accno1, int accno2, int amount) throws Exception
201       {
202
203            int s1=jt1.queryForInt("select bal from Account1 where accno=?",accno1);
204            //deducting amount from balance
205            int s2 = s1-amount;
206            if(s2<500)
207            {
208                 throw new Exception();
209            }
210            int s3=jt2.queryForInt("select bal from Account2 where accno=?",accno2);
211            //adding amount to balance
212            int s4 = s3+amount;
213
214            jt1.update("update Account1 set bal=? where accno=?",s2,accno1);
215            Thread.sleep(6000);
216            jt2.update("update Account2 set bal=? where accno=?",s4,accno2);
217            System.out.println("Transaction successful.....");
218       }
219  }
220  --------------------applicationContext.xml-----------------------------
221  <beans xmlns="http://www.springframework.org/schema/beans"
222
223  xmlns:tx="http://www.springframework.org/schema/tx"
224  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
225  xsi:schemaLocation="http://www.springframework.org/schema/beans
226  http://www.springframework.org/schema/beans/spring-beans-3.0.xsd
227  http://www.springframework.org/schema/tx
228  http://www.springframework.org/schema/tx/spring-tx-3.0.xsd">
229       <!-- business class configuration -->
230       <bean id="accimpl" class="com.sathya.spring.global.AccountImpl">
231        <property name="jt1" ref="jt1"/>
232        <property name="jt2" ref="jt2"/>
233     </bean>
234     <!-- JdbcTemplate -->
235     <bean id="jt1" class="org.springframework.jdbc.core.JdbcTemplate">
236      <property name="dataSource" ref="ds1"/>
237     </bean>
238     <bean id="jt2" class="org.springframework.jdbc.core.JdbcTemplate">
239      <property name="dataSource" ref="ds2"/>
240     </bean>
241     <!-- DataSource configuration -->
242     <bean id="ds1" class="com.atomikos.jdbc.AtomikosDataSourceBean"
243                    init-method="init" destroy-method="close">
244       <property name="uniqueResourceName" value="A"/>
245            <property name="xaDataSourceClassName"
246                      value="oracle.jdbc.xa.client.OracleXADataSource"/>
247            <property name="xaProperties">
248                 <props>
249                      <prop key="databaseName">xe</prop>
250                      <prop key="user">system</prop>
251                      <prop key="password">root</prop>
252                      <prop key="URL">jdbc:oracle:thin:@localhost:1521:xe</prop>
253                 </props>
254            </property>
255            <property name="poolSize" value="1"/>
256       </bean>
257       <bean id="ds2" class="com.atomikos.jdbc.AtomikosDataSourceBean"
258                      init-method="init" destroy-method="close">
259            <property name="uniqueResourceName" value="B"/>
260            <property name="xaDataSourceClassName"
261                      value="com.mysql.jdbc.jdbc2.optional.MysqlXADataSource"/>
262            <property name="xaProperties">
263                 <props>
264                      <prop key="databaseName">test</prop>
```

```
                    <prop key="user">root</prop>
                    <prop key="password">root</prop>
                    <prop key="URL">jdbc:mysql://localhost:3306/test</prop>
                </props>
            </property>
            <property name="poolSize" value="1"/>
        </bean>
        <bean id="atomikostm"  class="com.atomikos.icatch.jta.UserTransactionManager"
                                    init-method="init" destroy-method="close"/>
        <bean id="txm"  class="org.springframework.transaction.jta.JtaTransactionManager">
            <property name="transactionManager">
                <ref bean="atomikostm"/>
            </property>
        </bean>
        <tx:annotation-driven  transaction-manager="txm"/>
</beans>
--------------------------Main.java-----------------------
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import com.sathya.spring.global.AccountInter;
public class Main {
      public static void main(String args[ ])
      {
           ApplicationContext  ctx=new ClassPathXmlApplicationContext
                                                    ("applicationContext.xml");
           Object  o = ctx.getBean("accimpl");
           AccountInter  inter=(AccountInter)o;
           try
           {
           inter.transferMoney(101,102,200);
           }
           catch(Exception e)
           {
                System.out.println("U r transaction unsuccessful");
           }
      }
}
==============Mail Test1=========================
//TestBean.java
import  org.springframework.mail.*;
import  org.springframework.mail.javamail.*;
public class  TestBean
{
      private  JavaMailSenderImpl  mailSender;
      public  void  setMailSender(JavaMailSenderImpl  mailSender)
      {
           this.mailSender=mailSender;
      }
      public  void  sendEmail()
      {
           try
           {
                SimpleMailMessage  smm =new SimpleMailMessage();
                smm.setFrom("aabcd2011@gmail.com");
                smm.setTo("abcd@hotmail.com");
                smm.setSubject("Mail  From  Spring Application");
                smm.setText("This  is  a sample  mail  from  Spring Mail  Application");
                mailSender.send(smm);
                System.out.println("Mail  sent  successfully");
           }
           catch(Exception  e)
           {
                System.out.println(e);
           }
      }
}
```

```
==============spring-config.xml====================
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
  http://www.springframework.org/schema/beans/spring-beans-2.5.xsd">
<!-- spring-config.xml -->
<bean id="id1" class="org.springframework.mail.javamail.JavaMailSenderImpl">
 <property name="host" value="smtp.gmail.com"/>
 <property name="username" value="aabcd2011@gmail.com"/>
 <property name="password" value="sathyatechnologies"/>
 <property name="javaMailProperties">
 <props>
     <prop key="mail.transport.protocol">smtp</prop>
     <prop key="mail.smtp.auth">true</prop>
     <prop key="mail.smtp.port">465</prop>
     <prop key="mail.smtp.socketFactory.port">465</prop>
     <prop key="mail.smtp.socketFactory.class">javax.net.ssl.SSLSocketFactory</prop>
     <prop key="mail.smtp.socketFactory.fallback">false</prop>
     <prop key="mail.smtp.quitwait">false</prop>
     <prop key="mail.smtp.starttls.enable">true</prop>
 </props>
 </property>
</bean>
<bean id="id2" class="TestBean">
 <property name="mailSender" ref="id1"/>
</bean>
</beans>
------------------------Client.java--------------------------
//Client.java
import org.springframework.context.*;
import org.springframework.context.support.*;
public class Client
{
    public static void main(String args[ ])
    {
        ApplicationContext context =new ClassPathXmlApplicationContext
                                        ("spring-config.xml");
        Object o = context.getBean("id2");
        TestBean tb=(TestBean)o;
         tb.sendEmail();
    }
}
===============JMS Application============================
-----------------------sender-----------------------------------
//MyMessageCreator.java
import javax.jms.*;
import org.springframework.jms.core.*;
public class MyMessageCreator implements MessageCreator
{
    @Override
    public Message createMessage(Session session)
      {
        TextMessage message = null;
        try
        {
          message = session.createTextMessage();
          message.setText("Hello World");
        }
        catch (JMSException e)
        {
          e.printStackTrace();
        }
        return message;
    }
};
```

```
397 //MessageSender.java
398 import org.springframework.jms.core.*;
399 public class MessageSender {
400    private JmsTemplate jmsTemplate;
401    public void setJmsTemplate(JmsTemplate jmsTemplate)
402    {
403       this.jmsTemplate = jmsTemplate;
404    }
405    public void sendMessage()
406    {
407          MyMessageCreator creator=new MyMessageCreator();
408          jmsTemplate.send(creator);
409    }
410 }
411 ----------spring.xml----------
412 <beans xmlns="http://www.springframework.org/schema/beans"
413 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
414 xsi:schemaLocation="http://www.springframework.org/schema/beans
415 http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
416  <bean id="cf" class="org.springframework.jndi.JndiObjectFactoryBean">
417   <property name="jndiName" value="qcfactory"/>
418   <property name="jndiEnvironment">
419       <props>
420          <prop key="java.naming.factory.initial">
421          com.sun.enterprise.naming.impl.SerialInitContextFactory</prop>
422          <prop key="java.naming.provider.url">iiop://localhost:4848</prop>
423     </props>
424   </property>
425  </bean>
426  <bean id="q" class="org.springframework.jndi.JndiObjectFactoryBean">
427   <property name="jndiName" value="que"/>
428     <property name="jndiEnvironment">
429       <props>
430          <prop key="java.naming.factory.initial">
431          com.sun.enterprise.naming.impl.SerialInitContextFactory</prop>
432          <prop key="java.naming.provider.url">iiop://localhost:4848</prop>
433     </props>
434   </property>
435  </bean>
436  <bean id="jt" class="org.springframework.jms.core.JmsTemplate">
437   <property name="connectionFactory" ref="cf"/>
438   <property name="defaultDestination" ref="q"/>
439  </bean>
440  <bean id="ms" class="MessageSender">
441   <property name="jmsTemplate" ref="jt"/>
442  </bean>
443 </beans>
444 ----------Client.java----------
445 //Client.java
446 import org.springframework.context.ApplicationContext;
447 import org.springframework.context.support.ClassPathXmlApplicationContext;
448 public class Client
449 {
450     public static void main(String args[])
451     {
452          ApplicationContext ctx=new ClassPathXmlApplicationContext("spring.xml");
453          MessageSender ms =(MessageSender)ctx.getBean("ms");
454          ms.sendMessage();
455          System.out.println("done");
456     }
457 }
458 ----------receiver----------
459 //MessageReceiver.java
460 import javax.jms.*;
461 import org.springframework.jms.core.*;
462 public class MessageReceiver {
```

send() method internally create Message() of Messa -Creator. It returns message object send() will send that message to dest

JNDI properties [server to server it will change

```
463       private JmsTemplate jmsTemplate;
464     public void setJmsTemplate(JmsTemplate jmsTemplate)
465     {
466       this.jmsTemplate = jmsTemplate;
467     }
468     public void receiveMessage()
469     {
470       Message message = jmsTemplate.receive();
471       TextMessage textMessage = null;
472       if (message instanceof TextMessage)
473       {
474         textMessage = (TextMessage)message;
475         try
476         {
477          System.out.println(textMessage.getText());
478         }
479         catch (JMSException e)
480         {
481           e.printStackTrace();
482         }
483       }
484     }
485   }
486   ----------spring.xml----------
487   <beans xmlns="http://www.springframework.org/schema/beans"
488   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
489   xsi:schemaLocation="http://www.springframework.org/schema/beans
490   http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
491    <bean id="cf"  class="org.springframework.jndi.JndiObjectFactoryBean">
492      <property name="jndiName" value="qcfactory"/>
493      <property  name="jndiEnvironment">
494          <props>
495              <prop  key="java.naming.factory.initia!">
496              com.sun.enterprise.naming.impl.SerialInitContextFactory</prop>
497              <prop  key="java.naming.provider.url">iiop://localhost:4848</prop>
498       </props>
499      </property>
500    </bean>
501    <bean id="q"  class="org.springframework.jndi.JndiObjectFactoryBean">
502     <property name="jndiName" value="que"/>
503       <property  name="jndiEnvironment">
504         <props>
505             <prop  key="java.naming.factory.initial">
506             com.sun.enterprise.naming.impl.SerialInitContextFactory</prop>
507             <prop  key="java.naming.provider.url">iiop://localhost:4848</prop>
508       </props>
509     </property>
510    </bean>
511    <bean id="jt" class="org.springframework.jms.core.JmsTemplate">
512      <property name="connectionFactory" ref="cf"/>
513      <property name="defaultDestination"  ref="q"/>
514    </bean>
515    <bean id="mr"  class="MessageReceiver">
516     <property name="jmsTemplate"  ref="jt"/>
517    </bean>
518  </beans>
519  ----------Client.java----------
520  import  org.springframework.context.ApplicationContext;
521  import  org.springframework.context.support.ClassPathXmlApplicationContext;
522  public class Client
523  {
524       public static void main(String args[])
525       {
526            ApplicationContext  ctx=new  ClassPathXmlApplicationContext("spring.xml");
527            MessageReceiver mr =(MessageReceiver)ctx.getBean("mr");
528            mr.receiveMessage();
```

```
            System.out.println("done");
        }
}
============Spring RMI application=======================
--------------------Server----------------------------
//DemoInter.java
public interface DemoInter
{
     String sayHello(String uname);
}
//DemoImpl.java
public class DemoImpl implements DemoInter
{
     public String sayHello(String uname)
     {
          return "Hello ..... "+uname;
     }
};
----------------applicationContext.xml--------------------
<!-- applicationContext.xml -->
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
  <bean id="di"  class="DemoImpl"/>
  <bean id="serviceExporter"
  class="org.springframework.remoting.rmi.RmiServiceExporter">
    <property name="service" ref="di"/>
      <property name="serviceName" value="one"/>
      <property name="serviceInterface" value="DemoInter"/>
  </bean>
</beans>
----------------Main.java-------------------------
//Main.java
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
class Main
{
     public static void main(String[] args)
     {
          ApplicationContext ctx=new ClassPathXmlApplicationContext
            ("applicationContext.xml");
     }
}
-------------------------Client-----------------------------------
//TestBean.java
public class TestBean
{
     private DemoInter demoInter;
     //setter injection
     public void setDemoInter(DemoInter demoInter)
     {
          this.demoInter=demoInter;
     }
     public void businessMethod()
     {
          try
          {
               String msg = demoInter.sayHello("RAM");
               System.out.println(msg);
          }
          catch(Exception e)
          {
               e.printStackTrace();
          }
     }
```

```
595 }
596 //DemoInter.java
597 public interface DemoInter
598 {
599       String sayHello(String uname);
600 }
601 ---------------------applicationContext.xml---------------------
602       <!-- applicationContext.xml -->
603 <beans xmlns="http://www.springframework.org/schema/beans"
604 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
605 xsi:schemaLocation="http://www.springframework.org/schema/beans
606 http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
607  <bean id="tb" class="TestBean">
608   <property name="demoInter" ref="rpfb"/>
609  </bean>
610  <bean id="rpfb" class="org.springframework.remoting.rmi.RmiProxyFactoryBean">
611   <property name="serviceUrl" value="rmi://localhost:1099/one"/>
612   <property name="serviceInterface" value="DemoInter"/>
613  </bean>
614 </beans>
615 ---------------------Main.java-----------------------------
616 //Main.java
617 import org.springframework.context.ApplicationContext;
618 import org.springframework.context.support.ClassPathXmlApplicationContext;
619 class Main
620 {
621       public static void main(String[] args)
622       {
623             ApplicationContext ctx=new ClassPathXmlApplicationContext
624             ("applicationContext.xml");
625             TestBean testBean = (TestBean)ctx.getBean("tb");
626              testBean.businessMethod();
627       }
628 }
629 ===============Spring HTTP Invoker Service Application=====================
630 ---------------------------HttpInvokerService-------------------------
631 //DemoInter.java
632 public interface DemoInter
633 {
634       String sayHello(String uname);
635 }
636 //DemoImpl.java
637 public class DemoImpl implements DemoInter
638 {
639       public String sayHello(String uname)
640       {
641             return "Hello ..... "+uname;
642       }
643 };
644 ----------------------web.xml--------------------------
645 <web-app>
646   <servlet>
647       <servlet-name>spring</servlet-name>
648       <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
649       <load-on-startup>1</load-on-startup>
650   </servlet>
651   <servlet-mapping>
652     <servlet-name>spring</servlet-name>
653     <url-pattern>*.html</url-pattern>
654   </servlet-mapping>
655 </web-app>
656 ---------------------spring-servlet.xml---------------------
657  <!-- spring-servlet.xml -->
658 <beans xmlns="http://www.springframework.org/schema/beans"
659 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
660 xsi:schemaLocation="http://www.springframework.org/schema/beans
```

```
http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
  <bean id="simpleUrlMappings"
       class="org.springframework.web.servlet.handler.SimpleUrlHandlerMapping">
        <property name="mappings">
          <props>
              <prop key="/test.html">serviceExporter</prop>
           </props>
        </property>
    </bean>
  <bean id="di"  class="DemoImpl"/>
  <bean id="serviceExporter"
  class="org.springframework.remoting.httpinvoker.HttpInvokerServiceExporter">
    <property name="service" ref="di"/>
      <property name="serviceInterface" value="DemoInter"/>
  </bean>
</beans>
--------------------HttpInvoerClient-----------------------------------
//TestBean.java
public class TestBean
{
     private DemoInter demoInter;
     //setter injection
     public void setDemoInter(DemoInter demoInter)
     {
          this.demoInter=demoInter;
     }
     public void businessMethod()
     {
          try
          {
               String msg = demoInter.sayHello("Sathya");
               System.out.println(msg);
          }
          catch(Exception e)
          {
               e.printStackTrace();
          }
     }
}
//DemoInter.java
public interface DemoInter
{
     String sayHello(String uname);
}
--------------------applicationContext.xml-----------------------------
     <!-- applicationContext.xml -->
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
  <bean id="tb" class="TestBean">
    <property name="demoInter" ref="hipfb"/>
  </bean>
  <bean id="hipfb"
      class="org.springframework.remoting.httpinvoker.HttpInvokerProxyFactoryBean">
   <property name="serviceUrl" value="http://localhost:2015/Service/test.html"/>
   <property name="serviceInterface" value="DemoInter"/>
  </bean>
</beans>
------------------Main.java-------------------
//Main.java
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
class Main
{
     public static void main(String[] args)
```

BeanNameUrlHandlerMapping (or)

tomcat port

HttpInvokerService [project name]

```
727       {
728            ApplicationContext  ctx=new ClassPathXmlApplicationContext
729                                                  ("applicationContext.xml");
730            TestBean  testBean = (TestBean)ctx.getBean("tb");
731             testBean.businessMethod();
732       }
733  }
734  ===========Spring  Security  App1======================
735  <!--  index.jsp -->
736       <center>
737       <h1>
738        Any one Can  visit  This  Page
739     </h1>
740  <p><a href="secure/Admin.jsp">AdminPage</a></p>
741  <p><a href="secure/Faculty.jsp">FacultyPage</a></p>
742  </center>
743  <!--  Admin.jsp -->
744  <center>
745  Welcome To Admin Page  <br>
746  Today Date is : <%=new java.uti!.Date()%>
747  <p><a href="../j_spring_security_logout">Logout</a>
748  </center>
749  <!-- Faculty.jsp -->
750  Faculty  Page  visited.
751  <p><a href="../j_spring_security_logout">Logout</a>
752  <!-- logout.jsp -->
753  <center>
754  You have successfully LoggedOut<br>
755  <a href="index.jsp">start again</a>
756  </center>
757  ------------------web.xml------------------------------
758  <web-app>
759  <listener>
760     <listener-class>org.springframework.web.context.ContextLoaderListener
761     </listener-class>
762  </listener>
763  <context-param>
764       <param-name>contextConfigLocation</param-name>
765       <param-value>/WEB-INF/security-config.xml</param-value>
766  </context-param>
767  <welcome-file-list>
768        <welcome-file>index.jsp</welcome-file>
769  </welcome-file-list>
770  <filter>
771     <filter-name>springSecurityFilterChain</filter-name>
772     <filter-class>org.springframework.web.filter.DelegatingFilterProxy</filter-class>
773  </filter>
774  <filter-mapping>
775   <filter-name>springSecurityFilterChain</filter-name>
776   <url-pattern>/*</url-pattern>
777  </filter-mapping>
778  </web-app>
779  --------------------security-config.xml----------------------
780       <!-- security-config.xml -->
781  <beans xmlns:security="http://www.springframework.org/schema/security"
782     xmlns="http://www.springframework.org/schema/beans"
783     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
784     xsi:schemaLocation="http://www.springframework.org/schema/beans
785      http://www.springframework.org/schema/beans/spring-beans-3.0.xsd
786      http://www.springframework.org/schema/security
787      http://www.springframework.org/schema/security/spring-security-3.0.3.xsd">
788
789    <security:http use-expressions="true">
790     <security:intercept-url pattern="/index.jsp"   access="permitAll" />
791     <security:intercept-url pattern="/secure/Admin.jsp" access="hasRole('admin')" />
792     <security:intercept-url pattern="/secure/Faculty.jsp" access="hasRole('faculty')" />
```

```
793     <security:http-basic/>
794       <security:logout logout-success-url="/logout.jsp"/>
795    </security:http>
796    <security:authentication-manager>
797     <security:authentication-provider>
798      <security:user-service>
799       <security:user name="satya" password="tech" authorities="admin" />
800       <security:user name="ssreddy" password="ssreddy" authorities="faculty" />
801      </security:user-service>
802     </security:authentication-provider>
803    </security:authentication-manager>
804  </beans>
805  ============Spring Security  App2======================
806  <!-- index.jsp -->
807  <center>
808         Any one Can  visit  This  Page
809  <p><a href="secure/Admin.jsp">Admin Page</a></p>
810  <p><a href="secure/Faculty.jsp">Faculty Page</a></p>
811  </center>
812  <!-- login.jsp -->
813  <html>
814   <head>
815    <title>Security WebApp login page</title>
816   </head>
817   <body bgcolor="#cccccc">
818   <blockquote>
819   <h2>Please enter your user name and password:</h2>
820   <p>
821   <form method="POST" action="j_spring_security_check">
822   <table border=1>
823    <tr>
824     <td>Username:</td>
825     <td><input type="text" name="j_username"></td>
826    </tr>
827    <tr>
828     <td>Password:</td>
829     <td><input type="password" name="j_password"></td>
830    </tr>
831    <tr>
832     <td colspan=2 align=right><input type=submit
833                          value="Submit"></td>
834    </tr>
835   </table>
836   </form>
837   </blockquote>
838   </body>
839  </html>
840
841  <!-- login_failed.jsp -->
842  <h1> U r Credentials are not valid </h1>
843  <br>
844  <a href="login.jsp"> enter again </a>
845
846  <!--  logout.jsp  -->
847  <center>
848  You have successfully LoggedOut<br>
849  <a href="index.jsp">start again</a>
850  </center>
851
852  <!-- Faculty.jsp -->
853  Faculty  Page  visited.
854  <p><a href="../j_spring_security_logout">Logout</a>
855
856  <!-- Admin.jsp -->
857  <center>
858  Welcome To Admin Page  <br>
```

```
Today Date is : <%=new java.util.Date()%>
<p><a href="../j_spring_security_logout">Logout</a>
</center>
----------web.xml----------
<web-app>
<listener>
   <listener-class>org.springframework.web.context.ContextLoaderListener
   </listener-class>
 </listener>
<context-param>
     <param-name>contextConfigLocation</param-name>
     <param-value>
       /WEB-INF/security-config.xml,/WEB-INF/applicationContext.xml
     </param-value>
</context-param>
<welcome-file-list>
      <welcome-file>index.jsp</welcome-file>
</welcome-file-list>
<filter>
  <filter-name>springSecurityFilterChain</filter-name>
  <filter-class>org.springframework.web.filter.DelegatingFilterProxy</filter-class>
</filter>
<filter-mapping>
 <filter-name>springSecurityFilterChain</filter-name>
 <url-pattern>/*</url-pattern>
</filter-mapping>
</web-app>
----------security-config.xml----------
<beans xmlns:security="http://www.springframework.org/schema/security"
   xmlns="http://www.springframework.org/schema/beans"
   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
   xsi:schemaLocation="http://www.springframework.org/schema/beans
            http://www.springframework.org/schema/beans/spring-beans-4.0.xsd
            http://www.springframework.org/schema/security
            http://www.springframework.org/schema/security/spring-security-3.2.xsd">
   <security:http use-expressions="true">
     <security:intercept-url pattern="/index.jsp" access="permitAll" />
     <security:intercept-url pattern="/secure/Admin.jsp" access="hasRole('admin')"/>
     <security:intercept-url pattern="/secure/Faculty.jsp" access="hasRole('faculty')"/>
       <security:form-login    login-page="/login.jsp"
                               authentication-failure-url="/login_failed.jsp"/>
       <security:logout logout-success-url="/logout.jsp"/>
   </security:http>
   <security:authentication-manager>
     <security:authentication-provider>
      <security:jdbc-user-service  data-source-ref="ds"
      users-by-username-query="select username,password,enabled from users where
                               username=?"
      authorities-by-username-query=" select u.username, ur.authority from users u,
        user_roles ur where u.user_id = ur.user_id and u.username =?"/>
     </security:authentication-provider>
   </security:authentication-manager>
</beans>
----------applicationContext.xml----------
<beans xmlns="http://www.springframework.org/schema/beans"
   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
   xsi:schemaLocation="http://www.springframework.org/schema/beans
            http://www.springframework.org/schema/beans/spring-beans-4.0.xsd">
<bean id="ds" class="org.springframework.jdbc.datasource.DriverManagerDataSource">
      <property name="driverClassName" value="com.mysql.jdbc.Driver"/>
      <property name="url" value="jdbc:mysql://localhost:3306/test"/>
      <property name="username"  value="root"/>
      <property name="password"  value="root"/>
</bean>
</beans>
```